

LeetCode 150 算法套路

目录

LeetCode Top Interview 150 Python Tutorial	8
Study Routes	8
Completed Topics	8
Repository Map	9
Setup	9
Validate Metadata	9
Generate Scaffold	9
Run Tests	9
Complete One Problem	9
01 —Array / String（融合版）	10
一例速记	10
思维路径还原	10
学习目标	11
几何示意	11
方法变形（4 类）	36
思考路标（条件反射）	36
易错点	37
典型应用例题	37
自测题	39
题目全览（24 题）	40
融合版说明	41
02 —Two Pointers（融合版）	41
一例速记	42
思维路径还原	42
学习目标	43
几何示意	43
方法变形（4 类）	57
思考路标（条件反射）	58
易错点	58

典型应用例题	59
自测题	61
题目全览 (5 题)	61
融合版说明	62
03 —Sliding Window (融合版)	62
一例速记	63
思维路径还原	63
学习目标	64
几何示意	64
方法变形 (4 类)	88
思考路标 (条件反射)	89
易错点	89
典型应用例题	90
自测题	92
题目全览 (4 题)	93
融合版说明	93
04 —Binary Search (融合版)	94
一例速记	94
思维路径还原	94
学习目标	95
几何示意	95
方法变形 (4 类)	115
思考路标 (条件反射)	117
易错点	117
典型应用例题	118
自测题	120
题目全览 (7 题)	121
融合版说明	121
05 —Binary Search Tree (融合版)	122
一例速记	122
思维路径还原	122
学习目标	123
几何示意	123
方法变形 (4 类)	140
思考路标 (条件反射)	141
易错点	141
典型应用例题	142
自测题	144
题目全览 (3 题)	145

融合版说明	145
06 —Binary Tree BFS (融合版)	146
一例速记	146
思维路径还原	146
学习目标	146
几何示意	147
方法变形 (4 类)	153
思考路标 (条件反射)	153
易错点	154
典型应用例题	154
自测题	157
题目全览 (4 题)	157
融合版说明	158
07 —Binary Tree DFS (融合版)	158
一例速记	158
思维路径还原	159
学习目标	160
几何示意	160
方法变形 (4 类)	170
思考路标 (条件反射)	171
易错点	172
典型应用例题	172
自测题	174
题目全览 (14 题)	175
融合版说明	176
08 —Backtracking (融合版)	176
一例速记	176
思维路径还原	177
学习目标	177
几何示意	178
方法变形 (4 类)	188
思考路标 (条件反射)	189
易错点	189
典型应用例题	190
自测题	192
题目全览 (7 题)	193
融合版说明	193
09 —Divide & Conquer (融合版)	194
一例速记	194

思维路径还原	194
学习目标	195
几何示意	195
方法变形 (4 类)	204
思考路标 (条件反射)	205
易错点	205
典型应用例题	206
自测题	209
题目全览 (4 题)	209
融合版说明	209
10 —Dynamic Programming 1D (融合版)	210
一例速记	210
思维路径还原	211
学习目标	211
几何示意	212
方法变形 (4 类)	220
思考路标 (条件反射)	220
易错点	221
典型应用例题	222
自测题	223
题目全览 (5 题)	224
融合版说明	224
11 —Dynamic Programming Multidimensional (融合版)	225
一例速记	225
思维路径还原	225
学习目标	226
几何示意	227
方法变形 (4 类)	239
思考路标 (条件反射)	240
易错点	240
典型应用例题	241
自测题	243
题目全览 (9 题)	243
融合版说明	243
12 —Linked List (融合版)	244
一例速记	244
思维路径还原	245
学习目标	246
几何示意	246

方法变形（4 类）	259
思考路标（条件反射）	259
易错点	260
典型应用例题	260
自测题	262
题目全览（11 题）	262
融合版说明	263
13—Stack（融合版）	263
一例速记	263
思维路径还原	264
学习目标	264
几何示意	265
方法变形（3 类）	273
思考路标（条件反射）	274
易错点	275
典型应用例题	275
自测题	276
题目全览（5 题）	277
融合版说明	277
14—Hash Table（融合版）	278
一例速记	278
思维路径还原	278
学习目标	279
几何示意	280
方法变形（4 类）	289
思考路标（条件反射）	290
易错点	290
典型应用例题	291
自测题	292
题目全览（9 题）	293
融合版说明	293
15—Intervals（融合版）	294
一例速记	294
思维路径还原	295
学习目标	295
几何示意	296
方法变形（4 类）	303
思考路标（条件反射）	304
易错点	304

典型应用例题	305
自测题	307
题目全览 (4 题)	307
融合版说明	307
16 —Heap (融合版)	308
一例速记	308
思维路径还原	309
学习目标	309
几何示意	309
方法变形 (4 类)	321
思考路标 (条件反射)	322
易错点	322
典型应用例题	323
自测题	324
题目全览 (4 题)	324
融合版说明	325
17 —Kadane’s Algorithm (融合版)	326
一例速记	326
思维路径还原	326
学习目标	327
几何示意	327
方法变形 (3 类)	334
思考路标 (条件反射)	335
易错点	336
典型应用例题	336
自测题	337
题目全览 (2 题)	338
融合版说明	338
18 —Graph BFS (融合版)	339
一例速记	339
思维路径还原	339
学习目标	339
几何示意	340
方法变形 (3 类)	347
思考路标 (条件反射)	348
易错点	348
典型应用例题	348
自测题	351
题目全览 (3 题)	351

融合版说明	352
19 —Graph General (融合版)	352
一例速记	352
思维路径还原	353
学习目标	353
几何示意	354
方法变形 (4 类)	364
思考路标 (条件反射)	365
易错点	365
典型应用例题	366
自测题	368
题目全览 (6 题)	369
融合版说明	369
20 —Trie (融合版)	370
一例速记	370
思维路径还原	370
学习目标	370
几何示意	371
方法变形 (3 类)	380
思考路标 (条件反射)	381
易错点	381
典型应用例题	382
自测题	385
题目全览 (3 题)	385
融合版说明	385
21 —Bit Manipulation (融合版)	386
一例速记	386
思维路径还原	387
学习目标	387
几何示意	388
方法变形 (3 类)	396
思考路标 (条件反射)	397
易错点	397
典型应用例题	397
自测题	399
题目全览 (6 题)	399
融合版说明	400
22 —Math (融合版)	401
一例速记	401

思维路径还原	401
学习目标	402
几何示意	402
方法变形 (3 类)	410
思考路标 (条件反射)	410
易错点	411
典型应用例题	411
自测题	413
题目全览 (6 题)	413
融合版说明	414
23 —Matrix (融合版)	414
一例速记	415
思维路径还原	415
学习目标	416
几何示意	416
方法变形 (3 类)	426
思考路标 (条件反射)	427
易错点	427
典型应用例题	428
自测题	430
题目全览 (5 题)	430
融合版说明	430

LeetCode Top Interview 150 Python Tutorial

This repository is an English-first, Python-based tutorial for LeetCode's Top Interview 150 list. It is organized for advanced systematic learners who want reusable patterns, runnable examples, and a maintainable study workflow.

Study Routes

- Official order: docs/official-order.md
- Pattern roadmap: docs/pattern-roadmap.md

Completed Topics

Completed topics have final English tutorials, implemented Python solutions, and active pytest coverage.

- ☐ Two Pointers: 5 / 5 problems complete

Repository Map

- `data/top_interview_150.yaml`: canonical metadata for all 150 problems.
- `docs/problems/`: English tutorial pages grouped by pattern.
- `solutions/`: Python Solution placeholders grouped by pattern.
- `tests/`: skipped pytest examples for unimplemented problems.
- `scripts/validate_metadata.py`: validates metadata consistency.
- `scripts/generate_scaffold.py`: regenerates missing scaffold files.

Setup

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[dev]'
```

Validate Metadata

```
python scripts/validate_metadata.py data/top_interview_150.yaml
```

Generate Scaffold

```
python scripts/generate_scaffold.py data/top_interview_150.yaml --root .
```

The generator creates missing docs, solution files, tests, and indexes. It does not overwrite existing files unless `--force` is passed.

Run Tests

```
pytest
```

Unimplemented problem tests are skipped until their matching solution is completed.

Complete One Problem

1. Implement the method in `solutions/<pattern>/pNNN_slug.py`.
2. Remove the skip marker from `tests/<pattern>/test_pNNN_slug.py`.
3. Replace the problem doc's instructional TODO sections with final teaching content.
4. Run the problem-specific test file.
5. Run `pytest` before committing.

01 — Array / String (融合版)

难度: □□□□□ 题数: 24 核心套路: 原地修改、单次扫描、贪心、字符串技巧 本文件: 覆盖 array_string
24 题的算法套路总结 + 典型题精讲 + 自测

一例速记

数组原地修改: 双指针 (read/write) 一次扫描, $O(n)$ 时间 $O(1)$ 空间 (26、27、80、88) 多次扫描
预计算: 前缀积/后缀积 (238)、排序计数 (274)、前缀余量 (134) **Boyer-Moore** 投票: 候选 + 计数, 正负抵消 (169) 数组旋转: 三次翻转 (189) 或环形替换, $O(1)$ 空间 贪心策略: 跳跃游戏维护最远可达 (55/45)、加油站累计余量 (134)、糖果双向扫描 (135) 股票买卖: 单笔维护历史 min (121)、多笔累加上升段 (122) 接雨水: 双指针 left/right 维护两侧最大高度 (42) 字符串处理: 双指针相撞 (151)、模拟 (12/13/6/68)、KMP (28)、最长公共前缀 (14) 随机化数据结构: 哈希表 + 动态数组 (380)

思维路径还原

“看到 ‘in-place / $O(1)$ space’ 提示 → 想双指针 write/read 模式: read 从左到右扫描, write 只在满足条件时前进。26 题去重: read 比较 `nums[r]` 与 `nums[r-1]`, 不等则写入 `nums[w++]`。27 题移除元素: read 遇到非目标值才写。80 题允许至多 2 次重复: 把比较条件从 `nums[r] != nums[r-1]` 扩展为 `w < 2 or nums[r] != nums[w-2]`。

看到 88 题合并两个有序数组: 从后往前写! `nums1` 末尾是空位, 从大到小向后填, 不会覆盖未读区域。

看到 ‘majority / 出现 $> n/2$ 次’ → Boyer-Moore 投票法: 维护候选 `cand` 和计数 `cnt`, 遍历时 `cnt==0` 则换候选, 同候选则 +1, 否则 -1, 最终剩下的 `cand` 就是多数元素 (169)。

看到 ‘股票买卖’ → 121 单笔: 维护历史最低价 `min_price`, 每次更新当前利润; 122 多笔: 不需要状态机, 所有相邻上升段 (`prices[i] > prices[i-1]`) 的差值累加。

看到 ‘跳跃’ → 55 是否可达: 贪心维护 `farthest`, 遍历时若 `i > farthest` 则已无法前进; 45 最少跳数: BFS 思路, 维护当前段终点 `end` 和下一段最远 `farthest`, 到达 `end` 时跳数 +1。

看到 ‘加油站 / 环形’ → 134: 累计总余量 `total`, 若为负则无解; 贪心找起点: 一旦从某站出发累计余量为负, 则起点后移到下一站, 重置余量为 0。

看到 ‘接雨水’ → 42: 双指针 left/right, 分别维护 `max_left` 和 `max_right`, 每次移动较低一侧 (短板决定蓄水量), 当前蓄水 = 两侧最大值的 min - 当前高度。

看到‘字符串匹配 / **needle in haystack**’ → 28: 用 KMP, 预处理 needle 的 failure 数组 $O(m)$, 匹配 $O(n)$, 总体 $O(n+m)$ 而非暴力 $O(nm)$ 。

看到‘前缀 / 除自身以外的乘积’ → 238: 两次扫描, 第一次从左算前缀积存入 res, 第二次从右乘后缀积, 避免除法且 $O(1)$ 额外空间 (不含输出数组)。

看到‘罗马数字’ → 12/13: 贪心从大到小枚举 (整数转罗马), 或线性扫描逢小在大则减 (罗马转整数)。

看到‘文本对齐 / **zigzag** / 翻转单词’ → 模拟题, 关键是细心处理边界: 68 最后一行左对齐; 6 按行分组; 151 先 split 再 reverse join。”

学习目标

- 掌握数组原地修改的双指针 read/write 模板及其变体
 - 熟练运用贪心 + Boyer-Moore 投票 + 前缀积等单次扫描技巧
 - 字符串题的结构化处理: KMP、双指针对撞、模拟
 - 能识别”跳跃 / 加油站 / 接雨水”三类贪心题并直接套模板
 - 掌握 h-index (274) 和随机 $O(1)$ 数据结构 (380) 的设计思路
-

几何示意

图 read/write 双指针 (LC 26)

图 Boyer-Moore 投票 (LC 169)

图接雨水双指针 (LC 42)

抽象成方法 (标准模板代码)

套路
1:
read/write
双指
针
(去重
/ 移
除)
适用
题:
26、
27、
80

```
“python
# 26:
升序
数组
去重,
每元
素保
留 1
次 def
re-
move_duplicates_once(nums:
list[int])
-> int:
w = 0
for r in
range(len(nums)):
if r ==
0 or
nums[r]
!=
nums[r
- 1]:
nums[w]
=
nums[r]
w +=
1
return
w
```

```
# 80:
升序
数组
去重,
每元
素至
多保
留 2
次 def
re-
move_duplicates_twice(nums:
list[int])
-> int:
w = 0
for r in
range(len(nums)):
if w <
2 or
nums[r]
!=
nums[w
- 2]:
nums[w]
=
nums[r]
w +=
1
return
w
```

```

# 27:
移除
所有
值为
val 的
元素
def re-
move_element(nums:
list[int],
val:
int) ->
int: w
= 0
for r in
range(len(nums)):
if
nums[r]
!= val:
nums[w]
=
nums[r]
w +=
1
return
w ““
> 关
键规
律: w
< k
or
nums[r]
!=
nums[w
- k]
可泛
化为”
保留
至多
k 次”。

```

套路
2: 从
后向
前合
并
(88 合
并有序数
组)
适用
题:
88

```

python
def
merge_sorted(nums1:
list[int],
m:
int,
nums2:
list[int],
n:
int)
->
None:
"""
原地
合并,
从尾
部向
前填
充避
免覆
盖未
读区
域。
"""
p1,
p2, w
= m -
1, n
- 1,
m + n
- 1
while
p2 >=
0: if
p1 >=
0 and
nums1[p1]
>
nums2[p2]:
nums1[w]
=
nums1[p1]
p1 -=
1

```

###

套路

3:

Boyer-

Moore

投票

法

(多数

元素)

适用

题:

169

```
python
def
majority_element(nums:
list[int])
->
int:
"""
找出
出现
次数
> n/2
的元
素。
时间
O(n),
空间
O(1)。
"""
cand,
cnt =
nums[0],
0
for
x in
nums:
if
cnt
== 0:
cand
= x
cnt
+= 1
if x
==
cand
else
-1
return
cand
```

> 注意:
该算法依赖多数元素必然存在的前提;
若题目不保证则需第二次遍历验证。

套路
4: 贪心维护最远可达
(跳跃游戏)
适用题:
55
(判断可达)、
45
(最少跳数)

```
“python
# 55:
是否
能到
达末
尾 def
can_jump(nums:
list[int])
->
bool:
far-
thest
= 0
for i,
jump
in enu-
mer-
ate(nums):
if i >
far-
thest:
return
False
far-
thest =
max(farthest,
i +
jump)
return
True
```

```
# 45:
最少
跳跃
次数
(贪心
BFS)
def
jump_min(nums:
list[int])
-> int:
jumps
= 0
end =
0 # 当
前”一
跳”所
能覆
盖的
最远
终点
far-
thest
= 0 #
从当
前段
内任
意点
出发
的最
远覆
盖 for
i in
range(len(nums)
- 1):
far-
thest =
max(farthest,
i +
nums[i])
if i ==
end:
jumps
+= 1
end =
```

套路
5: 股
票买
卖贪
心
适用
题:
121
(单
笔)、
122
(多
笔)

```
“python
# 121:
只能
买卖
一次,
求最
大利
润 def
max_profit_once(prices:
list[int])
-> int:
    min_price
    =
    float(
    ‘inf’)
    profit
    = 0
    for p
    in
    prices:
        min_price
        =
        min(min_price,
        p)
        profit
        =
        max(profit,
        p -
        min_price)
    return
    profit
```

```
# 122:
可以
多次
买卖,
求最
大利
润
(累加
所有
上升
段)
def
max_profit_multi(prices:
list[int])
-> int:
    profit
    = 0
    for i
    in
    range(1,
    len(prices)):
        if
        prices[i]
        >
        prices[i
        - 1]:
            profit
            +=
            prices[i]
        -
        prices[i
        - 1]
    return
    profit
    ""

####
套路
6: 接
雨水
双指
针
```

适用
题:
42

```
python
def
trap_rain(height:
list[int])
->
int:
"""
双指
针，
短板
决定
当前
水位。
时间
O(n)，
空间
O(1)。
"""
left,
right
= 0,
len(height)
- 1
max_left
=
max_right
= 0
water
= 0
while
left
<
right:
if
height[left]
<=
height[right]:
if
height[left]
>=
max_left:
max_left
=
height[left]
```

套路
7: 前
缀积
+ 后
缀积
(除自
身以
外的
乘积)
适用
题:
238

```
python
```

```
def
```

```
product_except_self(nums:
```

```
list[int])
```

```
->
```

```
list[int]:
```

```
"""
```

```
两次
```

```
线性
```

```
扫描,
```

```
 $O(1)$ 
```

```
额外
```

```
空间
```

```
(不含
```

```
输出
```

```
数
```

```
组)。
```

```
""" n
```

```
=
```

```
len(nums)
```

```
res =
```

```
[1] *
```

```
n #
```

```
第一
```

```
次:
```

```
res[i]
```

```
=
```

```
nums[0]
```

```
* ...
```

```
*
```

```
nums[i-1]
```

```
prefix
```

```
= 1
```

```
for i
```

```
in
```

```
range(n):
```

```
res[i]
```

```
=
```

```
prefix
```

```
prefix
```

```
*=
```

```
nums[i]
```

```
# 第
```

```
二次:
```

```
...
```

###

套路

8:

KMP

字符

串匹

配

适用

题:

28

```

python
def
str_str_kmp(haystack:
str,
needle:
str)
->
int:
"""KMP
算法,
时间
 $O(n+m)$ ,
空间
 $O(m)$ 。
"""
if
not
needle:
return
0 #
构建
failure
数组
(最长
相同
前后
缀长
度)
m =
len(needle)
fail
= [0]
* m j
= 0
for i
in
range(1,
m):
while
j > 0
and
needle[i]
!=
needle[j]:

```

速查
表

| 题型

特征 |

套路 |

时间 |

空间 |

|—|—|

—|—|

| 原地

去重

(每元

素保

留

1 次) |

read/write

双指

针 |

 $O(n)$ | $O(1)$

|| 原

地去

重

(至多

k 次)

| $w <$

k or

nums[r]

!=

nums[w-k]

|

 $O(n)$ | $O(1)$

|| 合

并两

有序

数组

(原地)

| 从后

向前

双指

针 |

 $O(m+$ $n)$ | $O(1)$ |

| 多数

LC 26: 双指针去重 (Remove Duplicates)

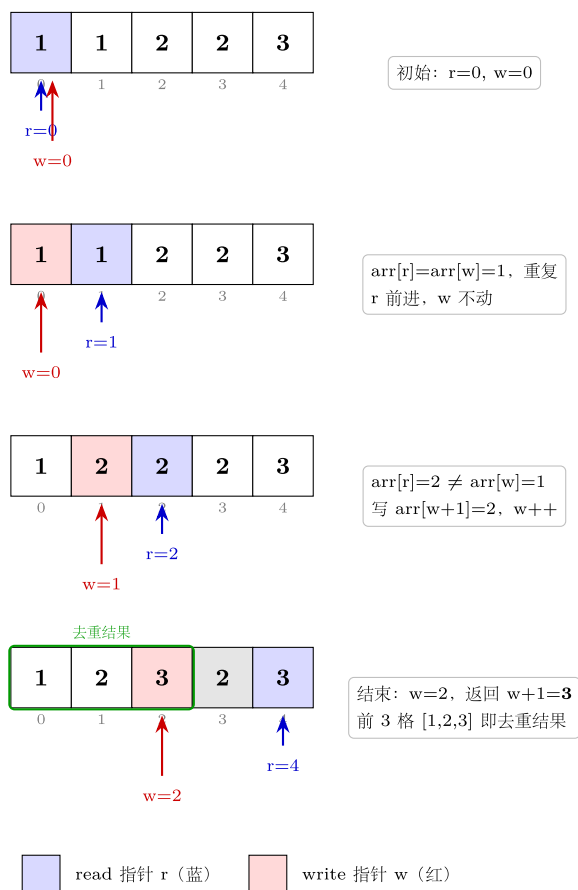


图 1: read/write 双指针 4 步演化示意

LC 169: Boyer-Moore 投票法 (Majority Element)



图 2: 7 步投票演化表

LC 42: 接雨水 (Trapping Rain Water) 双指针法

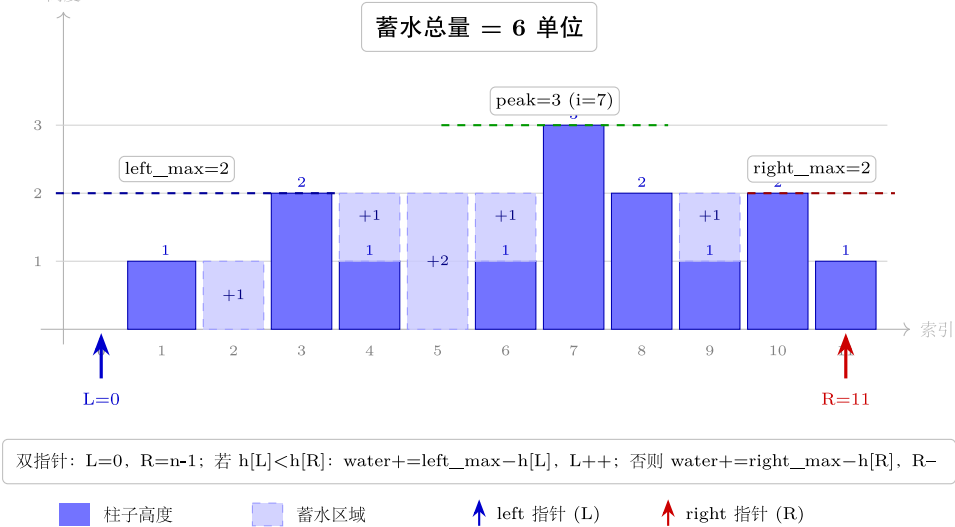


图 3: 柱状图 + left/right 指针 + 阴影蓄水

方法变形 (4 类)

变形 1: 原地修改系列

- **26** (每元素 1 次) → **80** (至多 2 次): 条件从 `nums[r] != nums[r-1]` 改为 `w < 2 or nums[r] != nums[w-2]`; 泛化为至多 k 次只需把 2 换成 k。
- **27** (移除指定值): 条件改为 `nums[r] != val`, write 指针同向推进。
- **88** (合并两数组): 逆向版 read/write, 从末尾向前写, 避免覆盖未处理元素。

变形 2: 股票买卖系列

- **121** (单次): 维护历史最低价, $O(n)$ 一次扫描。
- **122** (多次): 贪心累加所有相邻上升段, 不需要显式的状态机。
- 进阶扩展 (非本 category): **309** (含冷却期) → DP 三状态; **123** (至多 2 次) → DP; **188** (至多 k 次) → DP。

变形 3: 跳跃游戏系列

- **55** (是否能到达): 只需判断当前 index 是否超过 farthest, 返回 bool。
- **45** (最少跳数): BFS 思路, 当遍历指针触达当前”段终点”时, 跳数 +1, 段终点更新为目前已知的最远覆盖。
- 关键区分: 55 关注”能否”, 45 关注”多少次”; 45 的 end 变量代表”必须在本次跳跃内解决”的边界。

变形 4: 字符串重排系列

- **151** (翻转单词): `s.split()` 自动处理多余空格, `' '.join(reversed(...))` 重组, Python 两行搞定; 若要求原地则需先 reverse 整个数组再 reverse 每个单词。
- **6** (Zigzag 变换): 按行分组, 维护 row 和方向 direction, 遍历字符依次追加到对应行。
- **28** (字符串匹配): 暴力 $O(nm)$ → KMP $O(n+m)$; Python 内置 `str.find` 即为优化后的实现。
- **68** (文本对齐): 模拟贪心分行, 每行尽量多放单词; 空格分配: $(\text{maxWidth} - \text{总字符数}) // (\text{间隙数})$ 加余数分摊; 最后一行左对齐。

思考路标 (条件反射)

1. 看到 “in-place / $O(1)$ space” + 有序数组 → 双指针 read/write
2. 看到 “majority / 出现 $> n/2$ ” → Boyer-Moore 投票法
3. 看到 “product except self / 前缀积” → 两次线性扫描, 前缀积 + 后缀积
4. 看到 “jump / 最远可达” → 贪心维护 farthest; 求最少次数时加段终点 end

5. 看到 “gas station / 环形” → 累计总余量判断有无解；贪心找起点
 6. 看到 “trapping water / 接雨水” → 双指针，短板决定水位
 7. 看到 “find needle in haystack” → KMP ($O(n+m)$)
 8. 看到 “h-index / 排名阈值” → 降序排序后线性扫描，或计数数组 $O(n)$
 9. 看到 “ $O(1)$ insert / delete / getRandom” → 哈希表（值 → 下标）+ 动态数组 (380)
 10. 看到 “rotate array k steps” → 三次 reverse：全 → 前 k → 后 $n-k$
 11. 看到 “sliding window” 关键字 → 跳到 sliding_window category
 12. 看到 “two sum / 配对” → 跳到 hash_table category
 13. 看到 “sorted array + binary” → 跳到 binary_search category
 14. 看到 “Roman numerals” → 贪心贪心：整数 → 罗马从大到小减；罗马 → 整数逢小在大则减
-

易错点

1. read/write 边界： $r == 0$ 时无前驱可比，特判（或从 $r=1, w=1$ 开始），否则数组长度为 0 时越界。
 2. Boyer-Moore 假设：算法依赖“多数元素必然存在”；LeetCode 169 保证了这一点，但若题目不保证，必须第二轮遍历验证候选。
 3. 股票 121 vs 122：121 只能单次买卖，关键在维护历史最低价而非每日差值；122 多次买卖，每个上升段加总即可，不要把 122 的贪心用到 121 上。
 4. 跳跃 55 vs 45：55 一旦 $i > \text{farthest}$ 立即返回 False；45 的 end 是当前跳跃段内必须完成的终点，循环到 $\text{len}-2$ 而非 $\text{len}-1$ （最后一步不需要再跳）。
 5. 接雨水双指针方向：移动较低一侧（不是较高一侧），因为较低一侧的水量由它自身决定（短板）；混淆移动方向会导致结果错误。
 6. 189 旋转越界： k 可能大于数组长度，务必先取 $k = k \% n$ ，否则三次 reverse 的切分点错误。
 7. 68 文本对齐最后一行：最后一行不用均匀分散，直接单词间一个空格，末尾补齐 $\text{maxWidth} - \text{used}$ 个空格，与普通行逻辑不同，需要单独处理。
 8. 238 前缀积不用除法：若数组含 0 则除法不可行；模板中使用两次乘法遍历天然规避了这一问题。
 9. KMP failure 数组：fail[0] 恒为 0；构建时从 $i=1$ 开始；匹配成功后 $j = \text{fail}[j-1]$ 而非 $j = 0$ ，否则漏掉重叠匹配。
 10. 380 交换删除：删除时将目标元素与数组末尾元素交换，然后 pop()；记得同步更新哈希表中末尾元素的下标映射。
-

典型应用例题

例 1：26. Remove Duplicates from Sorted Array

题目：给定升序数组 nums，原地删除重复元素，使每个元素只出现一次，返回新长度 k，且 $\text{nums}[:k]$ 按升序排列。

思路：read/write 双指针。write 指针 w 记录下一个可写位置，read 指针 r 扫描所有元素，遇到与前一个不同的元素才写入。

解：

参考：solutions/array_string/p026_remove_duplicates_from_sorted_array.py

```
def removeDuplicates(nums: list[int]) -> int:
    if not nums:
        return 0
    w = 1
    for r in range(1, len(nums)):
        if nums[r] != nums[r - 1]:
            nums[w] = nums[r]
            w += 1
    return w
```

分析： $O(n)$ 时间， $O(1)$ 空间。 w 从 1 开始（第一个元素无需判断）， r 从 1 扫描到末尾， $\text{nums}[r] \neq \text{nums}[r-1]$ 确保只在新值出现时写入。

泛化到 80 题（至多保留 2 次）：把写条件改为 $w < 2 \text{ or } \text{nums}[r] \neq \text{nums}[w-2]$ ，即“write 指针还没写满 2 个，或当前值与 write 区域的倒数第二个不同”。参考 solutions/array_string/p080_remove_duplicates_from_sorted_array_

例 2：169. Majority Element

题目：给定长度 n 的数组，找出出现次数超过 $\lfloor n/2 \rfloor$ 的元素（保证存在）。时间 $O(n)$ ，空间 $O(1)$ 。

思路：Boyer-Moore 投票法。多数元素与所有其他元素“对消”后仍有剩余。维护候选 cand 和计数 cnt ：计数归零则换候选，同候选则 +1，否则 -1。

解：

参考：solutions/array_string/p169_majority_element.py

```
def majorityElement(nums: list[int]) -> int:
    cand, cnt = nums[0], 0
    for x in nums:
        if cnt == 0:
            cand = x
        cnt += 1 if x == cand else -1
    return cand
```

正确性直觉：多数元素出现次数 $> n/2$ ，即使所有少数元素联合对消，多数元素依然“剩余”。若题目不保证多数存在，则需第二次遍历统计 cand 出现次数做验证。

例 3: 134. Gas Station

题目：环形路上有 n 个加油站，第 i 站可以加 $gas[i]$ 升油，从 i 开到 $i+1$ 消耗 $cost[i]$ 升。若能从某站出发绕一圈回到出发点，返回该站下标；若无解返回 -1 。

思路：两个关键观察：1. 若总油量 $\sum gas < \sum cost$ ，则无论从哪里出发都无法完成，返回 -1 。2. 若总量足够，贪心从“累计余量首次变负后的下一站”出发，即可保证绕行成功。

解：

```
# 参考: solutions/array_string/p134_gas_station.py
def canCompleteCircuit(gas: list[int], cost: list[int]) -> int:
    total = 0    # 全局总余量
    tank = 0     # 从候选起点出发的累计余量
    start = 0    # 候选起点
    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total += diff
        tank += diff
        if tank < 0:    # 从 start 到 i 无法通过
            start = i + 1 # 起点后移
            tank = 0      # 重置余量
    return start if total >= 0 else -1
```

为什么贪心起点正确：若从 $start$ 到 i 累计余量为负，说明 $start$ 到 i 之间任意一个站点都不可能是起点（因为它们的前缀余量更差）；下一个候选只能是 $i+1$ 。若总余量 ≥ 0 ，贪心起点一定有效。

自测题

自测 1 (27 题 Remove Element) —— 给定数组 $nums$ 和值 val ，原地移除所有等于 val 的元素，返回新长度。

□ 提示：read/write 双指针，read 遇到目标值跳过不写，遇到其他值才写入 write 指针并后移。

自测 2 (189 题 Rotate Array) —— 将数组向右旋转 k 步，要求 $O(1)$ 空间。□ 提示：先取 $k = k \% n$ 防越界，然后三次 reverse：先 reverse 全部，再 reverse 前 k 个，再 reverse 后 $n-k$ 个。参考 solutions/array_string/p189_rotate_array.py。

自测 3 (121 题 Best Time to Buy and Sell Stock) —— 数组 $prices[i]$ 为第 i 天股价，只能买卖一次，求最大利润。□ 提示：维护历史最低价 min_price ，遍历时 $profit = \max(profit, price - min_price)$ ，不要用双重循环暴力 $O(n^2)$ 。参考 solutions/array_string/p121_best_time_to_buy_and_sell_stock.py。

自测 4 (45 题 Jump Game II) —— 数组 $nums[i]$ 为第 i 格最大跳跃步数，从下标 0 出发，求到达最后一格的最少跳跃次数（保证可达）。□ 提示：贪心 BFS，维护 end （当前跳跃段终点）和 $farthest$ （当前段内可达最远），遍历到 end 时跳数 +1 并更新 $end = farthest$ 。参考 solutions/array_string/p045_jump_game_ii.py。

自测 5 (42 题 Trapping Rain Water) ——给定高度数组, 求能接住的雨水总量, 要求 $O(n)$ 时间 $O(1)$ 空间。□ 提示: 双指针 left/right 从两端向中间移, 分别维护 max_left 和 max_right, 每次移动较低一侧, 当前蓄水 = 该侧最大值 - 当前高度。参考 solutions/array_string/p042_trapping_rain_water.py。

自测 6 (274 题 H-Index) ——给定引用次数数组, 求 h-index (至少有 h 篇论文各被引 $\geq h$ 次的最大 h)。□ 提示: 降序排序后线性扫描, citations[i] $\geq i+1$ 则 h 至少为 i+1; 或用计数数组 $O(n)$ 从大到小累积。参考 solutions/array_string/p274_h_index.py。

自测 7 (380 题 Insert Delete GetRandom $O(1)$) ——设计数据结构支持 $O(1)$ 的插入、删除、随机返回。□ 提示: 哈希表存 val $\rightarrow$ index, 动态数组存元素; 删除时把目标元素与末尾元素交换再 pop(), 记得更新哈希表中末尾元素的新下标。参考 solutions/array_string/p380_insert_delete_getrandom_o1.py。

题目全览 (24 题)

#	题目	套路分类	难度
88	Merge Sorted Array	从后向前双指针	Easy
27	Remove Element	read/write 双指针	Easy
26	Remove Duplicates from Sorted Array	read/write 双指针	Easy
80	Remove Duplicates II (至多 2 次)	read/write 双指针变体	Medium
169	Majority Element	Boyer-Moore 投票	Easy
189	Rotate Array	三次 reverse	Medium
121	Best Time to Buy and Sell Stock	贪心维护最小值	Easy
122	Best Time to Buy and Sell Stock II	贪心累加上升段	Medium
55	Jump Game	贪心维护 farthest	Medium
45	Jump Game II	贪心 BFS + 段终点	Medium
274	H-Index	排序 + 线性扫描	Medium
380	Insert Delete GetRandom $O(1)$	哈希表 + 动态数组	Medium
238	Product of Array Except Self	前缀积 + 后缀积	Medium
134	Gas Station	贪心累计余量	Medium
135	Candy	双向贪心扫描	Hard
42	Trapping Rain Water	双指针短板	Hard
13	Roman to Integer	线性扫描逢小在大则减	Easy
12	Integer to Roman	贪心从大到小枚举	Medium
58	Length of Last Word	从末尾跳过空格	Easy

#	题目	套路分类	难度
14	Longest Common Prefix	逐字符对比或排序取首尾	Easy
151	Reverse Words in a String	split + reverse join	Medium
6	Zigzag Conversion	按行分组模拟	Medium
28	Find the Index of First Occurrence	KMP	Easy
68	Text Justification	贪心分行 + 空格分配	Hard

融合版说明

段	来源	价值
一例速记	本文件	24 题套路一览，扫一眼知道要用什么
思维路径还原	本文件	从题目到代码的解题内心独白，模拟实战
抽象成方法	本文件	8 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展，覆盖系列题横向联系
思考路标	本文件	14 条题型识别条件反射，含跨 category 跳转
易错点	本文件	10 条高频踩坑，每条对应具体题目
典型应用例题	solutions/	3 道精讲（26、169、134），代码 + 正确性分析
自测题	leetcode	7 题带 ☐ 提示，链接 solutions 文件
题目全览	本文件	24 题完整列表，套路分类一览

02 —Two Pointers（融合版）

难度：□□□□□ 题数：5 核心套路：对撞双指针、三指针（固定 + 对撞）、快慢双指针 本文件：覆盖 two_pointers 5 题的算法套路总结 + 典型题精讲 + 自测

一例速记

对撞双指针 (头尾相向): left/right 从两端向中间收缩, 利用有序 / 对称性一次扫描, $O(n)$ 时间 $O(1)$ 空间 (125 / 167 / 11) 跳过无效字符: 内层 while 跳过不符条件的元素, 外层统一做判断 (125 回文校验) 短板决定贡献: 面积 = 宽度 $\times$ min(左高, 右高), 移动较低一侧才有机会变大 (11 盛水) 三指针 = 排序 + 固定 i + 对撞: 先排序, 枚举固定值, 内层对撞找互补对; 三处去重避免重复三元组 (15 三数之和) 快慢双指针: slow 追 fast, fast 每遇一个匹配字符就前进; slow 独立推进, 两指针速度不等 (392 子序列) vs 暴力: 暴力嵌套循环 $O(n^2)$, 双指针单次扫描 $O(n)$; 三指针 $O(n^2)$ 对比暴力 $O(n^3)$

思维路径还原

“看到 **125 Valid Palindrome**: 字符串回文校验, 但要忽略非字母数字字符、大小写不敏感 $\rightarrow$ 对撞双指针, left 从左找第一个 alnum, right 从右找第一个 alnum, 用内层 while 跳过非 alnum 字符, 再做 `s[left].lower() != s[right].lower()` 比较; 不匹配立即返回 False, 匹配则 left++, right--; 关键: 空字符串 / 全符号字符串在 left >= right 时直接返回 True, 无需额外判断。

看到 **167 Two Sum II** (有序数组): 数组已排序, 找两数之和等于 target $\rightarrow$ 对撞双指针, 两端收缩。sum < target 则 left++ (增大), sum > target 则 right-- (减小)。与哈希表 $O(n)$ 空间的 Two Sum I 不同, 有序数组让我们用 $O(1)$ 空间。题目保证恰好一个解, 所以 while left < right 一定能命中, 不用担心无解。

看到 **11 Container With Most Water**: height 数组, 找两条线使容纳水最多 $\rightarrow$ 面积 = (right - left) $\times$ min(height[left], height[right])。移动较高一侧无法增大面积 (高度已被较低一侧决定, 宽度只会减小); 移动较低一侧有可能找到更高的线从而增大面积。因此: height[left] < height[right] 则 left++, 否则 right--。

看到 **153 Sum**: 找所有不重复的三元组使之和为 0 $\rightarrow$ 暴力 $O(n^3)$ 不行。先排序, 枚举固定值 nums[i] (i 从 0 到 n-3), 内层对撞双指针在 [i+1, n-1] 找两数之和 = -nums[i]; 去重有三处: $\square$ 固定值 nums[i] == nums[i-1] 则 continue; $\square$ 找到解后 left++ / right-- 并继续跳过重复的 nums[left] / nums[right]; 若 nums[i] > 0 则直接 break (排序后右侧都更大, 不可能为 0)。

看到 **392 Is Subsequence**: 判断 s 是否是 t 的子序列 $\rightarrow$ 快慢双指针: s_index 追踪 s 的匹配进度 (慢指针), 遍历 t 的每个字符 (快指针); 遇到匹配字符 s_index++; 遍历完 t 后若 s_index == len(s) 则 s 完全匹配。两指针移动速度不同: t 每步必走, s 只在匹配时走。进阶: 若有大量 s 要验证同一个 t, 可对 t 建字符位置索引后用二分查找。”

学习目标

- 掌握对撞双指针模板及”移动哪一侧”的判断逻辑
- 理解”三指针 = 排序 + 固定 + 对撞”的组合套路及三处去重细节
- 能用快慢双指针处理子序列匹配问题
- 理解双指针从 $O(n^2)$ 到 $O(n)$ 的复杂度降低原因
- 识别”有序数组 + 两数之和”与”无序数组 + 哈希表”的适用场景差异

几何示意

图对撞双指针 (LC 167 Two Sum II)

LC 167: 对撞双指针 (Two Sum II)

升序数组 $[2, 7, 11, 15]$, $\text{target} = 9$

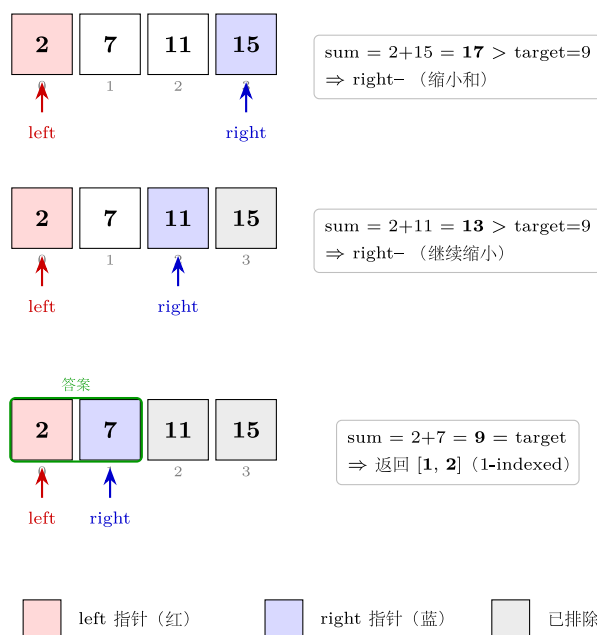
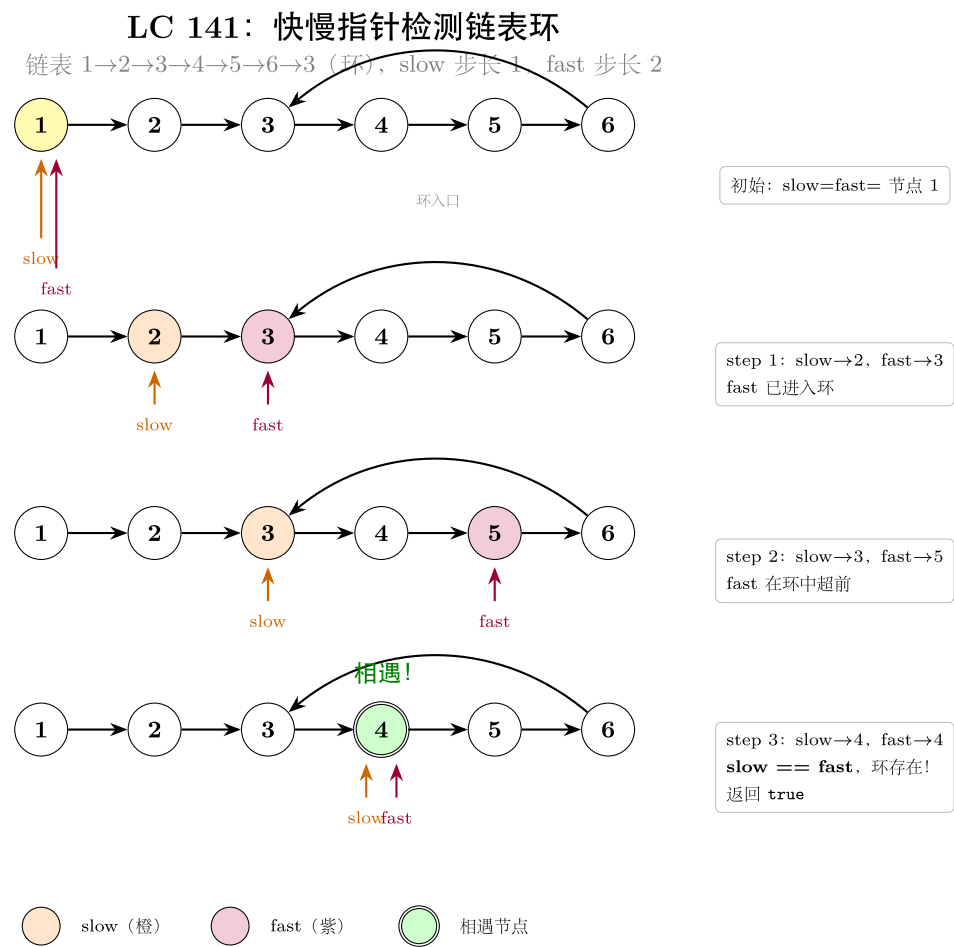


图 4: left/right 头尾收敛 3 步

图快慢指针检测环 (LC 141)



抽象方法
(标准模板代码)
套路
1: 对撞双指针
(基础)
适用题:
167
(有序数组
Two Sum)、
11
(盛最多水)

```

“python
# 167:
有序
数组
中找
两数
之和
等于
target
def
two_sum_sorted(numbers:
list[int],
target:
int) ->
list[int]:
""" “时
间
O(n),
空间
O(1)。
返回
1-
indexed
下
标。”
“left,
right
= 0,
len(numbers)
- 1
while
left <
right:
s =
num-
bers[left]
+ num-
bers[right]
if s ==
target:
return
[left +
1,
right +
1]
"""

```

11:

找两

条线

使容

器容

水最

多 def

max_area(height:

list[int])

-> int:

""" “时

间

O(n),

空间

O(1)。”

"""left,

right

= 0,

len(height)

- 1

best =

0

while

left <

right:

h =

min(height[left],

height[right])

best =

max(best,

(right -

left) *

h) if

height[left]

<

height[right]:

left

+= 1 #

移动

较低

一侧

才有

可能

提升

> 关键规律：
对撞双指针移动方向取决于”当前值与目标的偏差”；
11 中移动较低一侧是因为宽度必然减小，
只有拔高才可能补偿。

套路
2：对撞双指针（带过滤）

适用
题:
125
(回文
校验,
跳过
非 al-
num)

```

python
def
is_palindrome(s:
str)
->
bool:
"""
时间
O(n),
空间
O(1)。
跳过
非字
母数
字字
符,
大小
写不
敏感。
"""
left,
right
= 0,
len(s)
- 1
while
left
<
right:
# 跳
过左
侧非
alnum
while
left
<
right
and
not
s[left].isalnum():
left
+= 1
# 跳
过右
侧非
alnum
"""

```

> 内
层
while
必须
带
left
<
right
守卫,
防止
全为
符号
时指
针越
界交
叉后
仍继
续执
行。

套路
3: 三
指针
(固定
+ 对
撞)
适用
题: 15
(3Sum)

```
python
def
three_sum(nums:
list[int])
->
list[list[int]]:
"""
时间
 $O(n^2)$ ,
空间
 $O(1)$ 
(不含
输
出)。
找所
有和
为 0
的不
重复
三元
组。
"""
nums.sort()
result:
list[list[int]]
= []
for i
in
range(len(nums)
- 2):
# 去
重 1:
固定
值与
前一
个相
同则
跳过
if i
> 0
and
nums[i]
==
nums[i
```

###

套路

4: 快

慢双

指针

(子序

列匹

配)

适用

题:

392

(Is

Subse-

quence)

```
python
def
is_subsequence(s:
str,
t:
str)
->
bool:
"""
时间
O(n),
空间
O(1)。
s 慢
指针
只在
匹配
时推
进, t
快指
针每
步必
走。
"""
s_idx
= 0
for
ch in
t: if
s_idx
==
len(s):
break
# s
已全
部匹
配,
提前
退出
if
s[s_idx]
==
ch:
s_idx
```

```

> 进
阶
(大量
s 验证
同一
t): 预
处理 t
的字
符位
置字
典
pos[c]
=
sorted
list
of
indices,
对每
个字
符用
二分
查
找 (bi-
sect_left)
找最
近可
匹配
位置,
总体
 $O(|t| + k \cdot |s| \log |t|)$ 。
###
速查
表

```

| 题型
特征 |
套路 |
时间 |
空间 |
对应
题目 |
|—|
—|—|
—||
有序
数组
找两
数之
和 |
对撞
双指
针,
偏小
左移
偏大
右移 |
 $O(n)$
| $O(1)$
| 167 |
| 字符
串回
文校
验
(含噪
声) |
对撞
+ 内
层
while
过滤 |
 $O(n)$
| $O(1)$
| 125 |
| 两线
段围
成最
大面
积 |

方法变形 (4 类)

变形 1: 对撞双指针扩展系列

- 基础 (167): 有序数组两数之和, $O(1)$ 空间替代哈希表。
- 带过滤 (125): 内层 while 跳过无效字符, 模式可泛化为“在对撞过程中跳过特定元素”。
- 面积最大化 (11): 不是找目标值而是维护最优解; “移动较低一侧”是单调性贪心的核心。
- 进阶 (42 接雨水, 属 array_string): 同样是双端收缩 + 两侧最大值维护, 但每步更新蓄水量而非面积。

变形 2: 三指针系列 (nSum 通式)

- **3Sum $\rightarrow$ 4Sum** (18, 非本 category): 外层再套一个固定指针, 变成 $O(n^3)$; nSum 通用递归模板: 固定最外层, 递归降维到 2Sum。
- **3Sum Closest** (16): 同样排序 + 固定 + 对撞, 只是判断条件从 $== 0$ 改为维护 $\text{abs}(\text{diff})$ 最小值, 找到完美匹配可提前退出。
- 去重三关卡: 固定值去重 ($i > 0$ 判断)、解后 left 去重、解后 right 去重; 缺任何一关都会产生重复三元组。

变形 3: 快慢双指针系列

- **392 子序列**: s 是 t 的子序列 (字符顺序匹配, 无需连续)。
- 链表相关 (141 环检测、876 中点): slow 每次走 1 步, fast 每次走 2 步; 环检测看是否追上, 求中点看 fast 何时到达末尾。
- 滑动窗口退化: 当 slow 移动条件简单到“每次匹配才移动”时, 快慢指针与可变滑动窗口本质相同; 392 也可理解为窗口大小不固定的滑动窗口。

变形 4: 预处理优化

- **392 进阶** (海量 s 查询同一 t): 对 t 建字典 $\text{pos} = \text{defaultdict}(\text{list})$, $\text{pos}[c].\text{append}(i)$; 验证 s 时对每个字符用 bisect_left 找 $\geq \text{cur_pos}$ 的最小下标, 推进 cur_pos 。
 - **167 进阶** (需要所有对): 题目保证唯一解, 但若需要所有对则用哈希表存余数; 有序 + 唯一解时双指针更优。
 - **125 进阶** (判断最长回文子串): 中心扩展 (奇偶两种中心), 可以理解为“反方向对撞”——从中心向外扩展而非从两端向内收缩。
 - **AI 类比**: 对撞双指针 $\approx$ 双向束搜索 (Bidirectional BFS) ——从两端同时推进, 在中间汇合, 把搜索空间从 $O(b^d)$ 压缩到 $O(b^{d/2})$; 快慢双指针的“速度差” $\approx$ 在线算法中的“延迟缓冲”机制, 慢指针维护已确认的状态, 快指针探索新输入。
-

思考路标 (条件反射)

1. 看到 有序数组 + 找两数之和 $\rightarrow$ 对撞双指针, $O(n) O(1)$; 无序时才用哈希表
2. 看到 回文校验 + 忽略非字母数字 $\rightarrow$ 对撞 + 内层 while 过滤, `isalnum()` + `lower()`
3. 看到 两侧围住 / 容积 / 面积最大化 $\rightarrow$ 对撞双指针, 移动“不利”(较低/较小)的一侧
4. 看到 三数之和 / `nSum = target` $\rightarrow$ 排序 + 固定外层 + 内层对撞, 注意三处去重
5. 看到 `s` 是否为 `t` 的子序列 $\rightarrow$ 快慢双指针; 大量查询则预处理 `t` 的位置索引 + 二分
6. 看到 对撞双指针, 不知道移动哪侧 $\rightarrow$ 问自己“移动较优一侧能得到更好结果吗?”——通常不能, 移动较差一侧才是正确方向
7. 看到 **in-place $O(1)$** 空间 + 有序 $\rightarrow$ 先考虑 read/write 双指针 (`array_string` 套路); 若涉及配对则对撞
8. 看到 数组已排序 + 找多数组合 $\rightarrow$ 外层枚举 + 内层对撞, 时间 $O(n^{k-1})$ 对比暴力 $O(n^k)$
9. 看到 最长 / 最短子串 + 连续 $\rightarrow$ 优先考虑滑动窗口 (`sliding_window category`), 不是双指针
10. 看到 链表中间 / 链表环 $\rightarrow$ 快慢指针, `slow  $\times$  1`, `fast  $\times$  2`
11. 看到 **palindrome + expand from center** $\rightarrow$ 中心扩展, 奇偶各一次, $O(n^2)$; Manacher 算法 $O(n)$
12. 看到 **3Sum 超时** $\rightarrow$ 检查是否漏了 `nums[i] > 0` 的 `break` 剪枝, 以及三处去重
13. 看到 子序列而非子串 $\rightarrow$ 元素可不连续, 快慢指针; 子串要求连续, 用滑动窗口

易错点

1. 对撞双指针内层 **while** 缺少边界守卫: 125 题跳过非 `alnum` 时, 内层 `while not s[left].isalnum()` 必须带 `left < right`; 若字符串全为符号, 不带守卫会导致 `left > right` 后仍继续执行, 产生错误比较。
2. **11 题**移动方向: 移动较高一侧是错误的——高度由短板决定, 宽度还在减小, 面积只会更小; 必须移动较低一侧 (`height[left] < height[right]` 则 `left++`, 否则 `right--`; 等高时移动哪侧都行)。
3. **15 题**三处去重缺一不可: $\square$ 固定值 `if i > 0 and nums[i] == nums[i-1]: continue`, $\square$ 找到解后 `while nums[left] == nums[left-1]: left++`, $\square$ `while nums[right] == nums[right+1]: right--`; 只做其中一两处会漏掉或多出重复三元组。
4. **15 题**去重检查方向: 找到解后 `left++ / right--`之后再去做去重时, 比较的是 `nums[left] == nums[left-1]` (新 `left` 与刚用过的 `left`) 而非 `nums[left] == nums[left+1]`; 方向写反会跳过正确答案。
5. **392 题**返回条件: 循环结束后返回 `s_idx == len(s)` 而非 `s_idx >= len(s)` 或者在循环内 `return True` ——因为 `s` 可能是空字符串 (空串是任何字符串的子序列, 此时 `s_idx == 0 == len(s)` 正确返回 `True`)。
6. **167 题**下标偏移: 题目要求返回 1-indexed 答案, 即 `[left + 1, right + 1]`; 若直接返回 `[left, right]` 会错位。
7. **15 题**剪枝位置: `if nums[i] > 0: break` 需要放在内层对撞之前 (当 `fixed` 已大于 0, 后续所有三元组之和必然 > 0); 若放到内层会漏掉部分计算。
8. 对撞双指针的单调性前提: 对撞双指针有效的核心是“移动一侧能单调改变当前量”——167 中移动 `left` 单调增大 `sum`, 移动 `right` 单调减小 `sum`; 11 中移动任意一侧宽度单调减小。若数组无序且题目不满足单调性, 对撞双指针不能直接使用, 需先排序或换哈希表方案。
9. **125 题**输入边界: 空字符串 `s = ""` 或全为标点 `s = "!!!"` 时, `left/right` 内层 `while` 执行后 `left >= right`, 外层 `while` 条件为假, 直接返回 `True` (正确——空字符串和清洗后为空的字符串均视为回文); 不要在函

数开头特判 `if not s: return True`, 逻辑已经自然覆盖。

典型应用例题

例 1: 125. Valid Palindrome

题目: 给定字符串 `s`, 只保留字母和数字 (忽略大小写), 判断是否为回文串。

思路: 对撞双指针。left/right 从两端向中间收缩, 内层 while 跳过非 alnum 字符, 然后对当前对位字符做大小写不敏感比较。

解:

```
# 参考: solutions/two_pointers/p125_valid_palindrome.py
def isPalindrome(s: str) -> bool:
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

分析: $O(n)$ 时间, $O(1)$ 空间。每个字符最多被访问两次 (一次被内层 while 跳过, 一次被外层比较)。与“先过滤再比较”的 $O(n)$ 空间方案等价, 但省去了额外列表。

例 2: 15. 3Sum

题目: 给定整数数组 `nums`, 找出所有和为 0 的不重复三元组, 返回其列表。

思路: 先排序, 外层枚举固定值 `nums[i]`, 内层对撞双指针在剩余区间找两数之和为 `-nums[i]`; 三处去重保证无重复三元组。

解:

```
# 参考: solutions/two_pointers/p015_3sum.py
def threeSum(nums: list[int]) -> list[list[int]]:
```

```

nums.sort()
result: list[list[int]] = []
for i in range(len(nums) - 2):
    if i > 0 and nums[i] == nums[i - 1]: # 去重 1
        continue
    if nums[i] > 0:
        break # 剪枝
    left, right = i + 1, len(nums) - 1
    while left < right:
        s = nums[i] + nums[left] + nums[right]
        if s == 0:
            result.append([nums[i], nums[left], nums[right]])
            left += 1; right -= 1
            while left < right and nums[left] == nums[left - 1]:
                left += 1 # 去重 2
            while left < right and nums[right] == nums[right + 1]:
                right -= 1 # 去重 3
        elif s < 0:
            left += 1
        else:
            right -= 1
    return result

```

复杂度: $O(n^2)$ 时间 (排序 $O(n \log n)$ + 外层 $O(n) \times$ 内层 $O(n)$), $O(1)$ 额外空间 (不含输出)。比暴力三重循环 $O(n^3)$ 快一个数量级。

去重正确性: 排序后相同值连续排列, 跳过固定值的重复保证外层不重, 跳过 left/right 的重复保证内层不重; 三关缺一会产生重复三元组。

例 3: 11. Container With Most Water

题目: 给定高度数组 `height`, 选两条线使之与 `x` 轴构成的容器容水最多, 返回最大水量。

思路: 面积 = 宽度 $\times$ $\min$ (两端高度)。对撞双指针, 每次移动较低一侧: 因为宽度在收缩, 若移动较高一侧, 高度上限不变甚至变低, 面积只会变小; 移动较低一侧才有可能找到更高的线来补偿宽度损失。

解:

```

# 参考: solutions/two_pointers/p011_container_with_most_water.py
def maxArea(height: list[int]) -> int:
    left, right = 0, len(height) - 1
    best = 0

```

```

while left < right:
    w = right - left
    h = min(height[left], height[right])
    best = max(best, w * h)
    if height[left] < height[right]:
        left += 1
    else:
        right -= 1
return best

```

贪心正确性：设当前 $\text{height}[\text{left}] < \text{height}[\text{right}]$ ，则固定 right 时最优面积已经是 $(\text{right} - \text{left}) \times \text{height}[\text{left}]$ ； right 左移只会让宽度更小且高度上限不升，故 right 这个选项已被穷尽，移动 left 是唯一有意义的选择。

自测题

自测 1 (167 题 Two Sum II) —— 给定升序数组 `numbers` 和目标 `target`，找两数下标 (1-indexed) 使之和为 `target`。不允许使用哈希表。□ 提示：对撞双指针， $\text{sum} < \text{target}$ 则 $\text{left}++$ ， $\text{sum} > \text{target}$ 则 $\text{right}-$ ，命中返回 $[\text{left}+1, \text{right}+1]$ 。

自测 2 (125 题 Valid Palindrome) —— 判断字符串 (含空格和标点) 是否为回文，忽略非 `alnum` 字符，大小写不敏感。如 "A man, a plan, a canal: Panama" 应返回 `True`。□ 提示：对撞 + 内层 `while` 跳过非 `alnum`，注意内层 `while` 必须带 `left < right` 守卫。参考 `solutions/two_pointers/p125_valid_palindrome.py`。

自测 3 (11 题 Container With Most Water) —— 给定 `height = [1,8,6,2,5,4,8,3,7]`，求最大容水量。□ 提示：对撞双指针，移动较低一侧 (不是较高一侧)，维护 $\text{best} = \max(\text{best}, (\text{right}-\text{left}) \times \min(h[\text{left}], h[\text{right}]))$ 。参考 `solutions/two_pointers/p011_container_with_most_water.py`。

自测 4 (15 题 3Sum) —— 给定 `nums = [-1, 0, 1, 2, -1, -4]`，找所有和为 0 的不重复三元组。□ 提示：排序后固定 i ，内层对撞；三处去重：固定值去重、 left 去重、 right 去重； $\text{nums}[\text{i}] > 0$ 时 `break` 剪枝。参考 `solutions/two_pointers/p015_3sum.py`。

自测 5 (392 题 Is Subsequence) —— 判断 `s = "ace"` 是否为 `t = "abcde"` 的子序列 (字符顺序匹配，无需连续)。□ 提示：快慢双指针，遍历 t 的每个字符 (快)，只有在 $t[\text{i}] == s[\text{s_idx}]$ 时 s_idx 才推进 (慢)；循环结束后返回 $\text{s_idx} == \text{len}(s)$ 。参考 `solutions/two_pointers/p392_is_subsequence.py`。

题目全览 (5 题)

#	题目	套路分类	难度
125	Valid Palindrome	对撞双指针 + 字符过滤	Easy
167	Two Sum II —Input Array Is Sorted	对撞双指针	Medium
11	Container With Most Water	对撞双指针，移动较低侧	Medium
15	3Sum	排序 + 固定 + 对撞，三处去重	Medium
392	Is Subsequence	快慢双指针	Easy

融合版说明

段	来源	价值
一例速记	本文件	3 类套路一览，对比暴力复杂度
思维路径还原	本文件	5 题解题内心独白，含关键决策点
抽象成方法	本文件	4 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展（nSum / 链表 / 预处理优化 / AI 类比）
思考路标	本文件	13 条题型识别条件反射，含跨 category 跳转
易错点	本文件	9 条高频踩坑，每条对应具体题目
典型应用例题	solutions/	3 道精讲（125、15、11），代码 + 正确性分析
自测题	leetcode	5 题带 □ 提示，链接 solutions 文件
题目全览	本文件	5 题完整列表，套路分类一览

跨 category 导航：- 双指针用于 连续子串 / 子数组问题时，通常升级为滑动窗口 → 见 03-sliding-window.md - 有序数组 + 二分查找比双指针更快（子问题可缩减到 $O(\log n)$ ）→ 见 04-binary-search.md（待写）- 链表的快慢指针（环、中点）→ 见 linked_list category（待写）

03 —Sliding Window（融合版）

难度：□□□□□ 题数：4 核心套路：可变窗口双指针、哈希计数辅助、固定步长枚举 本文件：覆盖 sliding_window 4 题的算法套路总结 + 典型题精讲 + 自测

一例速记

可变窗口：left/right 指向窗口两端，right 每步右扩，条件不满足时 left 右缩；维护窗口内的聚合量（sum / 字符计数）随指针移动增量更新， $O(n)$ 时间 $O(1)$ 或 $O(k)$ 空间（209 / 3 / 76）哈希计数辅助：用 Counter 或 dict 记录窗口内字符频率，“满足条件”等价于 `need == 0` 或 `formed == required`；滑动时 left 缩小只在计数归零后才更新满足数（76 最小覆盖子串）固定窗口枚举：窗口大小固定为 $\text{len}(\text{word}) \times \text{num_words}$ ，步长为 $\text{len}(\text{word})$ ，每个起点独立用 Counter 比对（30 单词拼接）复杂度对比：暴力枚举所有子串 $O(n^2)$ （甚至 $O(n^2k)$ ）→ 滑动窗口 $O(n)$ 或 $O(n \cdot k)$ **AI 关联**：流式数据处理（实时特征窗口）、在线学习（mini-batch 滑动）、序列模型（时序特征提取）——滑动窗口是有界上下文的工程原型

思维路径还原

“看到 **209 Minimum Size Subarray Sum**：找最短连续子数组使之和 $\geq \text{target}$ → 子数组长度不固定，但窗口越大越可能满足条件，所以用可变窗口。right 右扩时 `window_sum += nums[right]`；当 `window_sum >= target` 时记录长度 `right-left+1`，然后 left 右缩并 `window_sum -= nums[left]`，继续尝试缩小；关键：left 缩小不是一次性缩到不满足，而是每次缩一步并立即记录，以捕捉所有满足条件的最短窗口。

看到 **3 Longest Substring Without Repeating Characters**：最长无重复字符子串 → 同样可变窗口，但维护的是“窗口内无重复”的约束。用 set 或 dict 记录窗口内字符，right 扩时若 `s[right]` 已在窗口中，left 一步一步右缩直到重复消除；每步更新 `best = max(best, right - left + 1)`。也可用 dict 存字符最近出现下标，left 跳到 `last_seen[s[right]] + 1` 而非逐步缩（效率更高但边界更难想）。

看到 **76 Minimum Window Substring**：s 中包含 t 所有字符的最短子串 → 需要字符频率计数。预处理 `need = Counter(t)`，`required = len(need)` 记录需要满足的不同字符数。right 扩时更新 `window[s[right]]++`；若新加字符的频率恰好等于 need 中的需求，`formed++`；当 `formed == required`（所有字符均已覆盖），记录当前窗口长度，然后 left 缩：`window[s[left]]--`，若某字符频率低于 need，`formed--`，left 缩到不满足为止。关键：formed 只在频率“恰好满足”或“恰好不满足”时变化，而非每次计数变化都更新。

看到 **30 Substring with Concatenation of All Words**：找所有起点使连续子串恰好是 words 的拼接 → 所有 word 等长（设为 L），窗口总长 $= L \times \text{len}(\text{words})$ 。固定窗口模式：枚举起点 i（0 到 L-1），步长 L 滑动；维护窗口内各词计数，与 `Counter(words)` 比对；当窗口内词数达到 $\text{len}(\text{words})$ 且计数匹配，记录左端为答案；left 缩时移除最左词并更新计数。注意：起点只需枚举 0 到 L-1（共 L 个），因为步长为 L，所有起点都能被覆盖。”

学习目标

- 掌握可变窗口的”右扩左缩”模板及窗口量的增量更新技巧
 - 理解哈希计数辅助（formed/required 技巧）在字符覆盖问题中的应用
 - 能用固定步长枚举处理等长单词拼接问题
 - 识别”子串/子数组 + 最长/最短/恰好满足”的问题与滑动窗口的对应关系
 - 理解滑动窗口与流式数据处理、在线学习窗口的工程联系
-

几何示意

图可变滑窗（LC 3）

图可变滑窗 + 哈希（LC 76）

抽象成方法（标准模板代码）
套路
1: 可变窗口（数值聚合）
适用题：
209（最短子数组和 $\geq$ target）
“python
import
math

```
def
min_subarray_len(target:
int,
nums:
list[int])
-> int:
""" “可
变窗
口：
找最
短子
数组
使之
和 >=
tar-
get。
时间
O(n),
空间
O(1)。”
""" left
= 0
win-
dow_sum
= 0
best =
math.inf
```

```
for
right
in
range(len(nums)):
win-
dow_sum
+=
nums[right]
# 右
扩:
增量
更新
聚合
量
while
win-
dow_sum
>=
target:
# 满
足条
件:
尝试
缩小
best =
min(best,
right -
left +
1)
win-
dow_sum
-=
nums[left]
# 左
缩:
减去
移出
元素
left
+= 1
```

```
return
0 if
best
==
math.inf
else
best “
> 关
键规
律:
for
right
+
while
left
是可
变窗
口的
经典
结构;
聚合
量随
指针
移动
做增
量更
新
(加/减
单个
元
素),
避免
每次
重新
扫描
整个
窗口。
```

###

套路

2: 可

变窗

口

(集合

约束)

适用

题: 3

(最长

无重

复字

符子

串)

```
“python
def
length_of_longest_substring(s:
str) ->
int: “
“可变
窗口
+ set
维护
唯一
性。
时间
O(n),
空间
O(min(n,
k)), k
为字
符集
大
小。”
“left
= 0
win-
dow:
set[str]
= set()
best =
0
```

```
for
right
in
range(len(s)):
    # 右
    扩:
    若字
    符已
    在窗
    口中,
    左缩
    直到
    消除
    重复
    while
    s[right]
    in
    win-
    dow:
    win-
    dow.remove(s[left])
    left
    += 1
    win-
    dow.add(s[right])
    best =
    max(best,
    right -
    left +
    1)
    return
    best
```

```

# 优化版:
dict
记录
最近
下标,
left 跳跃而
非逐步缩小
def
length_of_longest_substring_v2(s:
str) ->
int: """
“时间
O(n),
空间
O(k)。
left 直接跳
到重复字
符的
后一位。”
"""

last_seen:
dict[str,
int] =
{}
left
= 0
best =
0

```

```
for
right,
ch in
enu-
mer-
ate(s):
if ch
in
last_seen
and
last_seen[ch]
>=
left:
left =
last_seen[ch]
+ 1 #
跳跃:
跳过
重复
字符
last_seen[ch]
=
right
best =
max(best,
right -
left +
1)
return
best ““
```

> 注意 v2 中
last_seen[ch]
>=
left
的判断:
若上次出现位置在 left 左侧, 说明该字符已不在当前窗口中, 不需要缩左。

套路
3: 可变窗口
(哈希计数辅助)
适用题:
76
(最小覆盖子串)

```
“python
from
collec-
tions
im-
port
Counter
def
min_window(s:
str, t:
str) ->
str: ““
“可变
窗口
+ 频
率计
数。
时间
 $O(n + m)$ ,
空间
 $O(k)$ ,
k 为
字符
集大
小。”
“if
not t
or not
s:
return
“”
```

```
need =
Counter(t)
# 每
个字
符的
需求
量 re-
quired
=
len(need)
# 需
满足
的不
同字
符数
win-
dow:
dict[str,
int] =
{}
formed
= 0 #
当前
已满
足需
求的
不同
字符
数 left
= 0
best_len
=
math.inf
best_left
= 0
```

```
for
right
in
range(len(s)):
ch =
s[right]
win-
dow[ch]
= win-
dow.get(ch,
0) + 1
# 若
该字
符计
数恰
好达
到需
求,
formed
+1 if
ch in
need
and
win-
dow[ch]
==
need[ch]:
formed
+= 1
```

```
# 满
足覆
盖条
件:
尝试
缩小
while
formed
== re-
quired:
if
right -
left +
1 <
best_len:
best_len
=
right -
left +
1
best_left
= left
lch =
s[left]
win-
dow[lch]
-= 1 if
lch in
need
and
win-
dow[lch]
<
need[lch]:
formed
-= 1 #
该字
符不
再满
足需
求 left
+= 1
```

```

return
"""if
best_len
==
math.inf
else
s[best_left:
best_left
+
best_len]
"""

```

```

###
套路
4: 固
定步
长窗
口
(等长
单词
拼接)
适用
题:
30
(Sub-
string
with
Con-
cate-
nation
of All
Words)

```

```

“python
def
find_substring(s:
str,
words:
list[str])
->
list[int]:
““固
定步
长窗
口，
枚举
L 个
起点。
时间
O(n *
L),
空间
O(L *
k)。”
“if
not s
or not
words:
return
[] L =
len(words[0])
# 每
个单
词的
长度
num_words
=
len(words)
win-
dow_size
= L *
num_words
word_count
=
Counter(words)
result:
list[int]

```

```
# 起  
点只  
需枚  
举  
0..L-  
1, 步  
长 L  
即可  
覆盖  
所有  
位置  
for  
start  
in  
range(L):  
    left =  
    start  
    cur-  
    rent:  
    dict[str,  
    int] =  
    {}  
    count  
    = 0 #  
    窗口  
    内有  
    效词  
    数
```

```

for
right_word
in
range(start,
len(s)
- L +
1, L):
word
=
s[right_word:
right_word
+ L] if
word
in
word_count:
cur-
rent[word]
= cur-
rent.get(word,
0) + 1
count
+= 1 #
超出
该词
需求
数量:
左缩
到刚
好消
除多
余
while
cur-
rent[word]
>
word_count[word]:
left_word
=
s[left:
left +
L] cur-
rent[left_word]
-= 1
count

```

return
result
““

套路
5：可
变窗
口通
用框
架
适用
题：
所有
可变
窗口
题的
统一
模板

```

“python
def
slid-
ing_window_template(data:
list,
condi-
tion_fn,
up-
date_fn)
-> int:
"""可
变窗
口通
用框
架
(伪代
码风
格,
展示
结
构)。-
right
右扩:
更新
窗口
状态 -
条件
满足
时:
记录
答案,
left 右
缩并
更新
状态
时间
O(n):
每个
元素
最多
入窗
一次、
出窗
一
"""

```

```

for
right
in
range(len(data)):
up-
date_fn(state,
data[right],
“add”)
# 右
扩,
更新
状态
while
condi-
tion_fn(state):
# 条
件满
足
(或不
满足)
best =
max(best,
right -
left +
1) #
记录
答案
up-
date_fn(state,
data[left],
“re-
move”
) # 左
缩,
更新
状态
left
+= 1
return
best ““

```

> 框
架变
体：□
最长
问题：
条件
为”违
反约
束”时
缩左，
循环
外记
录答
案；□
最短
问题：
条件
为”满
足要
求”时
记录
答案
并缩
左。

速查
表

| 题型
特征 |
套路 |
维护
结构 |
时间 |
空间 |
|—|
—|—|
—||
最短
子数
组,
和 $\geq$
target
| 可变
窗口,
满足
时缩
左记
录 |
整数
sum |
 $O(n)$
| $O(1)$
|| 最
长子
串,
无重
复字
符 |
可变
窗口,
违反
时缩
左 |
set 或
last_seen
dict |
 $O(n)$
| $O(k)$
|| 最
小覆
盖子

LC 3: 可变滑窗 (Longest Substring Without Repeating)

字符串 “abcabcb”，蓝色窗 [l, r]，逐步扩张/收缩

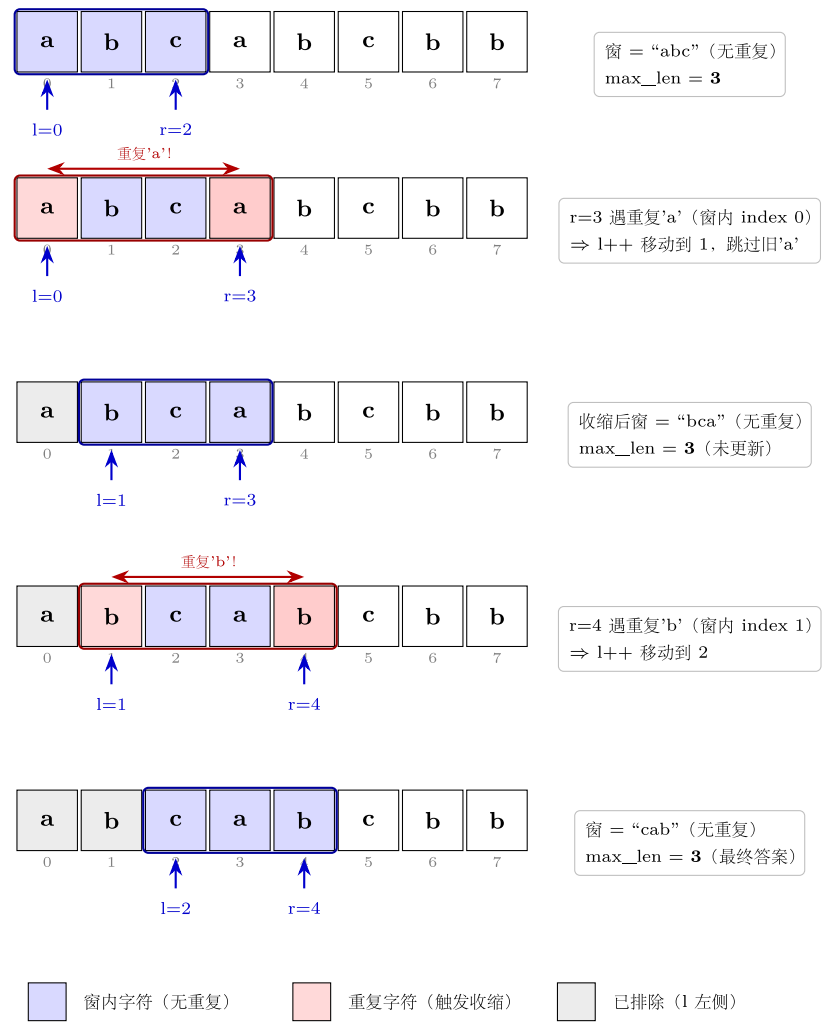


图 6: abcabcb 窗扩缩 4 步

LC 76: 固定/可变滑窗 (Minimum Window Substring)

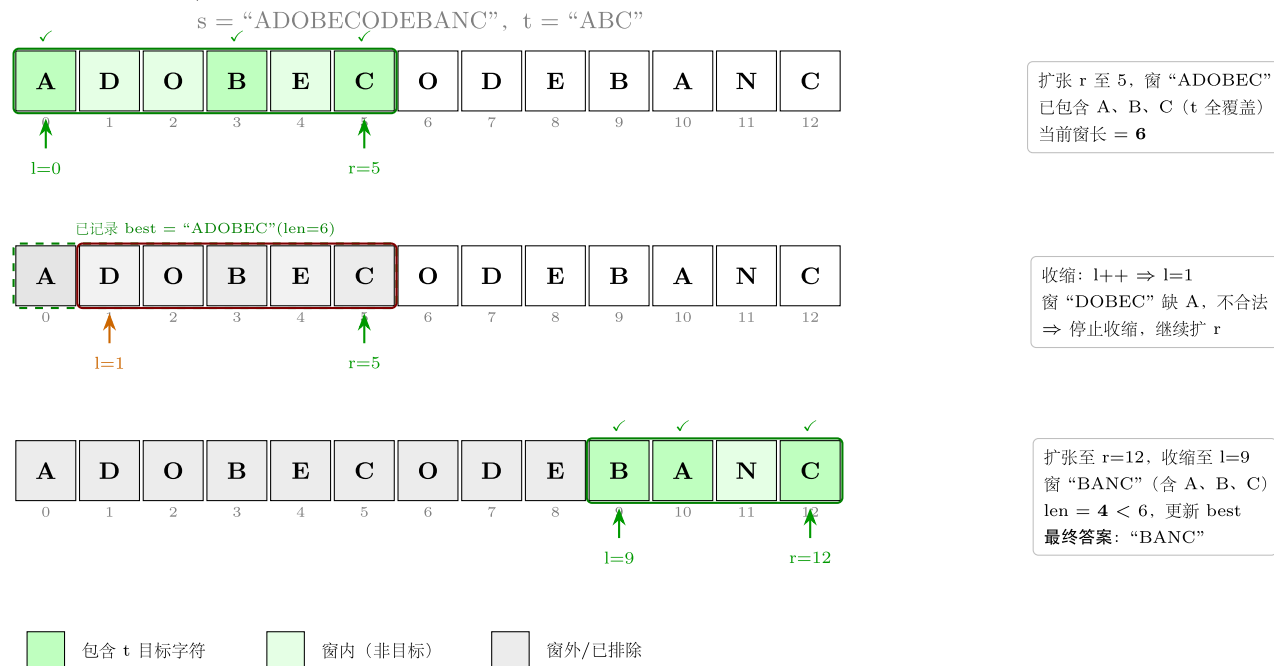


图 7: Min Window Substring 收缩到最小

方法变形 (4 类)

变形 1: 最短 vs 最长

- 最短 (209 / 76): 条件"满足"时记录并缩左, 找到最小长度。
- 最长 (3): 条件"违反"时缩左, 循环外每步记录最大长度。
- 记忆口诀: 最短 $\rightarrow$ 满足时缩; 最长 $\rightarrow$ 违反时缩。两者 while 的触发条件相反, 其余结构相同。

变形 2: 计数辅助的层次

- 简单计数 (3): set 判断唯一性, $O(1)$ 查询删除。
- 频率计数 (76): Counter + formed 变量, formed 在"恰好满足"和"恰好不满足"时更新, 避免每次 $O(|t|)$ 遍历检查是否全覆盖。
- 双 Counter 比对 (30): 窗口内 Counter 与 word_count 直接比较, 固定窗口所以每次比较 $O(k)$; 优化版用 formed 思路同样可降到 $O(1)$ 更新。

变形 3: 字符串 vs 数组

- 字符串 (3 / 76 / 30): 字符集有限 (128 或 26), $O(k)$ 空间通常可接受; dict 键为字符。
- 整数数组 (209): 窗口聚合量为数值 sum, 直接加减, $O(1)$ 空间。
- 混合场景 (438 找所有字母异位词, 非本 category): 同 76 的 formed 技巧, 但记录所有满足条件的起点。

变形 4: AI / 工程类比

- 流式特征窗口: 在线处理时序数据时, 只保留最近 W 步的特征; `left` 对应过期时间戳的移除, `right` 对应新数据的加入, 与 209 模板完全同构。
- 在线学习 **mini-batch**: 固定大小批次滑动处理序列样本, 对应固定窗口模式 (套路 4)。
- 序列模型注意力: Transformer 的局部注意力 (Local Attention) 限制每个 token 只关注距离 W 内的上下文, 等价于固定窗口滑动扫描。
- 监控告警: 实时统计最近 N 秒的错误率, 新事件入窗加计数, 超期事件出窗减计数, 满足阈值时触发告警——可变窗口的工程实例。

思考路标 (条件反射)

1. 看到“最短 / 最小子数组 + 满足条件” → 可变窗口, 满足条件时记录并缩左
2. 看到“最长 / 最大子串 + 约束” → 可变窗口, 违反约束时缩左, 每步记录最大
3. 看到“子串包含某字符集” → 哈希计数 + `formed/required` 技巧 (76 模板)
4. 看到“等长单词拼接 / 词语排列” → 固定窗口, 枚举 L 个起点按词步进 (30 模板)
5. 看到“子数组 / 子串” → 首先想滑动窗口; 若需要“前缀和”则想前缀和 + 哈希 (非本 category)
6. 看到“连续” → 滑动窗口; 若不要求连续则用双指针 (`two_pointers` category)
7. 看到“字符不重复” → 可变窗口 + `set`, 或 `dict` 存最近下标跳跃 (3 的两种写法)
8. 看到窗口题不知道 `while` 里记录还是外面记录 → 最短问题在 `while` 内记录, 最长问题在 `while` 外 (`for` 内) 记录
9. 看到“至多 k 个不同字符”的最长子串 → 可变窗口, `Counter` 记频率, `len(window) > k` 时缩左 (340, 非本 category 扩展)
10. 看到“固定窗口 / 固定步长” → 不需要 `while`, 直接移除最左元素加入新元素, $O(1)$ 更新
11. 看到时序数据 / 流式处理 → 联想可变窗口; 窗口 = 上下文范围, 指针 = 时间戳边界
12. 看到“**anagram** / 字母异位词” → `Counter` 比对或 `formed` 技巧; 字符串长度固定时用固定窗口

易错点

1. 可变窗口聚合量的增量更新: `right` 右扩时只加 `nums[right]`, `left` 左缩时只减 `nums[left]`; 不要每次重新求和 (否则 $O(n^2)$)。聚合量必须在指针移动的同时同步更新。
2. 76 题 **formed** 的更新条件: `formed` 只在 `window[ch] == need[ch]` (恰好满足) 时 +1, 在 `window[lch] < need[lch]` (刚刚不满足) 时 -1; 不要在每次计数变化时都更新 `formed`, 否则逻辑错误。
3. 3 题 **v2** 的 `left` 跳跃守卫: `if ch in last_seen and last_seen[ch] >= left`, 缺少 `>= left` 的判断会把左指针跳到已经在窗口左侧的位置, 错误地缩小了有效窗口 (已过期的下标不应再触发跳跃)。
4. 30 题起点枚举范围: 只需枚举 `range(L)` 共 L 个起点 (L = 单词长度), 不是枚举所有字符下标; 遇到不在 `words` 中的词时要重置左指针 (`left = right_word + L`), 不要只重置计数。

5. 30 题越界：内层 for 的范围是 `range(start, len(s) - L + 1, L)`，上界 `len(s) - L + 1` 确保切片 `s[right_word: right_word + L]` 不越界；写成 `len(s)` 会产生空字符串或短字符串。
6. 209 题返回值：若整个数组之和都 $< \text{target}$ ，`best` 仍为 `math.inf`，需返回 0；不要直接 `return best` 或 `return int(best)`（浮点数转换不直观），用 `return 0 if best == math.inf else best`。
7. 窗口更新顺序：`right` 扩时先更新状态再判断，`left` 缩时先记录答案再缩（最短问题）或先缩再记录（最长问题）；顺序颠倒会导致边界差 1 的错误。

典型应用例题

例 1：209. Minimum Size Subarray Sum

题目：给定正整数数组 `nums` 和正整数 `target`，找到最短的连续子数组使其和 $\geq \text{target}$ ，返回其长度；若不存在返回 0。

思路：可变窗口。`right` 右扩加入元素，窗口和满足条件时记录当前长度并尝试缩左（可能找到更短的窗口）。数组元素全为正数，保证左缩不会让窗口和“反弹”再次满足——这是可变窗口有效的单调性前提。

解：

参考：[solutions/sliding_window/p209_minimum_size_subarray_sum.py](#)

```
import math

def minSubArrayLen(target: int, nums: list[int]) -> int:
    left = 0
    window_sum = 0
    best = math.inf
    for right in range(len(nums)):
        window_sum += nums[right]
        while window_sum >= target:
            best = min(best, right - left + 1)
            window_sum -= nums[left]
            left += 1
    return 0 if best == math.inf else best
```

分析： $O(n)$ 时间，每个元素最多被 `right` 扫过一次、被 `left` 移出一次，共 $2n$ 步。 $O(1)$ 空间。与前缀和 + 二分的 $O(n \log n)$ 方案相比更优（前缀和方案适用于元素可能为负的情况）。

例 2：76. Minimum Window Substring

题目：给定字符串 `s` 和 `t`，找 `s` 中包含 `t` 所有字符（含重复）的最短子串；若不存在返回空串。

思路：可变窗口 + formed/required 计数技巧。required = len(Counter(t)) 表示需要满足的不同字符数，formed 追踪当前已满足数；right 扩时更新 formed，formed 达到 required 时记录并缩左，直到 formed 下降。

解：

参考：solutions/sliding_window/p076_minimum_window_substring.py

```
from collections import Counter
```

```
import math
```

```
def minWindow(s: str, t: str) -> str:
    if not t or not s:
        return ""
    need = Counter(t)
    required = len(need)
    window: dict[str, int] = {}
    formed = 0
    left = 0
    best_len = math.inf
    best_left = 0
    for right in range(len(s)):
        ch = s[right]
        window[ch] = window.get(ch, 0) + 1
        if ch in need and window[ch] == need[ch]:
            formed += 1
        while formed == required:
            if right - left + 1 < best_len:
                best_len = right - left + 1
                best_left = left
            lch = s[left]
            window[lch] -= 1
            if lch in need and window[lch] < need[lch]:
                formed -= 1
            left += 1
    return "" if best_len == math.inf else s[best_left: best_left + best_len]
```

formed 技巧正确性：formed 只在频率”恰好达到需求”(+1) 或”恰好低于需求”(-1) 时变化，确保每次 O(1) 判断是否满足全覆盖，而不用每次遍历 need 做全量比较（那样是 O(|t|)）。

例 3：3. Longest Substring Without Repeating Characters

题目：给定字符串 s，找最长不含重复字符的子串，返回其长度。

思路：可变窗口，维护”窗口内无重复”的约束。right 扩时若 s[right] 已在窗口中，left 右缩直到消除重复；每步更新最大长度。v2 优化版用 dict 存最近下标，left 跳跃而非逐步缩。

解：

```
# 参考: solutions/sliding_window/p003_longest_substring_without_repeating_characters.py
def lengthOfLongestSubstring(s: str) -> int:
    # 优化版: dict 存最近下标, left 跳跃
    last_seen: dict[str, int] = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1 # 跳跃到重复字符的后一位
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

两种写法对比：

写法	核心结构	适用场景
set + while 逐步缩	while s[right] in window: remove(s[left]); left++	更直观，易理解
dict + 跳跃	left = max(left, last_seen[ch] + 1)	更高效，减少 left 移动次数

两种写法时间复杂度同为 $O(n)$ ，跳跃版常数更小。

自测题

自测 1（209 题 Minimum Size Subarray Sum）——给定 target = 7，nums = [2,3,1,2,4,3]，找最短连续子数组使和 ≥ 7，答案为 2（子数组 [4,3]）。□ 提示：可变窗口，right 右扩加 nums[right]，窗口和 ≥ target 时记录长度并 left 右缩；注意全部扫完后可能无解（返回 0）。参考 solutions/sliding_window/p209_minimum_size_subarray_sum.py。

自测 2（3 题 Longest Substring Without Repeating Characters）——给定 s = "abcabcbb"，返回最长无重复子串的长度（答案 3，“abc”）。□ 提示：可变窗口 + set（违反时 while 缩左）或 dict 存最近下标（跳跃版）；dict 版注意 last_seen[ch] >= left 的守卫条件。参考 solutions/sliding_window/p003_longest_substring_without_repeating

自测 3（76 题 Minimum Window Substring）——给定 s = "ADOBECODEBANC"，t = "ABC"，返回最小覆盖子串（答案“BANC”）。□ 提示：need = Counter(t)，formed/required 追踪满足状态；right 扩时

若频率恰好满足需求则 `formed++`；满足全覆盖时记录窗口并缩左，缩到频率不足时 `formed -`。参考 `solutions/sliding_window/p076_minimum_window_substring.py`。

自测 4（30 题 Substring with Concatenation of All Words）——给定 `s = "barfoothefoobarman"`, `words = ["foo", "bar"]`, 返回所有起点下标(答案 `[0, 9]`)。□ 提示: `L = 单词长度 = 3`, 只需枚举 `range(L)` 共 3 个起点, 内层步长 `L` 滑动; 遇到非法词(不在 `words` 中)重置计数和左指针。参考 `solutions/sliding_window/p030_substring_with_con`

自测 5（综合设计题）——给定整数数组 `nums` 和整数 `k`, 找最长连续子数组使其中最多有 `k` 个不同的元素。□ 提示: 可变窗口 + Counter, `len(window) > k` 时左缩并减少频率（频率归零时从 Counter 删除该键），每步记录 `best = max(best, right - left + 1)`。这是 3 题（`k = 无限`）到 76 题（字符集约束）之间的中间形态。

题目全览（4 题）

#	题目	套路分类	难度
209	Minimum Size Subarray Sum	可变窗口，数值 sum	Medium
3	Longest Substring Without Repeating Characters	可变窗口，set/dict	Medium
76	Minimum Window Substring	可变窗口，formed/required	Hard
30	Substring with Concatenation of All Words	固定步长枚举，Counter	Hard

融合版说明

段	来源	价值
一例速记	本文件	3 类套路一览 + AI 工程类比
思维路径还原	本文件	4 题解题内心独白，含 <code>formed</code> 技巧和固定步长细节
抽象成方法	本文件	5 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体（最短 vs 最长 / 计数层次 / 字符串 vs 数组 / AI 类比）
思考路标	本文件	12 条题型识别条件反射，含 <code>while</code> 记录位置规则

段	来源	价值
易错点	本文件	7 条高频踩坑，每条对应具体题目
典型应用例题	solutions/	3 道精讲（209、76、3），代码 + 正确性分析
自测题	leetcode	5 题带 □ 提示，含 1 道综合设计题
题目全览	本文件	4 题完整列表，套路分类一览

04 —Binary Search（融合版）

难度：□□□□□ 题数：7 核心套路：标准二分、旋转数组二分、边界/区间二分、二分分割 本文件：覆盖 binary_search 7 题的算法套路总结 + 典型题精讲 + 自测

一例速记

标准二分（找位置）：l, r = 0, n, while l < r, mid = l + (r-l)//2, 条件满足时 r = mid 否则 l = mid+1; 返回 l（35 插入位置、74 二维矩阵）找峰值/边界（162）：while l < r, 比较 nums[mid] 与 nums[mid+1], 峰在右则 l = mid+1, 峰在左则 r = mid; 收敛到 l 即峰值下标 旋转数组（33 / 153）：先判断 mid 落在左段还是右段（比较 nums[mid] 与 nums[0]），再决定往哪侧收缩 找第一个 / 最后一个位置（34）：两次 lower_bound —第一次不变，第二次对 target+1 求 lower_bound 再 -1 中位数分割（4）：在较短数组上二分 partition, 用”最大左侧 ≤ 最小右侧”的不变式验证分割合法性, $O(\log \min(m, n))$ AI 关联：超参网格搜索 → 单调损失面上的二分；学习率调度（指数/余弦）→ 值域二分寻找最优点

思维路径还原

“看到 35 Search Insert Position：有序数组中找 target 的插入位置 → 就是 lower_bound：找第一个 target 的下标，不存在则返回 n。l=0, r=n（开区间右端 = 数组长度），while l < r, mid = l+(r-l)//2。nums[mid] >= target 则目标在左半（含 mid）→ r = mid; 否则 l = mid+1。循环结束 l == r, 返回 l。

看到 74 Search a 2D Matrix：每行升序，每行首元素大于上行尾元素 → 矩阵可视为展平的有序数组：把 mid 转换为行列 (mid // cols, mid % cols) 即可套标准二分。总长 m × n, 范围 [0, m*n-1]；比较 matrix[mid//n][mid%n] 与 target。

看到 162 Find Peak Element：nums[-1] = nums[n] = -∞, 找任意峰值下标 → 峰值不唯一；二分关键：nums[mid] < nums[mid+1] 说明右边坡度上升，峰在 mid 右侧 → l = mid+1; 否则峰在 mid

或左侧 $\rightarrow r = mid$ 。while $l < r$ 收敛后 l 即为答案。

看到 **33 Search in Rotated Sorted Array**: 数组旋转后在其中找 $target \rightarrow$ 先判断 mid 落在哪个升序段: $nums[mid] \geq nums[0]$ 说明 mid 在左段; $target$ 在 $[nums[0], nums[mid]]$ 范围内则收缩右端, 否则收缩左端; mid 在右段时对称处理。用 $l \leq r$ (闭区间) 更直观, 命中时直接返回 mid 。

看到 **34 Find First and Last Position**: 有序数组找 $target$ 第一个/最后一个下标 $\rightarrow lower_bound(target)$ 给第一个 $\geq target$ 的位置; $lower_bound(target+1) - 1$ 给最后一个 $\leq target$ 的位置。若 $lower_bound(target)$ 处值不等于 $target$, 则不存在, 返回 $[-1, -1]$ 。

看到 **153 Find Minimum in Rotated Sorted Array**: 无重复旋转数组找最小值 $\rightarrow$ 最小值就是旋转点左侧的那个元素; $nums[mid] > nums[r]$ 说明最小值在右段 $\rightarrow l = mid+1$; 否则最小值在左段 (含 mid) $\rightarrow r = mid$; while $l < r$ 收敛后 l 即答案。

看到 **4 Median of Two Sorted Arrays**: 两个有序数组, 要求 $O(\log \min(m, n)) \rightarrow$ 在较短数组上二分 partition: 设左侧共 $(m+n+1)/2$ 个元素, 枚举 $nums1$ 划分点 i , 验证 $maxLeft1 \leq minRight2$ 且 $maxLeft2 \leq minRight1$; 不满足时调整 i 。奇偶通用: 总长奇数时中位数 = $\max(maxLeft1, maxLeft2)$, 偶数时 = $(\max + \min) / 2$ 。”

学习目标

- 彻底搞清楚 $l \leq r$ 、 $l < r$ 两种循环终止条件的适用场景及其对应的返回值
- 掌握 $lower_bound$ / $upper_bound$ 两种边界模板并能用于 34 题
- 能识别旋转数组的“左段/右段”结构并写出正确的分支条件
- 理解“对函数单调性二分”的通用思想 (162 峰值、153 最小值)
- 掌握 4 题的分割二分思路, 理解 $(m+n+1)/2$ 的奇偶统一技巧

几何示意

图标准二分迭代 (LC 35)

图找左右边界 (LC 34)

抽象成方法 (标准模板代码)

###

套路

1:

lower_bound

—标

准左

边界

二分

适用

题:

35

(插入

位

置)、

74

(二维

矩阵

搜

索)、

34

(第一

个位

置)

```

"""python
def
lower_bound(nums:
list[int],
target:
int) ->
int: """
“返回
第一
个 >=
target
的下
标;
若所
有元
素 <
target
则返
回
len(nums)。
区间
语义:
[l, r),
循环
不变
式:
答案
在 [l,
r]
内。”
"""

l, r = 0,
len(nums)
# 注
意右
端是
len(nums),
不是
len(nums)-
1
while
l < r:
mid =
l + (r -

```

```
# 35:
搜索
插入
位置
def
searchIn-
sert(nums:
list[int],
target:
int) ->
int:
return
lower_bound(nums,
target)
```

```
# 74:
搜索
二维
矩阵
(每行
首元
素大
于上
一行
末尾)
def
search-
Ma-
trix(matrix:
list[list[int]],
target:
int) ->
bool:
m, n =
len(matrix),
len(matrix[0])
l, r =
0, m *
n
while
l < r:
mid =
l + (r -
l) // 2
row,
col =
div-
mod(mid,
n) if
ma-
trix[row][col]
<
target:
l =
mid +
1 elif
ma-
trix[row][col]
>
```

> 关
键：
右端
初始
化为
`len(nums)`
而非
`len(nums)-1`，
使结
果自
然覆
盖”插
入到
末尾”
的情
况。

套路
2：找
第一
个 /
最后
一个
位置
(`lower_bound`
+ `upper_bound`)
适用
题：
34
(`Find
First
and
Last
Position`)

```

“python
def
searchRange(nums:
list[int],
target:
int) ->
list[int]:
""" “时
间
O(log
n),
两次
二分
分别
找左
右边
界。”
""" def
lower_bound(t:
int) ->
int: """
“第一
个 >=
t 的位
置。”
"""

l, r = 0,
len(nums)
while
l < r:
mid =
l + (r -
l) // 2
if
nums[mid]
< t: l
= mid
+ 1
else: r
= mid
return
l

```

```

first =
lower_bound(target)
# 若
first
超界
或值
不匹
配,
说明
target
不存
在 if
first
==
len(nums)
or
nums[first]
!=
target:
return
[-1,
-1] last
=
lower_bound(target
+ 1) -
1 # 第
一个
>
target
的位
置减
1
return
[first,
last]
"""

```

>
 lower_bound(target+1)
 - 1
 等价
 于”最
 后一
 个 $\leq$
 target
 的位
 置”;
 比单
 独写
 up-
 per_bound
 更简
 洁。
 ###
 套路
 3: 找
 峰值 /
 边界
 ($1 < r$
 收敛
 型)
 适用
 题:
 162
 (Find
 Peak
 Ele-
 ment)、
 153
 (Find
 Mini-
 mum
 in Ro-
 tated
 Sorted
 Ar-
 ray)

```

"""python
# 162:
找任
意峰
值下
标 def
find-
PeakEle-
ment(nums:
list[int])
-> int:
""" “时
间
O(log
n)。
nums[-
1] =
nums[n]
= -inf,
峰值
必然
存
在。”
"""

l, r = 0,
len(nums)
- 1
while
l < r:
mid =
l + (r -
l) // 2
if
nums[mid]
<
nums[mid
+ 1]: l
= mid
+ 1 #
右坡
上升,
峰在
右侧
else: r

```

```

# 153:
旋转
数组
找最
小值
def
find-
Min(nums:
list[int])
-> int:
    """ “时
    间
    O(log
    n)。
    与右
    端
    nums[r]
    比较
    决定
    收缩
    方
    向。”
    """
    l, r = 0,
    len(nums)
    - 1
    while
    l < r:
        mid =
        l + (r -
        l) // 2
        if
        nums[mid]
        >
        nums[r]:
            l =
            mid +
            1 #
            mid
            在左
            段,
            最小
            值在
            右段

```

> 两
题都
用 l
< r
保证
收敛
时 l
== r ,
不需
要单
独判
断返
回值。

套路
4: 旋
转数
组搜
索 (l
≤ r
闭区
间型)
适用
题: 33
(Search
in Ro-
tated
Sorted
Ar-
ray)

```

python
def
search(nums:
list[int],
target:
int)
->
int:
"""
时间
O(log
n)。
先判
断
mid
在哪
个升
序段,
再决
定收
缩方
向。
"""
l, r
= 0,
len(nums)
- 1
while
l <=
r:
mid =
l +
(r -
l) //
2
if
nums[mid]
==
target:
return
mid
if
nums[l]
<=
nums[mid]:

```

> 关
键: 用
nums[1]
<=
nums[mid]
而非
nums[mid]
>
nums[0],
这样
处理
l ==
mid
的情
况更
安全。

套路
5: 分
割二
分
(中位
数)
适用
题: 4
(Me-
dian
of
Two
Sorted
Ar-
rays)

```

“python
def
find-
Medi-
an-
SortedAr-
rays(nums1:
list[int],
nums2:
list[int])
->
float:
""" “时
间
O(log
min(m,n))。
在较
短数
组上
二分
parti-
tion。”
"""# 保
证
nums1
是较
短的
数组
if
len(nums1)
>
len(nums2):
nums1,
nums2
=
nums2,
nums1
m, n =
len(nums1),
len(nums2)
half =
(m + n
+ 1) //
2 # 左

```

```
l, r =  
0, m  
while  
l <= r:  
    i = l +  
        (r - l)  
        // 2 #  
    nums1  
    左侧  
    取 i  
    个 j =  
        half - i  
    #  
    nums2  
    左侧  
    取 j  
    个
```

```
maxLeft1
=
float(
'-inf')
if i ==
0 else
nums1[i
- 1]
min-
Right1
=
float(
'-inf')
if i ==
m else
nums1[i]
maxLeft2
=
float(
'-inf')
if j ==
0 else
nums2[j
- 1]
min-
Right2
=
float(
'-inf')
if j ==
n else
nums2[j]
```

```

if
maxLeft1
<=
min-
Right2
and
maxLeft2
<=
min-
Right1:
# 分
割合
法 if
(m +
n) % 2
== 1:
return
float(max(maxLeft1,
maxLeft2))
else:
return
(max(maxLeft1,
maxLeft2)
+
min(minRight1,
min-
Right2))
/ 2.0
elif
maxLeft1
> min-
Right2:
r = i -
1 #
nums1
左侧
取多
了,
收缩
else: l
= i + 1
#
nums1
左侧

```

速查
表

| 题型
特征 |
套路 |
终止
条件 |
时间 |
空间 |
|—|
—|—|
—||
有序
数组
找插
入位
置 /
第一
个 $\geq$
target
|
lower_bound
| 1 <
r, 返
回 1 |
 $O(\log n)$
| $O(1)$
|| 找
第一
个 /
最后
一个
位置 |
lower_bound
 $\times 2 | 1$
< r |
 $O(\log n)$
| $O(1)$
|| 二
维矩
阵搜
索
(行首
递增)
| 展平
为

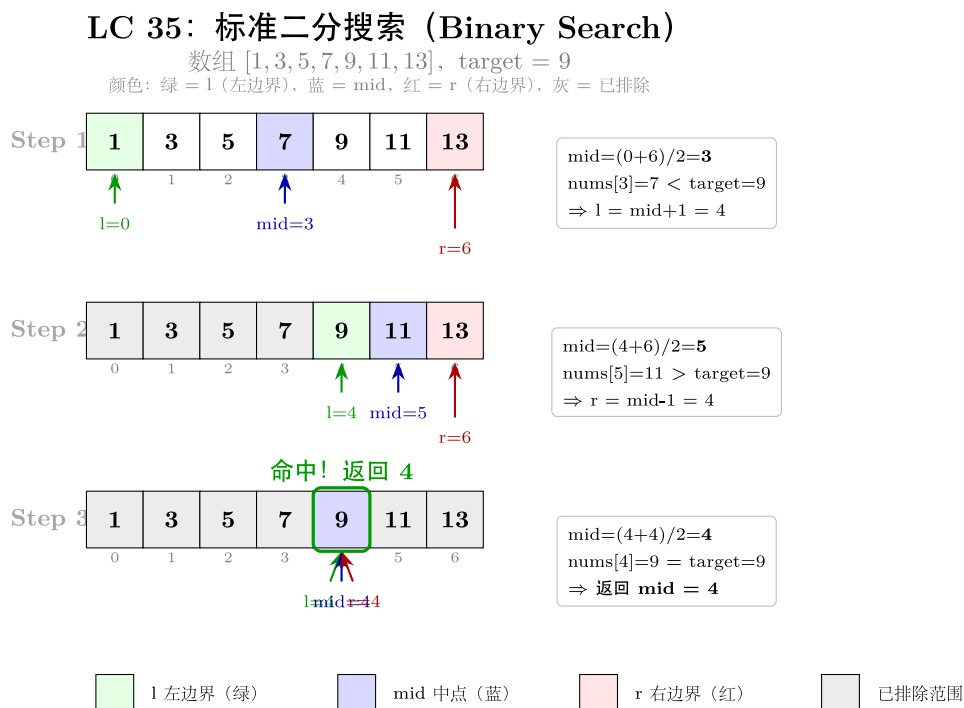


图 8: l/r/mid 三指针演化

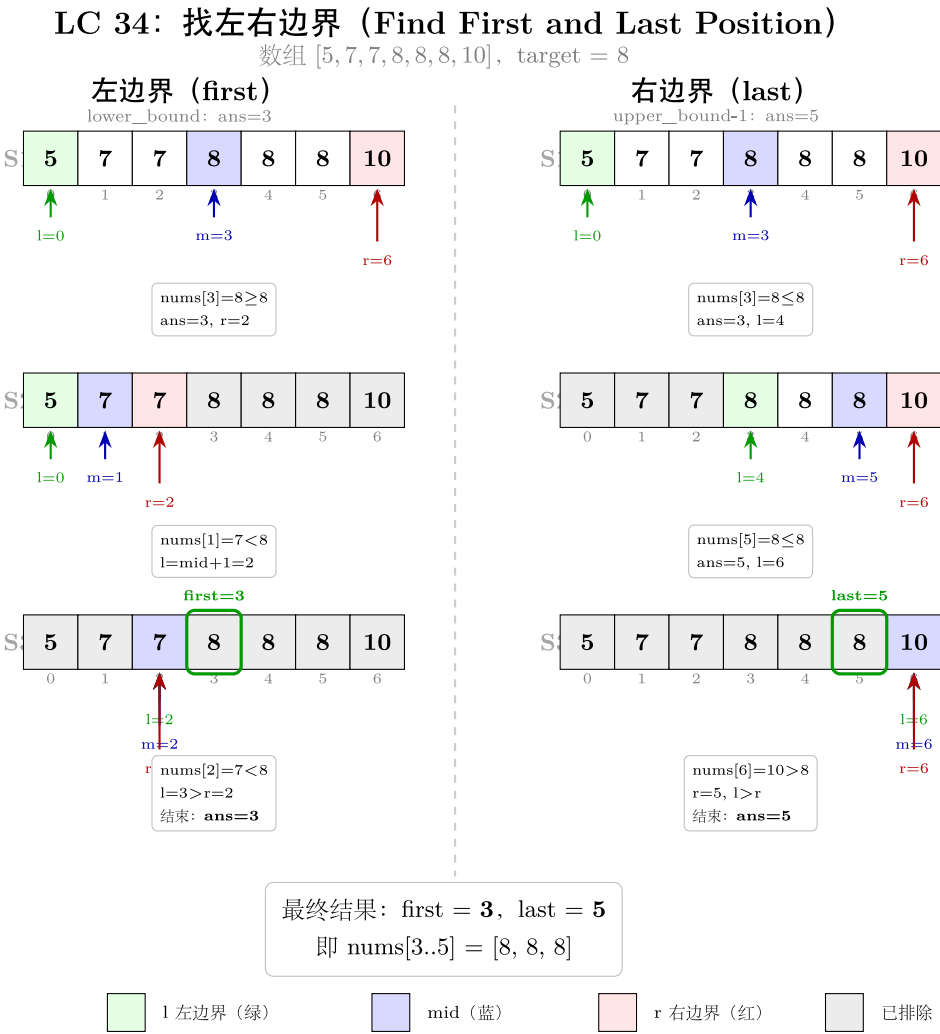
方法变形 (4 类)

变形 1: lower_bound 扩展系列

- **35** (插入位置): 直接返回 `lower_bound(target)`。
- **34** (第一/最后位置): `lower_bound(target) + lower_bound(target+1) - 1`, 两次调用复用同一模板。
- **74** (二维矩阵): 下标 `mid` 通过 `divmod(mid, n)` 映射为行列, 实质是 1D 标准二分的坐标变换。
- 泛化: 在“单调谓词”(条件从 False 变为 True 且只变一次) 上二分答案——适用于“最小化最大值 / 最大化最小值”等优化问题。

变形 2: 旋转数组系列

- **153** (只找最小值): 不需要知道 `target`, 只需与 `nums[r]` 比较找转折点, 用 $l < r$ 收敛。
- **33** (找目标值): 需要额外判断 `target` 在哪个段, 用 $l \leq r$ 闭区间, 命中时直接返回。
- **81** (含重复元素, 非本 category): `nums[l] == nums[mid] == nums[r]` 时三段相同无法判断, 需 `l++`, `r--` 线性退化, 最坏 $O(n)$ 。
- 关键区分: 153 用右端 `nums[r]` 比较, 33 用左端 `nums[l]` 判断左段——两种写法各有优势, 保持一致不要混用。



变形 3: 峰值系列

- **162** (任意峰值): 比较 `nums[mid]` 与 `nums[mid+1]`, 单侧比较即可 (题目保证边界为 $-\infty$)。
- **852** (山脉数组峰值, 非本 category): 同样的 $l < r$ 收敛, 结构完全一致。
- 对单调函数二分: 若 $f(\text{mid})$ 严格单调 (如超参搜索中损失函数), 直接套 `lower_bound` 模板找最优点。
- AI 关联: ternary search (三分搜索) 用于找单峰函数极值, 是 162 思路在连续域的推广。

变形 4: 分割 / 分治系列

- **4** (中位数): 分割二分, 核心是“左半总数固定”的不变式。
- 对答案二分 (非本 category): 875 Koko Eating Bananas、1011 Capacity To Ship Packages——在“可能的答案值域”上二分, 验证函数替代直接搜索。
- 分治归并: 两有序数组合并后取中位数的 $O(m+n)$ 做法是分治基础; 4 题要求 $O(\log \min(m, n))$ 才需要分割二分。

思考路标 (条件反射)

1. 看到 有序数组 + “找位置 / 插入 / 第一个满足条件” $\rightarrow$ `lower_bound`, $l=0$, $r=n$, `while l < r`, 返回 l
2. 看到 “找最后一个位置” $\rightarrow$ `lower_bound(target+1) - 1`, 复用同一模板
3. 看到 “二维矩阵 + 每行有序 + 行首递增” $\rightarrow$ `divmod(mid, n)` 展平为 1D 二分
4. 看到 “找任意峰值” $\rightarrow$ 比较 `nums[mid]` 与 `nums[mid+1]`, $l < r$ 收敛到峰值
5. 看到 “旋转数组 + 找最小值” $\rightarrow$ 与 `nums[r]` 比较, `while l < r`, 返回 `nums[l]`
6. 看到 “旋转数组 + 找目标值” $\rightarrow$ 先判断 `mid` 在哪个升序段, `while l <= r` 闭区间
7. 看到 “两有序数组 + 中位数 + $O(\log)$ ” $\rightarrow$ 在短数组上分割二分, 验证 $\text{maxLeft} \leq \text{minRight}$
8. 看到 循环写 $l \leq r \rightarrow$ 命中时直接返回, 未命中时 $l = \text{mid}+1$ 或 $r = \text{mid}-1$, 循环结束表示不存在
9. 看到 循环写 $l < r \rightarrow$ 收敛时 $l == r$, 该位置即答案; $r = \text{mid}$ (不减 1), 保证不遗漏
10. 看到 $\text{mid} = (l+r)//2 \rightarrow$ 改为 $\text{mid} = l + (r-l)//2$ (Python 无溢出, 但 C++/Java 必须这样写)
11. 看到 “最小化最大值 / 最大化最小值” $\rightarrow$ 对答案值域做二分, 写 `check` 函数判断可行性
12. 看到 “超参搜索 / 学习率调优” $\rightarrow$ 若损失曲线单调, 二分搜索最优超参值; 若单峰, 用三分搜索

易错点

1. $l < r$ vs $l \leq r$ 混用: $l < r$ 适合“收敛到唯一点”的场景 (结果不需要验证, 如找峰值/最小值); $l \leq r$ 适合“找目标值、可能不存在”的场景 (命中时 `return`, 结束时说明不存在)。两种混用会导致死循环或漏判。
2. $r = \text{mid}$ vs $r = \text{mid}-1$: 使用 $l < r$ 时 $r = \text{mid}$ (不减 1); 使用 $l \leq r$ 时 $r = \text{mid}-1$ (减 1)。混淆会导致死循环 (当 $l == \text{mid}$ 时 $r = \text{mid}$ 不变化, 永远循环)。

3. **lower_bound** 右端初始化: 应为 `r = len(nums)` 而非 `r = len(nums)-1`; 若用后者, “插入末尾”的情况 (`target` 比所有元素大) 会返回错误的 `n-1`。
4. 旋转数组的等号处理: `nums[l] <= nums[mid]` 的等号不能省; 若 `l == mid` (即数组只剩两个元素时), 没有等号会把 `mid` 归为右段, 导致边界判断错误。
5. 34 题不存在的情况: 调用 `lower_bound(target)` 后必须检查 `first == len(nums) or nums[first] != target`; 仅靠 `lower_bound` 不能区分 “`target` 不存在” 和 “`target` 就在边界”。
6. 4 题分割中 `half = (m+n+1)//2` 的作用: 奇偶通用——奇数总长时左半比右半多 1 个, 结果取 `maxLeft`; 偶数时各取一半, 结果取 `(maxLeft + minRight) / 2`。若写成 `(m+n)//2` 奇数情况会少取一个导致错误。
7. Python 的 `//` 向下取整: `mid = (l+r)//2` 在 `l, r` 均为非负数时等价于向下取整, 不会在 `l == r-1` 时让 `mid == r` (避免死循环)。但 `(l+r+1)//2` 用于 “向上取整 `mid`” 的写法有时用于 `l < r+1 = mid` 的模板, 需明确选哪种。
8. 162 峰值的越界风险: 比较 `nums[mid]` 与 `nums[mid+1]` 时, `l < r` 保证 `mid < r <= len-1`, 所以 `mid+1` 不越界; 若误用 `l <= r` 则当 `l == r` 时 `mid+1` 可能越界。

典型应用例题

例 1: 34. Find First and Last Position of Element in Sorted Array

题目: 给定升序数组 `nums` 和目标 `target`, 找 `target` 第一次和最后一次出现的下标; 若不存在返回 `[-1, -1]`。

思路: 两次 `lower_bound`。第一次求 `lower_bound(target)` 得到第一个 `>= target` 的位置, 检验值是否等于 `target`; 第二次求 `lower_bound(target+1) - 1` 得到最后一个 `<= target` 的位置。

解:

参考: [solutions/binary_search/p034_find_first_and_last_position_of_element_in_sorted_array.py](#)

```
def searchRange(nums: list[int], target: int) -> list[int]:
```

```
    def lower_bound(t: int) -> int:
```

```
        l, r = 0, len(nums)
```

```
        while l < r:
```

```
            mid = l + (r - l) // 2
```

```
            if nums[mid] < t:
```

```
                l = mid + 1
```

```
            else:
```

```
                r = mid
```

```
        return l
```

```
first = lower_bound(target)
```

```
if first == len(nums) or nums[first] != target:
```

```
    return [-1, -1]
```

```
last = lower_bound(target + 1) - 1
return [first, last]
```

分析： $O(\log n)$ 时间， $O(1)$ 空间。`lower_bound(target+1) - 1` 技巧避免另写 `upper_bound` 模板，代码精简且不易出错。

例 2: 33. Search in Rotated Sorted Array

题目：整数数组在某个轴点旋转，给定 `target`，返回其下标；若不存在返回 -1。所有元素不重复。

思路：旋转后数组分为两个有序段。二分时先用 `nums[l] <= nums[mid]` 判断 `mid` 落在哪段，再判断 `target` 是否在该段范围内，据此收缩区间。

解：

参考：[solutions/binary_search/p033_search_in_rotated_sorted_array.py](#)

```
def search(nums: list[int], target: int) -> int:
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if nums[mid] == target:
            return mid
        if nums[l] <= nums[mid]:          # 左段有序
            if nums[l] <= target < nums[mid]:
                r = mid - 1
            else:
                l = mid + 1
        else:                             # 右段有序
            if nums[mid] < target <= nums[r]:
                l = mid + 1
            else:
                r = mid - 1
    return -1
```

分析： $O(\log n)$ 。关键是两个嵌套 `if`——外层判断 `mid` 在哪段，内层判断 `target` 是否在该有序段的范围内；不满足则 `target` 必在另一段，向另侧收缩。

例 3: 4. Median of Two Sorted Arrays

题目：给定两个升序数组 `nums1`（长 `m`）和 `nums2`（长 `n`），找合并后的中位数，时间 $O(\log \min(m, n))$ 。

思路：在较短数组上二分 partition 位置 i ，nums2 的 partition 自动确定为 $j = \text{half} - i$ 。验证分割合法性： $\text{maxLeft1} \leq \text{minRight2}$ 且 $\text{maxLeft2} \leq \text{minRight1}$ 。奇偶统一：左半取 $(m+n+1)//2$ 个。

解：

参考：solutions/binary_search/p004_median_of_two_sorted_arrays.py

```
def findMedianSortedArrays(nums1: list[int], nums2: list[int]) -> float:
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    half = (m + n + 1) // 2
    l, r = 0, m
    while l <= r:
        i = l + (r - l) // 2
        j = half - i
        maxLeft1 = float('-inf') if i == 0 else nums1[i - 1]
        minRight1 = float('inf') if i == m else nums1[i]
        maxLeft2 = float('-inf') if j == 0 else nums2[j - 1]
        minRight2 = float('inf') if j == n else nums2[j]

        if maxLeft1 <= minRight2 and maxLeft2 <= minRight1:
            if (m + n) % 2 == 1:
                return float(max(maxLeft1, maxLeft2))
            return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) / 2.0
        elif maxLeft1 > minRight2:
            r = i - 1
        else:
            l = i + 1
    raise ValueError("unreachable")
```

分析： $O(\log \min(m, n))$ 。float('-inf')/float('inf') 作为哨兵处理边界划分 ($i=0$ 或 $i=m$) 时无需特殊分支。合法分割保证了左半最大值 $\leq$ 右半最小值，即中位数可以直接读出。

自测题

自测 1 (35 题 Search Insert Position) —— 有序数组 $[1, 3, 5, 6]$ ，target=5 返回 2，target=2 返回 1，target=7 返回 4。□ 提示：lower_bound 模板， $l=0$ ， $r=\text{len}(\text{nums})$ ，while $l < r$ ， $\text{nums}[\text{mid}] < \text{target}$ 则 $l=\text{mid}+1$ ，否则 $r=\text{mid}$ ，返回 l 。参考 solutions/binary_search/p035_search_insert_position.py。

自测 2 (74 题 Search a 2D Matrix) —— 矩阵 $[[1, 3, 5, 7], [10, 11, 16, 20], [23, 30, 34, 60]]$ ，target=3 返回 True，target=13 返回 False。□ 提示： $\text{mid} = l + (r - l) // 2$ ，行列 divmod(mid, n)，对 matrix[row][col] 与 target 做

三路比较。参考 `solutions/binary_search/p074_search_a_2d_matrix.py`。

自测 3（162 题 Find Peak Element）——`nums=[1,2,1,3,5,6,4]` 返回 1 或 5（任意峰值下标均可）。□ 提示：`while l < r`，比较 `nums[mid]` 与 `nums[mid+1]`：前者小则峰在右（`l=mid+1`），否则峰在左含 `mid`（`r=mid`），循环结束返回 `l`。参考 `solutions/binary_search/p162_find_peak_element.py`。

自测 4（153 题 Find Minimum in Rotated Sorted Array）——`nums=[3,4,5,1,2]` 返回 1，`nums=[4,5,6,7,0,1,2]` 返回 0。□ 提示：与 `nums[r]` 比较：`nums[mid] > nums[r]` 则 `l=mid+1`，否则 `r=mid`；收敛后返回 `nums[l]`。参考 `solutions/binary_search/p153_find_minimum_in_rotated_sorted_array.py`。

自测 5（34 题 Find First and Last Position）——`nums=[5,7,7,8,8,10]`，`target=8` 返回 `[3,4]`，`target=6` 返回 `[-1,-1]`。□ 提示：两次 `lower_bound`：`lower_bound(target)` 给首位，`lower_bound(target+1)-1` 给末位；别忘了检查首位处的值是否确实等于 `target`。参考 `solutions/binary_search/p034_find_first_and_last_position_of_element_in_sorted_array.py`。

题目全览（7 题）

#	题目	套路分类	难度
35	Search Insert Position	<code>lower_bound</code> 标准二分	Easy
74	Search a 2D Matrix	展平 1D + 标准二分	Medium
162	Find Peak Element	单侧比较收敛	Medium
33	Search in Rotated Sorted Array	旋转数组 + 左右段判断	Medium
153	Find Minimum in Rotated Sorted Array	旋转数组 + 右端比较	Medium
34	Find First and Last Position	<code>lower_bound</code> × 2	Medium
4	Median of Two Sorted Arrays	分割二分	Hard

融合版说明

段	来源	价值
一例速记	本文件	7 题 5 类套路一览，含 AI 关联
思维路径还原	本文件	6 道题的解题内心独白，含关键决策点
抽象成方法	本文件	5 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展（ <code>lower_bound</code> / 旋转 / 峰值 / 分割）
思考路标	本文件	12 条题型识别条件反射，含边界条件选择
易错点	本文件	8 条高频踩坑，覆盖 <code>l<r</code> vs <code>l<=r</code> 、越界等经典 bug

段	来源	价值
典型应用例题	solutions/	3 道精讲 (34、33、4)，代码 + 正确性分析
自测题	leetcode	5 题带 □ 提示，链接 solutions 文件
题目全览	本文件	7 题完整列表，套路分类一览

跨 category 导航：- 有序数组上的”配对”问题优先考虑双指针 → 见 02-two-pointers.md - BST 上的搜索利用中序有序性，逻辑类似二分但走树指针 → 见 05-binary-search-tree.md - “最小化最大值 / 最大化最小值”型 DP → 先写 check 函数再套 lower_bound 对答案二分

05 —Binary Search Tree (融合版)

难度：□□□□□ 题数：3 核心套路：中序遍历有序性、区间验证 BST 合法性 本文件：覆盖 binary_search_tree 3 题的算法套路总结 + 典型题精讲 + 自测

一例速记

中序遍历有序性：BST 中序遍历（左 → 根 → 右）结果严格递增； $O(n)$ 时间 $O(h)$ 空间 (h 为树高)，是 530 最小差、230 第 k 小的核心手段 迭代中序（栈模拟）：用显式栈替代递归，可随时中断——找到第 k 个元素即停，避免全量遍历；空间 $O(h)$ 区间验证：递归传递 (lo, hi) 区间，每个节点值必须在区间内；根节点区间为 $(-\infty, +\infty)$ ，向左传 $hi = node.val$ ，向右传 $lo = node.val$ (98 验证 BST) **BST 搜索**：从根出发，值小则走左子树，值大则走右子树， $O(h)$ 无需遍历全树；是普通二分查找的树形版本 **AI 关联**：决策树 (ID3/C4.5) 的划分条件 = BST 分支逻辑；XGBoost 每棵树的节点判断 = BST 搜索路径；排序数据库索引 (B-Tree) 是 BST 的磁盘友好扩展

思维路径还原

“看到 **530 Minimum Absolute Difference in BST**：BST 中任意两节点的最小绝对差 → BST 中序有序，所以最小差只可能出现在相邻节点之间。维护 `prev`（中序前驱节点值），每次访问节点时计算 `node.val - prev`，更新全局最小值。递归：先访问左子树，再处理当前节点（更新 `prev` 和 `min_diff`），再访问右子树。初始 `prev = -inf`，第一个节点与 `prev` 的差必然很大，不影响结果。

看到 **230 Kth Smallest Element in a BST**：BST 中第 k 小的元素 → 中序遍历是升序，第 k 个访问的节点就是答案。递推：先中序遍历左子树，每访问一个节点 $k--$ ， k 变为 0 时当前节点即为答案。优化：迭代中序用显式栈，找到第 k 个即停，不继续遍历剩余节点——若频繁调用可进一步用”带 rank 的增强 BST”。

看到 **98 Validate Binary Search Tree**: 验证是否是合法 BST → 关键陷阱: 不能只比较节点与左右子节点, 必须检查整棵子树的值域。传递 (lo, hi) 区间: 根节点为 (-inf, +inf); 向左递归时 hi = node.val; 向右递归时 lo = node.val。若 node.val <= lo 或 node.val >= hi 则非法 (注意严格不等号, BST 通常要求严格大于)。空节点返回 True (空树是合法 BST)。”

学习目标

- 掌握 BST 中序遍历 (递归 + 迭代) 的两种实现, 理解中序结果严格递增的来源
- 能用”中序前驱 prev”技巧 $O(1)$ 额外空间解决最小差类问题
- 理解”区间验证”替代”局部比较”的必要性, 避免 98 题最常见的错误思路
- 能写出迭代中序 (显式栈) 并在找到第 k 个时提前终止
- 认识 BST 与排序数据结构、决策树的工程联系

几何示意

图 BST 结构示意

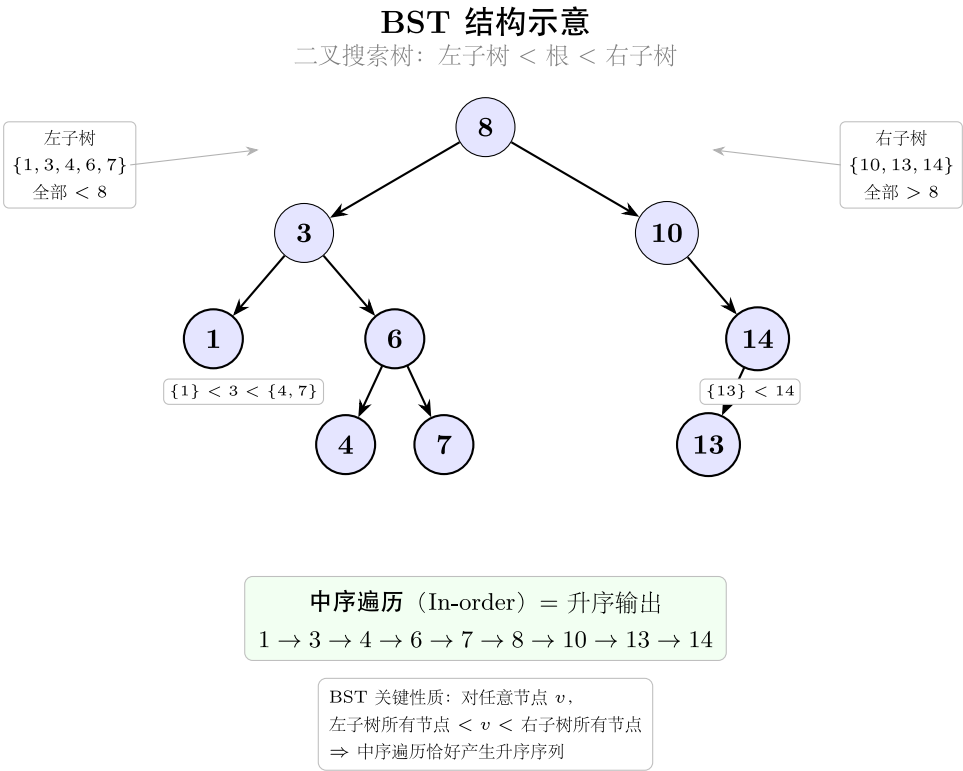


图 10: 根 / 左 < 根 < 右 / 中序有序

图 BST 验证 (LC 98)

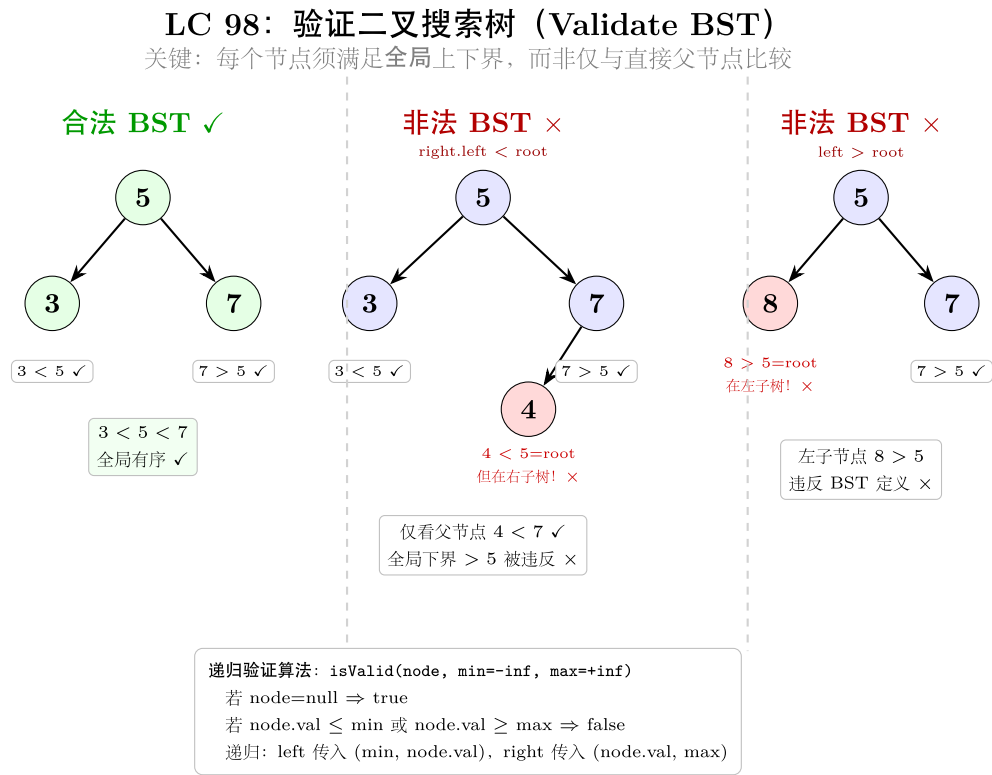


图 11: 合法 vs 两种非法情形

抽象成方法 (标准模板代码) ### 套路 1: 中序遍历 (递归) + 前驱追踪

适用
题:
530
(最小
绝对
差)
“python
假
设已
有
TreeN-
ode
定义
class
TreeN-
ode: #
def
init(self,
val=0,
left=None,
right=None):

self.val
= val;
self.left
= left;
self.right
=
right

```

def
get-
Mini-
mumD-
iffer-
ence(root)
-> int:
"""
“BST
中序
遍历,
相邻
节点
差的
最小
值。
时间
O(n),
空间
O(h)。”
"""
min_diff
=
float(
‘inf’)
prev =
float(
‘-inf’)
# 中
序前
驱的
值

```

```

def in-
order(node)
->
None:
nonlo-
cal
min_diff,
prev if
node
is
None:
return
in-
order(node.left)
# 先
遍历
左子
树
min_diff
=
min(min_diff,
node.val
- prev)
# 处
理当
前节
点
prev =
node.val
# 更
新前
驱 in-
order(node.right)
# 再
遍历
右子
树
inorder(root)
return
min_diff
"""

```

> 关
键:
`prev`
初始
化为
`float('-inf')`
使第
一个
节点
的差
足够
大不
影响
结
果; 用
`nonlocal`
在嵌
套函
数中
修改
外层
变量。

套路
2: 迭
代中
序
(显式
栈) +
计数
中断
适用
题:
230
(第 k
小元
素)

```
python
def
kthSmallest(root,
k:
int)
->
int:
"""
迭代
中序
遍历,
第 k
次访
问即
为答
案。
时间
 $O(h+k)$ ,
空间
 $O(h)$ 。
"""
stack
= []
cur =
root
count
= 0
while
cur
is
not
None
or
stack:
# 一
路走
到最
左
while
cur
is
not
None:
stack.append(cur)
```

> 迭
代中
序的”
一路
走左,
弹出
访问,
转右”
三步
循环
是标
准模
板,
可随
时中
断而
不必
遍历
整棵
树。

套路
3: 区
间验
证
(递归
传递
上下
界)
适用
题:
98
(Vali-
date
Bi-
nary
Search
Tree)

```

“python
def is-
ValidBST(root)
->
bool:
“““时
间
O(n),
空间
O(h)。
传递
(lo, hi)
区间,
每个
节点
值必
须严
格在
区间
内。”
“”def
vali-
date(node,
lo:
float,
hi:
float)
->
bool:
if
node
is
None:
return
True if
node.val
<= lo
or
node.val
>= hi:
return
False
return
(vali-

```

```

return
vali-
date(root,
float(
'-inf'),
float(
'inf'))
"""

```

$>$ 严格
 格不
 等号:
 标准
 BST
 定义
 为左
 子树
 所有
 值 严
 格小
 于根,
 右子
 树所
 有值
 严格
 大于
 根;
 区间
 用开
 区间
 (lo,
 hi)
 配严
 格不
 等号。

###

套路

4:

BST

搜索 /

插入

(路径

追踪)

适用

题:

搜索

与插

入基

础操

作

(本

cate-

gory 3

题的

底层

逻辑)

```
“python
def
bst_search(root,
target:
int):
"""
“BST
搜索,
时间
O(h),
空间
O(1)
(迭代
版)。”
“cur
= root
while
cur is
not
None:
if
target
==
cur.val:
return
cur
elif
target
<
cur.val:
cur =
cur.left
else:
cur =
cur.right
return
None
```

```

def
bst_insert(root,
val:
int):
"""
“BST
插入,
返回
根节
点
(递归
版)。”
"""
if
root is
None:
return
type(root)(val)
# 新
建节
点 if
val <
root.val:
root.left
=
bst_insert(root.left,
val)
elif
val >
root.val:
root.right
=
bst_insert(root.right,
val)
return
root """

```

> BST

搜索

是树

形二

分查

找:

每一

步排

除一

半子

树,

平衡

树时

$O(\log n)$,

退化

链时

$O(n)$ 。

####

套路

5: 中

序递

归

(通用

框架)

适用

题:

530、

230

(复用

此框

架)


```
"""python
def in-
order_collect(root)
->
list[int]:
"""“收
集
BST
中序
遍历
结果
(升序
列
表)。
仅用
于理
解，
生产
中可
提前
终
止。”
"""

result:
list[int]
= []
def
dfs(node)
->
None:
if
node
is
None:
return
dfs(node.left)
re-
sult.append(node.val)
dfs(node.right)
```

```

dfs(root)
return
result
# 530:
min(result[i+1]
- re-
sult[i]
for i
in
range(len(result)-
1)) #
230:
re-
sult[k-
1] ““
> 此
版本
便于
理解
但需
 $O(n)$ 
额外
空间
存储
结果;
套路
1/2 是
其空
间优
化版
本。
###
速查
表

```

题型
特征
套路
时间
空间
对应
题目
—
— —
—
BST
中任
意两
节点
最小
差
中序
递归
+ prev
追踪
$O(n)$
$O(h)$
530
BST
第 k
小的
元素
迭代
中序
+ 计
数中
断
$O(h +$
$k)$
$O(h)$
230
验证
是否
为合
法
BST
区间
验证
递归

$> h$ 为
 树高；
 平衡
 BST
 时
 $h =$
 $O(\log n)$ ，
 最坏
 退化
 链时
 $h =$
 $O(n)$ 。

方法变形 (4 类)

变形 1：中序前驱追踪系列

- **530** (最小绝对差)：中序前驱 `prev` 追踪，每次更新 `min_diff = min(min_diff, cur.val - prev)`。
- **783** (BST 节点最小距离，非本 category)：与 530 完全一致的模板，只是结果含义相同；LeetCode 两题实质相同。
- 中序转双向链表 (非本 category)：`prev` 不仅记值，还维护指针，遍历时修改节点 `left/right` 为前驱/后继。
- 平衡 BST 检查：中序收集后验证是否严格递增，等价于 98 的区间验证但空间更大。

变形 2：迭代中序 / 中断系列

- **230** (第 k 小)：计数到 k 即 `return`，不继续遍历。
- **BST 迭代器** (173，非本 category)：将迭代中序的栈封装为 `next()` / `hasNext()` 接口，每次调用只前进一步；空间 $O(h)$ 。
- 中序逆序 (右 $\rightarrow$ 根 $\rightarrow$ 左)：把所有 `left/right` 互换，得到降序遍历；用于“第 k 大”问题。
- **Morris** 中序遍历： $O(1)$ 空间替代栈，通过临时修改树结构实现；本 category 不需要，但是空间极限优化的参考。

变形 3：区间验证扩展系列

- **98** (验证 BST)：(`lo`, `hi`) 开区间传递，严格不等号。
- **BST 插入**：搜索时路径上每个节点天然维护了“此节点值的合法范围”——插入位置就是搜索失败的叶子位置。
- **BST 删除** (450，非本 category)：找到节点后，若有两个子节点，用中序后继 (右子树最左节点) 替代；是 98 区间逻辑的逆操作。

- 进阶：区间验证的 (lo, hi) 范围也可以用于剪枝——在搜索 / 计数问题中，若当前节点值已超出目标范围则整棵子树跳过。

变形 4：BST 与排序结构的工程联系

- **AI 类比—决策树**：每个内部节点是一个阈值判断 (`feature_val <= threshold`)，与 BST 分支完全同构；叶节点是分类/回归结果。
- **AI 类比—XGBoost 推理**：对单棵 CART 树做推理 = 从根沿 BST 路径走到叶节点， $O(\log n)$ 次比较。
- **数据库索引—B-Tree**：B-Tree 是 BST 的 m 叉推广，减少磁盘 I/O；B+Tree 所有数据在叶节点且相互链接，支持范围查询——对应 34 题的区间搜索。
- **平衡 BST (AVL / 红黑树)**：Python `sortedcontainers.SortedList` 底层是分块有序列表；C++ `std::map` / `std::set` 是红黑树；理解 BST 有助于分析其时间复杂度。

思考路标 (条件反射)

1. 看到 **BST** + “最小差 / 相邻节点” → 中序遍历，`prev` 前驱追踪，相邻访问差即候选
2. 看到 **BST** + “第 k 小 / 第 k 大” → 中序 (或逆中序) 迭代，计数到 k 立即返回
3. 看到 “验证是否为合法 **BST**” → 区间验证 (lo, hi) ，不要只比较父子节点
4. 看到 **BST** + “原地操作 / 频繁查询” → 迭代版优于递归 (无栈溢出风险，可中断)
5. 看到 “只比较 `node.val` 与 `node.left.val` / `node.right.val`” → 这是 98 题最常见的错误，反例：[5, 1, 4, null, null, 3, 6] 根节点 $4 < 5$ 不合法但局部比较通过
6. 看到 **BST** 搜索 → 从根出发比较大小走左/右，平均 $O(\log n)$ ，不需要遍历全树
7. 看到 “中序结果” → 快速收集：`inorder_collect(root)` 拿到有序列表；再做任何有序数组操作
8. 看到 树高 h → 平衡 BST $h = O(\log n)$ ，链状 BST $h = O(n)$ ；空间/时间分析时需说明哪种情况
9. 看到 **BST** + “范围查询 $[lo, hi]$ ” → 递归时若 `node.val < lo` 只走右子树，若 `node.val > hi` 只走左子树，剪枝 $O(\log n + k)$
10. 看到 “**BST** 转有序数组 / 链表” → 中序遍历即可；逆中序得到降序；修改指针可原地转双向链表
11. 看到 “决策树 / 随机森林推理” → 联想 BST 的搜索路径；特征阈值比较 = 节点分支；叶节点 = 预测结果

易错点

1. **98 题“局部比较”陷阱**：只比较 `node.val > node.left.val` 和 `node.val < node.right.val` 会通过反例 [5, 4, 6, null, null, 3, 7]——根节点 5 的右子树中有 $3 < 5$ ，但局部比较看不出。必须用区间 (lo, hi) 验证每个节点的全局合法性。
2. **BST 严格不等号**：标准 BST 定义中左子树值严格小于根，右子树值严格大于根 (无重复)；区间验证用 `node.val <= lo` or `node.val >= hi` 来拒绝等于边界的情况。若题目允许重复则需调整。

3. **530 / 230** 的 `prev` 初始化: `prev` 用于记录中序前驱值, 初始化为 `float('-inf')` (不是 0 或 `None`); 若初始化为 0, 当树只有一个节点时 `0 - 0 = 0` 可能错误地返回 0 差值。
4. 迭代中序的”一路走左”顺序: 外层 `while` 条件为 `cur is not None or stack`; 内层先把 `cur` 一路压栈到最左, 再弹出访问, 再转右——顺序不能颠倒, 否则访问顺序不是中序。
5. **230** 迭代版的 `count` 计数时机: `count += 1` 要在弹出节点后 (访问当前节点时) 而非压栈时; 压栈时节点尚未”访问”, 过早计数会导致结果偏移 `k` 位。
6. 平衡 **BST** vs 退化链: 时间复杂度通常标注为 $O(h)$, 而非 $O(\log n)$; 面试中应说明”若树平衡则 $O(\log n)$, 退化链则 $O(n)$ “, 不要直接说 $O(\log n)$ 。
7. **98** 题空节点处理: `if node is None: return True`——空节点是合法 BST (边界条件), 不要返回 `False` 或抛出异常; 遗漏此行会在空树时崩溃。

典型应用例题

例 1: 530. Minimum Absolute Difference in BST

题目: 给定 BST, 返回任意两节点之间最小绝对差。

思路: BST 中序有序, 相邻元素差最小。维护前驱节点值 `prev`, 遍历时计算 `node.val - prev` 更新答案。

解:

参考: [solutions/binary_search_tree/p530_minimum_absolute_difference_in_bst.py](#)

```
def getMinimumDifference(root) -> int:
    min_diff = float('inf')
    prev = float('-inf')

    def inorder(node) -> None:
        nonlocal min_diff, prev
        if node is None:
            return
        inorder(node.left)
        min_diff = min(min_diff, node.val - prev)
        prev = node.val
        inorder(node.right)

    inorder(root)
    return min_diff
```

分析: $O(n)$ 时间, $O(h)$ 空间 (递归调用栈)。利用 BST 中序严格递增的性质, 将”任意两点最小差”转化为”相邻中序元素最小差”, 一次遍历解决。

为什么相邻差最小：若数组 $a_1 < a_2 < \dots < a_n$ ，则对任意 $i < j$ ，有 $a_j - a_i = \sum_{k=i}^{j-1} (a_{k+1} - a_k) \geq a_{i+1} - a_i$ ，故最小差必在相邻元素之间。

例 2：230. Kth Smallest Element in a BST

题目：给定 BST 和整数 k，返回 BST 中第 k 个最小元素（1-indexed）。

思路：中序遍历升序访问节点，第 k 次访问即为答案。用迭代版（显式栈）避免全量遍历——找到第 k 个立即返回。

解：

参考：solutions/binary_search_tree/p230_kth_smallest_element_in_a_bst.py

```
def kthSmallest(root, k: int) -> int:
    stack = []
    cur = root
    count = 0
    while cur is not None or stack:
        while cur is not None:
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()
        count += 1
        if count == k:
            return cur.val
        cur = cur.right
    raise ValueError("k exceeds tree node count")
```

分析： $O(h + k)$ 时间， $O(h)$ 空间。与递归版（ $O(n)$ 时间，因为可能遍历全树）相比，迭代版在找到第 k 个后立即终止，对 k 较小或树较大的情况有显著优势。

进阶：若需要频繁查询不同的 k，可用“增强 BST”——每个节点额外存储左子树节点数，查询时根据左子树节点数与 k 的比较决定走哪侧， $O(\log n)$ 单次查询。

例 3：98. Validate Binary Search Tree

题目：给定二叉树的根节点，判断是否是有效的 BST（左子树所有节点严格小于根，右子树所有节点严格大于根，左右子树也均为合法 BST）。

思路：区间验证。递归时向下传递合法值域 (lo, hi)：访问左子节点时上界更新为 node.val，访问右子节点时下界更新为 node.val；任意节点值超出当前区间则非法。

解:

参考: `solutions/binary_search_tree/p098_validate_binary_search_tree.py`

```
def isValidBST(root) -> bool:
    def validate(node, lo: float, hi: float) -> bool:
        if node is None:
            return True
        if node.val <= lo or node.val >= hi:
            return False
        return (validate(node.left, lo, node.val) and
                validate(node.right, node.val, hi))

    return validate(root, float('-inf'), float('inf'))
```

分析: $O(n)$ 时间, $O(h)$ 空间。每个节点恰好访问一次; 区间验证确保了全局约束而非仅局部父子关系。

反例演示: 树 [5, 4, 6, null, null, 3, 7]——根 5、左子 4、右子 6 (根节点局部看合法), 但 6 的左子 3 < 5, 违反“右子树所有节点 > 5”; 局部比较会漏判, 区间验证在访问节点 3 时 $lo = 5$, $3 \leq 5$ 立即返回 False, 正确判断非法。

自测题

自测 1 (530 题 Minimum Absolute Difference)——BST [4, 2, 6, 1, 3], 返回最小绝对差 (答案为 1)。□ 提示: 中序递归, $prev = -inf$, 每次 $min_diff = \min(min_diff, node.val - prev)$, 再更新 $prev = node.val$; 递归结束返回 min_diff 。参考 `solutions/binary_search_tree/p530_minimum_absolute_difference_in_bst.py`。

自测 2 (230 题 Kth Smallest Element)——BST [3, 1, 4, null, 2], $k=1$ 返回 1, $k=3$ 返回 3。□ 提示: 迭代中序三步循环: “一路走左压栈 → 弹出访问计数 → 转向右子树”; $count == k$ 时立即 return 当前节点值。参考 `solutions/binary_search_tree/p230_kth_smallest_element_in_a_bst.py`。

自测 3 (98 题 Validate Binary Search Tree)——树 [5, 1, 4, null, null, 3, 6] 返回 False (右子树中有 $3 < 5$); 树 [2, 1, 3] 返回 True。□ 提示: 区间验证, 传 (lo , hi); 向左子树传 (lo , $node.val$), 向右子树传 ($node.val$, hi); 节点值必须满足 $lo < node.val < hi$ 。参考 `solutions/binary_search_tree/p098_validate_binary_search_tree.py`。

自测 4 (综合)——手动构造一棵 BST (插入序列 [5, 3, 7, 1, 4, 6, 8]), 验证其中序遍历是否严格递增, 并找出第 3 小的元素。□ 提示: 中序收集 `inorder_collect(root)` 得到 [1, 3, 4, 5, 6, 7, 8], 第 3 小为 `result[2] = 4`; 或用迭代中序计数到 3。

自测 5 (综合)——对树 [10, 5, 15, null, null, 6, 20] 调用 `isValidBST`: 局部来看 $15 > 10$ 合法, 但 15 的左子 $6 < 10$, 违反全局约束。手动追踪区间验证递归, 说明哪一步 lo/hi 导致判断为 False。□ 提示: 访问节点 6 时, 递归路径是 `root(10) → right(15) → left(6)`, 此时 $lo = 10$ (从 10 右转传来), $hi = 15$ (从 15 左转传来), $6 \leq 10$ 触发 `return False`。

题目全览（3 题）

#	题目	套路分类	难度
530	Minimum Absolute Difference in BST	中序递归 + prev 追踪	Easy
230	Kth Smallest Element in a BST	迭代中序 + 计数中断	Medium
98	Validate Binary Search Tree	区间验证递归	Medium

融合版说明

段	来源	价值
一例速记	本文件	3 题 2 类套路一览，含 AI/工程类比
思维路径还原	本文件	3 道题的解题内心独白，含核心陷阱提示
抽象成方法	本文件	5 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展（前驱追踪 / 迭代中断 / 区间验证 / AI 类比）
思考路标	本文件	11 条题型识别条件反射，含工程联系
易错点	本文件	7 条高频踩坑，覆盖局部比较陷阱、计数时机、空节点处理
典型应用例题	solutions/	3 道精讲（530、230、98），代码 + 正确性分析 + 反例演示
自测题	leetcode	5 题带 □ 提示，含手动追踪练习
题目全览	本文件	3 题完整列表，套路分类一览

跨 category 导航：- BST 搜索本质上是树形二分查找 → 二分查找理论基础见 04-binary-search.md
- BST 的遍历（前序 / 中序 / 后序 / 层序）→ 见 binary_tree_dfs / binary_tree_bfs category - 若需要
动态维护有序集合（插入 / 删除 / 查询第 k 小）→ 考虑平衡 BST（sortedcontainers.SortedList）
或堆（heap category）

06 — Binary Tree BFS (融合版)

难度: □□□□ 题数: 4 核心套路: deque 层序遍历、层级标记、之字形反向 本文件: 覆盖 binary_tree_bfs 4 题的算法套路总结 + 典型题精讲 + 自测 AI 关联: 图神经网络 (GNN) 消息传递、层级聚合 (Layer-wise Aggregation)

一例速记

标准 BFS (102 层序): deque + 每层快照, while deque, 先记 level_size = len(deque), 再弹 level_size 次, 每次把左右孩子入队 层级标记 (199 右视图): BFS 逐层, 取每层最后一个节点值即为右视图 层平均 (637): BFS 逐层, 层内求和 / 层节点数 之字形 (103 zigzag): BFS 逐层, 奇数层正向收集、偶数层反向 (或 appendleft)

思维路径还原

“看到 二叉树层序输出 → 立刻想到 BFS + deque: 根节点入队, 每次循环开始先记 level_size = len(q), 再循环 level_size 次弹节点、记录值、入队左右孩子。这样 deque 里始终存放‘当前层未处理节点 + 下一层已入队节点’, 每次 snapshot level_size 次就切出一层 (102)。”

看到 右视图 (199): 每层只取最后一个节点——直接在层级循环里判断 `i == level_size - 1` 时记录即可; 或者在弹节点时直接覆盖 `result`, 循环结束时 `result` 就是本层最后一个值。

看到 层平均 (637): 每层累计 `total`, 弹 level_size 次后 `total / level_size` 加入结果列表, 不需要额外空间。

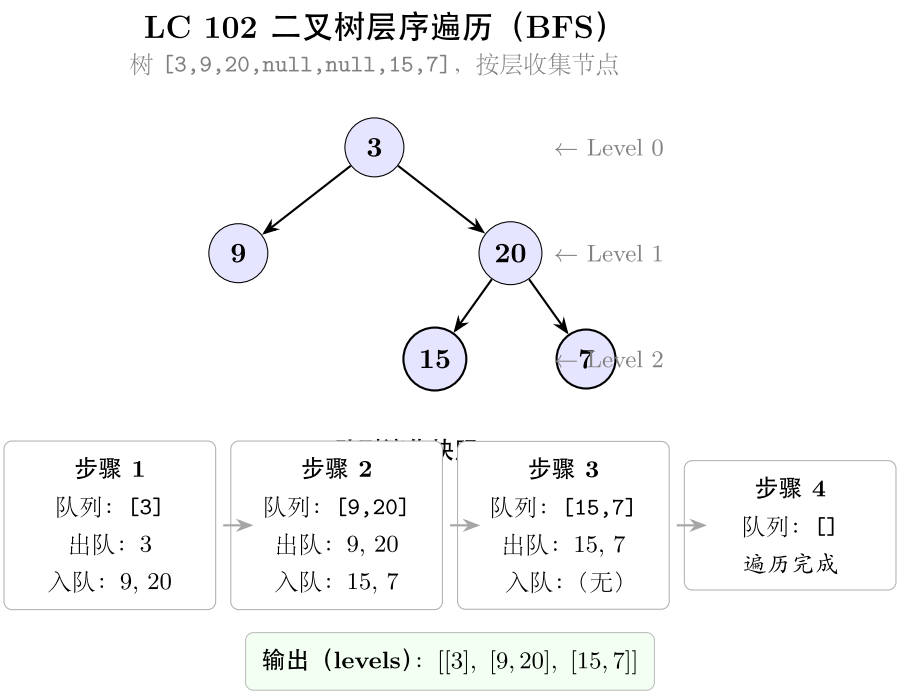
看到 之字形 (103): 奇偶层反向收集。最简洁: 用 Python deque 的 `appendleft`, 偶数层 (1-indexed) 把新节点从左插入当前层列表; 或者层结束后对奇数层 `level.reverse()`。另一种写法: 每层用 deque 收集, 奇数层从右弹 (正序), 偶数层从左弹 (反序)。”

学习目标

- 掌握 BFS + deque 的“层级快照”模板, 理解 level_size 的作用
 - 能从层序模板快速衍生出右视图、层平均、之字形等变体
 - 理解 BFS 与树深度、层数的关系: 第 k 层节点在 BFS 的第 k 轮弹出
 - 了解 GNN 层级聚合与树 BFS 的结构对应关系
-

几何示意

图层序遍历（LC 102）



“python

from

fu-

ture

im-

port

anno-

ta-

tions

from

collec-

tions

im-

port

deque

from

typing

im-

port

Op-

tional

```

class
TreeN-
ode:
def
init(self,
val:
int =
0, left:
Op-
tional[TreeNode]
=
None,
right:
Op-
tional[TreeNode]
=
None):
self.val
= val
self.left
= left
self.right
=
right

```

套路 1：标准 BFS 层序遍历

适用题：102（层序输出每层节点值列表）

```

def level_order(root: Optional[TreeNode]) -> list[list[int]]:
    """ 标准 BFS 层序，返回每层节点值的列表。时间  $O(n)$ ，空间  $O(n)$ 。 """
    if not root:
        return []
    result: list[list[int]] = []
    q: deque[TreeNode] = deque([root])
    while q:
        level_size = len(q)          # 本层节点数——快照关键
        level: list[int] = []
        for _ in range(level_size):

```

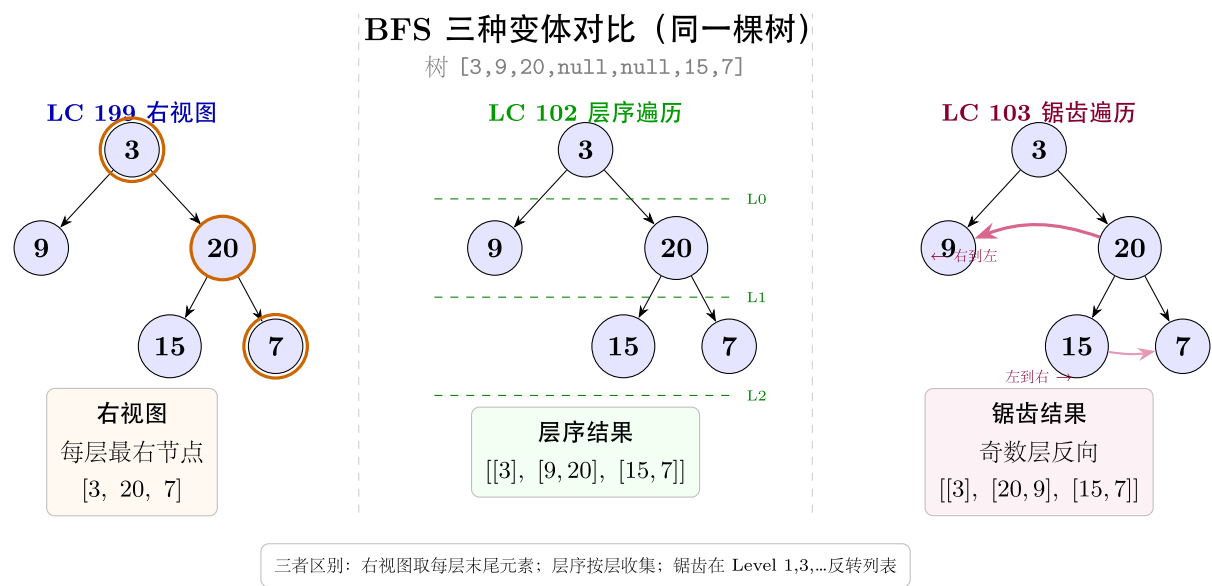


图 13: 三种 BFS 输出对比

```
node = q.popleft()
level.append(node.val)
if node.left:
    q.append(node.left)
if node.right:
    q.append(node.right)
result.append(level)
return result
```

关键：level_size = len(q) 在弹节点之前记录，确保本层只弹 level_size 次，下一层节点虽已入队但不在本轮处理范围内。

套路 2：层级标记——取层首 / 层尾节点

适用题：199（右视图，层尾）；变体：左视图（层首）

```
def right_side_view(root: Optional[TreeNode]) -> list[int]:
    """ 每层最后一个节点即右视图。时间 O(n)，空间 O(n)。 """
    if not root:
        return []
    result: list[int] = []
    q: deque[TreeNode] = deque([root])
    while q:
```

```

    level_size = len(q)
    for i in range(level_size):
        node = q.popleft()
        if i == level_size - 1:    # 本层最后一个
            result.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return result

```

```

def average_of_levels(root: Optional[TreeNode]) -> list[float]:
    """ 每层节点值求平均。时间  $O(n)$ , 空间  $O(n)$ 。 """
    if not root:
        return []
    result: list[float] = []
    q: deque[TreeNode] = deque([root])
    while q:
        level_size = len(q)
        total = 0.0
        for _ in range(level_size):
            node = q.popleft()
            total += node.val
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        result.append(total / level_size)
    return result

```

套路 3: 之字形 (Zigzag) 层序

适用题: 103 (奇数层正向, 偶数层反向, 1-indexed)

```

def zigzag_level_order(root: Optional[TreeNode]) -> list[list[int]]:
    """ 之字形层序: 奇数层从左到右, 偶数层从右到左。时间  $O(n)$ , 空间  $O(n)$ 。 """
    if not root:
        return []
    result: list[list[int]] = []

```

```
q: deque[TreeNode] = deque([root])
left_to_right = True # 第 1 层（根）正向
while q:
    level_size = len(q)
    level: list[int] = []
    for _ in range(level_size):
        node = q.popleft()
        level.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    if not left_to_right:
        level.reverse() # 偶数层反向
    result.append(level)
    left_to_right = not left_to_right
return result
```

变体：用 deque 替代 list 来收集 level，append vs appendleft 实现同效果，但 list.reverse() 更直观，不影响 O(n) 总复杂度。

速查表

题型特征	套路	时间	空间
树的层序遍历，每层一个列表	标准 BFS + level_size 快照	$O(n)$	$O(n)$
右视图 / 左视图（每层首 / 尾节点）	BFS + 层内下标判断	$O(n)$	$O(n)$
每层节点平均值	BFS + 层内累加 / level_size	$O(n)$	$O(n)$
之字形层序	BFS + 奇偶层 reverse	$O(n)$	$O(n)$
树的最大宽度（最宽层节点数）	BFS + 记录每层 level_size max	$O(n)$	$O(n)$

方法变形 (4 类)

变形 1: 右视图 / 左视图 / 层末值系列

- **199** (右视图, 层尾): $i == \text{level_size} - 1$ 时记录, 或弹节点时直接 `result[-1] = node.val` 再最终 `result.append` (后者每层只做一次 `append`)。
- 左视图 (非本 category, 但同构): $i == 0$ 时记录, 其余不变。
- 最深叶节点: BFS 遍历完毕时, 最后一个弹出的节点即最深 (或最深最右) 叶节点。

变形 2: 层统计系列

- **637** (层平均): 层内 `total += node.val`, 弹完后 `total / level_size`。
- 层最大值 / 最小值: 替换累加为 `max / min` 更新。
- 层节点数最大 (最宽层): 记录每层 `level_size`, 取最大值。
- 树的高度 (BFS 版): BFS 结束时循环了几轮 (维护计数器 `depth`), 最终 `depth` 即高度。

变形 3: 之字形扩展

- **103** (之字形, `reverse` 整层): 最简单, $O(n)$ 额外操作摊销。
- **deque** 双端插入: 用 `deque` 存 `level`, `left_to_right` 为真时 `level.append`, 否则 `level.appendleft`; 避免 `reverse`, 但代码略复杂。
- 层号直接控制下标: 预分配 `level = [0] * level_size`, 奇数层从 0 开始填, 偶数层从 `level_size-1` 开始填, 每次填一个位置—— $O(1)$ 每节点, 总 $O(n)$ 。

变形 4: BFS 遍历图 (跨 category)

- 树 BFS 是无环图 BFS 的特例: 把 `TreeNode` 换成图节点, 加入 `visited` 集合防止重复访问即可。
- GNN 层级聚合: 第 k 层神经元只聚合第 $k-1$ 层邻居的信息, 与树 BFS 每层只处理本层节点的思路完全对应。

思考路标 (条件反射)

1. 看到 “层序 / **level order** / 每层一个列表” → BFS + `deque`, `level_size = len(q)` 快照
2. 看到 “右视图 / **right side view**” → BFS + 每层取最后一个节点 ($i == \text{level_size} - 1$)
3. 看到 “层平均 / **average of levels**” → BFS + 层内求和再除以 `level_size`
4. 看到 “之字形 / **zigzag**” → BFS + `left_to_right` 标志位 + 偶数层 `reverse`
5. 看到 **BFS** 模板中忘写 `level_size` → 直接弹完整个 `deque`, 层级边界全部丢失
6. 看到 “树的高度 (BFS 做法)” → BFS 轮数即高度, `depth` 在每次 `while` 循环时 +1

7. 看到“树的最宽层 / 最大宽度” → BFS 记录每轮 `level_size`, 取 `max`
 8. 看到“最深叶节点 / 最右节点” → BFS 遍历到底, 最后弹出的即目标
 9. 看到“逐层处理 + 要知道当前是第几层” → 维护 `depth` 计数器, 与 `level_size` 配合
 10. 看到 **BFS** + 树 → 根节点入队时就 `append`, 不要先检查 `root.left != None` 才入队 (空节点检查在入队时做)
 11. 看到图 **BFS** (非树) → 加 `visited` 集合; 树 BFS 因为无环可省略
-

易错点

1. 忘记 `level_size` 快照: 若在弹节点的同时入队子节点, 直接用 `while q: for _ in range(len(q))` 会在每次 `for` 循环的 `range` 重新计算 `len(q)` (Python 的 `range` 在创建时求值, 此写法实际上是安全的, 但初学者常在其他语言中错误地直接循环 `q.size()`, 导致将下一层也弹入本层)。最安全的写法: 在 `while` 开头立即 `level_size = len(q)` 赋值。
 2. `popleft` 与 `pop` 混淆: BFS 要用 `deque.popleft()` (队首出队); 误用 `pop()` (栈顶出队) 会变成 DFS 先序遍历。
 3. **zigzag** 的奇偶层定义: LeetCode 103 要求根节点层 (第 1 层) 从左到右, 第 2 层从右到左。若 `left_to_right` 初始为 `True`, 第 1 层不 `reverse`, 第 2 层 `reverse` —— 确认初始值与 LeetCode 约定一致。
 4. 空树处理: `if not root: return []` 必须在第一行; 否则 `deque([root])` 会入队 `None`, 后续 `node.left` 会抛 `AttributeError`。
 5. **637** 整数除法: Python 3 中 `/` 自动返回 `float`, 无需 `float(total) / level_size`; 但如果用 `//` 则变成整数除法, 结果截断, 切勿混用。
-

典型应用例题

例 1: 102. Binary Tree Level Order Traversal

题目: 给定二叉树根节点 `root`, 返回其节点值的层序遍历 (每层为一个子列表)。

思路: 标准 BFS。根节点入队, 每轮开始时记录 `level_size = len(q)`, 弹 `level_size` 次形成本层快照, 左右孩子入队。

解:

```
# 参考: solutions/binary_tree_bfs/p102_binary_tree_level_order_traversal.py
from collections import deque

def levelOrder(root: Optional[TreeNode]) -> list[list[int]]:
    if not root:
        return []
```

```

result: list[list[int]] = []
q: deque[TreeNode] = deque([root])
while q:
    level_size = len(q)
    level: list[int] = []
    for _ in range(level_size):
        node = q.popleft()
        level.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    result.append(level)
return result

```

分析: $O(n)$ 时间 (每个节点入队一次、出队一次), $O(n)$ 空间 (最宽层至多 $n/2$ 个节点)。level_size 快照是核心——它在本层弹节点之前固定, 确保本轮循环只处理当前层。

例 2: 199. Binary Tree Right Side View

题目: 给定二叉树根节点, 从右侧看这棵树, 返回从上到下每层能看到的节点值 (即每层最右侧节点)。

思路: BFS 逐层, 每层只记录最后一个弹出的节点值。用 $i == \text{level_size} - 1$ 判断即可, 无需额外数据结构。

解:

参考: [solutions/binary_tree_bfs/p199_binary_tree_right_side_view.py](#)

```

from collections import deque

def rightSideView(root: Optional[TreeNode]) -> list[int]:
    if not root:
        return []
    result: list[int] = []
    q: deque[TreeNode] = deque([root])
    while q:
        level_size = len(q)
        for i in range(level_size):
            node = q.popleft()
            if i == level_size - 1:
                result.append(node.val)
            if node.left:

```

```

        q.append(node.left)
    if node.right:
        q.append(node.right)
    return result

```

分析: $O(n)$ 时间, $O(n)$ 空间。“右视图”的本质是每层最后弹出的节点——BFS 从左到右入队, 故层内最后弹出的就是最右节点。若想得到左视图, 把 `i == level_size - 1` 改为 `i == 0` 即可。

例 3: 103. Binary Tree Zigzag Level Order Traversal

题目: 给定二叉树, 返回节点值的之字形层序遍历: 第 1 层 (根) 从左到右, 第 2 层从右到左, 交替进行。

思路: BFS 逐层收集, 用 `left_to_right` 布尔标志控制当前层是否需要 `reverse`。每层结束后翻转标志。

解:

参考: [solutions/binary_tree_bfs/p103_binary_tree_zigzag_level_order_traversal.py](#)
`from collections import deque`

```

def zigzagLevelOrder(root: Optional[TreeNode]) -> list[list[int]]:
    if not root:
        return []
    result: list[list[int]] = []
    q: deque[TreeNode] = deque([root])
    left_to_right = True
    while q:
        level_size = len(q)
        level: list[int] = []
        for _ in range(level_size):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result

```

分析: $O(n)$ 时间 (每节点处理一次, `level.reverse()` 对每层摊销 $O(\text{level_size})$, 总和仍 $O(n)$), $O(n)$ 空间。

标志位 `left_to_right` 从 `True` 出发，根节点层正向排列符合题意。

自测题

自测 1(102 题 Binary Tree Level Order Traversal)——树 `[3,9,20,null,null,15,7]`，输出 `[[3],[9,20],[15,7]]`。
□ 提示：while 循环开头立即 `level_size = len(q)`，for 循环 `level_size` 次弹节点，完成后 `append` 整层列表。参考 `solutions/binary_tree_bfs/p102_binary_tree_level_order_traversal.py`。

自测 2（199 题 Binary Tree Right Side View）——树 `[1,2,3,null,5,null,4]`，输出 `[1,3,4]`。□ 提示：BFS 逐层，仅在 `i == level_size - 1` 时记录节点值，其余层内节点正常入队子节点即可。参考 `solutions/binary_tree_bfs/p199_binary_tree_right_side_view.py`。

自测 3（637 题 Average of Levels in Binary Tree）——树 `[3,9,20,15,7]`，输出 `[3.0, 14.5, 11.0]`。□ 提示：每层累加 `total`，弹完 `level_size` 个节点后除以 `level_size`，用 `/` 而非 `//` 保证返回 `float`。参考 `solutions/binary_tree_bfs/p637_average_of_levels_in_binary_tree.py`。

自测 4（103 题 Zigzag Level Order）——树 `[3,9,20,null,null,15,7]`，输出 `[[3],[20,9],[15,7]]`。□ 提示：`left_to_right` 初始为 `True`(根层正向)，收集完一层后若 `not left_to_right` 则 `level.reverse()`，最后翻转 `left_to_right`。参考 `solutions/binary_tree_bfs/p103_binary_tree_zigzag_level_order_traversal.py`。

自测 5（综合变体）——给定二叉树，求每层节点值的最大值（层最大），输出一个列表。□ 提示：把 637 题的 `total += node.val` 改为 `layer_max = max(layer_max, node.val)`，初始值设为 `float('-inf')`；最终 `append layer_max` 而非平均值。不需要单独的 `solutions` 文件，举一反三练习。

题目全览（4 题）

#	题目	套路分类	难度
102	Binary Tree Level Order Traversal	标准 BFS + <code>level_size</code> 快照	Medium
199	Binary Tree Right Side View	BFS + 层尾节点标记	Medium
637	Average of Levels in Binary Tree	BFS + 层内求和	Easy
103	Binary Tree Zigzag Level Order Traversal	BFS + 奇偶层 <code>reverse</code>	Medium

融合版说明

段	来源	价值
一例速记	本文件	4 题 3 类套路一览 + AI 关联，扫一眼知道要用什么
思维路径还原	本文件	从题目到代码的解题内心独白，模拟实战
抽象成方法	本文件	3 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展，覆盖层统计、视图、zigzag、图 BFS
思考路标	本文件	11 条题型识别条件反射，含跨 category 跳转
易错点	本文件	5 条高频踩坑，覆盖 level_size、popleft、奇偶层等
典型应用例题	solutions/	3 道精讲（102、199、103），代码 + 正确性分析
自测题	leetcode	5 题带 ☐ 提示，链接 solutions 文件
题目全览	本文件	4 题完整列表，套路分类一览

跨 category 导航：- 树的深度 / 路径问题 → 见 07-binary-tree-dfs.md（DFS 递归更自然）
- 图的 BFS 最短路 → 见 graph_bfs category（加 visited 集合）- BST 的中序遍历 → 见 05-binary-search-tree.md

07 —Binary Tree DFS（融合版）

难度：□□□□□ 题数：14 核心套路：自顶向下递归、后序整合、前/中/后序遍历、构造树、路径和、翻转/连接 本文件：覆盖 binary_tree_dfs 14 题的算法套路总结 + 典型题精讲 + 自测

一例速记

自顶向下（104 深度 / 100 same / 101 对称）：递归信息向下传，返回值向上汇总；框架：if not node: return base, left = dfs(node.left, ...), right = dfs(node.right, ...), 合并后序整合（124 max path / 222 count / 236 LCA）：先处理子树再使用结果；后序 = 左-右-根；全局变量 self.res 存跨根节点的答案 前中后序遍历（173 BST 迭代器 / 114 flatten）：迭代版用栈模拟；中序用于 BST 有序性；前序用于序列化 / 展开链表 构造树（105 前 + 中 / 106 中 + 后）：前序第一个

= 根；中序中根的位置切分左右子树；哈希表存中序下标，O(1) 查切分点 路径和 (112 / 129 / 124): DFS 参数携带”当前路径积累值”，叶节点处判断/更新答案 翻转/连接 (226 invert / 117 next right II): 后序翻转 or 层级迭代连接；117 用 BFS 更直观

思维路径还原

“看到 **104 最大深度**: $\text{maxDepth}(\text{root}) = 1 + \max(\text{maxDepth}(\text{left}), \text{maxDepth}(\text{right}))$ ，空节点返回 0。自顶向下框架：递归参数是当前节点，返回值是子问题答案（这里是深度），递归返回到父节点后取 $\text{max}+1$ ，自然汇总。

看到 **100 Same Tree**: 逐节点比较 $p.\text{val} == q.\text{val}$ 且两棵子树也相同；空节点需先判断：两个都空则 True，一个空一个非空则 False。

看到 **101 Symmetric Tree**: 转化为 $\text{isMirror}(\text{root}.\text{left}, \text{root}.\text{right})$ ；mirror 条件：两节点值相等，且左的左与右的右 mirror，左的右与右的左 mirror。

看到 **226 Invert Binary Tree**: 后序翻转——先翻左子树、翻右子树，再交换 $\text{root}.\text{left}, \text{root}.\text{right}$ ；或前序：先交换，再递归翻两侧（两种顺序均正确，后序更自然）。

看到 **124 Binary Tree Maximum Path Sum**: 路径可以不经过根，所以用全局变量 self.res 存最大值； $\text{dfs}(\text{node})$ 返回”过 node 且向下延伸的最大单侧贡献” $= \text{node}.\text{val} + \max(0, \text{dfs}(\text{left}), \text{dfs}(\text{right}))$ ；在 dfs 内部更新 $\text{self.res} = \max(\text{self.res}, \text{node}.\text{val} + \max(0, \text{left_gain}) + \max(0, \text{right_gain}))$ 。

看到 **236 LCA**: 若 root 是 p 或 q 直接返回 root；否则分别在左右子树找，两侧均非空则 root 是 LCA，一侧为空则返回非空那侧。

看到 **105 从前序+中序构造**: $\text{preorder}[0]$ 是根；在中序中找根的下标 idx ；左子树大小 $\text{left_size} = \text{idx} - \text{inorder_start}$ ；递归用 $\text{preorder}[1 : 1+\text{left_size}]$ 和 $\text{inorder}[:\text{idx}]$ 建左子树，其余建右子树。用哈希表 $\{\text{val}: \text{idx}\}$ 避免每次线性搜索，让总复杂度降至 $O(n)$ 。

看到 **173 BST Iterator**: 中序遍历的迭代版——初始化时把根一路向左压栈； $\text{next}()$ 弹栈顶（最小值），然后把其右子节点一路向左压栈； $\text{hasNext}()$ 检查栈是否非空。

看到 **114 Flatten Binary Tree to Linked List**: Morris 遍历 / 后序 / 前序迭代均可；最简洁：找右子树的前驱（左子树的最右节点），把右子树接在前驱后面，再把左子树移到右边，循环。

看到 **117 Populating Next Right Pointers II**: BFS 层序连接，或迭代用当前层的 next 指针遍历、建立下一层的链表；后者 $O(1)$ 额外空间。”

学习目标

- 掌握自顶向下 DFS 的递归框架（参数下传，返回值上汇）
- 理解后序整合模式：先拿子树结果，再在当前节点合并，适合路径和、LCA、计数
- 熟悉前/中/后序的迭代实现，尤其是 BST 迭代器（173）
- 掌握从两个遍历序列重建二叉树的切分思路（105、106）
- 识别”路径和”三题（112 / 129 / 124）的差异：是否需要到叶、路径是否穿根、全局 vs 参数
- 能用 Morris 遍历 O(1) 空间做中序，了解 114 的多种实现

几何示意

图三种 DFS 遍历对比

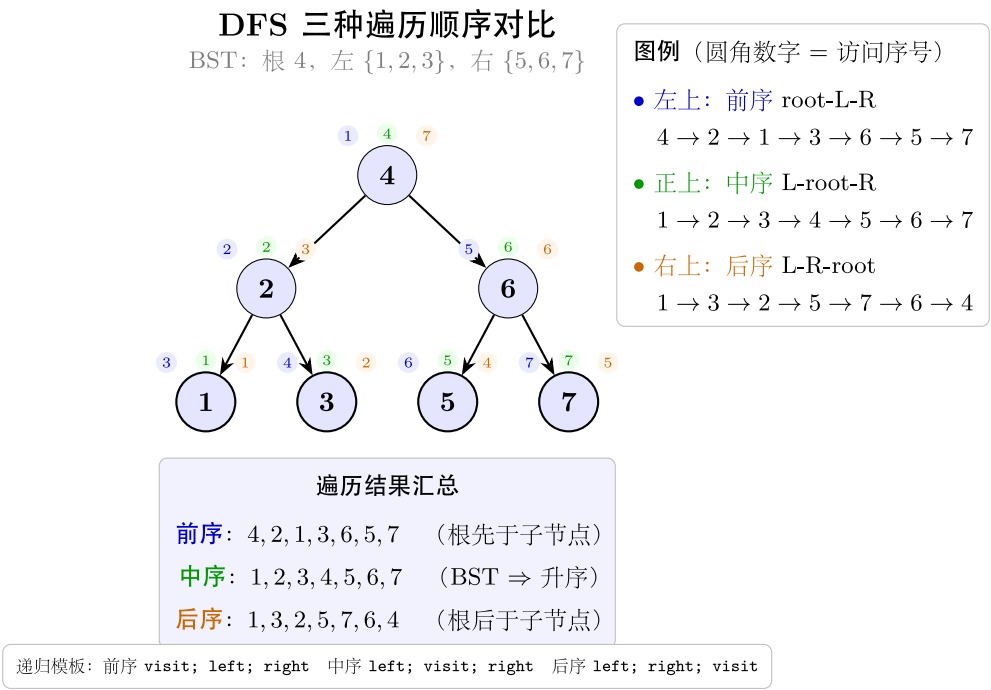


图 14: 前/中/后序访问顺序对比

图最大深度（LC 104）

图 LCA 最近公共祖先（LC 236）

LC 104 二叉树最大深度（DFS 后序）

递归: $\text{maxDepth}(\text{node}) = \max(\text{left}, \text{right}) + 1$, 叶子 = 1

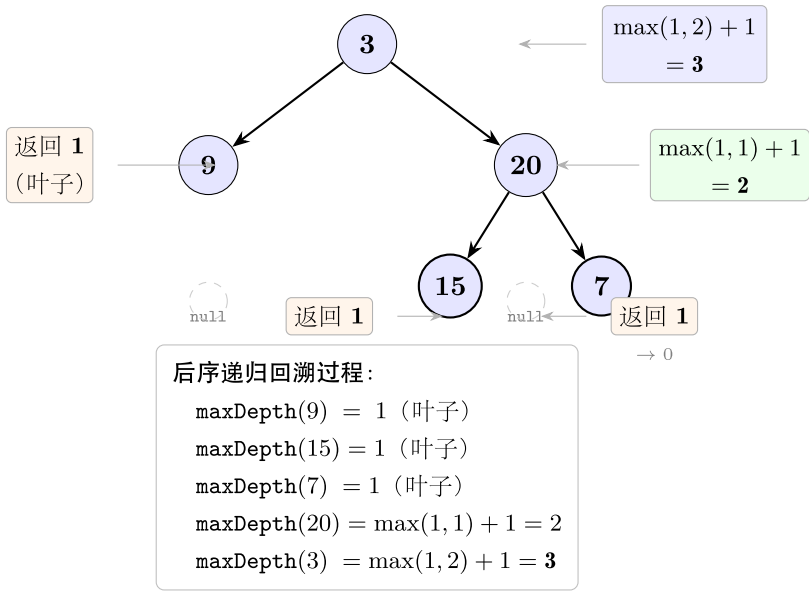


图 15: 叶子返回 1 + 内部 max+1

LC 236 最近公共祖先（LCA）

目标: $p = 5$ (红), $q = 1$ (红), $\text{LCA} = 3$ (绿框)

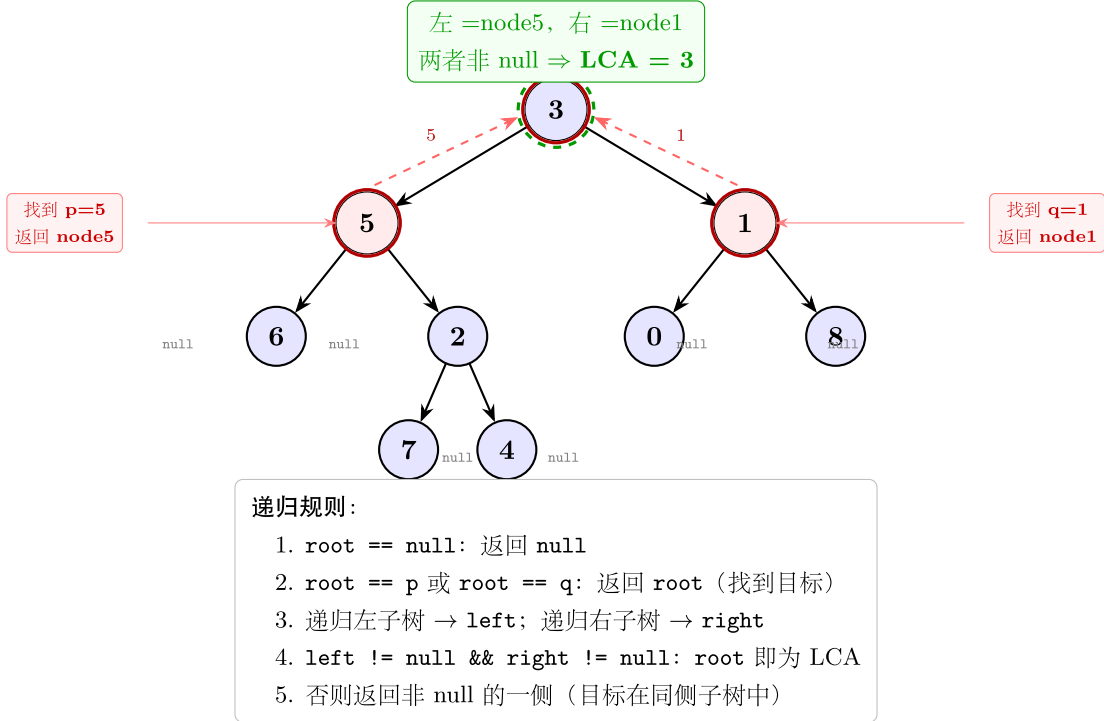


图 16: 递归从底向上找到 LCA

```
## 抽  
象成  
方法  
(标准  
模板  
代码)  
####  
TreeN-  
ode  
定义  
(所有  
二叉  
树题  
共用)  
“python  
from  
fu-  
ture  
im-  
port  
anno-  
ta-  
tions  
from  
typing  
im-  
port  
Op-  
tional
```

```

class
TreeN-
ode:
def
init(self,
val:
int =
0, left:
Op-
tional[TreeNode]
=
None,
right:
Op-
tional[TreeNode]
=
None):
self.val
= val
self.left
= left
self.right
=
right
““

```

套路 1：自顶向下递归

适用题：104（最大深度）、100（same tree）、101（symmetric）、226（invert）

104：最大深度

```

def maxDepth(root: Optional[TreeNode]) -> int:
    """ 时间  $O(n)$ ，空间  $O(h)$  ( $h$  为树高，最坏  $O(n)$ )。 """
    if root is None:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))

```

100：两棵树是否相同

```

def isSameTree(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:

```

```

    if p is None and q is None:
        return True
    if p is None or q is None:
        return False
    return p.val == q.val and isSameTree(p.left, q.left) and isSameTree(p.right, q.right)

```

101: 是否镜像对称 (转化为双指针同步 DFS)

```

def isSymmetric(root: Optional[TreeNode]) -> bool:
    def mirror(left: Optional[TreeNode], right: Optional[TreeNode]) -> bool:
        if left is None and right is None:
            return True
        if left is None or right is None:
            return False
        return (left.val == right.val
                and mirror(left.left, right.right)
                and mirror(left.right, right.left))
    return root is None or mirror(root.left, root.right)

```

226: 翻转二叉树 (后序)

```

def invertTree(root: Optional[TreeNode]) -> Optional[TreeNode]:
    if root is None:
        return None
    root.left = invertTree(root.left)
    root.right = invertTree(root.right)
    root.left, root.right = root.right, root.left
    return root

```

套路 2: 后序整合 (全局变量 + 单侧贡献)

适用题: 124 (max path sum)、222 (count complete nodes)、236 (LCA)

124: 二叉树最大路径和 (路径可不经根)

```

class MaxPathSum:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        self.res = float('-inf')

        def dfs(node: Optional[TreeNode]) -> int:
            """ 返回过 node 向下延伸的最大单侧贡献 (负则舍弃用 0)。"""

```

```

        if node is None:
            return 0
        left_gain = max(0, dfs(node.left))
        right_gain = max(0, dfs(node.right))
        # 更新全局最大 (路径穿过当前节点)
        self.res = max(self.res, node.val + left_gain + right_gain)
        # 向上只能选一侧
        return node.val + max(left_gain, right_gain)

    dfs(root)
    return self.res

```

236: 最近公共祖先

```

def lowestCommonAncestor(root: TreeNode,
                          p: TreeNode, q: TreeNode) -> TreeNode:
    """ 时间  $O(n)$ , 空间  $O(h)$ 。 """
    if root is None or root is p or root is q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:          #  $p$ 、 $q$  分属两侧,  $root$  是 LCA
        return root
    return left if left else right

```

222: 完全二叉树节点数 (利用完全二叉树性质, $O(\log_2 n)$)

```

def countNodes(root: Optional[TreeNode]) -> int:
    if root is None:
        return 0
    left_h, right_h = 0, 0
    l, r = root, root
    while l:
        left_h += 1
        l = l.left
    while r:
        right_h += 1
        r = r.right
    if left_h == right_h:          # 满二叉树
        return (1 << left_h) - 1  #  $2^h - 1$ 
    return 1 + countNodes(root.left) + countNodes(root.right)

```

套路 3: 路径和系列 (参数携带当前积累值)

适用题: 112 (路径和到叶)、129 (根到叶数字和)

112: 是否存在根到叶路径和等于 targetSum

```
def hasPathSum(root: Optional[TreeNode], targetSum: int) -> bool:
    if root is None:
        return False
    if root.left is None and root.right is None:    # 叶节点
        return root.val == targetSum
    remain = targetSum - root.val
    return hasPathSum(root.left, remain) or hasPathSum(root.right, remain)
```

129: 根到叶路径表示的数字之和 (如路径 1→2→3 表示数字 123)

```
def sumNumbers(root: Optional[TreeNode]) -> int:
    def dfs(node: Optional[TreeNode], cur: int) -> int:
        if node is None:
            return 0
        cur = cur * 10 + node.val
        if node.left is None and node.right is None:    # 叶节点
            return cur
        return dfs(node.left, cur) + dfs(node.right, cur)

    return dfs(root, 0)
```

套路 4: 前/中/后序遍历 (迭代版)

适用题: 173 (BST 迭代器, 迭代中序)、114 (flatten, 前序链表展开)

迭代中序遍历 (BST 迭代器核心)

```
def inorder_iterative(root: Optional[TreeNode]) -> list[int]:
    result: list[int] = []
    stack: list[TreeNode] = []
    curr = root
    while curr or stack:
        while curr:    # 一路向左压栈
            stack.append(curr)
```

```

        curr = curr.left
    curr = stack.pop()      # 弹出最左节点 (最小值)
    result.append(curr.val)
    curr = curr.right      # 转向右子树
return result

```

173: BST 迭代器 (懒加载中序)

```

class BSTIterator:
    def __init__(self, root: Optional[TreeNode]):
        self.stack: list[TreeNode] = []
        self._push_left(root)

    def _push_left(self, node: Optional[TreeNode]) -> None:
        while node:
            self.stack.append(node)
            node = node.left

    def next(self) -> int:
        node = self.stack.pop()
        self._push_left(node.right)  # 右子树的最左路径入栈
        return node.val

    def hasNext(self) -> bool:
        return bool(self.stack)

```

114: 展开为链表 (原地, 前序)

```

def flatten(root: Optional[TreeNode]) -> None:
    """ 将二叉树原地展开为前序遍历的 " 链表 " (right 指针串联, left 全为 None)。"""
    curr = root
    while curr:
        if curr.left:
            # 找左子树最右节点 (右子树的前驱)
            prev = curr.left
            while prev.right:
                prev = prev.right
            prev.right = curr.right  # 右子树接在前驱后
            curr.right = curr.left  # 左子树移到右边
            curr.left = None
        curr = curr.right

```

套路 5: 从遍历序列构造树

适用题: 105 (前序 + 中序)、106 (中序 + 后序)

105: 前序 + 中序 → 二叉树

```
def buildTree_pre_in(preorder: list[int], inorder: list[int]) -> Optional[TreeNode]:
    idx_map = {val: i for i, val in enumerate(inorder)} # 中序下标哈希

    def build(pre_l: int, pre_r: int, in_l: int, in_r: int) -> Optional[TreeNode]:
        if pre_l > pre_r:
            return None
        root_val = preorder[pre_l]
        root = TreeNode(root_val)
        mid = idx_map[root_val] # 根在中序中的位置
        left_size = mid - in_l # 左子树节点数
        root.left = build(pre_l + 1, pre_l + left_size, in_l, mid - 1)
        root.right = build(pre_l + left_size + 1, pre_r, mid + 1, in_r)
        return root

    n = len(preorder)
    return build(0, n - 1, 0, n - 1)
```

106: 中序 + 后序 → 二叉树

```
def buildTree_in_post(inorder: list[int], postorder: list[int]) -> Optional[TreeNode]:
    idx_map = {val: i for i, val in enumerate(inorder)}

    def build(post_l: int, post_r: int, in_l: int, in_r: int) -> Optional[TreeNode]:
        if post_l > post_r:
            return None
        root_val = postorder[post_r] # 后序最后一个为根
        root = TreeNode(root_val)
        mid = idx_map[root_val]
        left_size = mid - in_l
        root.left = build(post_l, post_l + left_size - 1, in_l, mid - 1)
        root.right = build(post_l + left_size, post_r - 1, mid + 1, in_r)
        return root

    n = len(inorder)
    return build(0, n - 1, 0, n - 1)
```


套路 6：层级连接 (next right pointer)

适用题：117 (Populating Next Right Pointers II, 任意二叉树)

```
# 117: 填充每个节点的下一个右侧节点指针 (非完美二叉树)
# Node 类含 val, left, right, next 字段
def connect(root) -> None:
    """O(1) 额外空间迭代法：利用已建立的 next 链遍历每层。"""
    curr = root # 当前层的某个节点 (从左到右遍历)
    while curr:
        dummy = object.__new__(object.__class__ if hasattr(object, '__class__') else type(curr))
        # 用哑节点简化下一层链表构建
        prev = type('_', (), {'next': None})() # dummy 哑节点
        head = prev
        node = curr
        while node:
            if node.left:
                prev.next = node.left
                prev = prev.next
            if node.right:
                prev.next = node.right
                prev = prev.next
            node = node.next # 通过当前层 next 链移动
        curr = head.next # 下一层从哑节点的 next 开始
```

注意：上方为示意，实际实现需要 Node 类定义。简洁版见下方精讲代码。

速查表

题型特征	套路	时间	空间
树的深度 / 节点比较 / 对称判断	自顶向下递归，返回 bool/int	$O(n)$	$O(h)$
最大路径和 (路径可不过根)	后序 + 全局变量 self.res	$O(n)$	$O(h)$
最近公共祖先	后序 + 两侧返回值合并	$O(n)$	$O(h)$
完全二叉树节点数	左右高度比较 + 二分	$O(\log^2 n)$	$O(\log n)$

题型特征	套路	时间	空间
根到叶路径和 / 数字	参数携带当前积累值，叶节点判断	$O(n)$	$O(h)$
前序 + 中序构造树	哈希表切分 + 区间递归	$O(n)$	$O(n)$
中序 + 后序构造树	后序末尾取根 + 哈希表切分	$O(n)$	$O(n)$
BST 迭代器（中序懒加载）	栈 + 一路向左压栈	$O(1)$ 均摊	$O(h)$
展开为链表（flatten）	迭代找前驱 + 原地改指针	$O(n)$	$O(1)$
翻转二叉树	后序递归，翻转子树后交换	$O(n)$	$O(h)$
填充 next 右侧指针	利用 next 链迭代建下一层	$O(n)$	$O(1)$

方法变形（4 类）

变形 1：自顶向下递归系列

- **104**（最大深度）→ **111**（最小深度，非本 category）：最小深度需注意单侧子树为空时不能直接取 min，否则会把空子树的深度 0 算进去；需判断左右孩子的空否。
- **100**（两棵树相同）→ **101**（对称）：same tree 比较 p.left, q.left 和 p.right, q.right；symmetric 比较 left.left, right.right 和 left.right, right.left——镜像对称将同侧改为交叉侧。
- **112**（路径和，到叶）→ **113**（路径和，所有路径，非本 category）→ **129**（路径数字和）：三题均用参数下传积累值，区别在于叶节点的判断逻辑（是否收集整条路径）。
- **226**（翻转）：后序翻转与前序翻转效果相同，后序更符合”先处理子树”的思维定式。

变形 2：后序整合系列

- **124**(max path sum): dfs 返回”单侧最大贡献”，全局变量记录”穿根路径最大值”。负贡献 max(0, dfs(...)) 截断，相当于”不走这侧”。
- **236** (LCA)：后序遍历中，若两侧返回值均非空则当前节点是 LCA；若只有一侧非空则答案在那侧。这个模式也可用于”计算子树中满足条件的节点数”。
- **222** (count complete nodes)：普通二叉树 $O(n)$ 遍历；完全二叉树利用左高 == 右高判断是否满二叉树，将 $O(n)$ 优化到 $O(\log^2 n)$ 。

变形 3: 遍历顺序的选择

- 前序 (根-左-右): 适合序列化、路径记录、114 展开链表 (按前序顺序链接)。
- 中序 (左-根-右): BST 专属, 产生升序序列 (173 迭代器就是懒加载中序)。
- 后序 (左-右-根): 适合“先拿子树结果、再在当前节点合并”的场景 (124、236); 也用于 114 展开 (Morris)。
- 迭代 vs 递归: 递归简洁但栈深度受树高限制; 迭代用显式栈, 适合超深树或需要中途暂停 (173 迭代器)。

变形 4: 构造树系列

- **105** (前序 + 中序) → **106** (中序 + 后序): 结构对称——前序取首元素为根, 后序取尾元素为根; 切分点均在中序中查找。两题核心代码几乎相同, 仅“取根”和“递归区间”的偏移方向不同。
- 哈希表加速: 不用哈希表时, `inorder.index(root_val)` 是 $O(n)$, 总复杂度 $O(n^2)$; 用哈希表预处理变为 $O(1)$ 查找, 总 $O(n)$ 。

思考路标 (条件反射)

1. 看到“树的深度 / 最大 / 最小深度” → 自顶向下, $1 + \max(\text{dfs}(\text{left}), \text{dfs}(\text{right}))$, 空节点返回 0
2. 看到“两棵树是否相同” → 同步 DFS 双指针, 先判空, 再比 val, 再递归左右
3. 看到“树是否对称” → 转化为 `mirror(root.left, root.right)`, 交叉比较 (左左 vs 右右, 左右 vs 右左)
4. 看到“翻转二叉树” → 后序: 先翻左右子树, 再 `root.left, root.right = root.right, root.left`
5. 看到“路径和 + 必须到叶节点” → 递归传入剩余 `targetSum`, 叶节点处判断是否等于 `node.val`
6. 看到“根到叶路径表示的数字” (129) → 递归传入当前数字 `cur = cur * 10 + node.val`, 叶节点返回 `cur`
7. 看到“最大路径和 (路径可任意)” → 后序 + 全局变量; `dfs` 返回单侧贡献, 负贡献截 0; 全局记录穿根路径最大值
8. 看到“最近公共祖先” → 后序; 若节点是 p 或 q 则返回自身; 两侧均非空则当前节点是 LCA
9. 看到“前序 + 中序构造树” → 前序首元素是根; 中序中根的位置切分左右; 哈希表加速查找
10. 看到“中序 + 后序构造树” → 后序末尾是根; 其余与 105 对称
11. 看到“BST 中序迭代器” → 栈 + 初始化一路向左压栈; `next()` 弹栈顶后把右子树一路向左压栈
12. 看到“展开为链表 (前序)” → 迭代找左子树最右节点 (前驱), 把右子树接到前驱后, 左子树移右
13. 看到“填充 next 指针” → BFS 层序 (加 `visited` 空间) 或利用已建 next 链 $O(1)$ 空间迭代
14. 看到“完全二叉树节点数” → 比较左高与右高: 相等则左子树满 ($2^{\text{left}_h} - 1$) + 递归右; 否则右子树满 + 递归左

易错点

1. 空节点的判断顺序：在 `isSameTree`、`isSymmetric` 等中，必须先判断两节点都空 (True)、一空一非空 (False)，再访问 `p.val`；若顺序颠倒会对 `None.val` 抛 `AttributeError`。
2. 124 全局变量初始化：`self.res = float('-inf')` 而非 0——当所有节点均为负数时，最大路径和是某个负数，初始为 0 会导致返回 0 (错误)。
3. LCA 的 `is vs ==`：236 要判断节点对象是否为 `p` 或 `q`，用 `root is p or root is q` (身份判断)，而非 `root.val == p.val` (值判断，若有重复值会出错)。
4. 构造树的区间偏移：105 题中 `left_size = mid - in_l` (不是 `mid`)，递归右子树的前序区间起点是 `pre_l + left_size + 1` (不是 `pre_l + mid + 1`)——偏移量基于左子树大小，与绝对中序下标无关。
5. 114 `flatten` 的左子树转移：必须先把右子树接到左子树最右节点后，再把 `curr.right = curr.left`，最后 `curr.left = None`；若先 `curr.left = None` 再找前驱，左子树已丢失。
6. 173 `BST` 迭代器的 `next()` 与 `hasNext()`：`next()` 在弹出节点后必须把该节点右子树的最左路径压栈，否则右子树的节点永远不会被访问。
7. 222 完全二叉树的高度计算：左高向左走 (`l = l.left`)，右高向右走 (`r = r.right`)；判断是否满二叉树时两高相等才能用 $2^h - 1$ ，不要混淆走左还是走右。
8. 112 vs 129 叶节点判断：两题都要检测叶节点 (`left is None and right is None`)；若漏掉这个判断，中间节点的剩余值恰好为 0 时会提前返回 `True` (112) 或错误累加 (129)。

典型应用例题

例 1：104. Maximum Depth of Binary Tree

题目：给定二叉树，返回其最大深度（从根到最远叶节点的路径上的节点数）。

思路：自顶向下递归。`maxDepth(root) = 1 + max(maxDepth(left), maxDepth(right))`，空节点返回 0。代码一行，直接映射定义。

解：

```
# 参考: solutions/binary_tree_dfs/p104_maximum_depth_of_binary_tree.py
def maxDepth(root: Optional[TreeNode]) -> int:
    if root is None:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

分析： $O(n)$ 时间（每个节点访问一次）， $O(h)$ 空间（递归栈深度 = 树高，最坏退化为链表时 $O(n)$ ）。这是最简洁的自顶向下模板——没有额外变量，递归框架与问题定义直接对应。

BFS 迭代版（备用）：BFS 统计轮数即深度，若想规避递归栈溢出可改写为 BFS，但递归版更简洁。

例 2: 124. Binary Tree Maximum Path Sum

题目：二叉树中每个节点都有一个整数值（可为负）。路径是从某节点出发经过若干边到达另一节点的序列，每个节点至多出现一次。返回所有路径中节点值之和的最大值。

思路：后序 DFS + 全局变量。关键观察：路径可以不经过根节点，因此需要在 DFS 中途更新全局最大值。

定义 $\text{dfs}(\text{node})$ = “从 node 出发、仅向下延伸的单侧最大贡献”。由于负贡献不如不选，取 $\max(0, \text{dfs}(\text{child}))$ 。在每个节点处，穿过该节点的路径最大值 = $\text{node.val} + \max(0, \text{dfs}(\text{left})) + \max(0, \text{dfs}(\text{right}))$ ，用它更新全局 self.res 。但向父节点返回时只能选一侧，因此返回 $\text{node.val} + \max(\text{left_gain}, \text{right_gain})$ 。

解：

参考: [solutions/binary_tree_dfs/p124_binary_tree_maximum_path_sum.py](#)

class Solution:

```
def maxPathSum(self, root: Optional[TreeNode]) -> int:
    self.res = float('-inf')

    def dfs(node: Optional[TreeNode]) -> int:
        if node is None:
            return 0
        left_gain = max(0, dfs(node.left))
        right_gain = max(0, dfs(node.right))
        # 穿过 node 的路径（可能是最终答案）
        self.res = max(self.res, node.val + left_gain + right_gain)
        # 向上只能贡献一侧
        return node.val + max(left_gain, right_gain)

    dfs(root)
    return self.res
```

分析： $O(n)$ 时间， $O(h)$ 空间。两个关键设计：1. $\max(0, \dots)$ 截断负贡献——相当于“不经过这条边”；2. self.res 在内部更新（两侧相加），但 dfs 返回时只给出单侧（防止路径“折返”经过同一节点两次）。

例 3: 105. Construct Binary Tree from Preorder and Inorder Traversal

题目：给定前序遍历 preorder 和中序遍历 inorder，重建二叉树（节点值唯一）。

思路：前序首元素是当前子树的根。在中序数组中找到该根的位置 mid，则 $\text{inorder}[:\text{mid}]$ 是左子树， $\text{inorder}[\text{mid}+1:]$ 是右子树；对应到前序数组，左子树有 $\text{left_size} = \text{mid} - \text{in_l}$ 个节点，据此切分前序区间递归。用哈希表预存中序下标，使查找 $O(1)$ 。

解：

```
# 参考: solutions/binary_tree_dfs/p105_construct_binary_tree_from_preorder_and_inorder_traversal.py
def buildTree(preorder: list[int], inorder: list[int]) -> Optional[TreeNode]:
    idx_map = {val: i for i, val in enumerate(inorder)}

    def build(pre_l: int, pre_r: int, in_l: int, in_r: int) -> Optional[TreeNode]:
        if pre_l > pre_r:
            return None
        root_val = preorder[pre_l]
        root = TreeNode(root_val)
        mid = idx_map[root_val]
        left_size = mid - in_l
        root.left = build(pre_l + 1, pre_l + left_size,
                          in_l, mid - 1)
        root.right = build(pre_l + left_size + 1, pre_r,
                           mid + 1, in_r)

        return root

    n = len(preorder)
    return build(0, n - 1, 0, n - 1)
```

分析: $O(n)$ 时间 (哈希表 $O(1)$ 查找, 每节点创建一次), $O(n)$ 空间 (哈希表+递归栈)。递归区间的偏移关系: - 左子树前序: $[pre_l+1, pre_l+left_size]$ (跳过根, 取 $left_size$ 个) - 右子树前序: $[pre_l+left_size+1, pre_r]$ (左子树后的剩余部分) - 偏移量 $left_size = mid - in_l$ 基于中序切分位置与区间左端之差, 与绝对下标无关。

106 题对称版: 后序末尾是根 ($postorder[post_r]$), 左子树后序 $[post_l, post_l+left_size-1]$, 右子树后序 $[post_l+left_size, post_r-1]$, 其余逻辑完全相同。

自测题

自测 1 (100 题 Same Tree) —— 两棵树 $p=[1,2,3]$ 和 $q=[1,2,3]$ 返回 True, $p=[1,2]$ 和 $q=[1,null,2]$ 返回 False。□ 提示: 先判断 p is None and q is None (True), 再判断 p is None or q is None (False), 最后比较 $p.val == q.val$ 并递归两侧。参考 solutions/binary_tree_dfs/p100_same_tree.py。

自测 2 (101 题 Symmetric Tree) —— 树 $[1,2,2,3,4,4,3]$ 返回 True, $[1,2,2,null,3,null,3]$ 返回 False。□ 提示: 转化为 $mirror(root.left, root.right)$, 比较方式是“左的左 vs 右的右”+“左的右 vs 右的左” (交叉比较, 非同侧)。参考 solutions/binary_tree_dfs/p101_symmetric_tree.py。

自测 3 (112 题 Path Sum) —— 树 $[5,4,8,11,null,13,4,7,2,null,null,null,1]$, $targetSum=22$, 存在路径 $5 \rightarrow 4 \rightarrow 11 \rightarrow 2$ 返回 True。□ 提示: $hasPathSum$ 每层减去 $root.val$, 叶节点处判断 $root.val == targetSum$ (即剩余 0); 非叶节点时递归左右子树取 or。参考 solutions/binary_tree_dfs/p112_path_sum.py。

自测 4(236 题 Lowest Common Ancestor)——树 [3,5,1,6,2,0,8,null,null,7,4], p=5,q=1, 答案是节点 3。□
提示:后序;若 root is p or root is q 直接返回 root;否则在左右子树找,两侧均非空则 root 是 LCA,一侧为
空则返回非空侧。参考 solutions/binary_tree_dfs/p236_lowest_common_ancestor_of_a_binary_tree.py。

自测 5(173 题 BST Iterator)——树 [7,3,15,null,null,9,20], 依次 next() 返回 3,7,9,15,20; hasNext() 在最后
一次 next() 后返回 False。□ 提示:初始化时从根一路向左压栈;next() 弹栈顶节点,再把其右子节点一路向左压
栈;hasNext() 检查栈非空即可。参考 solutions/binary_tree_dfs/p173_binary_search_tree_iterator.py。

题目全览（14 题）

#	题目	套路分类	难度
104	Maximum Depth of Binary Tree	自顶向下递归	Easy
100	Same Tree	自顶向下递归（双树同步）	Easy
226	Invert Binary Tree	后序翻转	Easy
101	Symmetric Tree	自顶向下递归（镜像比较）	Easy
112	Path Sum	参数携带剩余值，叶节点判断	Easy
129	Sum Root to Leaf Numbers	参数携带当前数字	Medium
114	Flatten Binary Tree to Linked List	迭代找前驱，原地改指针	Medium
105	Construct Binary Tree from Preorder and Inorder	哈希表 + 区间递归	Medium
106	Construct Binary Tree from Inorder and Postorder	哈希表 + 区间递归（对称）	Medium
117	Populating Next Right Pointers in Each Node II	利用 next 链 O(1) 空间迭代	Medium
173	Binary Search Tree Iterator	迭代中序（懒加载栈）	Medium
222	Count Complete Tree Nodes	左右高比较 + 递归 ($O(\log^2 n)$)	Medium
124	Binary Tree Maximum Path Sum	后序 + 全局变量 + 单侧贡献	Hard
236	Lowest Common Ancestor of a Binary Tree	后序 + 两侧返回值合并	Medium

融合版说明

段	来源	价值
一例速记	本文件	14 题 6 类套路一览，扫一眼知道要做什么
思维路径还原	本文件	9 道题的解题内心独白，含关键决策点
抽象成方法	本文件	8 个标准模板代码 + 速查表，可直接运行
方法变形	本文件	4 类变体扩展，覆盖递归系列、后序整合、遍历顺序、构造树
思考路标	本文件	14 条题型识别条件反射，覆盖全部 14 题
易错点	本文件	8 条高频踩坑，含空节点判断、全局变量初始化、区间偏移等
典型应用例题	solutions/	3 道精讲（104、124、105），代码 + 正确性分析
自测题	leetcode	5 题带 □ 提示，链接 solutions 文件
题目全览	本文件	14 题完整列表，套路分类一览

跨 category 导航：- 树的层序遍历、右视图、之字形 → 见 06-binary-tree-bfs.md - BST 专属操作（插入 / 删除 / 验证有效性）→ 见 05-binary-search-tree.md - 图的 DFS（连通分量、拓扑排序）→ 见 graph_general category - 路径和类 DP（非树结构）→ 见 dynamic_programming_1d / dynamic_programming_multidimensional

08 —Backtracking（融合版）

难度：□□□□□ 题数：7 核心套路：选择 → 递归 → 撤销（三件套）、组合、排列、约束满足 本文件：覆盖 backtracking 7 题的算法套路总结 + 典型题精讲 + 自测

一例速记

回溯三件套:for choice in choices: path.append(choice) → recurse(...) → path.pop(); 对应”选择 → 递归 → 撤销”三步，共享同一 path 列表 组合 (77 / 39): 从 start 开始枚举，避免重复；77 选 k 个不重复数，39 可重复选但剪 sum 排列 (46): 全排列用 used[] 标记已选元素，元素不固定起点 字母组合 (17): 电话号码每位独立选一个字母，DFS 深度 = 号码长度，宽度 = 按键字

母数 **N-Queens (52)**: 用列 / 正对角线 / 反对角线三个集合做冲突检测, 不需要检查每行 (一行放一个) 生成括号 **(22)**: 参数携带 open / close 计数, 约束: $\text{open} \leq n$, $\text{close} \leq \text{open}$ 网格搜索 **(79)**: 在二维格子上 DFS, 用“原地修改标记 + 回退”替代 visited 数组 **AI** 关联: 约束满足问题 (CSP) / 搜索剪枝 / 超参组合网格搜索 / 神经架构搜索 (NAS)

思维路径还原

“看到 **77 Combinations**: 从 $[1..n]$ 中取 k 个数的所有组合 $\rightarrow$ 回溯, start 指针保证不重复, path 长度到 k 时收集结果。剪枝: 若剩余数字数 $n - \text{start} + 1 < k - \text{len}(\text{path})$, 即使全选也凑不够, 直接 return。

看到 **39 Combination Sum**: 给定 candidates (无重复), 找所有和为 target 的组合, 可重复选 $\rightarrow$ 回溯时不推进 start (允许重复选同一元素), 但用 $\text{target} - \text{candidates}[i]$ 剪负值分支。排序后若 $\text{candidates}[i] > \text{target}$ 则后续全剪。

看到 **46 Permutations**: 给定无重复整数, 求所有全排列 $\rightarrow$ 每层从全集中选一个未用的元素, 用 used 布尔数组标记, path 长度等于 n 时收集。不需要 start——排列关心顺序, 每次从 0 开始扫。

看到 **17 Letter Combinations**: 电话号码 $\rightarrow$ 字母组合 $\rightarrow$ DFS 深度 = 号码长度, 第 idx 层选 $\text{digit_map}[\text{digits}[\text{idx}]]$ 中的一个字母; 到底时收集。注意空输入时直接返回 []。

看到 **52 N-Queens II**: $n \times n$ 棋盘放 n 个不攻击的皇后, 返回方案数 $\rightarrow$ 按行放置, 每行恰好放一个; 用集合 cols、diag1 (行-列)、diag2 (行+列) 三个集合检测冲突; 到第 n 行时计数 +1。三个集合加/删对应选择/撤销。

看到 **22 Generate Parentheses**: n 对括号的所有合法组合 $\rightarrow$ 递归参数 open 和 close: $\text{open} < n$ 时可加左括号, $\text{close} < \text{open}$ 时可加右括号; $\text{open} == \text{close} == n$ 时收集结果。约束代替回溯撤销——直接传参不修改全局状态更简洁。

看到 **79 Word Search**: 在字符网格中找目标单词 $\rightarrow$ 对每个格子尝试作为起点, DFS 四方向匹配单词; 原地标记已访问 ($\text{board}[r][c] = \text{'\#'}'$), 递归结束后恢复 ($\text{board}[r][c] = \text{tmp}$); 匹配完整单词返回 True 立即终止。”

学习目标

- 掌握回溯“选择 $\rightarrow$ 递归 $\rightarrow$ 撤销”三件套, 能徒手写出标准框架
- 区分组合 (有 start 去重) 与排列 (无 start、有 used 标记) 的写法差异
- 理解剪枝的两类手段: 约束剪 (sum 超 target、凑不够 k 个) 和状态剪 (visited / 集合冲突)
- 掌握 N-Queens 的三集合冲突检测方案, 避免逐行逐格扫描
- 理解网格回溯中“原地标记”的技巧, 与 visited 数组的等价性

- 识别生成括号问题”参数约束”替代”撤销”的简洁写法

几何示意

图排列回溯决策树（LC 46）

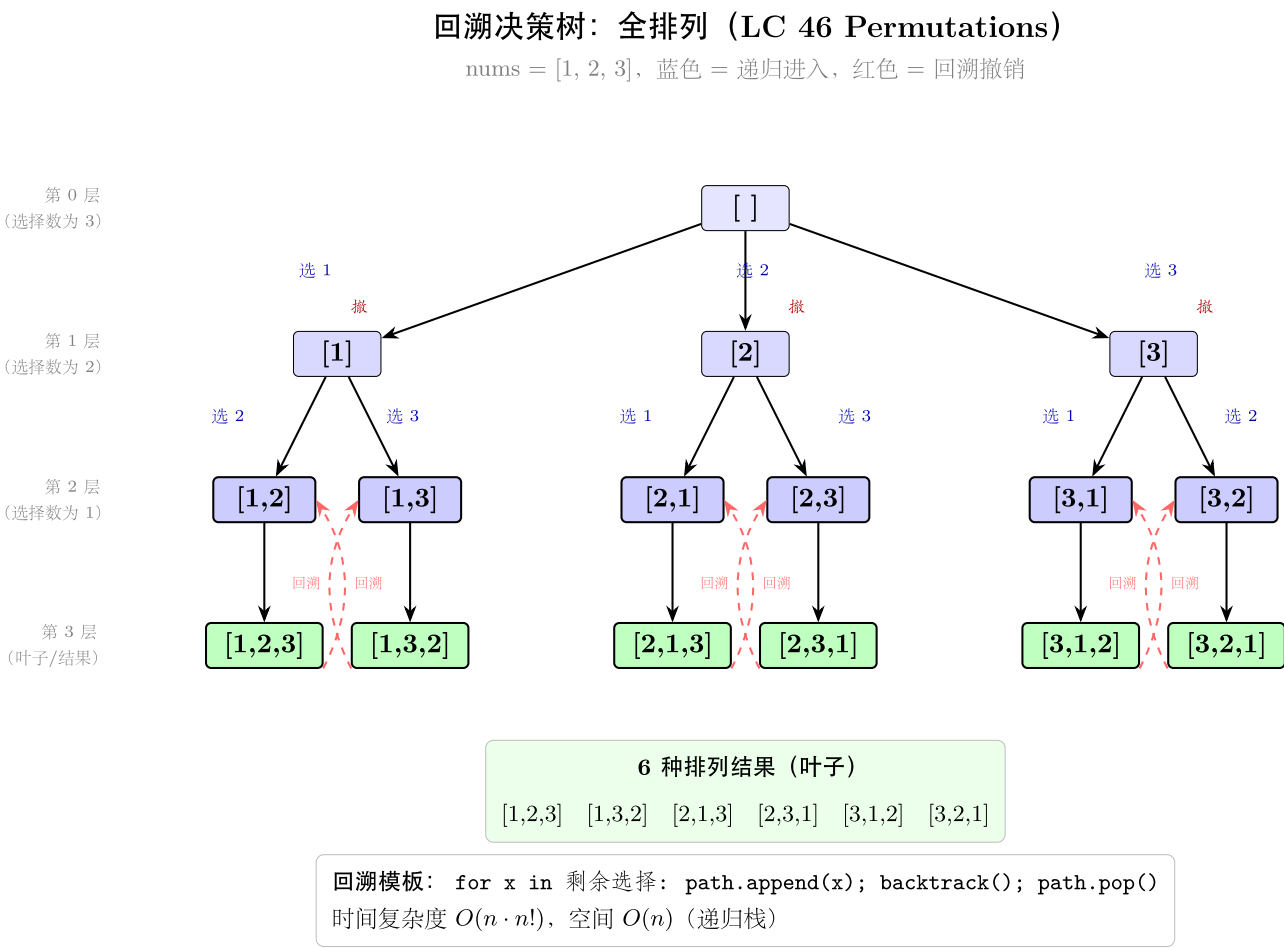


图 17: 完整 3 层决策树 6 叶子

图组合回溯（LC 77）

图 N-Queens（LC 52）

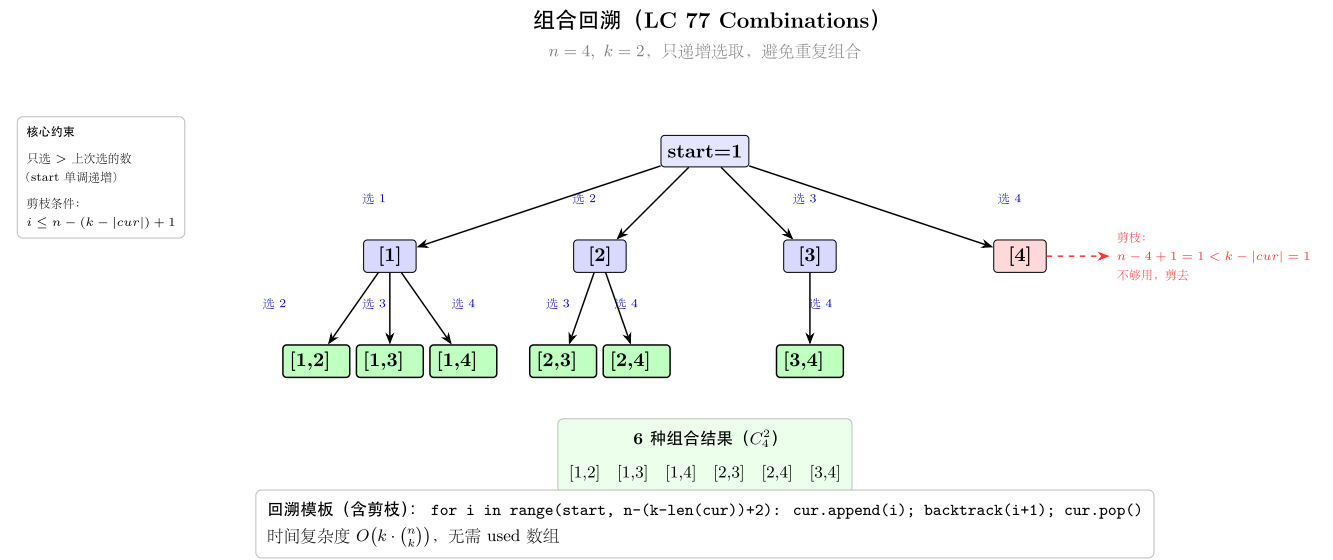


图 18: 递增组合决策树 + 剪枝

抽象方法 (标准模板代码) ### 回溯通用骨架 “python from typing import List

```

def
back-
track_skeleton(nums:
List[int])
->
List[List[int]]:
    """ “回
    溯三
    件套
    骨架:
    选择
    → 递
    归 →
    撤
    销。”
    """

    result:
    List[List[int]]
    = []
    path:
    List[int]
    = []

```

```
def
back-
track(start:
int) ->
None:
# 终
止条
件
(视题
目而
定) if
len(path)
==
len(nums):
re-
sult.append(path[:])
# 收
集当
前路
径的
快照
return
for i
in
range(start,
len(nums)):
path.append(nums[i])
# 选
择
back-
track(i
+ 1) #
递归
(组合
用
i+1,
排列
用 0)
path.pop()
# 撤
销
```

```

backtrack(0)
return
result
"""

```

套路 1: 组合类 (77/39)

适用题: 77 (Combinations)、39 (Combination Sum)

77: 从 $[1..n]$ 中取 k 个数的所有组合

```

def combine(n: int, k: int) -> List[List[int]]:
    """ 时间  $O(C(n,k) \cdot k)$ , 空间  $O(k)$  (递归栈 + path)。"""
    result: List[List[int]] = []
    path: List[int] = []

    def backtrack(start: int) -> None:
        if len(path) == k:
            result.append(path[:])
            return
        # 剪枝: 剩余候选数不足以凑成  $k$  个则跳出
        for i in range(start, n - (k - len(path)) + 2):
            path.append(i)
            backtrack(i + 1)
            path.pop()

    backtrack(1)
    return result

```

39: 候选数可重复选, 找和为 $target$ 的所有组合

```

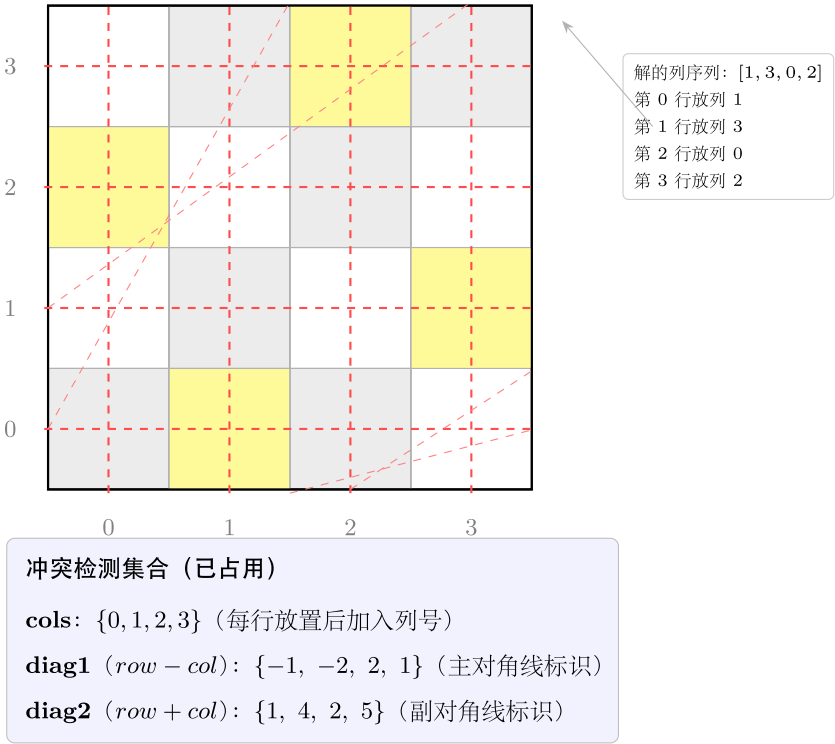
def combinationSum(candidates: List[int], target: int) -> List[List[int]]:
    """ 时间  $O(n^{(target/min)})$ , 空间  $O(target/min)$  (最大递归深度)。"""
    candidates.sort()
    # 排序后可提前剪枝
    result: List[List[int]] = []
    path: List[int] = []

    def backtrack(start: int, remaining: int) -> None:
        if remaining == 0:
            result.append(path[:])
            return

```

N 皇后 (LC 52 N-Queens II)

4 × 4 棋盘，解：列 [1, 3, 0, 2]，标红线为攻击范围



放置合法条件: $col \notin \mathbf{cols}$ 且 $row - col \notin \mathbf{diag1}$ 且 $row + col \notin \mathbf{diag2}$

图 19: 4x4 棋盘 + 攻击线 + 三集合

```

    for i in range(start, len(candidates)):
        if candidates[i] > remaining: # 剪枝: 后续更大, 全部跳过
            break
        path.append(candidates[i])
        backtrack(i, remaining - candidates[i]) # i 不 +1, 允许重复选
        path.pop()

backtrack(0, target)
return result

```

套路 2: 排列类 (46)

适用题: 46 (Permutations)

46: 无重复整数的全排列

```

def permute(nums: List[int]) -> List[List[int]]:
    """ 时间  $O(n! \cdot n)$ , 空间  $O(n)$ 。 """
    result: List[List[int]] = []
    path: List[int] = []
    used = [False] * len(nums)

    def backtrack() -> None:
        if len(path) == len(nums):
            result.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return result

```

套路 3: 字母组合 / 逐位选择 (17)

适用题: 17 (Letter Combinations of a Phone Number)

17: 电话号码字母组合

```
def letterCombinations(digits: str) -> List[str]:
    """ 时间  $O(4^n \cdot n)$ , 空间  $O(n)$  ( $n$  为 digits 长度, 最多 4 字母/键)。"""
    if not digits:
        return []

    digit_map = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz',
    }
    result: List[str] = []
    path: List[str] = []

    def backtrack(idx: int) -> None:
        if idx == len(digits):
            result.append(''.join(path))
            return
        for ch in digit_map[digits[idx]]:
            path.append(ch)
            backtrack(idx + 1)
            path.pop()

    backtrack(0)
    return result
```

套路 4: N-Queens 三集合约束 (52)

适用题: 52 (N-Queens II)

52: N 皇后方案数

```
def totalNQueens(n: int) -> int:
    """ 时间  $O(n!)$ , 空间  $O(n)$  (三个集合各最多  $n$  个元素)。"""
    count = 0
    cols: set[int] = set()
    diag1: set[int] = set() # row - col (主对角线标识)
    diag2: set[int] = set() # row + col (副对角线标识)
```

```

def backtrack(row: int) -> None:
    nonlocal count
    if row == n:
        count += 1
        return
    for col in range(n):
        if col in cols or (row - col) in diag1 or (row + col) in diag2:
            continue
        cols.add(col)
        diag1.add(row - col)
        diag2.add(row + col)
        backtrack(row + 1)
        cols.remove(col)
        diag1.remove(row - col)
        diag2.remove(row + col)

backtrack(0)
return count

```

套路 5：生成括号（参数约束型回溯）(22)

适用题：22 (Generate Parentheses)

22: 生成所有合法括号组合

```

def generateParenthesis(n: int) -> List[str]:
    """ 时间  $O(4^n / n^{(3/2)})$  (卡特兰数), 空间  $O(n)$  (递归深度  $2n$ )。 """
    result: List[str] = []

    def backtrack(path: str, open_cnt: int, close_cnt: int) -> None:
        if len(path) == 2 * n:
            result.append(path)
            return
        if open_cnt < n:
            backtrack(path + '(', open_cnt + 1, close_cnt)
        if close_cnt < open_cnt:
            backtrack(path + ')', open_cnt, close_cnt + 1)

    backtrack('', 0, 0)
    return result

```

套路 6：网格 DFS + 原地标记（79）

适用题：79（Word Search）

```
# 79: 单词搜索
def exist(board: List[List[str]], word: str) -> bool:
    """ 时间  $O(m \cdot n \cdot 4^L)$  ( $L$  为 word 长度), 空间  $O(L)$  (递归栈)。"""
    rows, cols = len(board), len(board[0])
    directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

    def dfs(r: int, c: int, idx: int) -> bool:
        if idx == len(word):
            return True
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return False
        if board[r][c] != word[idx]:
            return False
        tmp = board[r][c]
        board[r][c] = '#' # 原地标记: 防止同一路径重复访问
        for dr, dc in directions:
            if dfs(r + dr, c + dc, idx + 1):
                board[r][c] = tmp # 恢复 (回溯)
                return True
        board[r][c] = tmp # 恢复 (回溯)
        return False

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False
```

速查表

题目	套路类型	去重方式	关键剪枝	时间复杂度
77 Combinations	组合，有 start	start 推进	剩余元素不足 k 个	$O(C(n, k) \cdot k)$
39 Combination Sum	组合，可重选	start 不推进	<code>candidates[i] > remaining</code>	$O(n^{t/m})$
46 Permutations	排列，有 used[]	used[i] 标记	无（全排列）	$O(n! \cdot n)$

题目	套路类型	去重方式	关键剪枝	时间复杂度
17 Letter Combinations	逐位 DFS	按位推进 idx	无	$O(4^n \cdot n)$
52 N-Queens II	按行放置	三集合冲突检测	col/diag1/diag2 命中跳过	$O(n!)$
22 Generate Parentheses	参数约束型	open <= n, close <= open	约束即剪枝	$O(4^n / n^{3/2})$
79 Word Search	网格 DFS	原地 # 标记	字符不匹配立即返回	$O(mn \cdot 4^L)$

方法变形（4 类）

变形 1：组合系列

- 77(C(n,k)) → 39(重复选, 剪 sum) → 40(有重复元素, 排序后跳过 candidates[i] == candidates[i-1], 非本 category): 三题均用 start 指针控制枚举起点, 核心差别在于”是否允许重选”和”是否有重复元素”。
- 剪枝力度对比: 77 剪”剩余不足 k”, 39 剪”超过 target”, 重复元素题还要跳过同层重复项。
- 泛化: 所有”从集合中选若干元素、顺序无关”的组合类问题, 首先考虑 start 指针 + 递归 + 剪枝框架。

变形 2：排列系列

- 46（无重复元素）→ 47（有重复元素, used[i-1] 控制同层跳过, 非本 category): 46 用 used[] 数组, 47 额外需要排序 + 同层去重。
- 交换法 (swap): 另一种实现排列的方式是 nums[i], nums[start] = nums[start], nums[i], 再递归 backtrack(start+1), 最后 swap 回来。代码更简洁但思路略难理解。
- 排列 vs 组合的区分: 排列关心顺序, [1,2] 和 [2,1] 是不同答案, 不用 start 但要 used; 组合不关心顺序, 用 start 避免重复。

变形 3：约束满足（N-Queens / 生成括号）

- 52（N-Queens II, 计数）→ 51（N-Queens, 返回棋盘, 非本 category): 51 额外在 count += 1 处构造并收集棋盘字符串。
- 三集合判断: 行不需要集合（每行恰好一个）, 列 col in cols, 主对角 row-col, 副对角 row+col——这三个标识在同一直线上的值相同, 用 set 检测 O(1)。
- 22 的”无撤销”写法: generateParenthesis 用字符串拼接（不可变）传参, 天然无需撤销步骤, 比维护 path 列表更简洁——适用于状态量小且用不可变对象表示的场景。

变形 4: 网格搜索

- **79 (单词匹配)** → **200 (岛屿数量, 不含本 category, 属 graph)**: 79 每次 DFS 从当前格出发匹配单词, 找到则立即返回; 200 DFS 用来“淹没”整个连通分量, 两者 DFS 框架相同, 语义不同。
- **原地标记 vs visited 数组**: 原地改 `board[r][c] = '#'` 节省空间, 但必须在函数退出前还原; visited 数组更安全但需 $O(m \cdot n)$ 额外空间。
- **剪枝机会**: 79 可提前检查 word 中各字符在 board 中的频率——若 board 里某字符出现次数少于 word 中的需求量, 可直接返回 False ($O(mn)$ 预处理, 跳过大量无效搜索)。

思考路标 (条件反射)

1. 看到“所有组合 / 子集” → 回溯 + start 指针, 去重靠“不往前看”
2. 看到“组合 + 可重复选”(39) → 递归时 `backtrack(i, ...)` 而非 `backtrack(i+1, ...)`
3. 看到“全排列”(46) → `used[]` 数组标记, 每层从 0 扫到 `n-1`, 跳过 `used[i]`
4. 看到“按键字母映射”(17) → DFS 深度 = `digits` 长度, 第 `idx` 层选 `digit_map[digits[idx]]`
5. 看到“N 皇后 / 数独类约束” → 三集合 (列 / 两对角线) 冲突检测, 加前删后
6. 看到“合法括号”(22) → 参数携带 `open_cnt / close_cnt`, 约束即剪枝, 无需撤销
7. 看到“网格中找路径 / 单词”(79) → 对每格起点 DFS, 原地 # 标记防回头, 退出时还原
8. 看到“搜索树中有多余分支” → 想剪枝: 组合类剪“后续不够”, 和类剪“超过目标”, 排列类剪“已用”
9. 看到“`result.append(path[:])`” → 必须是 `path[:]` (快照), 否则结果全是同一引用
10. 看到“结果集中无重复”但候选有重复 → 排序后跳过同层 `candidates[i] == candidates[i-1]`
11. 看到“NAS / 超参组合搜索”(AI 场景) → 本质是 `combinationSum / combinations` 框架 + 约束剪枝
12. 看到“CSP 问题 (约束满足)” → 回溯框架 + 前向检验 (forward checking) = 本节所有技巧的工程化版本

易错点

1. **path 快照**: `result.append(path)` 和 `result.append(path[:])` 的区别——前者只存引用, 回溯撤销后 `path` 变化, 最终结果全为空或相同; 必须用 `path[:]` 或 `list(path)` 拷贝当前状态。
2. 排列中忘写 `used[i] = False`: 选择之后设 `used[i] = True`, 递归后必须 `used[i] = False`; 若忘记撤销, 后续层看到元素被占用, 全排列结果会严重缺失。
3. **39 题 start 不推进**: 可重复选时递归传 `i` (不是 `i+1`); 若误传 `i+1` 则每个元素只能选一次, 变成 77 题逻辑。
4. **77 题剪枝边界**: 上界应为 `n - (k - len(path)) + 2` (Python range 右开), 而非 `n + 1`; 若上界过大, 虽然结果正确但剪枝无效, 时间复杂度退化。
5. **22 题字符串拼接**: `path + '('` 创建新字符串 (不可变), 不需要撤销; 若改成 `path.append('(')` + `path.pop()` 的列表写法, 逻辑等价但要记得撤销。

6. 52 题对角线标识：主对角线用 `row - col`（同一主对角线上差值相同），副对角线用 `row + col`（同一副对角线上和相同）；两者容易混淆，写错则约束失效导致放置冲突皇后。
7. 79 题标记未还原：`board[r][c] = '#'` 之后，无论 DFS 结果如何（True 或 False），退出前都必须 `board[r][c] = tmp`；若在返回 True 时忘记还原，下一个起点的搜索会受污染。
8. 17 题空输入：`digits = ""` 时应返回 `[]` 而非 `[""]`；需在入口处显式判断 `if not digits: return []`，否则 `backtrack(0)` 在 `idx == len(digits)` 时会 `append` 一个空字符串。

典型应用例题

例 1：46. Permutations

题目：给定无重复整数数组 `nums`，返回所有全排列。

思路：每次从未使用的元素中选一个加入 `path`，`used` 数组标记已选。递归深度达到 `n` 时收集结果。没有 `start` 参数——排列关心顺序，每层都从头扫描全集。

解：

参考：[solutions/backtracking/p046_permutations.py](#)

```
def permute(nums: List[int]) -> List[List[int]]:
    result: List[List[int]] = []
    path: List[int] = []
    used = [False] * len(nums)

    def backtrack() -> None:
        if len(path) == len(nums):
            result.append(path[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return result
```

分析： $O(n! \cdot n)$ 时间（ $n!$ 个叶节点，每个收集 $O(n)$ ）， $O(n)$ 空间（`path` + `used` + 递归栈均 $O(n)$ ）。与组合题

的核心区别：无 `start` 参数，每层对所有元素重新扫描，靠 `used` 而非位置避免重复。

例 2: 52. N-Queens II

题目： $n \times n$ 棋盘放 n 个互不攻击的皇后，返回方案总数。

思路：按行放置（一行一个），对每列检查是否与已放皇后冲突。冲突检测：同列（`col in cols`）、主对角线（`row-col` 相同）、副对角线（`row+col` 相同）。三个集合的增删对应选择与撤销。

解：

参考： [solutions/backtracking/p052_n_queens_ii.py](#)

```
def totalNQueens(n: int) -> int:
    count = 0
    cols: set[int] = set()
    diag1: set[int] = set() # row - col
    diag2: set[int] = set() # row + col

    def backtrack(row: int) -> None:
        nonlocal count
        if row == n:
            count += 1
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            backtrack(row + 1)
            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)

    backtrack(0)
    return count
```

分析： $O(n!)$ 时间（实际因剪枝远小于 $n!$ ）， $O(n)$ 空间。三集合检测 $O(1)$ ，比逐格扫描快。对角线标识 `row±col` 是经典技巧：同一对角线上所有格子的行列差（或和）相同，一个整数即可表示整条对角线。

例 3: 79. Word Search

题目：给定字符矩阵 `board` 和字符串 `word`，判断 `word` 是否存在于矩阵中（每格只能用一次，可向上下左右移动）。

思路：枚举所有可能的起点，对每个起点 DFS 匹配单词。访问标记用原地修改 (`board[r][c] = '#'`)，避免额外 `visited` 数组；回退时还原。单词匹配完整 (`idx == len(word)`) 时返回 `True`，立即终止后续搜索。

解：

参考: [solutions/backtracking/p079_word_search.py](#)

```
def exist(board: List[List[str]], word: str) -> bool:
    rows, cols = len(board), len(board[0])

    def dfs(r: int, c: int, idx: int) -> bool:
        if idx == len(word):
            return True
        if r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != word[idx]:
            return False
        tmp, board[r][c] = board[r][c], '#'
        found = (dfs(r+1, c, idx+1) or dfs(r-1, c, idx+1) or
                 dfs(r, c+1, idx+1) or dfs(r, c-1, idx+1))
        board[r][c] = tmp # 回溯：恢复原始字符
        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False
```

分析： $O(m \cdot n \cdot 4^L)$ 时间 ($m \cdot n$ 个起点，每个起点 DFS 深度 L ，每层最多 4 方向)， $O(L)$ 空间 (递归栈)。原地标记的回溯确保了正确性：每条 DFS 路径上已访问的格子不会被再次选取，且不影响其他起点的搜索。

自测题

自测 1 (77 题 Combinations) —— $n=4$, $k=2$ 输出 `[[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]` (顺序不限)。□ 提示: `backtrack(start)` 中循环 `range(start, n-(k-len(path))+2)`, `path` 长度到 k 时收集 `path[:]`。参考 [solutions/backtracking/p077_combinations.py](#)。

自测 2 (39 题 Combination Sum) —— `candidates=[2,3,6,7]`, `target=7` 输出 `[[2,2,3],[7]]`。□ 提示: 排序后循环, `candidates[i] > remaining` 时 `break`; 递归传 i (不是 $i+1$) 允许重复选; `remaining == 0` 时收集。参考 [solutions/backtracking/p039_combination_sum.py](#)。

自测 3 (17 题 Letter Combinations) —— `digits="23"` 输出 `["ad","ae","af","bd","be","bf","cd","ce","cf"]` (顺序不限)。□ 提示: DFS 第 `idx` 层遍历 `digit_map[digits[idx]]`, `idx == len(digits)` 时 `result.append(''.join(path))`。空输入直接返回 `[]`。参考 [solutions/backtracking/p017_letter_combinations_of_a_phone_number.py](#)。

自测 4 (22 题 Generate Parentheses) ——n=3 输出 ["((()))","(()())","(())()","()(())","()()()"] (顺序不限)。□ 提示: open_cnt < n 可加左括号, close_cnt < open_cnt 可加右括号, len(path) == 2*n 时收集; 用字符串拼接不需撤销。参考 solutions/backtracking/p022_generate_parentheses.py。

自测 5(52 题 N-Queens II)——n=4 输出 2,n=1 输出 1。□ 提示: 三集合 cols、diag1(row-col)、diag2(row+col); 按行递归, 第 n 行时 count += 1; 选择后 add, 递归后 remove。参考 solutions/backtracking/p052_n_queens_ii.py。

题目全览（7 题）

#	题目	套路分类	难度
17	Letter Combinations of a Phone Number	逐位 DFS，字母映射	Medium
77	Combinations	组合回溯，start 去重	Medium
46	Permutations	全排列，used[] 标记	Medium
39	Combination Sum	组合可重选，剪 sum	Medium
52	N-Queens II	三集合约束，按行放置	Hard
22	Generate Parentheses	参数约束型回溯	Medium
79	Word Search	网格 DFS，原地标记	Medium

融合版说明

段	来源	价值
一例速记	本文件	7 题 6 类套路一览 + AI 关联，扫一眼知要用什么
思维路径还原	本文件	7 道题的解题内心独白，含关键决策点
抽象成方法	本文件	6 个标准模板代码（骨架 + 5 类子套路）+ 速查表
方法变形	本文件	4 类变体扩展（组合 / 排列 / 约束满足 / 网格）
思考路标	本文件	12 条题型识别条件反射，覆盖全部 7 题 + AI 场景
易错点	本文件	8 条高频踩坑（path 快照 / 撤销遗漏 / 对角线标识等）
典型应用例题	solutions/	3 道精讲（46、52、79），代码 + 正确性分析

段	来源	价值
自测题	leetcode	5 题带 ☐ 提示，链接 solutions 文件
题目全览	本文件	7 题完整列表，套路分类一览

跨 category 导航：- 图的 DFS（连通分量、拓扑排序）与网格搜索结构相同但语义不同 → 见 graph_general category - 动态规划可视为”回溯 + 记忆化”：若搜索树有大量重叠子问题，把 backtrack 的参数作为 key 存 memo → 见 09-dynamic-programming 系列 - 二叉树 DFS 是回溯的特殊形式（树形搜索空间）→ 见 07-binary-tree-dfs.md - 约束满足问题（CSP）工程化：arc consistency + backtracking = 本节技巧在 AI Solver 中的应用

09 —Divide & Conquer（融合版）

难度：□□□□ 题数：4 核心套路：归并排序思想、树形分治、二分递归、合并有序序列 本文件：覆盖 divide_conquer 4 题的算法套路总结 + 典型题精讲 + 自测

一例速记

归并排序思想 (148 / 23)：split → 左右递归 → merge；148 对链表用快慢指针找中点拆分，23 把 k 路归并转化为反复两两归并 树形分治 (108 / 427)：问题天然具有树结构，每次在中点或四等分处划分，返回子树根节点后自底向上组装 108 sorted array → BST：每次取中间元素为根 (mid = (l + r) // 2)，左半建左子树，右半建右子树，保证高度平衡 427 Quad Tree：若区域内值全相同则建叶节点，否则四等分递归建四个子节点，再聚合 23 Merge k Lists：分治两两归并，深度 $O(\log k)$ ，总时间 $O(N \log k)$ (N 为总节点数) 148 Sort List：链表归并排序；快慢指针找中点，断开两段后递归；merge 函数与数组归并一致 AI 关联：MapReduce (split → 分布式处理 → reduce/merge) / 并行计算（数据分片）/ 分层神经网络推理

思维路径还原

“看到 108 Convert Sorted Array to BST：升序数组建高度平衡 BST → 分治：取中间元素为根（保证左右子树大小尽量均等），左半数组建左子树，右半数组建右子树。区间 [l, r]，mid = (l + r) // 2, root.left = build(l, mid-1), root.right = build(mid+1, r)。递归直到 l > r 返回 None，共 $O(n)$ 次节点创建。

看到 **427 Construct Quad Tree**: 二维网格, 值只有 0/1, 要求建四叉树 → 先判断当前区域是否全为同一值 (遍历检查或前缀和 $O(1)$ 查询); 若是, 则建叶节点 (`isLeaf=True`, `val` 为该值); 若否, 则四等分成左上/右上/左下/右下四个子区域递归, 返回四个子节点, 组装非叶节点。

看到 **23 Merge k Sorted Lists**: k 个有序链表合并 → 分治: 把 k 个链表两两配对, 每轮合并后变成 $k/2$ 个, 重复 $O(\log k)$ 轮; 每轮合并两个有序链表用经典双指针 `merge` 函数, $O(m+n)$ 时间。总时间 $O(N \log k)$ (N 为所有节点总数)。另法: 最小堆, 每次弹出最小节点后把其后继推入堆, 同样 $O(N \log k)$, 常数略大。

看到 **148 Sort List**: 对链表原地排序, 要求 $O(n \log n)$ 时间 $O(1)$ 额外空间 → 归并排序: 找中点 (快慢指针 `slow/fast`, `slow` 停在前半末尾), 断开两段链表; 递归排序两半, 再 `merge`; 空间分析: 递归栈 $O(\log n)$ (非严格 $O(1)$, 但常视为满足要求)。Bottom-up 迭代归并 (步长从 1 倍增) 可做到严格 $O(1)$ 额外空间。”

学习目标

- 掌握“分治三步”框架: Divide (拆分) → Conquer (递归子问题) → Combine (合并结果)
- 理解链表归并排序中快慢指针找中点、断链、递归、`merge` 的完整流程
- 能用分治两两归并实现 k 路合并, 理解其时间复杂度 $O(N \log k)$ 的推导
- 掌握有序数组建平衡 BST 的中点划分递归, 理解“高度平衡”的来源
- 理解四叉树构建中“叶节点条件”与“四等分递归”的结构
- 能识别 MapReduce / 并行计算与分治框架的对应关系

几何示意

图归并排序递归树 (LC 148)

抽象方法
(标准模板代码)
分治通用骨架

```
“python
def di-
vide_and_conquer(problem,
l: int,
r: int):
“““分
治三
步框
架骨
架。”
“”# 1.
边界:
问题
规模
足够
小,
直接
求解
if l >=
r:
return
base_case(problem,
l, r)
# 2.
Di-
vide:
找分
割点
mid =
l + (r -
l) // 2
```

```

# 3.
Con-
quer:
递归
左右
子问
题
left_result
= di-
vide_and_conquer(problem,
l, mid)
right_result
= di-
vide_and_conquer(problem,
mid +
1, r)
# 4.
Com-
bine:
合并
子问
题结
果
return
com-
bine(left_result,
right_result)
"""

```

套路 1: 有序数组建平衡 BST (108)

适用题: 108 (Convert Sorted Array to Binary Search Tree)

```

from __future__ import annotations
from typing import Optional, List

class TreeNode:
    def __init__(self, val: int = 0,
                 left: Optional[TreeNode] = None,

```

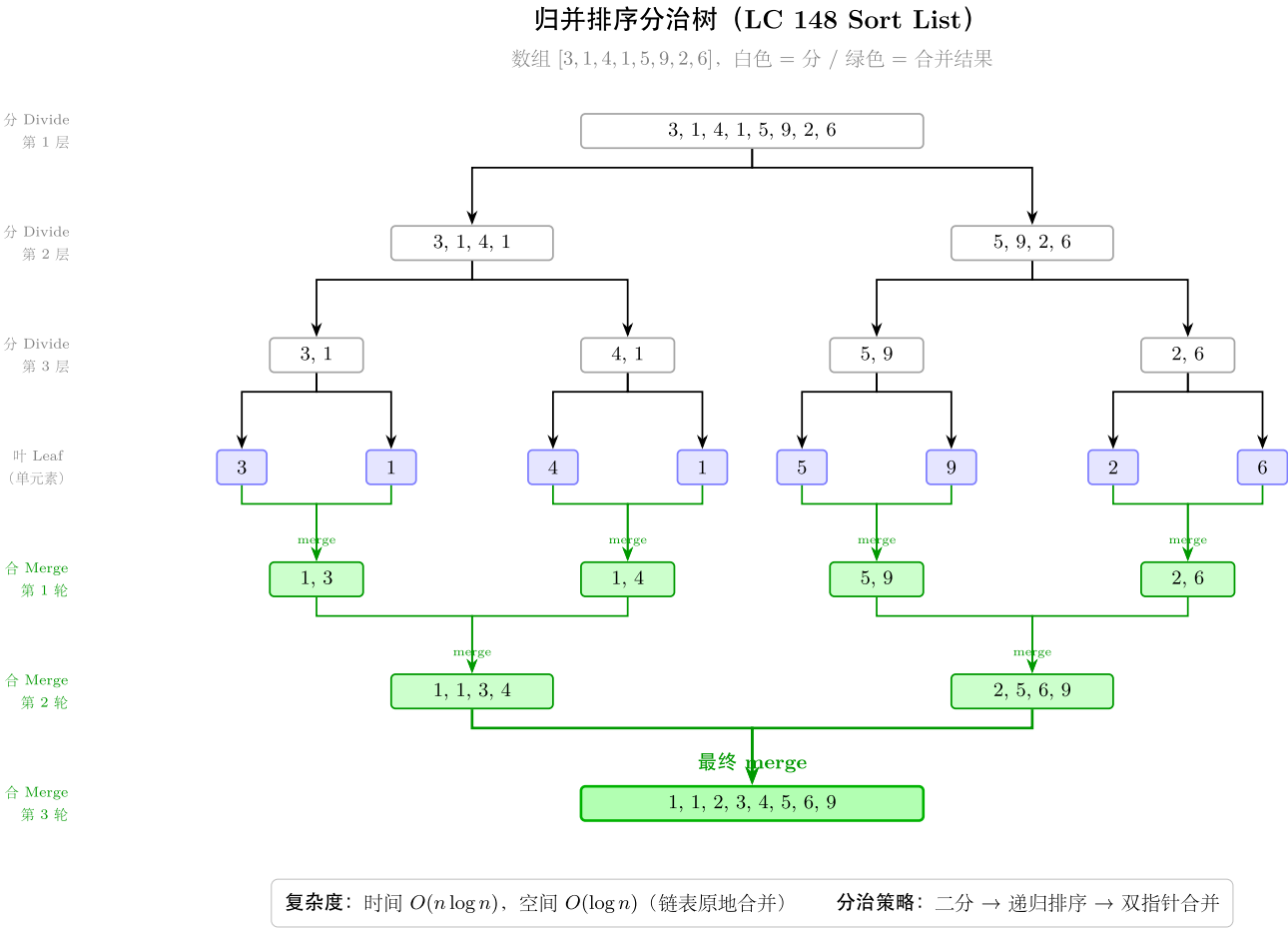


图 20: 分治 + 合并双向递归树

```

        right: Optional[TreeNode] = None):
    self.val = val
    self.left = left
    self.right = right

# 108: 升序数组 → 高度平衡 BST
def sortedArrayToBST(nums: List[int]) -> Optional[TreeNode]:
    """ 时间  $O(n)$  (每个元素创建一次节点), 空间  $O(\log n)$  (递归栈深度 = 树高)。"""
    def build(l: int, r: int) -> Optional[TreeNode]:
        if l > r:
            return None
        mid = l + (r - l) // 2          # 取中间元素为根 (保证左右均等)
        root = TreeNode(nums[mid])
        root.left = build(l, mid - 1)
        root.right = build(mid + 1, r)
        return root

    return build(0, len(nums) - 1)

```

套路 2: 四叉树构建 (427)

适用题: 427 (Construct Quad Tree)

```

class QuadNode:
    def __init__(self, val: bool, isLeaf: bool,
                 topLeft=None, topRight=None,
                 bottomLeft=None, bottomRight=None):
        self.val = val
        self.isLeaf = isLeaf
        self.topLeft = topLeft
        self.topRight = topRight
        self.bottomLeft = bottomLeft
        self.bottomRight = bottomRight

```

427: 构建四叉树

```

def construct(grid: List[List[int]]) -> QuadNode:
    """ 时间  $O(n^2 \log n)$  (每层每个格子最多访问一次), 空间  $O(\log n)$  (递归栈)。
    若用前缀和预处理则时间降至  $O(n^2)$ 。

```

```

"""
n = len(grid)

def is_uniform(r: int, c: int, size: int) -> bool:
    """ 检查左上角 (r,c)、边长 size 的正方形区域是否全为同一值。"""
    val = grid[r][c]
    for i in range(r, r + size):
        for j in range(c, c + size):
            if grid[i][j] != val:
                return False
    return True

def build(r: int, c: int, size: int) -> QuadNode:
    if is_uniform(r, c, size):
        return QuadNode(bool(grid[r][c]), isLeaf=True)
    half = size // 2
    return QuadNode(
        val=True,
        isLeaf=False,
        topLeft = build(r, c, half),
        topRight = build(r, c + half, half),
        bottomLeft = build(r + half, c, half),
        bottomRight = build(r + half, c + half, half),
    )

return build(0, 0, n)

```

427 优化版：前缀和 $O(1)$ 区域均匀判断

```

def construct_optimized(grid: List[List[int]]) -> QuadNode:
    """ 时间  $O(n^2)$ , 空间  $O(n^2)$  (前缀和数组)。"""
    n = len(grid)
    # prefix[i][j] = grid[0..i-1][0..j-1] 的元素和
    prefix = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            prefix[i][j] = (grid[i-1][j-1]
                            + prefix[i-1][j]
                            + prefix[i][j-1]
                            - prefix[i-1][j-1])

```



```

def region_sum(r: int, c: int, size: int) -> int:
    """ 区域 [r, r+size) × [c, c+size) 的元素和, O(1)。 """
    return (prefix[r + size][c + size]
            - prefix[r][c + size]
            - prefix[r + size][c]
            + prefix[r][c])

def build(r: int, c: int, size: int) -> QuadNode:
    s = region_sum(r, c, size)
    if s == 0 or s == size * size: # 全 0 或全 1
        return QuadNode(bool(s > 0), isLeaf=True)
    half = size // 2
    return QuadNode(
        val=True, isLeaf=False,
        topLeft    = build(r,          c,          half),
        topRight   = build(r,          c + half, half),
        bottomLeft = build(r + half, c,          half),
        bottomRight = build(r + half, c + half, half),
    )

return build(0, 0, n)

```

套路 3: 链表归并排序 (148)

适用题: 148 (Sort List)

```

class ListNode:
    def __init__(self, val: int = 0, next: Optional[ListNode] = None):
        self.val = val
        self.next = next

# 148: 链表排序 (归并排序)
def sortList(head: Optional[ListNode]) -> Optional[ListNode]:
    """ 时间  $O(n \log n)$ , 空间  $O(\log n)$  (递归栈)。 """
    # 边界: 空或只有一个节点, 无需排序
    if head is None or head.next is None:
        return head

    # Divide: 快慢指针找中点, 断开两段

```

```

slow, fast = head, head.next
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
mid = slow.next
slow.next = None          # 断链: 前半以 slow 结尾

```

Conquer: 递归排序两半

```

left = sortList(head)
right = sortList(mid)

```

Combine: 合并两个有序链表

```

return merge_two_lists(left, right)

```

```

def merge_two_lists(l1: Optional[ListNode],
                    l2: Optional[ListNode]) -> Optional[ListNode]:
    """ 合并两个有序链表, 时间  $O(m+n)$ , 空间  $O(1)$ 。 """
    dummy = ListNode(0)
    cur = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            cur.next = l1
            l1 = l1.next
        else:
            cur.next = l2
            l2 = l2.next
        cur = cur.next
    cur.next = l1 if l1 else l2
    return dummy.next

```

套路 4: k 路归并分治 (23)

适用题: 23 (Merge k Sorted Lists)

23: 合并 k 个有序链表 (分治两两归并)

```

def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    """ 时间  $O(N \log k)$  ( $N$  为总节点数,  $k$  为链表数), 空间  $O(\log k)$  (递归栈)。 """
    if not lists:
        return None

```

```

    return merge_range(lists, 0, len(lists) - 1)

def merge_range(lists: List[Optional[ListNode]],
                 l: int, r: int) -> Optional[ListNode]:
    if l == r:
        return lists[l]
    mid = l + (r - l) // 2
    left = merge_range(lists, l, mid)
    right = merge_range(lists, mid + 1, r)
    return merge_two_lists(left, right)    # 复用上方 merge_two_lists

# 23 备选：最小堆（同样  $O(N \log k)$ ，适合  $k$  较大时减少常数）
import heapq

def mergeKLists_heap(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    """ 时间  $O(N \log k)$ ，空间  $O(k)$ （堆大小）。 """
    dummy = ListNode(0)
    cur = dummy
    heap: list[tuple[int, int, Optional[ListNode]]] = []

    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    while heap:
        val, i, node = heapq.heappop(heap)
        cur.next = node
        cur = cur.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))

    return dummy.next

```

题目	分治模式	划分方式	Combine 操作	时间	空间
108 Sorted Array to BST	数组二分	取中点为根	左右子树拼接	$O(n)$	$O(\log n)$
427 Construct Quad Tree	二维四分	正方形四分	四子节点拼接	$O(n^2 \log n)$	$O(\log n)$
148 Sort List	链表二分	快慢指针中点	merge 有序链表	$O(n \log n)$	$O(\log n)$
23 Merge k Lists	k 路两两归并	下标区间对折	merge 两有序链表	$O(N \log k)$	$O(\log k)$

方法变形 (4 类)

变形 1: 有序序列建树系列 (108)

- **108** (数组 $\rightarrow$ 平衡 BST) $\rightarrow$ **109** (链表 $\rightarrow$ 平衡 BST, 非本 category): 数组版 $O(1)$ 随机访问直接取中点; 链表版需先快慢指针找中点 ($O(n)$), 或先转为数组再建树。
- 中点选取策略: $\text{mid} = (\text{l} + \text{r}) // 2$ (左中位数, 偏左) 和 $\text{mid} = (\text{l} + \text{r} + 1) // 2$ (右中位数) 均合法; LeetCode 108 接受多种答案, 但实践中习惯取左中位数。
- 泛化: 任何“升序序列建平衡二叉搜索结构”的问题都可套此模板——关键是“取中点为根, 两侧递归”。

变形 2: 归并排序系列 (148)

- **148** (链表) $\rightarrow$ **912** (数组, 非本 category): 数组归并排序需额外 $O(n)$ 辅助数组做 merge; 链表归并可原地修改指针, 不需要额外空间存元素 (仅需 $O(1)$ 辅助变量)。
- **Bottom-up** 迭代归并: 从步长 1 开始, 每轮将相邻步长区间两两合并, 步长翻倍, 共 $O(\log n)$ 轮——实现严格 $O(1)$ 额外空间 (无递归栈)。
- 快慢指针找中点: $\text{fast} = \text{head.next}$ (而非 $\text{fast} = \text{head}$) 使 slow 停在前半末尾, 方便断链; 若 $\text{fast} = \text{head}$ 则 slow 会多走一步, 在偶数长度链表上导致两段长度不均。

变形 3: k 路归并系列 (23)

- **23** (分治两两归并) vs 堆解法: 分治时间复杂度相同 ($O(N \log k)$), 但分治递归栈 $O(\log k)$ 而堆维护 k 个节点 $O(k)$; k 极大时堆空间占用更显著, 分治更优。
- 逐一归并 (Naive): 把 k 个链表依次归并到结果中, 时间 $O(N \cdot k)$, 当 k 大时劣于分治。
- **AI 关联 / MapReduce**: 23 题是 Reduce 阶段的典型范例——多个 worker 各自输出有序结果 (多个有序链表), coordinator 做多路 merge; 分治两两归并对应 tree aggregation (树形归约)。

变形 4: 四叉树 / 空间分治系列 (427)

- 427 (值全同则叶, 否则四分) → **BSP Tree** (二叉空间分割, 3D 渲染中的类似概念): 相同的“均匀则叶, 否则继续分”思路在计算机图形学中广泛应用。
- 前缀和优化: 朴素的 `is_uniform` 需 $O(\text{size}^2)$, 导致总时间 $O(n^2 \log n)$; 预处理二维前缀和后, 区域检测 $O(1)$, 总时间降至 $O(n^2)$ (每个格子最多分配到一个叶节点, 总工作量 $O(n^2)$)。
- 四叉树的应用: GIS 中的点查询 (空间索引)、图像压缩 (均匀区域直接存值, 非均匀区域继续分) —— 都是 427 题思路的工程化。

思考路标 (条件反射)

1. 看到“升序数组 + 建平衡 **BST**”(108) → 取中间元素为根, 左半建左子树, 右半建右子树, `mid = (l+r)//2`
2. 看到“二维网格 + 均匀判断 + 四叉树”(427) → 先判断区域是否全同, 是则叶节点; 否则四等分递归, 组装四子节点
3. 看到“链表 + $O(n \log n)$ 排序”(148) → 快慢指针找中点断链, 递归排左右, `merge` 两有序链表
4. 看到“合并 k 个有序链表”(23) → 分治两两归并 (深度 $O(\log k)$, 总时间 $O(N \log k)$); 或最小堆
5. 看到“分治后需要 **combine**” → 思考 `combine` 操作是否能 $O(n)$ 完成 (归并是 $O(n)$, 不能退化为 $O(n^2)$)
6. 看到“ k 路 **reduce** / 多数据源合并” → 联系 23 题, 分治两两合并而非逐一合并
7. 看到“链表找中点” → `slow = head, fast = head.next, while fast and fast.next, slow` 停在前半末尾
8. 看到“**merge** 两有序链表” → 哑节点 `dummy + cur` 指针, 逐一比较头节点, 拼接, 最后接剩余段
9. 看到“二维区域均匀判断 + 需要高效” → 二维前缀和预处理, $O(1)$ 查询任意矩形区域的元素和
10. 看到“**MapReduce** / 分布式聚合”(AI 场景) → `Map` = 分治拆分, `Reduce` = `merge`; 树形归约对应分治归并
11. 看到“并行计算 + 子任务无依赖” → 分治框架: 左右子问题可并行执行, `combine` 串行
12. 看到“分层神经网络推理” → 每层独立计算 + 层间激活合并, 对应分治的 `conquer + combine` 结构

易错点

1. **148** 快慢指针的起始位置: `fast = head.next` (不是 `head`); 若写 `fast = head` 则对偶数长度链表, `slow` 会停在中点右侧, 导致两段不均且无法正常断链。
2. **148** 断链顺序: `mid = slow.next; slow.next = None` —— 必须先保存 `mid` 再断链; 若先 `slow.next = None` 则 `mid` 信息丢失。
3. **108** `mid` 取左或右中位数: $(l + r) // 2$ 和 $(l + r + 1) // 2$ 结果可能不同, 两者都合法; 但在代码中必须与递归区间 `build(l, mid-1)` 和 `build(mid+1, r)` 保持一致, 不能混用两种计算方式。
4. **23** 逐一归并 (**Naive**) 的性能陷阱: 把 k 个链表一条一条合并 (`result = merge(result, lists[i])`) 是 $O(Nk)$, 当 k 很大时会 TLE; 应使用分治两两归并或最小堆。

5. 427 叶节点 `val` 的含义：对于非叶节点，`val` 可以设为任意值（通常设 `True`），LeetCode 仅检查叶节点的 `val`；但代码中不要把非叶节点的 `val` 误设为区域内第一个格子的值（可能误导）。
6. `merge` 两有序链表后忘接剩余：while `l1` and `l2` 循环结束后，`l1` 或 `l2` 可能仍有剩余；必须 `cur.next = l1 if l1 else l2` 将剩余部分接上，否则尾部截断。
7. 427 前缀和边界：`prefix[i][j]` 表示 `grid[0..i-1][0..j-1]` 的和（下标偏移 1），`region_sum(r, c, size)` 的参数是左上角 `(r, c)` 和区域大小 `size`，对应前缀和的下标是 `prefix[r+size][c+size] - ...` ——下标换算容易差 1，写完后用小例子验证。
8. 分治递归的空间计入：148 的“ $O(1)$ 额外空间”说法指的是 `merge` 操作本身；递归栈有 $O(\log n)$ 深度，若面试要求严格 $O(1)$ 空间需改用 bottom-up 迭代归并。

典型应用例题

例 1: 148. Sort List

题目：给定链表头节点 `head`，将链表按升序排序并返回排序后的链表头节点。要求时间 $O(n \log n)$ ，尽量使用 $O(1)$ 额外空间。

思路：链表归并排序。三步：□ 快慢指针找中点并断链（Divide）；□ 递归排序两半（Conquer）；□ 合并两个有序链表（Combine）。

`merge` 函数用哑节点简化头节点处理：比较两链表当前节点的值，小的接入结果链，循环直至一方为空，最后接上剩余段。

解：

```
# 参考: solutions/divide_conquer/p148_sort_list.py
def sortList(head: Optional[ListNode]) -> Optional[ListNode]:
    if head is None or head.next is None:
        return head

    # Divide: 快慢指针找前半末尾 slow
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    mid = slow.next
    slow.next = None    # 断开前半和后半

    left = sortList(head)
    right = sortList(mid)
```

```

# Combine: 合并两个有序链表
dummy = ListNode(0)
cur = dummy
while left and right:
    if left.val <= right.val:
        cur.next = left; left = left.next
    else:
        cur.next = right; right = right.next
    cur = cur.next
cur.next = left if left else right
return dummy.next

```

分析: $O(n \log n)$ 时间 ($\log n$ 层, 每层 merge 共 $O(n)$), $O(\log n)$ 空间 (递归栈)。关键在于链表 merge 可以原地修改指针, 无需 $O(n)$ 辅助数组, 是链表相比数组在归并排序上的优势。

例 2: 23. Merge k Sorted Lists

题目: 给你 k 个升序链表, 将它们合并为一个升序链表并返回。

思路: 分治两两归并。把 k 个链表的下标区间 $[1, r]$ 递归拆成 $[1, \text{mid}]$ 和 $[\text{mid}+1, r]$, 分别合并后再合并两侧结果。树形递归深度 $O(\log k)$, 每层所有 merge 共处理 N 个节点, 总时间 $O(N \log k)$ 。

解:

```

# 参考: solutions/divide_conquer/p023_merge_k_sorted_lists.py
def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    if not lists:
        return None

    def merge_two(l1: Optional[ListNode],
                  l2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0)
        cur = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                cur.next = l1; l1 = l1.next
            else:
                cur.next = l2; l2 = l2.next
            cur = cur.next
        cur.next = l1 if l1 else l2
        return dummy.next

```

```
def solve(lo: int, hi: int) -> Optional[ListNode]:
    if lo == hi:
        return lists[lo]
    mid = lo + (hi - lo) // 2
    return merge_two(solve(lo, mid), solve(mid + 1, hi))

return solve(0, len(lists) - 1)
```

分析: $O(N \log k)$ 时间, $O(\log k)$ 空间 (递归栈)。与逐一归并 ($O(Nk)$) 对比: 分治将每个节点参与 merge 的次数从 $k - 1$ 次降低到 $\log k$ 次。时间复杂度与最小堆解法相同, 但分治不需要额外的堆结构, 常数更小。

例 3: 108. Convert Sorted Array to Binary Search Tree

题目: 给定升序整数数组 `nums`, 将其转换为高度平衡的二叉搜索树 (每个节点两侧子树高度差 ≤ 1)。

思路: 取中间元素为根, 保证左右两侧元素数量尽量均等 (高度平衡的来源)。左半数组递归建左子树, 右半数组递归建右子树。

解:

参考: [solutions/divide_conquer/p108_convert_sorted_array_to_binary_search_tree.py](#)

```
def sortedArrayToBST(nums: List[int]) -> Optional[TreeNode]:
    def build(l: int, r: int) -> Optional[TreeNode]:
        if l > r:
            return None
        mid = l + (r - l) // 2
        root = TreeNode(nums[mid])
        root.left = build(l, mid - 1)
        root.right = build(mid + 1, r)
        return root

    return build(0, len(nums) - 1)
```

分析: $O(n)$ 时间 (每个元素恰好创建一次节点), $O(\log n)$ 空间 (递归栈深度等于树高, 平衡树高度 $O(\log n)$)。正确性: BST 性质由升序数组保证 (左半 $<$ 中点 $<$ 右半); 平衡性由取中点保证 (两侧大小差 ≤ 1 , 高度差 ≤ 1)。

自测题

自测 1 (108 题 Convert Sorted Array to BST) ——nums=[-10,-3,0,5,9] 建出高度平衡 BST, 根为 0 (或其他合法答案)。□ 提示: build(0, n-1) 中 mid = (l+r)//2, root = TreeNode(nums[mid]), 递归 build(l, mid-1) 和 build(mid+1, r); l > r 返回 None。参考 solutions/divide_conquer/p108_convert_sorted_array_to_binary_sea

自测 2 (427 题 Construct Quad Tree) ——grid=[[0,1],[1,0]] 建四叉树, 根非叶, 四个子节点均为叶节点, 值分别为 False/True/True/False。□ 提示: build(0, 0, 2) 中 is_uniform 返回 False, 四等分 half=1, 递归各 1×1 子区域; 1×1 区域必为均匀, 直接建叶节点。参考 solutions/divide_conquer/p427_construct_quad_tree.py。

自测 3 (148 题 Sort List) ——head=[4,2,1,3] 排序后输出 [1,2,3,4]。□ 提示: 快慢指针 slow=head, fast=head.next; 断链后递归两半; merge 时哑节点简化操作; 别忘 slow.next = None 断链。参考 solutions/divide_conquer/p148_sort_list.py。

自测 4 (23 题 Merge k Sorted Lists) ——lists=[[1,4,5],[1,3,4],[2,6]] 输出 [1,1,2,3,4,4,5,6]。□ 提示: 分治 solve(lo, hi) 在 lo==hi 时返回 lists[lo]; merge_two 合并两有序链表; 中点 mid = (lo+hi)//2, 递归 solve(lo, mid) 和 solve(mid+1, hi) 后 merge。参考 solutions/divide_conquer/p023_merge_k_sorted_lists.py。

自测 5 (综合) ——将 [-9,-3,0,5,9,11] (6 个元素) 建平衡 BST, 验证: 根为 nums[2]=0 或 nums[3]=5, 树高不超过 3。□ 提示: 6 个元素时 mid = (0+5)//2 = 2, 根 = nums[2] = 0; 左子树区间 [0,1], 右子树区间 [3,5]; 继续递归验证高度均为 2, 满足平衡条件。

题目全览（4 题）

#	题目	套路分类	难度
108	Convert Sorted Array to Binary Search Tree	数组二分, 取中点建 BST	Easy
427	Construct Quad Tree	二维四等分, 均匀判断建叶	Medium
148	Sort List	链表快慢指针 + 归并排序	Medium
23	Merge k Sorted Lists	k 路分治两两归并	Hard

融合版说明

段	来源	价值
一例速记	本文件	4 题 4 类套路一览 + AI 关联 (MapReduce / 并行计算)
思维路径还原	本文件	4 道题的解题内心独白，含关键决策点
抽象成方法	本文件	5 个标准模板（骨架 + 4 类子套路 + 堆备选）+ 速查表
方法变形	本文件	4 类变体扩展（建树 / 归并 / k 路 / 四叉树）
思考路标	本文件	12 条题型识别条件反射，覆盖全部 4 题 + AI 场景
易错点	本文件	8 条高频踩坑（快慢指针起点 / 断链顺序 / 剩余段接入等）
典型应用例题	solutions/	3 道精讲（148、23、108），代码 + 正确性分析
自测题	leetcode	5 题带 ☐ 提示，链接 solutions 文件
题目全览	本文件	4 题完整列表，套路分类一览

跨 category 导航：- 二叉树 DFS（递归子树、LCA、路径和）→ 见 07-binary-tree-dfs.md（分治是 DFS 的泛化）- 二叉搜索树（插入 / 删除 / 验证）→ 见 05-binary-search-tree.md（108 建出的是 BST）- 有序数组二分搜索 → 见 04-binary-search.md（分治思路与二分同源）- 动态规划：若分治子问题有重叠，加记忆化变为 DP → 见 dynamic_programming 系列 - 链表操作（双指针、反转、删节点）→ 见 linked_list category（148 用到链表基础操作）

10 — Dynamic Programming 1D (融合版)

难度：□□□□□ 题数：5 核心套路：一维状态转移、滚动数组优化、LIS 单序列、完全背包分段 本文件：覆盖 dynamic_programming_1d 5 题的算法套路总结 + 典型题精讲 + 自测

一例速记

DP 三问：☐ 状态是什么（dp[i] 代表什么）☐ 转移方程（当前状态如何由历史状态推出）☐ 初始化 + 边界 **70 爬楼梯：** dp[i] = dp[i-1] + dp[i-2]，到第 i 阶 = 从 i-1 迈一步 + 从 i-2 迈两步；可用两个变量滚动 **198 打家劫舍：** dp[i] = max(dp[i-1], dp[i-2] + nums[i])，不偷当前屋 vs 偷当前屋；滚动到 O(1) **300 LIS：** dp[i] = 以 nums[i] 结尾的最长递增子序列长度；O(n²) 转移或 O(n log n) 贪心 + 二分 **139 单词拆分：** dp[i] = s[:i] 能否被字典拆分；对每个 j < i，若 dp[j] 为真且 s[j:i]

在字典中则 $dp[i] = \text{True}$ **322 零钱兑换**: 完全背包, $dp[i] = \min(dp[i], dp[i - c] + 1)$; 枚举每种硬币, 取最少硬币数 **AI 关联**: Viterbi 算法 (最大概率路径) 是 HMM 解码的 DP 形式; 序列标注模型 (CRF) = LIS/Word Break 在概率图上的推广

思维路径还原

“看到 **70 Climbing Stairs**: 每次可走 1 或 2 步, 求走到第 n 阶的方法数 $\rightarrow$ 第 n 阶只能从第 $n-1$ 阶走 1 步, 或从第 $n-2$ 阶走 2 步到达。因此 $dp[i] = dp[i-1] + dp[i-2]$, 初始化 $dp[1]=1$, $dp[2]=2$ (或 $dp[0]=1$)。只用到两个历史值, 用两个变量 $prev2$, $prev1$ 即可 $O(1)$ 空间。

看到 **198 House Robber**: 数组相邻元素不能同时取, 求最大和 $\rightarrow$ 对第 i 栋房子: 要么不偷 (保留前 $i-1$ 栋的最优结果 $dp[i-1]$), 要么偷 (前 $i-2$ 栋最优 + 当前金额 $dp[i-2] + \text{nums}[i]$)。 $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ 。同样只依赖两步前, 滚动数组降至 $O(1)$ 。

看到 **300 Longest Increasing Subsequence**: 找最长严格递增子序列长度 $\rightarrow O(n^2)$: $dp[i]$ = 以 $\text{nums}[i]$ 结尾的 LIS 长度; 转移: $\text{for } j \text{ in range}(i): \text{if } \text{nums}[j] < \text{nums}[i]: dp[i] = \max(dp[i], dp[j]+1)$ 。 $O(n \log n)$ 优化: 维护 tails 数组 (每个长度的 LIS 的最小末尾值), 对每个 $\text{nums}[i]$ 二分找第一个 $\geq \text{nums}[i]$ 的位置并替换, tails 长度即答案。

看到 **139 Word Break**: 字符串 s 能否被字典中的词拼接覆盖 $\rightarrow dp[i] = s[:i]$ 能否被拆分 (布尔); $dp[0] = \text{True}$ (空串可拆)。转移: $\text{for } j \text{ in range}(i): \text{if } dp[j] \text{ and } s[j:i] \text{ in word_set}: dp[i] = \text{True}$ 。将字典转为 set 以 $O(1)$ 查询; 可提前剪枝 (只枚举 j 满足 $s[j:i]$ 长度在字典最短/最长词长度范围内)。

看到 **322 Coin Change**: 无限量硬币凑 amount , 求最少硬币数 $\rightarrow$ 完全背包: $dp[0]=0$, 其余 $dp[i]=\text{inf}$; 对每个 amount 值 i , 枚举所有硬币 c , 若 $i \geq c$ 则 $dp[i] = \min(dp[i], dp[i-c]+1)$ 。最终 $dp[\text{amount}]$ 若仍为 inf 则返回 -1, 否则返回该值。”

学习目标

- 掌握 DP 三问 (状态定义 / 转移方程 / 初始化), 能从问题描述直接导出方程
- 熟练写出“自底向上递推”和“自顶向下记忆化搜索”两种形式并互相转换
- 理解滚动数组优化 ($O(n) \rightarrow O(1)$) 的条件: 转移只依赖常数步前
- 掌握 LIS 的 $O(n^2)$ DP 解法和 $O(n \log n)$ 贪心 + 二分解法
- 理解完全背包框架 (322) 及其与 0/1 背包的区别 (内层循环方向)
- 能识别 Word Break 的“切割型 DP”模式, 并用 set 加速字典查询

几何示意

图一维 DP 状态转移（LC 70/198）

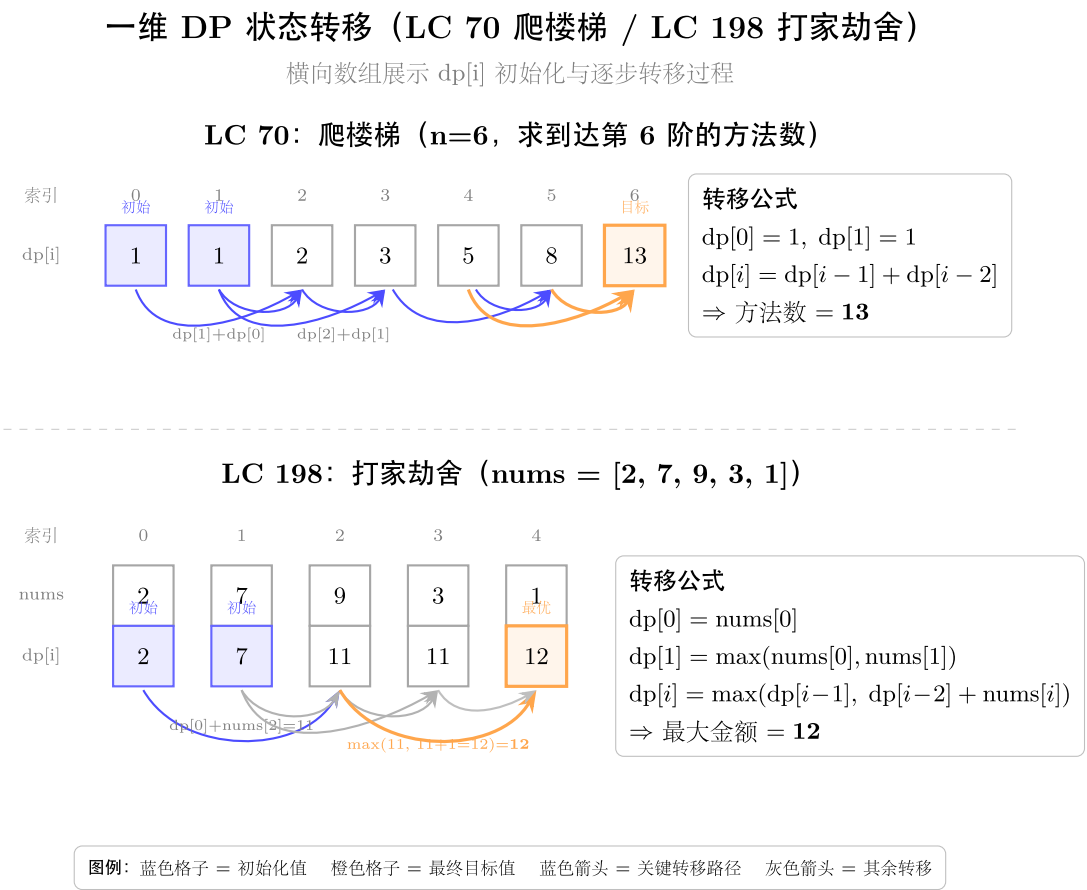


图 21: 爬楼梯/打家劫舍 dp 演化

图 LIS 最长上升子序列（LC 300）

抽象成方法（标准模板代码）

模板
1: 自
底向
上递
推
(标准
dp 数
组)
适用
所有
题;
以 70
爬楼
梯为
模型。
“python
from
typing
im-
port
List

```

def
climb-
ing_stairs_tabulation(n:
int) ->
int: """
    “70:
    自底
    向上
    递推。
    时间
    O(n),
    空间
    O(n)。”
    “”if n
    <= 2:
    return
    n dp =
    [0] *
    (n + 1)
    dp[1]
    = 1
    dp[2]
    = 2
    for i
    in
    range(3,
    n + 1):
    dp[i]
    = dp[i
    - 1] +
    dp[i -
    2] #
    状态
    转移
    方程
    return
    dp[n]
    """

```



图 22: dp 数组 + 跳跃箭头转移

模板 2: 自顶向下记忆化搜索 (@lru_cache)

适用 300、139 等有递归子问题结构的题。

```
from functools import lru_cache

def climbing_stairs_memo(n: int) -> int:
    """70: 自顶向下记忆化。时间  $O(n)$ , 空间  $O(n)$  (调用栈 + 缓存)。"""
    @lru_cache(maxsize=None)
    def dfs(i: int) -> int:
        if i <= 2:
            return i
        return dfs(i - 1) + dfs(i - 2)
    return dfs(n)
```

模板 3: 滚动数组优化 ($O(n) \rightarrow O(1)$ 空间)

适用套路 1: 转移只依赖 $dp[i-1]$ 和 $dp[i-2]$ (70 爬楼梯、198 打家劫舍)。

```
def rolling_array_template(n: int) -> int:
    """通用滚动数组骨架 (以爬楼梯为例)。时间  $O(n)$ , 空间  $O(1)$ 。"""
    if n <= 2:
        return n
    prev2, prev1 = 1, 2          #  $dp[i-2]$ ,  $dp[i-1]$ 
    for _ in range(3, n + 1):
        curr = prev1 + prev2     # 状态转移
        prev2, prev1 = prev1, curr
    return prev1

def house_robber_rolling(nums: List[int]) -> int:
    """198: 打家劫舍, 滚动数组。时间  $O(n)$ , 空间  $O(1)$ 。"""
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]
    prev2, prev1 = 0, 0
    for num in nums:
        curr = max(prev1, prev2 + num)
        prev2, prev1 = prev1, curr
    return prev1
```


套路 1: 线性状态转移 (70 / 198)

```
# 70: 爬楼梯 (标准 Fibonacci DP)
def climbStairs(n: int) -> int:
    """ 时间  $O(n)$ , 空间  $O(1)$  (滚动)。
         $dp[i] = dp[i-1] + dp[i-2]$ : 第  $i$  阶 = 从  $i-1$  阶走 1 步 + 从  $i-2$  阶走 2 步。
    """
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1

# 198: 打家劫舍
def rob(nums: List[int]) -> int:
    """ 时间  $O(n)$ , 空间  $O(1)$  (滚动)。
         $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$ : 不偷 vs 偷当前。
    """
    prev2 = prev1 = 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + num)
    return prev1
```

套路 2: LIS —— 单序列最长递增子序列 (300)

```
# 300: LIS  $O(n^2)$  版本
def lengthOfLIS_n2(nums: List[int]) -> int:
    """ 时间  $O(n^2)$ , 空间  $O(n)$ 。
         $dp[i]$  = 以  $nums[i]$  结尾的 LIS 长度, 初始为 1 (只含自身)。
    """
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
```

```

        dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

```

```
import bisect
```

300: LIS $O(n \log n)$ 版本 —— 贪心 + 二分

```

def lengthOfLIS(nums: List[int]) -> int:
    """ 时间  $O(n \log n)$ , 空间  $O(n)$ 。
        tails[k] = 所有长度为 k+1 的 LIS 中末尾元素的最小值。
        对每个 nums[i], 二分找 tails 中第一个 >= nums[i] 的位置并替换 (维持单调性)。
        tails 的长度即为答案。
    """
    tails: List[int] = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)  # 找第一个 >= num 的位置
        if pos == len(tails):
            tails.append(num)                # 扩展序列
        else:
            tails[pos] = num                  # 替换, 维护最小末尾
    return len(tails)

```

套路 3: 切割型 DP —— Word Break (139)

139: 单词拆分

```

def wordBreak(s: str, wordDict: List[str]) -> bool:
    """ 时间  $O(n^2 \cdot m)$  ( $n$  为  $s$  长度,  $m$  为最长单词长度), 空间  $O(n + \text{vert } D \text{ vert})$ 。
        dp[i] = s[:i] 能否被字典拆分。
        dp[0] = True (空串), 转移: 若 dp[j] 且 s[j:i] in word_set, 则 dp[i] = True。
    """
    word_set = set(wordDict)
    min_len = min(len(w) for w in wordDict)
    max_len = max(len(w) for w in wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(max(0, i - max_len), i - min_len + 1):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True

```

```
        break          # 已找到一种拆法，无需继续枚举 j
    return dp[n]
```

套路 4：完全背包——Coin Change（322）

```
# 322: 零钱兑换（完全背包）
def coinChange(coins: List[int], amount: int) -> int:
    """ 时间  $O(\text{amount} \cdot |\text{coins}|)$ ，空间  $O(\text{amount})$ 。
     $dp[i]$  = 凑出金额  $i$  所需的最少硬币数； $dp[0] = 0$ ，其余初始化为  $\infty$ 。
    完全背包：对每个  $i$  枚举每种硬币  $c$ ， $dp[i] = \min(dp[i], dp[i-c]+1)$ 。
    内层循环顺序（正向）允许同一硬币重复使用。
    """
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for c in coins:
            if i >= c and dp[i - c] != INF:
                dp[i] = min(dp[i], dp[i - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

速查表

题目	dp 定义	转移方程	空间	时间
70 Climbing Stairs	$dp[i]$ = 到第 i 阶的方法数	$dp[i-1] + dp[i-2]$	$O(1)$ 滚动	$O(n)$
198 House Robber	$dp[i]$ = 前 i 栋能偷的最大金额	$\max(dp[i-1], dp[i-2]+nums[i])$	$O(1)$ 滚动	$O(n)$
300 LIS	$dp[i]$ = 以 $nums[i]$ 结尾的 LIS 长度	$\max(dp[j]+1 \text{ for } nums[j] < nums[i])$	$O(n)$	$O(n^2) / O(n \log n)$
139 Word Break	$dp[i] = s[:i]$ 能否拆分（布尔）	$dp[j]$ and $s[j:i]$ in dict	$O(n)$	$O(n^2m)$
322 Coin Change	$dp[i]$ = 凑出金额 i 的最少硬币数	$\min(dp[i-c]+1 \text{ for each } c)$	$O(\text{amount})$	$O(\text{amount} \cdot \text{coins})$

方法变形 (4 类)

变形 1: 线性转移的泛化

- **70 (1/2 步)** → 若每次可走 $1 \sim k$ 步, 则 $dp[i] = \sum(dp[i-j] \text{ for } j \text{ in range}(1, k+1) \text{ if } i-j \geq 0)$, 时间 $O(nk)$ 。
- **198 (不相邻)** → **213 House Robber II (首尾相连)**: 拆成两个子问题 $[0..n-2]$ 和 $[1..n-1]$, 各跑一次 198, 取最大值。
- **740 Delete and Earn (非本 category)**: 等价于打家劫舍——先将 `nums` 转换为“取得值 i 的总分”的数组, 再跑 198。
- 本质: 任何“相邻互斥选择 + 最大化”结构 → 先确认状态只依赖前 $1 \sim 2$ 步, 再选滚动/数组。

变形 2: LIS 家族

- **300 LIS (严格递增)** → 改为非严格: `bisect_left` 换为 `bisect_right`。
- 最长不降子序列 (**LNDS**): 同上, 将 $<$ 改为 $\leq$ 。
- **354 Russian Doll Envelopes (非本 category)**: 先按宽升序、高降序排列, 再对高度跑 LIS——排序去掉“宽度相同不能嵌套”的冲突。
- **673 Number of LIS (非本 category)**: 维护 $dp[i]$ (LIS 长度) 和 $cnt[i]$ (该长度的方案数), 同时更新两个数组。

变形 3: 切割型 DP 扩展

- **139 Word Break I (能否拆分)** → **140 Word Break II (所有拆分方案, 非本 category)**: 用 $dp[i]$ 记录所有分割点, 再回溯重建路径, 时间指数级。
- **343 Integer Break (非本 category)**: $dp[i] = \max(j*(i-j), j*dp[i-j])$ for j in $\text{range}(2, i)$ ——切一刀后, 右侧可以继续切 (用 dp) 或不切 (直接乘)。
- 切割型通式: $dp[i] = f(dp[j], s[j:i])$ for $j < i$, 外层 i 循环, 内层 j 枚举切割点。

变形 4: 完全背包 vs 0/1 背包

- **322 Coin Change (完全背包, 可重复用)**: 内层 `coins` 循环正向枚举 `amount`。
- **416 Partition Equal Subset Sum (0/1 背包, 非本 category)**: 内层 `amount` 循环逆向枚举, 防止同一元素用两次。
- 区分关键: 完全背包内层正向 (允许重复), 0/1 背包内层逆向 (每个元素仅用一次)。

思考路标 (条件反射)

1. 看到“爬楼梯 / Fibonacci 类” → $dp[i] = dp[i-1] + dp[i-2]$, 两个变量滚动, $O(1)$ 空间

2. 看到“不能选相邻元素 + 最大化” → 打家劫舍: $dp[i] = \max(dp[i-1], dp[i-2] + val[i])$
3. 看到“最长递增子序列” → $O(n^2)$ DP 或 $O(n \log n)$ tails + bisect_left
4. 看到“字符串能否被字典拆分” → $dp[i]$ = 布尔, 枚举切割点 j , 查 set
5. 看到“最少硬币 / 无限取用 + 凑成目标” → 完全背包, 正向枚举 amount
6. 看到 转移只依赖 $dp[i-1]$ 和 $dp[i-2]$ → 立刻想到滚动数组, 空间 $O(n) \rightarrow O(1)$
7. 看到 递归有大量重叠子问题 → 加 @lru_cache, 自顶向下 = 自底向上同等效率
8. 看到 **dp** 初始化 $dp[0] = \text{True}$ 或 $dp[0] = 0$ → 空串/零金额是 DP 的边界锚点, 务必单独初始化
9. 看到 切割型 DP 的 j 范围可剪枝 → 只枚举 $s[j:i]$ 长度在 $[\text{min_len}, \text{max_len}]$ 之间的 j
10. 看到“Viterbi / HMM 解码”(AI 场景) → 本质是 DP, 状态 = 当前时刻 + 隐藏状态, 转移 = 发射概率 $\times$ 转移概率
11. 看到“序列标注 / NER / POS tagging” → CRF 的 Viterbi 解码 = Word Break 的概率版, $dp[i][\text{label}]$ 存最优路径
12. 看到“ $dp[i] = \text{inf}$ 且答案仍为 inf ” → 返回 -1 (322 Coin Change 的约定)

易错点

1. **dp 数组索引偏移**: $dp[i]$ 表示“前 i 个元素”时数组长度为 $n+1$, $dp[0]$ 是边界; 若混用“第 i 个元素”则索引相差 1, 导致越界或漏初始化。
2. 滚动变量赋值顺序: $\text{prev2}, \text{prev1} = \text{prev1}, \text{prev1} + \text{prev2}$ 中 Python 右侧同时求值, 安全; 若用临时变量则必须先算 $\text{curr} = \text{prev1} + \text{prev2}$ 再更新, 顺序反了会读到已改写的值。
3. **198 题初始化**: $\text{prev2} = \text{prev1} = 0$ (均为 0), 不是 $\text{nums}[0]$; 若初始化为 $\text{nums}[0]$, 循环逻辑需调整, 容易出 off-by-one。
4. **LIS 的 bisect_left vs bisect_right**: 严格递增用 bisect_left (找第一个 $\geq \text{num}$ 的位置), 非严格递增用 bisect_right (允许等于); 混用导致 LIS 长度错误。
5. **Word Break 的字典查询**: wordDict 是列表, 查询 $O(n)$; 必须先转为 set, 否则总时间从 $O(n^2)$ 变为 $O(n^2 \cdot |\text{dict}|)$, 大字典下超时。
6. **322 的 INF 初始化**: $dp[i] = \text{float}('inf')$; 若初始化为 -1 (常见误写), 后续 $dp[i-c] + 1$ 会算出 0, 错误地认为可凑成。
7. 完全背包内层方向: 枚举 coins 在外 (或枚举 amount 在外再枚举 coins 在内), 均为正向, 允许重用; 若内层 amount 逆向枚举则退化为 0/1 背包, 每枚硬币只能用一次。
8. **tails 数组的含义**: tails 不是 LIS 本身 (元素顺序不对应原数组), 只是维护各长度 LIS 末尾的最小值; 若需重建实际 LIS, 需额外维护 parent 数组。

典型应用例题

例 1: 198. House Robber

题目：一排房子，相邻不能同时偷，求能偷到的最大金额。

思路：对第 i 栋，两个选择：跳过（取 $dp[i-1]$ ）或偷（取 $dp[i-2] + \text{nums}[i]$ ）。转移只依赖两步前，滚动数组 $O(1)$ 空间。

解：

```
# 参考: solutions/dynamic_programming_1d/p198_house_robber.py
def rob(nums: List[int]) -> int:
    prev2 = prev1 = 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + num)
    return prev1
```

分析： $O(n)$ 时间， $O(1)$ 空间。prev2 对应“跳过两栋前的最优”，prev1 对应“跳过一栋前的最优”。每次循环后语义不变，循环结束后 prev1 即为覆盖全部房屋的最优解。

例 2: 300. Longest Increasing Subsequence

题目：给定整数数组，求最长严格递增子序列的长度。

思路（两种）：- $O(n^2)$ ：dp[i] 为以 $\text{nums}[i]$ 结尾的 LIS 长度，枚举所有 $j < i$ 且 $\text{nums}[j] < \text{nums}[i]$ 。- $O(n \log n)$ ：维护 tails，对每个新数用 bisect_left 找替换位置。

解（ $O(n \log n)$ ）：

```
# 参考: solutions/dynamic_programming_1d/p300_longest_increasing_subsequence.py
import bisect

def lengthOfLIS(nums: List[int]) -> int:
    tails: List[int] = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

分析: $O(n \log n)$, $O(n)$ 空间 (tails 最长为 n)。tails[k] 始终是所有长度为 $k+1$ 的递增子序列中末尾的最小值——这个贪心性质保证替换操作不会错过更长的 LIS。

例 3: 322. Coin Change

题目: 给定硬币面值数组 (可无限使用), 求凑成 amount 的最少硬币数; 无法凑成返回 -1。

思路: 完全背包。dp[i] = 凑成金额 i 的最少硬币数, dp[0] = 0, 其余 inf。对每个 i 枚举每种硬币 c: dp[i] = min(dp[i], dp[i-c] + 1)。

解:

```
# 参考: solutions/dynamic_programming_1d/p322_coin_change.py
def coinChange(coins: List[int], amount: int) -> int:
    INF = float('inf')
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for c in coins:
            if i >= c and dp[i - c] < INF:
                dp[i] = min(dp[i], dp[i - c] + 1)
    return dp[amount] if dp[amount] < INF else -1
```

分析: $O(\text{amount} \cdot |\text{coins}|)$ 时间, $O(\text{amount})$ 空间。内层正向枚举 (完全背包特征) 允许同一硬币多次使用。与 0/1 背包的唯一区别: 内层枚举方向为正向。

自测题

自测 1 (70 题 Climbing Stairs) —— $n=5$ 返回 8, $n=1$ 返回 1, $n=2$ 返回 2。提示: prev2, prev1 = 1, 2, 循环 range(3, n+1) 中 prev2, prev1 = prev1, prev1+prev2, 返回 prev1; $n \leq 2$ 直接返回 n。参考 solutions/dynamic_programming_1d/p070_climbing_stairs.py。

自测 2 (198 题 House Robber) —— nums=[2,7,9,3,1] 返回 12 (偷 2+9+1), nums=[1,2,3,1] 返回 4 (偷 1+3)。提示: prev2=prev1=0; 对每个 num, prev2, prev1 = prev1, max(prev1, prev2+num); 返回 prev1。参考 solutions/dynamic_programming_1d/p198_house_robber.py。

自测 3 (300 题 LIS) —— nums=[10,9,2,5,3,7,101,18] 返回 4 (LIS 为 [2,3,7,101])。提示: tails=[], 对每个 num 做 bisect_left(tails, num): 若 pos==len(tails) 则 append, 否则 tails[pos]=num; 返回 len(tails)。参考 solutions/dynamic_programming_1d/p300_longest_increasing_subsequence.py。

自测 4（139 题 Word Break）——`s="leetcode"`，`wordDict=["leet","code"]` 返回 `True`；`s="catsandog"`，`wordDict=["cats","dog","sand","and","cat"]` 返回 `False`。提示：`word_set=set(wordDict)`，`dp=[False]*(n+1)`，`dp[0]=True`；双重循环，`dp[j]` and `s[j:i]` in `word_set` 则 `dp[i]=True`。参考 `solutions/dynamic_programming_1d/p139_word_break.py`。

自测 5（322 题 Coin Change）——`coins=[1,5,11]`，`amount=15` 返回 3（`5+5+5` 比 `11+1+1+1+1` 少），`coins=[2]`，`amount=3` 返回 -1。提示：`dp=[inf]*(amount+1)`，`dp[0]=0`；双重循环 `dp[i]=min(dp[i], dp[i-c]+1)`；`dp[amount]` 仍为 `inf` 则返回 -1。参考 `solutions/dynamic_programming_1d/p322_coin_change.py`。

题目全览（5 题）

#	题目	套路分类	难度
70	Climbing Stairs	线性转移 + 滚动数组	Easy
198	House Robber	线性转移 + 滚动数组	Medium
300	Longest Increasing Subsequence	LIS， $O(n \log n)$ 贪心 + 二分	Medium
139	Word Break	切割型 DP，字典 set	Medium
322	Coin Change	完全背包	Medium

融合版说明

段	来源	价值
一例速记	本文件	5 题 4 类套路一览 + DP 三问框架 + AI 关联
思维路径还原	本文件	5 道题的解题内心独白，含状态定义决策
抽象成方法	本文件	4 个标准模板（自底向上/记忆化/滚动/套路专项）+ 速查表
方法变形	本文件	4 类变体（线性/LIS/切割/背包）及其扩展题
思考路标	本文件	12 条题型识别条件反射，含 AI/ML 场景
易错点	本文件	8 条高频踩坑（索引偏移/滚动顺序/INF 初始化等）
典型应用例题	<code>solutions/</code>	3 道精讲（198、300、322），代码 + 正确性分析
自测题	<code>leetcode</code>	5 题带提示，链接 <code>solutions</code> 文件

段	来源	价值
题目全览	本文件	5 题完整列表，套路分类一览

跨 **category** 导航：- 多维 DP（双序列 / grid / 状态机）→ 见 11-dp-multidim.md - 回溯 + 记忆化 = DP 的等价形式：把回溯参数作为 key 加入 cache → 见 08-backtracking.md - 二分搜索在 LIS $O(n \log n)$ 优化中的应用 → 见 04-binary-search.md (lower_bound 模板) - Viterbi 算法（HMM/CRF 解码）是 DP 在概率图模型中的核心应用，与 Word Break 结构同源

11 — Dynamic Programming Multidimensional (融合版)

难度：□□□□□ 题数：9 核心套路：二维 DP、双序列对齐、状态机 DP、区间 DP、滚动行压缩 本文件：覆盖 dynamic_programming_multidimensional 9 题的算法套路总结 + 典型题精讲 + 自测

一例速记

DP 三问 (多维版)：□ 状态 $dp[i][j]$ 含义（两个维度各自代表什么）□ 转移（当前格如何从上/左/左上推出）□ 初始化（第 0 行/列的边界）**63/64 grid path**： $dp[i][j]$ = 到达 (i,j) 的路径数/最小代价，从 $dp[i-1][j]$ 和 $dp[i][j-1]$ 推出，滚动行压缩 $O(n)$ 空间 **120 三角形**：自底向上， $dp[i][j] = \min(dp[i+1][j], dp[i+1][j+1]) + triangle[i][j]$ ；原地修改或额外 $O(n)$ 数组 **72 编辑距离**： $dp[i][j]$ = word1[:i] 到 word2[:j] 的最少操作数；字符匹配则继承左上角，否则取三邻 +1 **97 交错字符串**： $dp[i][j] = s1[:i]$ 和 $s2[:j]$ 能否交错组成 $s3[:i+j]$ ；两个转移分支取 OR **5 最长回文子串**：区间 DP， $dp[i][j] = s[i..j]$ 是否为回文；也可中心扩展 $O(n)$ 空间 **123/188 股票多次交易**：状态机 DP，状态 = (天数, 已完成交易次数, 是否持股)，转移为买入/持有/卖出三条边 **221 最大正方形**： $dp[i][j]$ = 以 (i,j) 为右下角的最大全 1 正方形边长； $\min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ **AI 关联**：BLEU 评分（LCS/n-gram 匹配）= 编辑距离的变体；DTW（动态时间规整）= grid path DP；Needleman-Wunsch = 编辑距离在生物信息学的应用

思维路径还原

“看到 **63 Unique Paths II**：带障碍物的 $m \times n$ 网格，求左上到右下的不同路径数 → $dp[i][j]$ = 到达 (i,j) 的路径数；障碍物格子设为 0，跳过不更新。初始化第 0 行/列：遇到障碍则后续全 0（障碍物挡住整行/列）。转移： $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ （障碍格直接为 0）。滚动优化：只用一维数组， $dp[j] += dp[j-1]$ ，原地更新同一行。

看到 **64 Minimum Path Sum**: $m \times n$ 网格每格有代价, 求左上到右下的最小代价路径 $\rightarrow dp[i][j] = \min(dp[i-1][j], dp[i][j-1]) + grid[i][j]$ 。第 0 行只能从左来, 第 0 列只能从上来, 单独初始化前缀和。滚动: 一维数组, $dp[j] = \min(dp[j], dp[j-1]) + grid[i][j]$, 左 $dp[j]$ 是更新后的当前行, $dp[j-1]$ 同行左邻。

看到 **120 Triangle**: 三角形自顶向下, 找最小路径和 $\rightarrow$ 自底向上更省心: $dp[i][j] = \min(dp[i+1][j], dp[i+1][j+1]) + triangle[i][j]$, 最终 $dp[0][0]$ 即答案。可直接原地修改 `triangle`, $O(1)$ 额外空间。

看到 **72 Edit Distance**: 两个字符串的最少操作数 (增删改) $\rightarrow dp[i][j] = word1[:i]$ 变为 $word2[:j]$ 的最少操作数。若 $word1[i-1] == word2[j-1]$: $dp[i][j] = dp[i-1][j-1]$ (不需操作); 否则: $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ (删/插/替换)。初始化: $dp[0][j] = j$ (空串变为 $word2[:j]$ 需插入 j 次), $dp[i][0] = i$ 。

看到 **97 Interleaving String**: 判断 $s3$ 是否由 $s1$ 和 $s2$ 交错组成 $\rightarrow dp[i][j] = s1[:i]$ 和 $s2[:j]$ 能否交错组成 $s3[:i+j]$ 。转移: $dp[i][j] = (dp[i-1][j] \text{ and } s1[i-1] == s3[i+j-1]) \text{ or } (dp[i][j-1] \text{ and } s2[j-1] == s3[i+j-1])$ 。若 $\text{len}(s1) + \text{len}(s2) != \text{len}(s3)$ 直接返回 `False`。

看到 **5 Longest Palindromic Substring**: 找最长回文子串 $\rightarrow$ 区间 DP: $dp[i][j] = s[i..j]$ 是否为回文; 枚举子串长度 l , $dp[i][j] = (s[i] == s[j]) \text{ and } (l \leq 2 \text{ or } dp[i+1][j-1])$ 。时间 $O(n^2)$ 空间 $O(n^2)$; 中心扩展可降至 $O(1)$ 空间, 同样 $O(n^2)$ 时间。

看到 **123 Best Time III**: 最多 2 笔交易, 求最大利润 $\rightarrow$ 状态机 DP: 4 个状态 (持 1 / 卖 1 / 持 2 / 卖 2), 每天更新; 也可拆成两次 121 题 (前缀最大、后缀最大相加)。

看到 **188 Best Time IV**: 最多 k 笔交易 $\rightarrow dp[t][0/1] =$ 完成 t 笔交易后不持股/持股的最大利润; 外层枚举 t , 内层枚举天数; 若 $k \geq n/2$ 则等价于无限次 (贪心)。

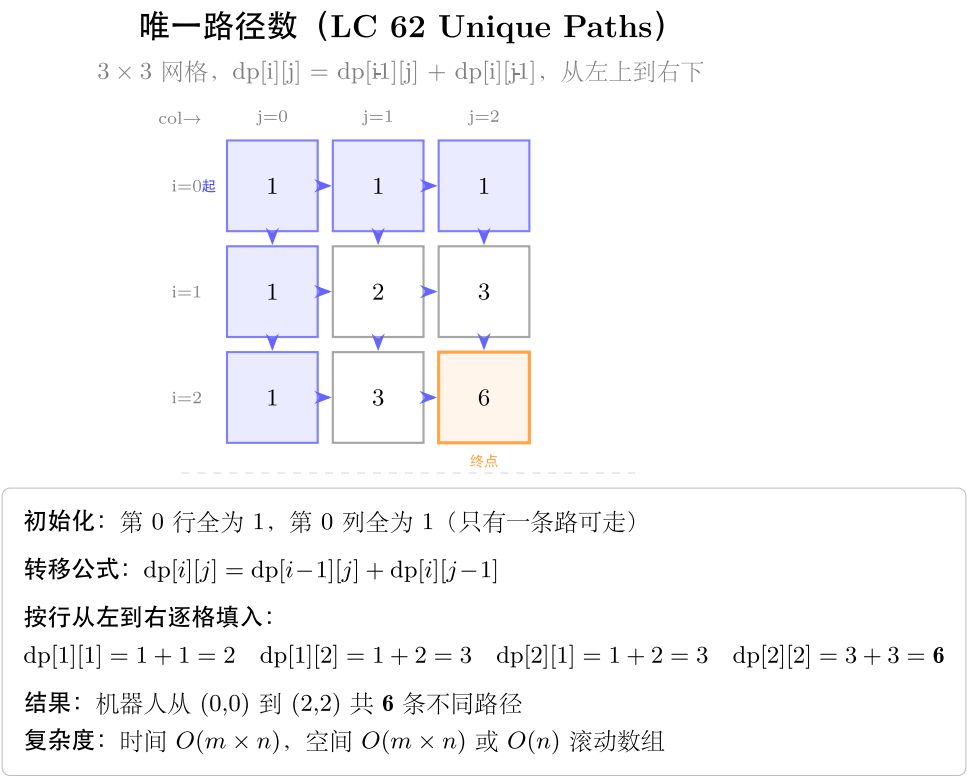
看到 **221 Maximal Square**: 二值矩阵中最大全 1 正方形面积 $\rightarrow dp[i][j] =$ 以 (i,j) 为右下角的最大正方形边长; $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ (当 $matrix[i][j] == '1'$); 答案 $= \max(dp[i][j])^2$ 。”

学习目标

- 掌握二维 DP 的初始化方法 (第 0 行/列的前缀积/前缀和) 及滚动行压缩
- 理解双序列对齐 DP (编辑距离 / 交错字符串) 的二维状态机写法
- 掌握区间 DP (最长回文子串) 的枚举顺序: 先枚举长度, 再枚举起点
- 掌握状态机 DP (股票多次交易): 明确区分“状态 = 系统属性”和“决策 = 边”
- 理解最大正方形的“木桶原理”递推, 能从几何直觉推导转移方程
- 能识别“滚动行优化”的适用条件 (当前行只依赖上一行), 将 $O(mn)$ 空间降至 $O(n)$
- 理解 DTW / BLEU / Needleman-Wunsch 与编辑距离/LCS 的关联

几何示意

图二维路径 DP (LC 62)



模板
1: 二
维 DP
grid
(自底
向上
递推)
适
用: 63
Unique
Paths
II、64
Mini-
mum
Path
Sum
“python
from
typing
im-
port
List

```

def
grid_dp_template(grid:
List[List[int]])
-> int:
    """二
    维
    grid
    DP 骨
    架
    (以最
    小路
    径和
    为
    例)。
    时间
    O(mn),
    空间
    O(mn)。”
    """m, n
    =
    len(grid),
    len(grid[0])
    dp =
    [[0] *
    n for _
    in
    range(m)]
    # —
    初始
    化边
    界—
    dp[0][0]
    =
    grid[0][0]
    for j
    in
    range(1,
    n):
    dp[0][j]
    =
    dp[0][j
    - 1] +
    grid[0][j]
    # 第 0
    行

```

编辑距离 (LC 72 Edit Distance)

"horse" → "ros", dp[i][j] = 最少操作数 (插入/删除/替换)

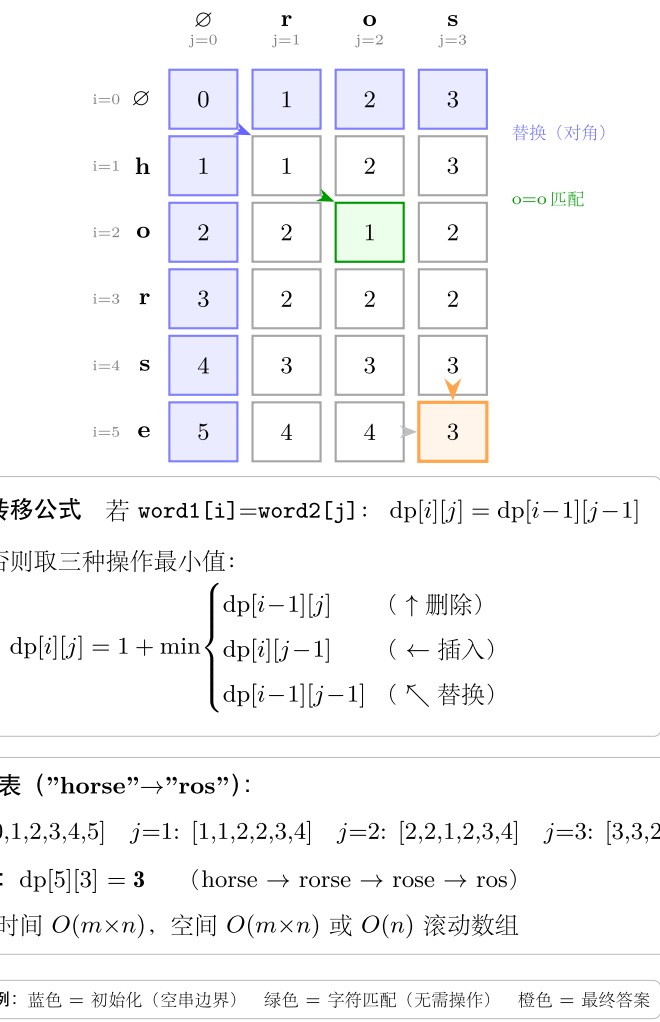


图 24: 6x4 dp 表 + min 三方向

最长公共子序列 LCS (LC 1143)

s1="abcde", s2="ace", dp[i][j] = LCS 长度, 红色 = 回溯路径

	∅ j=0	a j=1	c j=2	e j=3
i=0 ∅	0	0	0	0
i=1 a	0	1	1	1
i=2 b	0	1	1	1
i=3 c	0	1	2	2
i=4 d	0	1	2	2
i=5 e	0	1	2	3

转移公式 若 s1[i]=s2[j]: dp[i][j] = dp[i-1][j-1] + 1
否则: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
回溯: 从 dp[5][3] 沿红色箭头追踪对角匹配, 还原 LCS = "ace"

完整 dp 表:
j=0: [0,0,0,0,0,0] j=1: [0,1,1,1,1,1] j=2: [0,1,1,2,2,2] j=3: [0,1,1,2,2,3]
LCS = 3, 公共子序列为 "ace" (红色路径: a@(1,1), c@(3,2), e@(5,3))
复杂度: 时间 O(m×n), 空间 O(m×n) 或 O(n) 滚动数组

图例: 蓝色 = 初始化 (空串) 红色格 + 箭头 = 回溯匹配路径 橙色 = 最终答案格

图 25: 6x4 dp 表 + 回溯路径

模板 2: 滚动行压缩 ($O(mn) \rightarrow O(n)$ 空间)

适用: 63、64、221 (凡是”当前格只依赖上一行 + 当前行左侧”的题)

```
def rolling_row_template(grid: List[List[int]]) -> int:
    """ 滚动行优化: 只保留一行 dp。时间  $O(mn)$ , 空间  $O(n)$ 。 """
    m, n = len(grid), len(grid[0])
    dp = [0] * n
    # 初始化第 0 行
    dp[0] = grid[0][0]
    for j in range(1, n):
        dp[j] = dp[j - 1] + grid[0][j]
    # 从第 1 行开始滚动更新
    for i in range(1, m):
        dp[0] += grid[i][0]          # 第 0 列只能从上来
        for j in range(1, n):
            # dp[j] 此刻仍是上一行的值 (来自上方), dp[j-1] 是本行左侧 (已更新)
            dp[j] = min(dp[j], dp[j - 1]) + grid[i][j]
    return dp[n - 1]
```

模板 3: 自顶向下记忆化 (双序列)

适用: 72 Edit Distance、97 Interleaving String

```
from functools import lru_cache

def edit_distance_memo(word1: str, word2: str) -> int:
    """ 72: 自顶向下记忆化。时间  $O(mn)$ , 空间  $O(mn)$ 。 """
    m, n = len(word1), len(word2)

    @lru_cache(maxsize=None)
    def dp(i: int, j: int) -> int:
        if i == 0:
            return j          # word1 前 0 个字符变为 word2[:j], 需插入 j 次
        if j == 0:
            return i          # word2 前 0 个字符, 需删除 i 次
        if word1[i - 1] == word2[j - 1]:
            return dp(i - 1, j - 1)
        return 1 + min(dp(i - 1, j),          # 删除 word1[i-1]
                       dp(i, j - 1),          # 插入 word2[j-1]
                       dp(i - 1, j - 1))      # 替换
```



```
return dp(m, n)
```

套路 1: 二维 grid DP (63 / 64 / 120)

63: 带障碍的不同路径数

```
def uniquePathsWithObstacles(obstacleGrid: List[List[int]]) -> int:
    """ 时间  $O(mn)$ , 空间  $O(n)$  (滚动行)。
         $dp[j]$  = 到达当前行第  $j$  列的路径数; 障碍格置 0 并停止更新。
    """
    m, n = len(obstacleGrid), len(obstacleGrid[0])
    dp = [0] * n
    # 初始化第 0 行: 遇到障碍后全为 0
    dp[0] = 1 if obstacleGrid[0][0] == 0 else 0
    for j in range(1, n):
        dp[j] = dp[j - 1] if obstacleGrid[0][j] == 0 else 0
    # 逐行更新
    for i in range(1, m):
        if obstacleGrid[i][0] == 1:
            dp[0] = 0
        for j in range(1, n):
            dp[j] = 0 if obstacleGrid[i][j] == 1 else dp[j] + dp[j - 1]
    return dp[n - 1]
```

64: 最小路径和

```
def minPathSum(grid: List[List[int]]) -> int:
    """ 时间  $O(mn)$ , 空间  $O(n)$  (滚动行)。
         $dp[j]$  = 到达当前行第  $j$  列的最小路径和。
    """
    m, n = len(grid), len(grid[0])
    dp = [0] * n
    dp[0] = grid[0][0]
    for j in range(1, n):
        dp[j] = dp[j - 1] + grid[0][j]
    for i in range(1, m):
        dp[0] += grid[i][0]
        for j in range(1, n):
            dp[j] = min(dp[j], dp[j - 1]) + grid[i][j]
    return dp[n - 1]
```

120: 三角形最小路径和 (自底向上)

```
def minimumTotal(triangle: List[List[int]]) -> int:
    """ 时间  $O(n^2)$ , 空间  $O(n)$  (原地修改最底行, 逐层向上)。
         $dp[j]$  = 从底层到当前位置的最小路径和; 自底向上避免正向的路径选择歧义。
    """
    dp = triangle[-1][:]          # 拷贝最后一行作为初始状态
    for i in range(len(triangle) - 2, -1, -1):
        for j in range(len(triangle[i])):
            dp[j] = min(dp[j], dp[j + 1]) + triangle[i][j]
    return dp[0]
```

套路 2: 双序列对齐 DP (72 / 97)

72: 编辑距离 (自底向上, 空间 $O(n)$)

```
def minDistance(word1: str, word2: str) -> int:
    """ 时间  $O(mn)$ , 空间  $O(n)$  (滚动行)。
         $dp[j]$  =  $word1[:i]$  变为  $word2[:j]$  的最少操作数。
        关键: 需提前保存  $dp[i-1][j-1]$  (左上角) 再更新  $dp[j]$ 。
    """
    m, n = len(word1), len(word2)
    dp = list(range(n + 1))          # 初始化:  $dp[0][j] = j$ 
    for i in range(1, m + 1):
        prev = dp[0]                  #  $prev = dp[i-1][j-1]$  (左上角)
        dp[0] = i                     #  $dp[i][0] = i$ 
        for j in range(1, n + 1):
            temp = dp[j]              # 保存  $dp[i-1][j]$  供下次迭代用作  $prev$ 
            if word1[i - 1] == word2[j - 1]:
                dp[j] = prev
            else:
                dp[j] = 1 + min(prev,      # 替换 (左上)
                                dp[j],      # 删除 (上方, 即旧的  $dp[i-1][j]$ )
                                dp[j - 1])  # 插入 (左方, 即新的  $dp[i][j-1]$ )
            prev = temp
    return dp[n]
```

97: 交错字符串

```
def isInterleave(s1: str, s2: str, s3: str) -> bool:
```

```

""" 时间  $O(mn)$ , 空间  $O(n)$  (滚动行)。
 $dp[j]$  =  $s1[:i]$  和  $s2[:j]$  能否交错组成  $s3[:i+j]$ 。
两个分支: 来自  $s1$  (上方) 或来自  $s2$  (左方)。
"""
m, n, l = len(s1), len(s2), len(s3)
if m + n != l:
    return False
dp = [False] * (n + 1)
dp[0] = True
for j in range(1, n + 1):
    dp[j] = dp[j - 1] and s2[j - 1] == s3[j - 1]
for i in range(1, m + 1):
    dp[0] = dp[0] and s1[i - 1] == s3[i - 1]
    for j in range(1, n + 1):
        dp[j] = ((dp[j] and s1[i - 1] == s3[i + j - 1]) or
                  (dp[j - 1] and s2[j - 1] == s3[i + j - 1]))
return dp[n]

```

套路 3: 区间 DP —— 最长回文子串 (5)

5: 最长回文子串 (区间 DP)

```

def longestPalindrome_dp(s: str) -> str:
    """ 时间  $O(n^2)$ , 空间  $O(n^2)$ 。
     $dp[i][j]$  =  $s[i..j]$  是否为回文; 枚举子串长度  $l$ , 再枚举起点  $i$ 。
    """
    n = len(s)
    dp = [[False] * n for _ in range(n)]
    start, max_len = 0, 1
    # 单字符必然是回文
    for i in range(n):
        dp[i][i] = True
    # 子串长度从 2 开始枚举
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                dp[i][j] = (length == 2) or dp[i + 1][j - 1]
                if dp[i][j] and length > max_len:
                    start, max_len = i, length
    return s[start: start + max_len]

```

5: 最长回文子串 (中心扩展, $O(1)$ 空间)

```
def longestPalindrome(s: str) -> str:
    """ 时间  $O(n^2)$ , 空间  $O(1)$ 。每个字符 (奇) / 每两个相邻字符 (偶) 作为中心向外扩展。 """
    n = len(s)
    start, max_len = 0, 1

    def expand(l: int, r: int) -> None:
        nonlocal start, max_len
        while l >= 0 and r < n and s[l] == s[r]:
            l -= 1
            r += 1
        length = r - l - 1
        if length > max_len:
            start, max_len = l + 1, length

    for i in range(n):
        expand(i, i)          # 奇数长度
        expand(i, i + 1)      # 偶数长度

    return s[start: start + max_len]
```

套路 4: 状态机 DP —— 股票买卖 (123 / 188)

123: 至多 2 笔交易 (4 状态机)

```
def maxProfit_k2(prices: List[int]) -> int:
    """ 时间  $O(n)$ , 空间  $O(1)$ 。
    4 个状态: buy1 (第一次买入后最大净值)、sell1 (第一次卖出后)、
            buy2 (第二次买入后)、sell2 (第二次卖出后)。
    每天用当日价格更新 4 个状态, 转移对应买入/持有/卖出。
    """
    buy1 = buy2 = float('-inf')
    sell1 = sell2 = 0
    for price in prices:
        buy1 = max(buy1, -price)          # 第一次买入: 花费 price
        sell1 = max(sell1, buy1 + price)  # 第一次卖出
        buy2 = max(buy2, sell1 - price)   # 第二次买入 (依赖 sell1)
        sell2 = max(sell2, buy2 + price)  # 第二次卖出
    return sell2
```

188: 至多 k 笔交易 (通用状态机)

```
def maxProfit(k: int, prices: List[int]) -> int:
    """ 时间  $O(kn)$ , 空间  $O(k)$ 。
     $k \geq n/2$  时等价于无限次交易 (贪心);
    否则  $buy[t]$ 、 $sell[t]$  分别为完成  $t$  次交易时持股/不持股的最大利润。
    """
    n = len(prices)
    if k >= n // 2:
        # 无限次交易: 贪心累加上升段
        return sum(max(0, prices[i] - prices[i - 1]) for i in range(1, n))
    buy = [float('-inf')] * (k + 1)
    sell = [0] * (k + 1)
    for price in prices:
        for t in range(1, k + 1):
            buy[t] = max(buy[t], sell[t - 1] - price)
            sell[t] = max(sell[t], buy[t] + price)
    return sell[k]
```

套路 5: 最大正方形 (221)

221: 最大全 1 正方形

```
def maximalSquare(matrix: List[List[str]]) -> int:
    """ 时间  $O(mn)$ , 空间  $O(n)$  (滚动行)。
     $dp[j]$  = 以  $(i, j)$  为右下角的最大全 1 正方形边长。
    转移 (木桶原理):  $dp[i][j] = \min(\text{上方}, \text{左方}, \text{左上方}) + 1$ 。
    滚动时需额外保存左上角  $prev$  (= 更新前的  $dp[j]$ )。
    """
    if not matrix:
        return 0
    m, n = len(matrix), len(matrix[0])
    dp = [0] * (n + 1)
    max_side = 0
    for i in range(m):
        prev = 0
        # 对应  $dp[i-1][j-1]$  (左上角)
        for j in range(1, n + 1):
            temp = dp[j]
            # 保存  $dp[i-1][j]$  供下轮循环作为  $prev$ 
            if matrix[i][j - 1] == '1':
                dp[j] = min(dp[j],
                            # 上方
                            dp[j - 1],
                            # 左方
                            prev) + 1
            prev = temp
    return max_side
```

```
        prev) + 1    # 左上角
    max_side = max(max_side, dp[j])
else:
    dp[j] = 0
prev = temp
return max_side * max_side
```

速查表

题目	dp 定义	空间（优化后）	时间	关键转移
63 Unique Paths II	到达 (i,j) 的路径数	O(n) 滚动行	O(mn)	dp[j] + dp[j-1], 障碍置 0
64 Min Path Sum	到达 (i,j) 的最小代价	O(n) 滚动行	O(mn)	min(dp[j], dp[j-1]) + grid[i][j]
120 Triangle	从底到当前的最小路径	O(n) 单行	O(n²)	min(dp[j], dp[j+1]) + tri[i][j]
72 Edit Distance	word1[:i] 变 word2[:j] 的操作数	O(n) 滚动行	O(mn)	匹配继承左上, 否则三邻 +1
97 Interleaving	s1[:i], s2[:j] 能否交错成 s3[:i+j]	O(n) 滚动行	O(mn)	上方 OR 左方
5 Palindrome	s[i..j] 是否回文 (或中心扩展)	O(n²) 或 O(1)	O(n²)	两端相等 + 内部回文
123 Stock k=2	4 状态机: buy1/sell1/buy2/sell2	O(1)	O(n)	逐天更新 4 个状态
188 Stock k=k	buy[t]/sell[t] 第 t 次持/不持股	O(k)	O(kn)	逐天逐交易数更新
221 Max Square	以 (i,j) 为右下角的最大正方形边长	O(n) 滚动行	O(mn)	min(上, 左, 左上) + 1

方法变形 (4 类)

变形 1: 二维 grid DP 系列

- **63** (有障碍) $\rightarrow$ **62** (无障碍, 非本 category): 无障碍时 $dp[i][j] = C(m+n-2, m-1)$, 组合数公式 $O(1)$; 62 仍用 DP 更直观。
- **64** (最小路径和) $\rightarrow$ **174** (地下城 (Dungeon Game), 非本 category): 从终点向起点做 DP, $dp[i][j] = \max(1, \min(dp[i+1][j], dp[i][j+1]) - \text{dungeon}[i][j])$ 。
- **120** (三角形) $\rightarrow$ 自底向上 vs 自顶向下: 自顶向下需要记录每一行的状态 (或用滚动), 自底向上每次只用下一行, 代码更简洁。
- **DTW** (动态时间规整) (AI 场景): $dp[i][j] = \text{dist}(s1[i], s2[j]) + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$, 与编辑距离同构, 用于序列对比 (语音识别、时序数据)。

变形 2: 双序列对齐系列

- **72 Edit Distance** $\rightarrow$ **1143 LCS** (最长公共子序列, 非本 category): $dp[i][j] = dp[i-1][j-1] + 1$ (匹配) 或 $\max(dp[i-1][j], dp[i][j-1])$ (不匹配)。
- **97 Interleaving** $\rightarrow$ 若 $s3$ 长度不等于 $s1+s2$ 则直接 False, 这是快速过滤的必要前置检查。
- **LCS 与 BLEU 的关系** (AI 场景): BLEU 分数 = n -gram 精确度的几何平均 $\times$ 简洁惩罚, n -gram 匹配本质是 LCS 在 n -gram 级别的推广; MT 系统评价中大量使用。
- **Needleman-Wunsch** (生物信息学): 编辑距离 + 权重 (不同替换代价不同) = 序列比对算法, dp 框架完全相同。

变形 3: 状态机 DP 系列

- **121** (1 次交易) $\rightarrow$ **122** (无限次) $\rightarrow$ **123** (2 次) $\rightarrow$ **188** (k 次): 状态机维度随 k 变化, $k=1$ 时退化为只需追踪 `min_price`; 无限次时退化为贪心。
- **309 Cooldown** (非本 category): 状态机增加“冷却”状态, 每次卖出后必须等 1 天; 三个状态 (持股/卖出当天/冷却)。
- **714 With Fee** (非本 category): 买入时多扣一次手续费, $\text{sell} = \max(\text{sell}, \text{buy} + \text{price} - \text{fee})$ 。
- 状态机设计原则: 明确区分“状态” (系统当前处于什么情况) 和“决策” (今天做什么), 每条“决策边”对应一个转移方程, 状态数量决定空间复杂度。

变形 4: 区间 DP / 形状 DP

- **5 Longest Palindrome** $\rightarrow$ **516 Longest Palindromic Subsequence** (非本 category) : $dp[i][j] = dp[i+1][j-1] + 2$ (两端匹配) 或 $\max(dp[i+1][j], dp[i][j-1])$ (不匹配)。
- **221 Max Square** $\rightarrow$ **85 Maximal Rectangle** (非本 category): 以每行为底, 维护高度直方图, 再用单调栈; 221 的木桶转移方程是 85 的特殊化。
- 区间 DP 枚举顺序: 必须先枚举区间长度 (从小到大), 再枚举起点, 确保计算 $dp[i][j]$ 时 $dp[i+1][j-1]$ 已就绪; 若枚举顺序错误则访问未初始化的格。

- **221 木桶原理**: 正方形由左方、上方、左上方三个方向“同时约束”, 最短那条边决定最大正方形边长, $\min(\text{三方向})$ 直接体现了这一几何约束。

思考路标 (条件反射)

1. 看到“**grid** + 只能向右/向下 + 计数或最优值” → 二维 DP, $dp[i][j]$ 从上方和左方推出, 滚动行 $O(n)$ 空间
2. 看到“有障碍的路径计数” → 障碍格置 0 且不更新, 初始化时遇到障碍后续全 0
3. 看到“三角形 / 从外到内层层收缩” → 自底向上 DP, $dp[j] = \min(dp[j], dp[j+1]) + val$
4. 看到“两个字符串 + 最少操作 / 编辑” → 编辑距离, $dp[i][j]$ 依赖三个方向 (上/左/左上)
5. 看到“**s3** 是否由 **s1, s2** 交错构成” → 交错字符串 DP, 先检查长度, 再二维 DP 或滚动行
6. 看到“最长回文子串 (返回子串本身)” → 中心扩展 $O(1)$ 空间; 若需统计所有回文则用区间 DP
7. 看到“股票 + 至多 **k** 次交易” → 状态机 DP, 维护 $buy[t] / sell[t]$; $k \geq n/2$ 时贪心
8. 看到“买卖股票 + 最多 **2** 次” ($k=2$) → 4 状态变量 ($buy1/sell1/buy2/sell2$), $O(1)$ 空间逐日更新
9. 看到“二值矩阵 + 最大全 1 正方形面积” → 221 木桶 DP, $\min(\text{上}, \text{左}, \text{左上})+1$, 答案取平方
10. 看到“ $dp[i][j]$ 依赖 $dp[i-1][j-1]$ ” → 滚动行时需额外变量 $prev$ 保存左上角 (否则已被覆盖)
11. 看到“区间 DP / 子串回文” → 枚举长度从 1 到 n , 确保子问题先于父问题计算
12. 看到“**BLEU / DTW / Needleman-Wunsch**” (AI 场景) → 对应 LCS/grid DP/编辑距离, 调权重或对齐窗口即可适配

易错点

1. 滚动行中左上角丢失: $dp[i][j]$ 依赖 $dp[i-1][j-1]$; 滚动后更新 $dp[j]$ 之前, 旧的 $dp[j](=dp[i-1][j])$ 会被覆盖, 而 $dp[i-1][j-1]$ 也随之丢失。必须在进入内层循环之前用 $prev$ 保存, 并在循环体内先 $temp = dp[j]$, 处理后 $prev = temp$ 。
2. **63 题第 0 行初始化中断**: 第 0 行遇到障碍后, 后续格子全为 0 (障碍挡住了唯一路径); 初始化时若未及时截断, 会错误地传递路径数。
3. **72 题三方向顺序**: $\min(prev, dp[j], dp[j-1])$ 中 $prev$ = 左上角 (替换), $dp[j]$ = 上方 (删除), $dp[j-1]$ = 左方 (插入); 混淆方向导致错误的操作语义。
4. **97 题长度检查**: $\text{len}(s1) + \text{len}(s2) \neq \text{len}(s3)$ 时直接返回 False, 否则下标计算 $s3[i+j-1]$ 会越界。
5. **5 题区间 DP 枚举顺序**: 外层必须枚举长度 $length$ (从 2 到 n), 内层枚举起点 i ; 若外层枚举 i 、内层枚举 j , 则 $dp[i+1][j-1]$ 尚未填充, 读到错误的初始值 (False)。
6. **123 题 buy2 依赖 sell1 的顺序**: 每天应按 $buy1 \rightarrow sell1 \rightarrow buy2 \rightarrow sell2$ 顺序更新; 若调换 $buy2$ 和 $sell1$ 的更新顺序, $buy2$ 会使用当天刚更新的 $sell1$, 逻辑上允许同一天内多次操作 (不合题意)。
7. **188 题 k 过大时的退化**: $k \geq n // 2$ 时等价于无限次交易, 不退化处理则 $buy/sell$ 数组大小为 $O(k)$, k 可能达到 10^9 导致 MLE; 必须提前做贪心处理。
8. **221 题答案取平方**: max_side 存的是边长, $面积 = max_side^2$; 直接返回 max_side 是常见笔误。

典型应用例题

例 1: 72. Edit Distance

题目：给定 word1 和 word2，求将 word1 转为 word2 所需的最少操作数（插入/删除/替换各算 1 步）。

思路：dp[i][j] = word1[:i] 变为 word2[:j] 的最少操作数。字符匹配时不需操作（继承左上角），否则三邻取 min 后 +1。初始化 dp[i][0] = i, dp[0][j] = j。滚动行 O(n) 空间，需 prev 保存左上角。

解：

参考：solutions/dynamic_programming_multidimensional/p072_edit_distance.py

```
def minDistance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = list(range(n + 1))
    for i in range(1, m + 1):
        prev = dp[0]
        dp[0] = i
        for j in range(1, n + 1):
            temp = dp[j]
            if word1[i - 1] == word2[j - 1]:
                dp[j] = prev
            else:
                dp[j] = 1 + min(prev, dp[j], dp[j - 1])
            prev = temp
    return dp[n]
```

分析：O(mn) 时间，O(n) 空间。prev 是进入当前列之前的 dp[j] (= 上一行同列的值，即 dp[i-1][j-1] 的角色)，这是滚动行中保存左上角的标准写法。

例 2: 123. Best Time to Buy and Sell Stock III

题目：最多完成 2 笔交易（同一时刻最多持有一股），求最大利润。

思路：4 状态机。buy1 = 第一次买入后的最大净值，sell1 = 第一次卖出后，buy2 = 第二次买入后，sell2 = 第二次卖出后。每天价格都尝试“更新”每个状态，相当于选择今天是否操作。

解：

参考：solutions/dynamic_programming_multidimensional/p123_best_time_to_buy_and_sell_stock_iii.py

```
def maxProfit(prices: List[int]) -> int:
```

```

buy1 = buy2 = float('-inf')
sell1 = sell2 = 0
for price in prices:
    buy1 = max(buy1, -price)
    sell1 = max(sell1, buy1 + price)
    buy2 = max(buy2, sell1 - price)
    sell2 = max(sell2, buy2 + price)
return sell2

```

分析: $O(n)$ 时间, $O(1)$ 空间。初始化 $buy1 = buy2 = -inf$ 表示“尚未发生第一/二次买入”, $sell1 = sell2 = 0$ 表示未卖出时利润为 0。每次循环按固定顺序更新, 保证 $buy2$ 使用的是“本次循环之前”的 $sell1$, 不会在同一天内完成买卖。

例 3: 221. Maximal Square

题目: 给定二值字符矩阵, 求最大全 1 正方形的面积。

思路: $dp[i][j]$ = 以 (i,j) 为右下角的最大全 1 正方形边长。若 $matrix[i][j] == '0'$ 则 $dp[i][j] = 0$; 否则 $dp[i][j] = \min(\text{上}, \text{左}, \text{左上}) + 1$ 。木桶原理: 三方向的最小值决定了可以形成的正方形大小。

解:

```

# 参考: solutions/dynamic_programming_multidimensional/p221_maximal_square.py
def maximalSquare(matrix: List[List[str]]) -> int:
    m, n = len(matrix), len(matrix[0])
    dp = [0] * (n + 1)
    max_side, prev = 0, 0
    for i in range(m):
        for j in range(1, n + 1):
            temp = dp[j]
            if matrix[i][j - 1] == '1':
                dp[j] = min(dp[j], dp[j - 1], prev) + 1
                max_side = max(max_side, dp[j])
            else:
                dp[j] = 0
            prev = temp
    return max_side * max_side

```

分析: $O(mn)$ 时间, $O(n)$ 空间 (滚动行)。prev 对应左上角 $dp[i-1][j-1]$, 在 j 循环开始前保存旧的 $dp[j]$, 在本列更新后移至下一列时充当新的左上角。

自测题

自测 1 (63 题 Unique Paths II) ——obstacleGrid=[[0,0,0],[0,1,0],[0,0,0]] 返回 2 (障碍在中心)。提示: 一维 dp, 障碍格直接置 0; 第 0 行遇障后续全 0; 每行滚动 dp[j] += dp[j-1] (障碍则 dp[j]=0)。参考 solutions/dynamic_programming_multidimensional/p063_unique_paths_ii.py。

自测 2 (64 题 Minimum Path Sum) ——grid=[[1,3,1],[1,5,1],[4,2,1]] 返回 7 (路径 1→3→1→1→1)。提示: 一维 dp, 初始化第 0 行前缀和; 滚动时 dp[0] += grid[i][0], dp[j] = min(dp[j], dp[j-1]) + grid[i][j]。参考 solutions/dynamic_programming_multidimensional/p064_minimum_path_sum.py。

自测 3 (72 题 Edit Distance) ——word1="horse", word2="ros" 返回 3; word1="", word2="abc" 返回 3。提示: dp=list(range(n+1)); 外层 i 循环中 prev=dp[0]; dp[0]=i; 内层匹配时 dp[j]=prev, 否则 dp[j]=1+min(prev,dp[j],dp[j-1]);循环末 prev=temp。参考 solutions/dynamic_programming_multidimensional/p072_edit_distance.py。

自测 4 (123 题 Stock III) ——prices=[3,3,5,0,0,3,1,4] 返回 6 (买 0 卖 3 + 买 1 卖 4); prices=[1,2,3,4,5] 返回 4。提示: buy1=buy2=-inf; sell1=sell2=0; 每天按序更新 4 个状态; 返回 sell2。参考 solutions/dynamic_programming_multidimensional/p123_stock_iii.py。

自测 5 (221 题 Maximal Square) ——matrix=[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","1","1","1","1"]] 返回 4 (2×2 正方形, 面积=4)。提示: dp=[0]*(n+1); prev=0; 内层 j: temp=dp[j]; '1' 则 dp[j]=min(dp[j],dp[j-1],prev)+1 则 dp[j]=0; prev=temp; 答案 max_side²。参考 solutions/dynamic_programming_multidimensional/p221_maximal_square.py。

题目全览 (9 题)

#	题目	套路分类	难度
5	Longest Palindromic Substring	区间 DP / 中心扩展	Medium
63	Unique Paths II	二维 grid DP, 障碍处理	Medium
64	Minimum Path Sum	二维 grid DP, 最小代价	Medium
72	Edit Distance	双序列对齐 DP	Medium
97	Interleaving String	双序列对齐 DP	Medium
120	Triangle	自底向上 DP, 原地优化	Medium
123	Best Time to Buy and Sell Stock III	状态机 DP (4 状态)	Hard
188	Best Time to Buy and Sell Stock IV	状态机 DP (通用 k)	Hard
221	Maximal Square	二维 DP, 木桶原理	Medium

融合版说明

段	来源	价值
一例速记	本文件	9 题 5 类套路一览 + AI 关联 (BLEU/DTW)
思维路径还原	本文件	9 道题的解题内心独白，含状态定义推导过程
抽象成方法	本文件	5 个标准模板（grid/滚动行/记忆化/双序列/状态机/区间/最大正方形）+ 速查表
方法变形	本文件	4 类变体（grid/双序列/状态机/区间）及 AI 应用
思考路标	本文件	12 条题型识别条件反射，含滚动行/状态机/区间枚举陷阱
易错点	本文件	8 条高频踩坑（左上角丢失/枚举顺序/状态机顺序/k 过大退化）
典型应用例题	solutions/	3 道精讲（72、123、221），代码 + 正确性分析
自测题	leetcode	5 题带提示，链接 solutions 文件
题目全览	本文件	9 题完整列表，套路分类一览

跨 category 导航：- 一维 DP（线性转移 / LIS / 完全背包）→ 见 10-dp-1d.md - 回溯加记忆化 = DP 的等价形式，当搜索树有大量重叠子问题时加 @lru_cache → 见 08-backtracking.md - 股票系列状态机与有限自动机（DFA）的联系：状态 = 自动机节点，交易决策 = 转移边，DP = 在 DFA 上找最优路径 - BLEU 评分（MT 评价）、DTW（语音/时序对齐）、Needleman-Wunsch（生物序列比对）均是本节 grid DP / 编辑距离的直接工程应用

12 —Linked List（融合版）

难度：□□□□□ 题数：11 核心套路：dummy 头节点、快慢指针、迭代/递归翻转、合并、LRU 缓存
本文件：覆盖 linked_list 11 题的算法套路总结 + 典型题精讲 + 自测

一例速记

dummy 头节点（哨兵）：在真正的头节点前加一个值为 0 的假头 dummy，使”对头节点的操作”和”对中间节点的操作”写法统一，消除 null 检查（21 合并 / 82 去重 / 86 分区 / 2 两数相加）**快慢指针**：slow 每次走 1 步，fast 每次走 2 步（或 n 步）；用于找中点（奇数个节点 slow 停在正中，偶数个停在前半末尾）、检测环（141）、找倒数第 k 个节点（19）**迭代翻转**：三指针 prev=None，cur=head，

每步 `nxt=cur.next; cur.next=prev; prev=cur; cur=nxt` (92 区间翻转 / 25 k 组翻转) 递归翻转: `tail = reverse(head.next); head.next.next = head; head.next = None`, 尾节点变新头 (92 / 25) 合并: 21 两路合并 = 比较两头取小, $O(m+n)$; 23 k 路合并 = 最小堆或分治 (分治 = 两两归并, $O(n \log k)$) **LRU 缓存** (146): 哈希表 (key→node) + 双向链表 (按访问顺序), get/put 均 $O(1)$ **AI 关联**: streaming 数据窗口 / Token 流中间层可视为带指针的链表; LRU = KV cache 逐出策略 (vLLM paged attention 的核心思想)

思维路径还原

“看到链表题 头节点可能被修改 (如删除头 / 分区 / 合并) → 立刻加 dummy 头节点: `dummy = ListNode(0); dummy.next = head`, 操作完返回 `dummy.next`, 永远不用单独处理头节点是否为 None 的情况。

看到‘找链表 midpoint’ → 快慢指针: `slow = fast = head, while fast and fast.next`, 循环结束时 `slow` 停在中点 (奇数个节点停在正中, 偶数个停在前半末)。876 是纯找中点, 翻转链表的第二步也用这个。

看到 **141 Linked List Cycle** (判断有环) → 快慢指针: 若有环, `fast` 终究追上 `slow` (数学上追及问题); 若无环, `fast` 先到 None。时间 $O(n)$, 空间 $O(1)$ 。

看到 **19 Remove Nth Node From End** → 快慢指针, `fast` 先走 `n` 步, 然后 `fast, slow` 同步走, `fast.next == None` 时 `slow` 正好在倒数第 `n+1` 个节点, 执行 `slow.next = slow.next.next` 删除目标。配合 dummy 处理删除头节点的情形。

看到 翻转整个链表 (**206 基础题**) → 三指针迭代: `prev=None, cur=head`; 循环 `nxt=cur.next; cur.next=prev; prev=cur; cur=nxt`; 结束时 `prev` 是新头。

看到 **92 Reverse Linked List II** (翻转 `[left, right]` 区间) → 先走到 `left-1` 的位置 (用 dummy 避免越界), 记录 `tail_of_left = cur` (翻转后它会变成区间尾), 对 `right - left + 1` 个节点执行内层翻转循环, 最后重新接线。

看到 **25 Reverse Nodes in K-Group** → 每次取 `k` 个, 检查剩余是否有 `k` 个 (不足则停止), 翻转 `k` 个, 接回主链, `prev` 指针移到翻转后区间的末尾 (即原来的头), 继续。

看到 **21 Merge Two Sorted Lists** → dummy 头 + 比较两头取小, $O(m+n)$ 线性合并; 23 Merge K Sorted Lists → 最小堆 (`heapq.merge` 或 `heap` 维护 `k` 个头) $O(n \log k)$, 或分治: 两两合并 $\log k$ 轮, 每轮 $O(n)$, 总 $O(n \log k)$ 。

看到 **2 Add Two Numbers** → dummy 头 + 模拟逐位相加, 注意进位 `carry`; 两条链表不等长时短的补 0; 最后 `carry != 0` 时补一个新节点。

看到 **61 Rotate List** → 先算链表长度 `n`, `k = k % n`; 尾节点接回头构成环, 从新头位置 (倒数第 `k` 个的前一个) 断开。快慢指针找断点。

看到 **82 Remove Duplicates from Sorted List II** → dummy + 双指针: prev 指向”确认非重复”的最后一个节点, cur 向前试探; 若发现重复 (`cur.val == cur.next.val`), 跳过所有同值节点, 然后 `prev.next = cur.next` (跳过整个重复块)。

看到 **86 Partition List** → dummy + 两条子链: less_dummy 收集 $< x$ 的节点, greater_dummy 收集 $\geq x$ 的节点, 最后将 less 链的尾接到 greater 链头, 返回 less_dummy.next。

看到 **138 Copy List with Random Pointer** → 三步法: □ 将每个节点的复制节点插到它的后面 (形成 $A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow \dots$); □ 复制 random 指针 (`node.next.random = node.random.next`); □ 拆分两条链表, 还原原链表。O(n) 时间, O(1) 额外空间 (不算输出)。或哈希表: old→new 映射, 两次遍历, O(n) 空间。

看到 **146 LRU Cache** → 哈希表 {key: node} + 双向链表 (表头 = 最近访问, 表尾 = 最久未访问); get: 命中则移到表头, 返回值; put: 已存在则更新并移到表头; 不存在则插入表头, 若超容量则删除表尾节点及其哈希表条目。”

学习目标

- 掌握 dummy 头节点技巧, 能识别何时必须使用 (头节点可能被删除 / 插入位置在头部)
 - 熟练写快慢指针模板: 找中点、找环、找倒数第 n 个
 - 能徒手写链表区间翻转 (迭代版) 并正确接线, 尤其是 92 和 25 的”接线”步骤
 - 理解 23 题的两种解法 (堆 vs 分治) 并能分析复杂度
 - 掌握 LRU Cache 的哈希表 + 双向链表设计, 能在 15 分钟内手写完整实现
 - 理解 138 三步法的核心思路 (利用原链表结构编码 random 关系)
-

几何示意

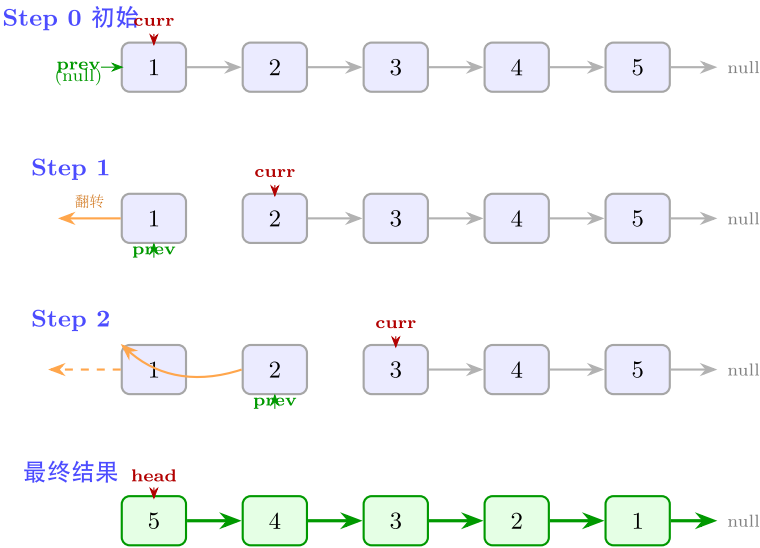
图链表反转 (LC 206)

图快慢指针找中点 (LC 876)

图 LRU 缓存 (LC 146)

反转链表 (LC 206 Reverse Linked List)

三指针迭代法: prev / curr / next, 逐步翻转箭头方向



三指针迭代 ($O(n)$ 时间, $O(1)$ 空间):

1. 初始化 $prev = null$, $curr = head$
2. 每步: $next = curr.next$ (保存后继)
 $curr.next = prev$ (翻转箭头)
 $prev = curr$ (prev 前移)
 $curr = next$ (curr 前移)
3. $curr == null$ 时, $prev$ 即为新头节点

图 26: prev/curr/next 三指针 4 步

链表中间 (LC 876 Middle of Linked List)

slow (红) 每次走 1 步, fast (蓝) 每次走 2 步; fast 到末尾时 slow 在中间

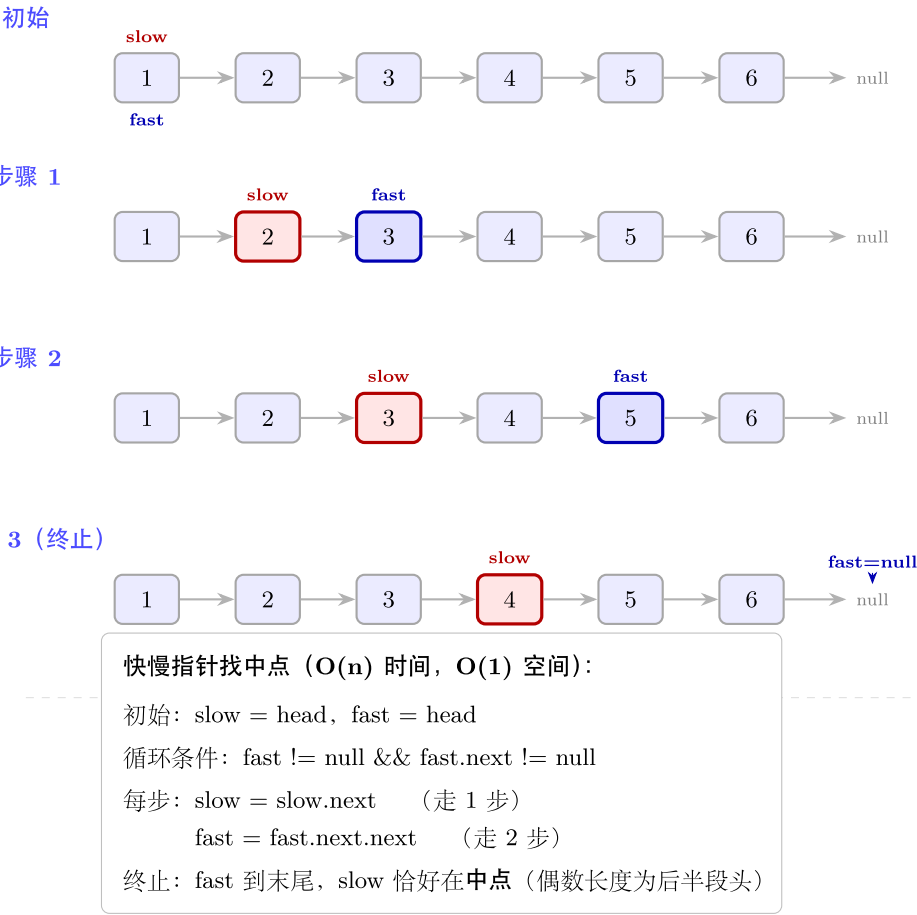
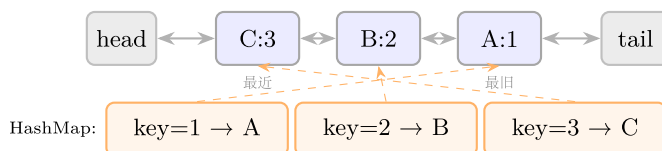


图 27: 6 节点链表 slow/fast 追及

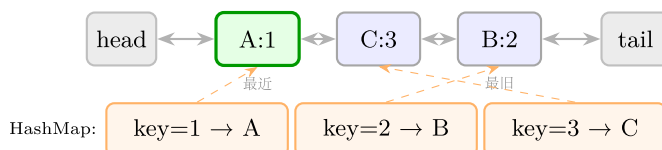
LRU 缓存 (LC 146 LRU Cache)

哈希表 + 双向链表：最近使用移到链表头，淘汰从链表尾移除

快照 1：初始（容量 = 3）



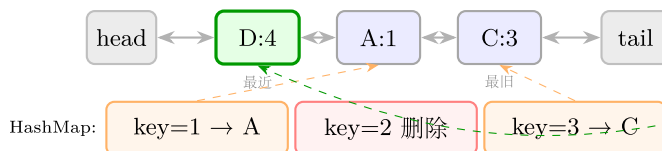
快照 2：get(1) → 移 A 到头



get(1):

1. HashMap 查 key=1
2. 找到节点 A
3. 从原位置摘除
4. 插到 head 后

快照 3：put(4,v) → 满，淘汰 B，插入 D



put(4,v):

1. size==capacity
2. 淘汰 tail 前节点 B
3. HashMap 删 key=2
4. 新建节点 D
5. 插到 head 后

数据结构：HashMap<key, Node> + 带哨兵头尾的双向链表

get(key): O(1) 查表 → 节点移到 head 后 → 返回 val (未找到返回 -1)

put(key, val): O(1) 若 key 存在则更新并移头；否则新建节点插头；
若超容量则删 tail 前节点并从 HashMap 移除对应 key

图 28: 哈希表 + 双向链表 + 3 操作

抽象方法
(标准模板代码)

ListN-ode
定义
(所有链表题共用)
“python
from
future
import
annotations
from
typing
import
Optional

```

class
ListN-
ode:
def
init(self,
val:
int =
0,
next:
Optional[ListNode]
=
None):
self.val
= val
self.next
= next
““

```

套路 1: dummy 头节点 (哨兵)

适用题: 2、19、21、82、86、92、138

```

def dummy_head_template(head: Optional[ListNode]) -> Optional[ListNode]:
    """ 任何可能修改头节点的操作, 先加 dummy。 """
    dummy = ListNode(0)
    dummy.next = head
    cur = dummy          # cur 从 dummy 出发, 操作 cur.next
    while cur.next:
        # ... 各种操作 (插入 / 删除 / 分区 / 进位)
        cur = cur.next
    return dummy.next    # 返回真正的头节点

```

关键: dummy 的 next 始终指向链表真实头, 操作完直接返回 dummy.next, 无需处理头节点特殊情况。

套路 2: 快慢指针

适用题: 141 (环检测)、19 (倒数第 n 个)、876 (找中点, 非本 category 但同模板)

141: 判断链表是否有环

```
def hasCycle(head: Optional[ListNode]) -> bool:
    """ 时间  $O(n)$ , 空间  $O(1)$ 。快指针追上慢指针 有环。 """
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

19: 删除倒数第 n 个节点 (配合 *dummy*)

```
def removeNthFromEnd(head: Optional[ListNode], n: int) -> Optional[ListNode]:
    """ 时间  $O(L)$ , 空间  $O(1)$ 。fast 先走  $n$  步, 然后同步走, fast.next=None 时 slow 在目标前一位。 """
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n):          # fast 先走  $n$  步
        fast = fast.next
    while fast.next:           # 同步走直到 fast 到达最后一个节点
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next  # 删除目标节点
    return dummy.next
```

找链表中点 (通用)

```
def find_middle(head: Optional[ListNode]) -> Optional[ListNode]:
    """ 奇数个节点: slow 停在正中; 偶数个节点: slow 停在前半末尾。 """
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

套路 3: 迭代翻转 (标准版 + 区间版)

适用题: 92 (区间翻转)、25 (k 组翻转)

翻转整个链表 (基础, 206 题)

```
def reverseList(head: Optional[ListNode]) -> Optional[ListNode]:
    """ 时间  $O(n)$ , 空间  $O(1)$ 。三指针原地翻转。 """
    prev, cur = None, head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev  # prev 是新头
```

92: 翻转 $[left, right]$ 区间

```
def reverseBetween(head: Optional[ListNode], left: int, right: int) -> Optional[ListNode]:
    """ 时间  $O(n)$ , 空间  $O(1)$ 。dummy + 定位 + 区间翻转 + 接线。 """
    dummy = ListNode(0, head)
    pre = dummy
    for _ in range(left - 1):  # pre 走到 left-1 位置
        pre = pre.next
    # tail_of_left 翻转后会变成区间尾
    tail_of_left = pre.next
    prev, cur = None, pre.next
    for _ in range(right - left + 1):  # 翻转 right-left+1 个节点
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    # 接线: pre -> (新头 = prev) -> (中间) -> (tail_of_left = 区间尾) -> cur (后续节点)
    pre.next = prev
    tail_of_left.next = cur
    return dummy.next
```

25: 每 k 个节点一组翻转

```
def reverseKGroup(head: Optional[ListNode], k: int) -> Optional[ListNode]:
    """ 时间  $O(n)$ , 空间  $O(1)$ 。每次检查剩余是否有  $k$  个, 再原地翻转。 """
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        # 检查剩余是否有  $k$  个
        kth = group_prev
```

```

for _ in range(k):
    kth = kth.next
    if not kth:
        return dummy.next
group_next = kth.next # 下一组的开头
# 翻转 [group_prev.next, kth]
prev, cur = group_next, group_prev.next
while cur is not group_next:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
# 接线
tmp = group_prev.next # 翻转后变成组尾 (原来的组头)
group_prev.next = kth # group_prev → kth (翻转后的新组头)
group_prev = tmp      # 移动 group_prev 到组尾, 准备下一组

```

套路 4: 递归翻转 (用于 92 / 25 的递归版思路)

```

def reverseList_recursive(head: Optional[ListNode]) -> Optional[ListNode]:
    """ 递归翻转整个链表。时间  $O(n)$ , 空间  $O(n)$  (递归栈)。"""
    if not head or not head.next:
        return head
    new_head = reverseList_recursive(head.next)
    head.next.next = head # 让 head.next (原来的下一个) 反指 head
    head.next = None     # 断开原来的 head→next 指向
    return new_head

```

迭代版优先 ($O(1)$ 空间), 递归版用于理解“翻转的递归拆解”思路。

套路 5: 合并有序链表

适用题: 21 (两路)、23 (k 路分治)

```

from typing import Optional, List
import heapq

```

21: 合并两个有序链表

```

def mergeTwoLists(l1: Optional[ListNode],
                  l2: Optional[ListNode]) -> Optional[ListNode]:
    """ 时间  $O(m+n)$ , 空间  $O(1)$  (不含输出)。 """
    dummy = ListNode(0)
    cur = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            cur.next = l1
            l1 = l1.next
        else:
            cur.next = l2
            l2 = l2.next
        cur = cur.next
    cur.next = l1 or l2  # 把剩余那条直接接上
    return dummy.next

# 23: 合并 k 个有序链表 (分治版)
def mergeKLists(lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    """ 分治: 两两合并,  $\log k$  轮, 每轮  $O(n)$ , 总  $O(n \log k)$ 。 """
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            l1 = lists[i]
            l2 = lists[i + 1] if i + 1 < len(lists) else None
            merged.append(mergeTwoLists(l1, l2))
        lists = merged
    return lists[0]

```

套路 6: LRU Cache (哈希表 + 双向链表)

适用题: 146

```

class DLinkedNode:
    """ 双向链表节点。 """
    def __init__(self, key: int = 0, val: int = 0):
        self.key = key
        self.val = val

```

```

self.prev: Optional[DLinkedListNode] = None
self.next: Optional[DLinkedListNode] = None

```

```
class LRUCache:
```

```
    """146: LRU Cache。get / put 均 O(1)。
```

```
    数据结构：哈希表 (key→node) + 双向链表 (头 = 最近访问，尾 = 最久未访问)。
```

```
    头尾各设一个哨兵节点，避免边界判断。
```

```
    """
```

```
def __init__(self, capacity: int):
```

```
    self.cap = capacity
```

```
    self.cache: dict[int, DLinkedListNode] = {}
```

```
    # 哨兵头尾
```

```
    self.head = DLinkedListNode()
```

```
    self.tail = DLinkedListNode()
```

```
    self.head.next = self.tail
```

```
    self.tail.prev = self.head
```

```
# ---- 内部辅助 ----
```

```
def _remove(self, node: DLinkedListNode) -> None:
```

```
    """ 从双向链表中删除节点。 """
```

```
    node.prev.next = node.next
```

```
    node.next.prev = node.prev
```

```
def _add_to_head(self, node: DLinkedListNode) -> None:
```

```
    """ 将节点插入到哨兵头的后面 (最近访问位置)。 """
```

```
    node.prev = self.head
```

```
    node.next = self.head.next
```

```
    self.head.next.prev = node
```

```
    self.head.next = node
```

```
def _move_to_head(self, node: DLinkedListNode) -> None:
```

```
    self._remove(node)
```

```
    self._add_to_head(node)
```

```
def _remove_tail(self) -> DLinkedListNode:
```

```
    """ 删除并返回最久未使用的节点 (尾哨兵的 prev)。 """
```

```
    node = self.tail.prev
```

```
    self._remove(node)
```

```
    return node
```



```

# ---- 公开接口 ----
def get(self, key: int) -> int:
    if key not in self.cache:
        return -1
    node = self.cache[key]
    self._move_to_head(node)    # 访问后移到头
    return node.val

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        node = self.cache[key]
        node.val = value
        self._move_to_head(node)
    else:
        node = DLinkedNode(key, value)
        self.cache[key] = node
        self._add_to_head(node)
        if len(self.cache) > self.cap:
            tail = self._remove_tail()
            del self.cache[tail.key]    # 同步删除哈希表条目

```

套路 7：三步法复制随机指针链表

适用题：138

```

class RandomNode:
    def __init__(self, val: int = 0,
                 next: Optional['RandomNode'] = None,
                 random: Optional['RandomNode'] = None):
        self.val = val
        self.next = next
        self.random = random

def copyRandomList(head: Optional[RandomNode]) -> Optional[RandomNode]:
    """O(n) 时间, O(1) 额外空间 (不算输出)。
    步骤:
    1. 在每个节点后插入它的副本: A→A'→B→B'→...
    2. 设置副本的 random: node.next.random = node.random.next
    3. 拆分两条链表, 恢复原链表。

```

```
"""
if not head:
    return None
# 步骤 1: 插入副本节点
cur = head
while cur:
    copy = RandomNode(cur.val, cur.next)
    cur.next = copy
    cur = copy.next
# 步骤 2: 复制 random 指针
cur = head
while cur:
    if cur.random:
        cur.next.random = cur.random.next
    cur = cur.next.next
# 步骤 3: 拆分
cur = head
new_head = head.next
while cur:
    copy = cur.next
    cur.next = copy.next
    if copy.next:
        copy.next = copy.next.next
    cur = cur.next
return new_head
```

速查表

题型特征	套路	时间	空间
头节点可能被修改	dummy 头节点	$O(n)$	$O(1)$
找中点 / 判环 / 倒数第 n	快慢指针	$O(n)$	$O(1)$
翻转整段 / 区间	三指针迭代	$O(n)$	$O(1)$
翻转 k 组	迭代 + 接线	$O(n)$	$O(1)$
合并两个有序链表	dummy + 比较取小	$O(m+n)$	$O(1)$
合并 k 个有序链表	分治两两归并	$O(n \log k)$	$O(\log k)$
LRU Cache	哈希表 + 双向链表	$O(1)$	$O(\text{cap})$
复制带随机指针链表	三步法（穿插 + 复制 + 拆分）	$O(n)$	$O(1)$

方法变形 (4 类)

变形 1: 快慢指针的扩展

- **141 (判断有环) → 142 Linked List Cycle II (找环的入口)**: 快慢指针相遇后, 将 `slow` 重置到 `head`, `fast` 留在相遇点, 两者同速走, 再次相遇处即为环入口 (Floyd 判圈定理)。
- **找中点 (876) → 将 `fast` 初始从 `head.next` 出发**, 则偶数时 `slow` 停在前半末尾; 用于链表中间分割。
- **倒数第 `n` 个 (19) → 泛化**: `fast` 先走 `k` 步, 同步走后 `slow` 在倒数第 `k+1` 个节点, 可删除 / 插入。

变形 2: 翻转系列

- **206 (翻转整个) → 92 (翻转区间) → 25 (k 组翻转)**: 难度递进, 但核心三指针翻转子程序不变; 差异在“定位区间入口”和“接线”步骤。
- **234 Palindrome Linked List (非本 category)**: 找中点 → 翻转后半 → 逐节点比较 → 还原后半 (若需保持结构)。
- **递归翻转**: 在 LeetCode 25 题中递归版比迭代版更简洁但空间 $O(n/k)$, 面试首选迭代。

变形 3: 合并 / 拆分

- **21 (两路合并) 是 23 (k 路合并) 的基础子程序**; 分治时每轮调用 $\log k$ 次 21 的逻辑。
- **86 Partition List**: 不是合并, 而是“分裂”——将原链表按阈值分成两条, 再拼接; 与合并逻辑对称。
- **82 Remove Duplicates II**: 去掉所有出现过重复的节点 (不保留任何一个); 区别于 83 (保留一个)。

变形 4: LRU 扩展到 LFU

- **146 LRU**: 双向链表按“最近访问时间”排序 → 逐出最旧的。
- **460 LFU Cache (非本 category)**: 按“访问频率”排序, 同频率按 LRU 规则; 用两个哈希表 (`key→node` 和 `freq→有序链表`) + 维护 `min_freq` 变量, `get/put` 仍 $O(1)$ 。
- **AI 场景**: vLLM 的 Paged Attention 用 LRU 决定 KV Cache 的 page 逐出; Transformer 推理的 KV cache 管理本质是 LRU/LFU 变体。

思考路标 (条件反射)

1. 看到“**head** 可能被删除 / 插入在最前面” → 立刻加 `dummy`, 返回 `dummy.next`
2. 看到“**找中点 / 判环 / 倒数第 `k` 个**” → 快慢指针, `fast` 走 2 步 `slow` 走 1 步
3. 看到“**翻转 / reverse**” → 三指针迭代: `prev, cur = None, head`; 循环 `nxt=cur.next; cur.next=prev; prev=cur; cur=nxt`
4. 看到“**区间翻转 [`l, r`]**” → 先定位到 `l-1` (用 `dummy`), 记录区间两端, 翻转后重新接线 (4 根指针)

5. 看到“每 k 个翻转”→先检查剩余够不够 k 个，不够则停止；再翻转，再移动 `group_prev`
6. 看到“合并 k 个有序链表”→分治（两两合并） $O(n \log k)$ ，优于朴素 $O(nk)$
7. 看到“ **$O(1)$ get/put 缓存 + 最近最少使用**”→LRU：哈希表 + 双向链表 + 头尾哨兵
8. 看到“复制带 **random** 指针”→三步法：穿插副本 → 复制 `random` → 拆分（ $O(1)$ 空间）；或哈希表（ $O(n)$ 空间）
9. 看到“两数相加 / 进位”→dummy 头 + 逐位相加 `carry`，最后检查 `carry != 0` 是否需加节点
10. 看到“旋转链表 k 位”→计算长度 n ， $k \% n$ ；接尾到头成环，从 $n-k$ 处断开

易错点

1. 快慢指针的初始化：`slow = fast = head` 还是 `slow = head; fast = head.next`？偶数节点找“前半末”用后者，找“后半头”用前者。统一格式：`while fast and fast.next` 是安全写法。
2. 区间翻转接线顺序（92）：翻转后原头变成区间尾（`tail_of_left`），必须先 `pre.next = prev`（接新头），再 `tail_of_left.next = cur`（接后续）；顺序反了会成环或断链。
3. 25 题检查够不够 k 个：`kth` 走 k 步若遇到 `None` 则直接 `return dummy.next`，不要翻转残余。
4. LRU 删除节点忘记同步哈希表：`_remove_tail()` 删链表节点后，必须 `del self.cache[tail.key]`，否则哈希表和链表不一致。
5. 138 拆分步骤的顺序：拆分时先处理 `cur.next = copy.next`（恢复原链表），再处理 `copy.next = copy.next.next`（提取副本链），不能颠倒，否则 `copy.next` 丢失。
6. 82 去重 II 漏掉末尾重复块：用 `while cur.next and cur.val == cur.next.val` 跳过重复，循环结束后 `prev.next = cur.next`（跳过 `cur` 本身），不是 `prev.next = cur`。
7. 2 题进位在最终节点：两条链表都遍历完后，若 `carry == 1` 需再 `append(ListNode(1))`，常见漏写。
8. 61 旋转 k 步取模： $k = k \% n$ ；若 $k == 0$ 可提前返回 `head`； n 要先遍历一次链表计算，并记住尾节点以便构成环。

典型应用例题

例 1：92. Reverse Linked List II

题目：翻转链表从位置 `left` 到 `right` 的节点（1-indexed）。

思路：加 dummy 头，先走 `left-1` 步到达区间前驱节点 `pre`；记录 `tail_of_left = pre.next`（翻转后它会成为区间尾），然后对 `right - left + 1` 个节点执行标准三指针翻转；最后 `pre.next = prev`（接新头），`tail_of_left.next = cur`（接后续）。

解：

参考：[solutions/linked_list/p092_reverse_linked_list_ii.py](#)

```
def reverseBetween(head: Optional[ListNode], left: int, right: int) -> Optional[ListNode]:
```

```

dummy = ListNode(0, head)
pre = dummy
for _ in range(left - 1):
    pre = pre.next
tail_of_left = pre.next
prev, cur = None, pre.next
for _ in range(right - left + 1):
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
pre.next = prev
tail_of_left.next = cur
return dummy.next

```

分析： $O(n)$ 时间， $O(1)$ 空间。pre 走 $\text{left}-1$ 步 + 翻转 $\text{right}-\text{left}+1$ 步 = 最多 right 步，且每个节点只访问一次。

例 2: 146. LRU Cache

题目：设计 LRU 缓存，容量为 capacity ，get/put 均要求 $O(1)$ 。

思路： $O(1)$ get $\rightarrow$ 哈希表。 $O(1)$ 维护访问顺序 + $O(1)$ 逐出最旧 $\rightarrow$ 双向链表（插头删尾 $O(1)$ ）。两者结合：哈希表存 $\text{key} \rightarrow \text{node}$ ，双向链表保持访问顺序。头尾各设哨兵节点，省去边界判断。

解：见模板代码”套路 6 LRU Cache”。

分析：get/put 均 $O(1)$ （哈希表查找 + 双向链表插入/删除，均 $O(1)$ ）；空间 $O(\text{capacity})$ （链表 + 哈希表各存至多 capacity 个节点）。

例 3: 25. Reverse Nodes in K-Group

题目：每 k 个节点一组翻转链表，不足 k 个的末尾部分保留原顺序。

思路：外层循环：每次先探查剩余是否有 k 个（走 k 步，碰到 None 就停），若有则翻转这 k 个，将 group_prev 移到组尾（原来的组头，翻转后变组尾），循环继续。翻转的核心是标准三指针，只是把”下一组的开头”作为 prev 的初始值，这样翻转结束后新组尾自然指向了下一组。

解：见模板代码”套路 3 迭代翻转—reverseKGroup”。

分析：每个节点被访问常数次数（一次探查，一次翻转），总 $O(n)$ ；空间 $O(1)$ （只用固定数量指针变量）。

自测题

自测 1 (141 Linked List Cycle) ——head=[3,2,0,-4], pos=1 (尾节点连接到 index 1) 返回 True; head=[1], pos=-1 返回 False。提示: slow=fast=head, while fast and fast.next, slow=slow.next; fast=fast.next.next,若 slow is fast 返回 True。参考 solutions/linked_list/p141_linked_list_cycle.py。

自测 2 (19 Remove Nth Node From End) ——head=[1,2,3,4,5], n=2 返回 [1,2,3,5]; head=[1], n=1 返回 []。提示: dummy + fast 先走 n 步, slow=dummy, 同步走到 fast.next=None, slow.next=slow.next.next。参考 solutions/linked_list/p019_remove_nth_node_from_end_of_list.py。

自测 3 (21 Merge Two Sorted Lists) ——l1=[1,2,4], l2=[1,3,4] 返回 [1,1,2,3,4,4]。提示: dummy + cur, while l1 and l2 取小者接上,循环后 cur.next = l1 or l2。参考 solutions/linked_list/p021_merge_two_sorted_lists.py。

自测 4 (146 LRU Cache) ——capacity=2, put(1,1)、put(2,2)、get(1)=1、put(3,3)(逐出 key=2)、get(2)=-1、get(3)=3。提示: 哈希表 + 双向链表 + 头尾哨兵; get/put 每次移到头; 超容量时删尾。参考 solutions/linked_list/p146_lru_cache.py。

自测 5 (92 Reverse Linked List II) ——head=[1,2,3,4,5], left=2, right=4 返回 [1,4,3,2,5]。提示: dummy + pre 走 left-1 步, 记 tail_of_left=pre.next, 翻转 right-left+1 个, pre.next=prev, tail_of_left.next=cur。参考 solutions/linked_list/p092_reverse_linked_list_ii.py。

题目全览 (11 题)

#	题目	套路分类	难度
2	Add Two Numbers	dummy + 逐位进位	Medium
19	Remove Nth Node From End of List	快慢指针 + dummy	Medium
21	Merge Two Sorted Lists	dummy + 两路合并	Easy
25	Reverse Nodes in K-Group	迭代翻转 + 接线	Hard
61	Rotate List	成环 + 快慢/计数断点	Medium
82	Remove Duplicates from Sorted List II	dummy + 跳过重复块	Medium
86	Partition List	dummy + 两路分裂拼接	Medium
92	Reverse Linked List II	dummy + 区间翻转	Medium
138	Copy List with Random Pointer	三步法 (穿插 + 复制 + 拆分)	Medium
141	Linked List Cycle	快慢指针判环	Easy
146	LRU Cache	哈希表 + 双向链表	Medium

融合版说明

段	来源	价值
一例速记	本文件	5 大套路一览 + AI 场景关联
思维路径还原	本文件	11 道题的解题内心独白，含指针操作决策
抽象成方法	本文件	7 个标准模板（dummy/快慢/翻转/递归翻转/合并/LRU/三步法）+ 速查表
方法变形	本文件	4 类变体（快慢指针/翻转/合并拆分/LRU→LFU）
思考路标	本文件	10 条题型识别条件反射，含 AI/ML 场景
易错点	本文件	8 条高频踩坑（接线顺序/哈希表同步/拆分顺序等）
典型应用例题	solutions/	3 道精讲（92、146、25），代码 + 正确性分析
自测题	leetcode	5 题带提示，链接 solutions 文件
题目全览	本文件	11 题完整列表，套路分类一览

跨 category 导航：- 快慢指针的”找中点”在 234（Palindrome Linked List）中配合翻转使用 → 见本文件套路 2+3 - 双向链表是 LRU 的底层结构；堆（minheap）是 23 题的另一种实现 → 见 10-heap.md（若有）- 递归翻转的空间复杂度 O(n) 与 DFS 递归栈同理 → 见 07-binary-tree-dfs.md - LRU/LFU 是 AI 推理引擎（vLLM、FlashAttention）KV Cache 管理的核心机制

13 —Stack（融合版）

难度：□□□□□ 题数：5 核心套路：括号匹配、逆波兰表达式求值、单调栈、辅助栈、路径/表达式栈 本文件：覆盖 stack 5 题的算法套路总结 + 典型题精讲 + 自测

一例速记

括号匹配 (20)：遇左括号压栈，遇右括号弹栈验证配对，最终栈空则合法；O(n) 时间，O(n) 空间 路径简化 (71)：按 / 分割，用栈处理各部件：.. 弹栈，. 忽略，普通名压栈，最后 '/' + '/' .join(stack) 拼接 逆波兰表达式 RPN (150)：操作数压栈，遇运算符弹两个操作数计算后压回，最终栈顶即结果；O(n) 时间 最小栈 (155)：辅助栈同步记录”当前栈的最小值历史”，push/pop

时双栈联动, `getMin()` $O(1)$ 基础计算器 (224): 处理 $+/-/()$ 的带括号表达式求值; 栈保存”遇到左括号时的 $(result, sign)$ “, 右括号时恢复并合并; 单次扫描 $O(n)$ 时间 **AI** 关联: 计算图的算子优先级调度 $\approx$ 逆波兰表达式; 编译器/解释器的函数调用栈帧 $\approx$ 括号嵌套匹配; Transformer 层的残差连接可类比”栈式”状态保存

思维路径还原

“看到 **20 Valid Parentheses** (括号合法性检查) $\rightarrow$ 典型栈匹配: 遇 $(, [, \{$ 压栈; 遇 $),], \}$ 弹栈验证是否配对。边界: 弹栈时栈为空 (右括号多) $\rightarrow$ False; 遍历完栈非空 (左括号多) $\rightarrow$ False; 否则 True。Python 用字典 `{')': '(', ']' : '[' , '}' : '{'}` 简化配对查找。

看到 **71 Simplify Path** (UNIX 路径简化) $\rightarrow$ 按 $/$ 分割得到各部件, 栈处理规则: `'..'` $\rightarrow$ 若栈非空则 `pop()` (返回上一级); `'.'` 或空串 $\rightarrow$ 忽略 (当前目录或多余的斜杠); 其他 $\rightarrow$ `push` (正常目录名)。最终 `'/' + '/'.join(stack)` 即为简化路径。

看到 **150 Evaluate Reverse Polish Notation** (逆波兰表达式) $\rightarrow$ 顺序遍历 `tokens`: 数字压栈; $+/-/*//$ 弹出两个操作数 `b, a = pop(), pop()` (注意顺序, `b` 是后压的, 是右操作数), 计算 `a op b` 后压回。Python 整除向零取整用 `int(a / b)` (不能用 `a // b`, 负数时方向不同)。

看到 **155 Min Stack** (支持 $O(1)$ `getMin` 的栈) $\rightarrow$ 维护辅助栈 `min_stack`: `push` 时同时压入 `min(x, min_stack[-1])` (当前全局最小); `pop` 时双栈同步弹; `getMin` 返回 `min_stack[-1]`。初始化时 `min_stack` 预置一个 `inf` (防止首次 `push` 时无法比较)。

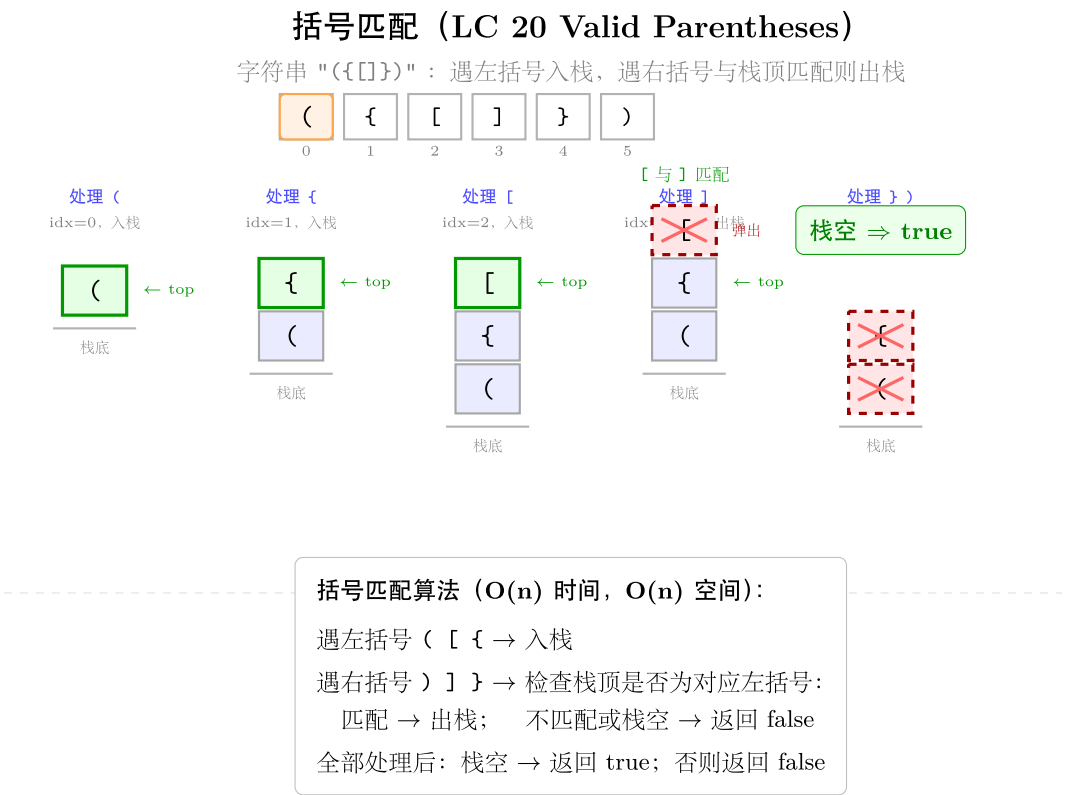
看到 **224 Basic Calculator** (带括号的 $+/-$ 表达式求值) $\rightarrow$ 核心: 括号里的表达式递归求值。用栈保存进入括号前的状态 $(result, sign)$: 遇 $($ $\rightarrow$ 把当前 $(result, sign)$ 压栈, `result = 0, sign = 1` (重新开始子表达式); 遇 $)$ $\rightarrow$ 计算子表达式结果, 弹出 $(prev_result, prev_sign)$, `result = prev\_result + prev\_sign * result` (将子结果以正确符号加回父结果); 遇数字 $\rightarrow$ 读完整数, `result += sign * num`; 遇 $+$ $\rightarrow$ `sign = 1`; 遇 $-$ $\rightarrow$ `sign = -1`。”

学习目标

- 掌握括号匹配的通用栈模板, 能扩展到多种括号类型及嵌套
- 熟练用栈模拟路径操作 (71), 理解 `...`、`..`、空串三类分支
- 理解 RPN (逆波兰表达式) 的计算过程, 注意操作数弹出顺序 (后弹的是左操作数)
- 掌握”辅助栈”设计模式 (155), 能推广到”支持 $O(1)$ 最大值”的变体
- 掌握带括号表达式的栈求值 (224), 理解进入 (时保存状态、遇 $)$ 时合并的机制
- 理解单调栈 (虽本 category 5 题未直接包含, 但作为 `stack` 核心知识点补充)

几何示意

图括号匹配（LC 20）



套路
1: 括
号匹
配通
用模
板
适用
题:
20

```
python
def
isValid(s:
str)
->
bool:
"""20:
括号
合法
性检
查。
时间
 $O(n)$ ,
空间
 $O(n)$ 。
左括
号压
栈;
右括
号与
栈顶
匹配,
不匹
配或
栈空
则非
法;
遍历
完栈
必须
为空。
"""
matching
=
{'(':
'(',
']':
'[',
'}':
'{'
stack:
list[str]
= []
for
```

> 泛
化：
若括
号类
型增
多，
只需
扩展
matching
字典；
若需
找第
一个
非法
位
置，在
return
False
前记
录 i。

套路 2：路径栈（文件系统路径简化）

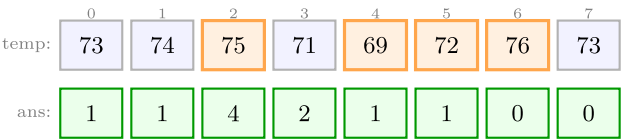
适用题：71

```
def simplifyPath(path: str) -> str:
    """71: UNIX 路径简化。时间  $O(n)$ ，空间  $O(n)$ 。
    按 '/' 分割，栈处理各部件：'..' 弹栈，'.' 或空串忽略，其他压栈。
    """
    stack: list[str] = []
    for part in path.split('/'):
        if part == '..':
            if stack:
                stack.pop()
        elif part and part != '.':
            stack.append(part)
    return '/' + '/'.join(stack)
```

关键：path.split('/') 会在多个连续 / 处产生空串，elif part and part != '.' 自然过滤。

每日温度（LC 739 Daily Temperatures）

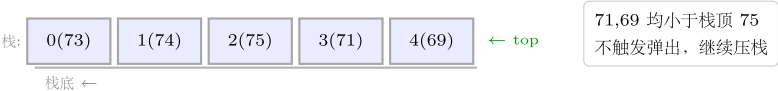
单调递减栈（存索引），弹出时记录等待天数；红箭头连被弹出元素与当前元素



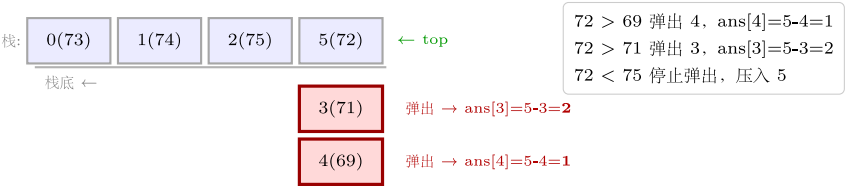
快照 1: i=0→2 全部压栈



快照 2: i=3(71),4(69) 压栈



快照 3: i=5(72)，弹出 4(69) 和 3(71)



快照 4: i=6(76)，弹出所有，压 6；i=7(73) 压栈



单调递减栈（O(n) 时间，O(n) 空间）：

维护单调递减栈（存索引 i），遍历 temp 数组：

当 temp[i] > temp[stack.top()] 时循环弹出 j=stack.pop()，
记录 ans[j] = i - j（等待天数）

将 i 压栈；遍历结束，栈中剩余索引对应 ans=0

图 30: 4 步演化 + 弹出对应 ans 更新

套路 3：逆波兰表达式求值（RPN）

适用题：150

```
def evalRPN(tokens: list[str]) -> int:
```

"""150: 逆波兰表达式。时间 $O(n)$ ，空间 $O(n)$ 。

操作数压栈；运算符弹两个操作数（后弹的 b 是左操作数，先弹的 a 是右操作数）

——注意： $tokens$ 中 b 先于 a 出现，压栈顺序 b 先 a 后，所以 pop 得到 a （右），再 pop 得到 b （左）。整除向零取整用 $int(b / a)$ 而非 $b // a$ （Python 对负数 $//$ 向下取整，与题意不符）。

"""

```
stack: list[int] = []
```

```
ops = {'+', '-', '*', '/'}
```

```
for tok in tokens:
```

```
    if tok in ops:
```

```
        a = stack.pop() # 右操作数（后压入的）
```

```
        b = stack.pop() # 左操作数（先压入的）
```

```
        if tok == '+':
```

```
            stack.append(b + a)
```

```
        elif tok == '-':
```

```
            stack.append(b - a)
```

```
        elif tok == '*':
```

```
            stack.append(b * a)
```

```
        else: # '/', 向零取整
```

```
            stack.append(int(b / a))
```

```
    else:
```

```
        stack.append(int(tok))
```

```
return stack[0]
```

套路 4：辅助栈（Min Stack）

适用题：155

```
class MinStack:
```

"""155: 支持 $O(1)$ $getMin$ 的栈。

两个栈同步维护： $main_stack$ 存元素， min_stack 存对应前缀最小值。

$push(x)$: $main_stack.append(x)$; $min_stack.append(min(x, min_stack[-1]))$ 。

$pop()$: 两栈同步 pop 。

$getMin()$: $min_stack[-1]$ 。

"""

```
def __init__(self):
```

```
    self.stack: list[int] = []
```

```

        self.min_stack: list[int] = [float('inf')] # 哨兵，防止首次 push 无法比较

    def push(self, val: int) -> None:
        self.stack.append(val)
        self.min_stack.append(min(val, self.min_stack[-1]))

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1] # O(1)

```

关键：min_stack[-1] 存的是“从栈底到当前位置”的前缀最小值，而非全局历史最小（前缀最小随 pop 正确回退）。

套路 5：带括号表达式求值（Basic Calculator）

适用题：224

```

def calculate(s: str) -> int:
    """224: 带括号的 +/- 表达式求值。时间 O(n)，空间 O(n)。
    栈保存进入括号前的 (result, sign)，遇 ')' 时弹出并合并。
    """
    stack: list[tuple[int, int]] = [] # (result, sign) 进入括号前的状态
    result = 0
    sign = 1 # 当前符号：+1 或 -1
    i = 0
    n = len(s)
    while i < n:
        ch = s[i]
        if ch.isdigit():
            num = 0
            while i < n and s[i].isdigit():
                num = num * 10 + int(s[i])
                i += 1
            result += sign * num
            continue # i 已经由内层循环推进，不要再 i+=1

```

```

elif ch == '+':
    sign = 1
elif ch == '-':
    sign = -1
elif ch == '(':
    # 保存当前状态，重置子表达式
    stack.append((result, sign))
    result = 0
    sign = 1
elif ch == ')':
    # 子表达式结束，合并到父表达式
    prev_result, prev_sign = stack.pop()
    result = prev_result + prev_sign * result
i += 1
return result

```

`prev_sign * result`: `prev_sign` 是进入括号之前外层的符号，例如 `3 - (2 + 1)` 中 `-` 就是 `prev_sign = -1`，子表达式结果 `3` 应以 `-3` 加回。

套路 6：单调栈通用模板（补充知识点）

虽然本 category 5 题未直接考查单调栈，但它是栈的核心扩展套路，在 739 (Daily Temperatures)、84 (Largest Rectangle)、85 (Maximal Rectangle) 等题大量使用。

单调递增栈：找每个元素 " 右侧第一个更小的元素 "

```

def next_smaller(nums: list[int]) -> list[int]:
    """ 结果 res[i] = 右侧第一个 < nums[i] 的下标，不存在则为 -1。时间 O(n)。 """
    n = len(nums)
    res = [-1] * n
    stack: list[int] = [] # 存下标，栈内对应值单调递增
    for i in range(n):
        # 当前元素比栈顶小 → 栈顶元素找到了 " 右侧第一个更小 "
        while stack and nums[i] < nums[stack[-1]]:
            idx = stack.pop()
            res[idx] = i
        stack.append(i)
    return res

```

单调递减栈：找每个元素 " 右侧第一个更大的元素 " (739 Daily Temperatures 变体)

```

def next_greater(nums: list[int]) -> list[int]:

```



```
""" 结果 res[i] = 右侧第一个 > nums[i] 的下标，不存在则为 -1。时间 O(n)。 """
n = len(nums)
res = [-1] * n
stack: list[int] = [] # 存下标，栈内对应值单调递减
for i in range(n):
    while stack and nums[i] > nums[stack[-1]]:
        idx = stack.pop()
        res[idx] = i
    stack.append(i)
return res
```

记忆：单调栈 = 维护”还没找到答案”的元素，新元素到来时，凡是满足条件的都出栈（找到答案），其余压栈继续等待。每个元素最多入栈/出栈各一次，总 O(n)。

速查表

题目	核心结构	关键操作	时间	空间
20 Valid Parentheses	单栈，存左括号	遇右括号弹栈验证	O(n)	O(n)
71 Simplify Path	单栈，存路径部件	.. 弹栈，. 忽略，其他压栈	O(n)	O(n)
150 Evaluate RPN	单栈，存操作数	运算符弹两个数计算后压回	O(n)	O(n)
155 Min Stack	双栈（主栈 + 辅助最小栈）	push/pop 双栈联动	O(1) 均摊	O(n)
224 Basic Calculator	单栈，存 (result, sign)	（压状态，）弹状态合并	O(n)	O(n)

方法变形（3 类）

变形 1：括号类扩展

- **20（合法性判断）** → **22 Generate Parentheses**（回溯生成合法括号，非本 category）：DFS 维护”未配对左括号数”，控制左括号数量 ≤ n，右括号数量 ≤ 已用左括号数。
- **20 扩展**：若括号有权重（如 { } 得 3 分），遍历时在匹配处累加分值；若需找最外层括号数量，维护深度计数而非用栈。
- **32 Longest Valid Parentheses**（非本 category）：栈存下标；入栈标记规则：左括号压下标，右括号若能匹配则弹栈，否则压自身下标作为”分隔符”；最长有效子串 = 相邻分隔符之间的区间长度。

变形 2：表达式类扩展

- **224 (+/-/()) → 227 Basic Calculator II (+/-/*/, 无括号, 非本 category)**: 遇数字时根据”当前运算符”决定压栈时的值; * 和 / 直接修改栈顶 (高优先级立刻算), +/- 则直接压带符号的值, 最终求和。
- **150 RPN (后缀表达式) → 中缀转后缀 (Shunting-Yard 算法)**: 用两个栈 (操作符栈 + 输出栈), 按优先级决定何时弹操作符到输出。
- **辅助栈的”最大值版”**: 将 min_stack 中的 min 改为 max, 即可实现 O(1) getMax 的栈。

变形 3：单调栈扩展（重点）

问题类型	单调栈方向	典型题
右侧第一个更大元素	单调递减栈	739 Daily Temperatures
右侧第一个更小元素	单调递增栈	901 Online Stock Span (反向)
柱状图最大矩形	单调递增栈 (存柱高)	84 Largest Rectangle in Histogram
接雨水 (栈法)	单调递减栈 (存高度)	42 Trapping Rain Water (同 01-array-string 双指针法)

单调栈的”方向”规则: 栈内维持单调递增 → 用于寻找”更小值”(新更小元素入栈时能弹出未满足的大值); 单调递减 → 寻找”更大值”。

思考路标（条件反射）

1. 看到 “括号合法性 / 配对” → 左压栈, 右弹栈验证; 结尾栈必须为空
2. 看到 “UNIX 路径 / 文件系统路径简化” → 按 / 分割 + 栈: .. 弹, . 忽略, 普通名压栈
3. 看到 “逆波兰 / 后缀表达式” → 操作数压栈, 运算符弹两个计算后压回, 注意弹出顺序 (后弹 = 左操作数)
4. 看到 “O(1) 最小值 / 最大值 + 支持 push/pop” → 辅助栈同步记录前缀 min/max
5. 看到 “带括号的 +/- 表达式” → 栈保存 (result, sign): (压,) 弹并合并; 遇数字直接 result += sign * num
6. 看到 “右侧第一个更大/更小” → 单调栈: 新元素触发弹栈 (找到答案), 未满足的继续等待
7. 看到 “柱状图 / 接雨水” → 单调栈 (栈法) 或双指针 (数组法), 时间均 O(n)
8. 看到 “操作符优先级 / 表达式解析” → Shunting-Yard 算法 (操作符栈 + 输出栈), AI 中等价于 tokenizer 的优先级调度
9. 看到 “函数调用 / 递归展开” → 显式栈模拟递归: 把”递归参数 + 返回地址”压栈, 用循环代替递归; 节省系统栈空间

易错点

1. **RPN** 操作数弹出顺序：`a = stack.pop()` 是右操作数（后压入），`b = stack.pop()` 是左操作数（先压入），计算 `b op a`。写成 `a op b` 对减法 / 除法结果错误（顺序敏感）。
 2. **RPN** 整除向零取整：Python `//` 对负数向下取整（`-7 // 2 = -4`），LeetCode 要求向零（`-7 / 2 = -3`），应用 `int(b / a)` 或 `math.trunc(b / a)`。
 3. **224** 进入括号时符号重置：进入（后 `result = 0, sign = 1`；若忘记重置 `sign = 1`，子表达式首个数字可能继承外层符号。
 4. **224** `prev_sign` 的含义：`prev_sign` 是括号之前外层的运算符，不是括号内首个运算符；例如 `1 - (2 - 3)` 中 `prev_sign = -1`，子结果 `-1` 要乘以 `prev_sign` 再加回。
 5. **224** 数字读取后 `i` 的移动：内层 `while` 循环读完整数后 `i` 已指向非数字字符，此时用 `continue` 跳过末尾的 `i += 1`，否则 `i` 会多走一步，跳过当前字符。
 6. **155** 辅助栈初始化：`min_stack = [float('inf')]` 作为哨兵，否则第一次 `push` 时 `min_stack[-1]` 报 `IndexError`；`pop` 时两栈必须同步，不能只弹 `stack`。
 7. **71** 路径末尾不加斜杠：`'/' + '/'.join(stack)`，注意只有开头有 `/`，`join` 本身不在结尾加 `/`；根路径（`stack` 为空）结果正确是 `'/'`。
 8. **20** 空栈弹操作：弹栈前必须检查 `not stack`，否则遇到纯右括号字符串（如 `"]"`）会 `IndexError`；Python `list.pop()` 在空列表上抛异常。
-

典型应用例题

例 1：155. Min Stack

题目：实现栈，支持 `push`、`pop`、`top`、`getMin`，所有操作均 $O(1)$ 。

思路： $O(1)$ `push/pop` → 普通 `list`。 $O(1)$ `getMin` → 不能每次遍历，需要辅助栈保存历史最小值。辅助栈 `min_stack[i]` = 主栈前 `i+1` 个元素的最小值；`pop` 时同步弹，自然回退到弹前的最小值。

解：见模板代码”套路 4 辅助栈 `MinStack`”。

分析：所有操作均 $O(1)$ ；空间 $O(n)$ （两个栈各存 n 个元素）。辅助栈和主栈等长是关键——这保证了 `pop` 后最小值正确回退。

例 2：224. Basic Calculator

题目：计算含 `+`、`-`、`(`、`)` 和空格的合法表达式字符串的值。

思路：遇到括号时，表达式在括号内”重新开始”；出括号时，把子结果以正确符号合并回父结果。用栈保存 `(result, sign)` 状态对——进入（时压，遇到）时弹。空格直接跳过，数字可能多位，需循环读完。

解：见模板代码”套路 5 带括号表达式求值”。

正确性验证：1 + (2 - (3 + 4))

- 初始：result=0, sign=1, stack=[]
- 读 1：result=1
- 读 +：sign=1
- 读 (：stack=[(1,1)], result=0, sign=1
- 读 2：result=2
- 读 -：sign=-1
- 读 (：stack=[(1,1),(2,-1)], result=0, sign=1
- 读 3：result=3
- 读 +：sign=1
- 读 4：result=7
- 读)：弹 (2,-1), result = 2 + (-1)*7 = -5
- 读)：弹 (1,1), result = 1 + 1*(-5) = -4

输出 -4，正确。

例 3：150. Evaluate Reverse Polish Notation

题目：给出逆波兰表达式（后缀表达式）的 tokens 数组，求其值。整除向零取整。

思路：RPN 的规律：数字压栈，运算符弹两个数字计算后压回；每个 token 处理一次，时间 O(n)。弹出顺序：第一次 pop 得到右操作数（后进先出），第二次 pop 得到左操作数。

解：见模板代码”套路 3 逆波兰表达式求值”。

验证：tokens = ["2","1","+","3","*"] - 压 2，压 1，遇 +：弹 1（右），弹 2（左），压 3 - 压 3，遇 *：弹 3（右），弹 3（左），压 9

输出 9，正确 ((2+1)*3 = 9)。

自测题

自测 1 (20 Valid Parentheses) ——s="() [] {}" 返回 True; s="[]" 返回 False; s="([])" 返回 False; s="{[]}" 返回 True。提示：matching = {'(':')', '[':']', '{':'}'，左括号压栈，右括号比对栈顶，结尾 len(stack)==0。参考 solutions/stack/p020_valid_parentheses.py。

自测 2 (71 Simplify Path) ——path="/home/" 返回 "/home"; path="/../" 返回 "/"; path="/home//foo/" 返回 "/home/foo"。提示：path.split('/') 后栈处理，'..' 弹栈，'.' 和空串忽略，最后 '/' + '/' + '.join(stack)。参考 solutions/stack/p071_simplify_path.py。

自测 3（150 Evaluate RPN）——tokens=["2","1","+","3","*"] 返回 9；tokens=["4","13","5","/","+"] 返回 6（4+(13/5)=4+2=6）；tokens=["10","6","9","3","+","-11","*","/","*","17","+","5","+"] 返回 22。提示：操作数压栈，运算符弹两个 a=pop()（右）、b=pop()（左），计算 b op a，整除用 int(b/a)。参考 solutions/stack/p150_evaluate_reverse_polish_notation.py。

自测 4（155 Min Stack）——push(-2)、push(0)、push(-3)、getMin()=-3、pop()、top()=0、getMin()=-2。提示：双栈，min_stack 初始含哨兵 inf；push 时 min_stack.append(min(val, min_stack[-1]))；pop 时两栈同步弹；getMin 返回 min_stack[-1]。参考 solutions/stack/p155_min_stack.py。

自测 5（224 Basic Calculator）——s="1 + 1" 返回 2；s=" 2-1 + 2 " 返回 3；s="(1+(4+5+2)-3)+(6+8)" 返回 23。提示：遇（压(result, sign), result=0, sign=1；遇）弹并 result = prev + prev_sign * result；遇数字读完整数累加；注意 continue 跳过多余的 i+=1。参考 solutions/stack/p224_basic_calculator.py。

题目全览（5 题）

#	题目	套路分类	难度
20	Valid Parentheses	括号匹配，单栈	Easy
71	Simplify Path	路径栈，按 / 分割	Medium
150	Evaluate Reverse Polish Notation	RPN 求值，操作数栈	Medium
155	Min Stack	辅助栈同步最小值	Medium
224	Basic Calculator	带括号 +/- 表达式，状态栈	Hard

融合版说明

段	来源	价值
一例速记	本文件	5 题 5 种栈套路一览 + AI 场景关联
思维路径还原	本文件	5 道题的解题内心独白，含栈状态管理决策
抽象成方法	本文件	6 个标准模板（括号/路径/RPN/辅助栈/表达式/单调栈）+ 速查表
方法变形	本文件	3 类变体（括号扩展/表达式扩展/单调栈扩展）含对照表
思考路标	本文件	9 条题型识别条件反射，含 AI/编译器场景
易错点	本文件	8 条高频踩坑（RPN 弹出顺序/整除/符号重置/空栈弹操作等）

段	来源	价值
典型应用例题	solutions/	3 道精讲（155、224、150），含手动验证过程
自测题	leetcode	5 题带提示，链接 solutions 文件
题目全览	本文件	5 题完整列表，套路分类一览

跨 category 导航：- 单调栈（本文套路 6）在 array_string 的接雨水（42）中有双指针替代实现 → 见 01-array-string.md - 递归 DFS 的显式栈改写（用 list 模拟调用栈）与此处栈求值思路一致 → 见 07-binary-tree-dfs.md - 带括号表达式（224）的”括号递归”结构与回溯的递归调用栈同理 → 见 08-backtracking.md - 编译器的词法分析（tokenizer）和语法分析（parser）都依赖栈；AI 中 ONNX/TorchScript 的算子图构建用 Shunting-Yard 调度优先级

14 —Hash Table（融合版）

难度：□□□□□ 题数：9 核心套路：两数之和模式、频次计数、滑窗哈希、集合去重 / 路径检测 本文件：覆盖 hash_table 9 题的算法套路总结 + 典型题精讲 + 自测

一例速记

两数之和模式：哈希表存”已见过的值 → 下标”，遍历时检查 `target - x` 是否在表中，一次扫描完成配对（1）频次计数：Counter 或 `defaultdict(int)` 统计字符/元素出现次数，然后比较两份计数表（242 / 383 / 49 / 205 / 290）字符 **26** 桶：全小写字母可用长度 26 的 list 代替哈希表，以 `tuple(bucket)` 作为 key（49 group anagrams）滑动窗口 + 哈希 **O(1)** 查找：在窗口内维护一个集合 / 字典，检查重复时 **O(1)**（219）集合去重：用 `set` 检查”序列的起始点”，避免重复展开，将 **O(n²)** 降到 **O(n)**（128）快慢指针 / 集合检环：判断数列是否进入循环，集合记录已出现的状态（202）双射映射：双向字典确保一一对应，双向约束必须同时检查（205 同构字符串 / 290 单词模式）**AI** 关联：feature hashing（ML 特征降维）/ Bloom Filter（近似集合查询，Redis / 推荐系统去重）/ 词频 TF-IDF

思维路径还原

“看到‘两数之和 / 配对’ → 立刻想哈希表：`seen = {}`，遍历时先查 `target - x` 是否在 `seen` 里，在则返回 `[seen[target - x], i]`，不在则 `seen[x] = i`。一次扫描，时间 **O(n)**，空间 **O(n)**（1 Two Sum）。

看到‘判断两个字符串互为 **anagram**’ → 频次计数: `Counter(s) == Counter(t)` 一行搞定 (Python 内置)。若不让用库, 用长度 26 的 bucket, 遍历 `s` 时 +1, 遍历 `t` 时 -1, 最终全零则是 **anagram** (242)。

看到‘**Ransom Note**’ (383) → 同频次计数, 但方向是单向检测: 统计 `magazine` 的字符频次, 然后检查 `ransomNote` 的每个字符是否都能从 `magazine` 中“借”到, 缺一个字符就返回 `False`, 不需要 `Counter` 比较。

看到‘**Group Anagrams**’ (49) → 分组: 用排序后的字符串或 26-bucket tuple 作 key, 相同 key 的字符串归为一组。 `defaultdict(list)`, 遍历一次, $O(n \cdot m \log m)$ (m 为最长串长度)。

看到‘同构字符串’ (205) → 双射映射: `s→t` 和 `t→s` 两个字典同时维护, 遇到冲突 (`s[i]` 已映射到不同的 `t[i]`, 或反之) 则返回 `False`。仅单向映射会漏掉 `'ab'/'aa'` 这类反向不一致的情况。

看到‘**Word Pattern**’ (290) → 与 205 几乎完全相同: `pattern` 字符 ↔ `word` 双射, 用同样的双向字典模板即可, 同构字符串的直接复用。

看到‘**Contains Duplicate II**’ (219) → 滑动窗口 + 哈希: 维护一个最多 $k+1$ 大小的窗口集合 `window`; 遍历时, 若 `nums[i]` 在 `window` 里则找到了距离 $\leq k$ 的重复, 否则将 `nums[i]` 加入 `window`; 若 `window` 大于 k 则删掉 `nums[i-k]`。 $O(n)$ 时间, $O(k)$ 空间。

看到‘**Happy Number**’ (202) → 集合检环: 计算平方和序列, 用 `set` 记录已出现的值, 若出现重复则说明进入了循环 (不会到 1), 返回 `False`; 若值变为 1 则返回 `True`。等价写法: 快慢指针 (Floyd 判环), 慢指针每次算一步, 快指针每次算两步, 相遇则有环。

看到‘**Longest Consecutive Sequence**’ (128) → 先 $O(n)$ 建 `num_set = set(nums)`, 然后遍历 `nums`, 只对“序列起始点” (`num - 1` 不在 `set` 里的数) 展开计数, 每次从起始点向后累加连续整数个数, 总体 $O(n)$ (每个数最多被访问两次)。”

学习目标

- 掌握两数之和模式: 哈希表“以空间换时间”, 将配对检测从 $O(n^2)$ 降到 $O(n)$
 - 熟练用 `Counter` / `defaultdict` / 26-bucket 做频次统计并比较
 - 理解“双射映射”约束: 同构 / 单词模式需要双向字典, 单向不够
 - 滑动窗口 + 哈希: 维护固定大小的窗口集合, $O(1)$ 查重
 - 集合检环 (202) 与快慢指针检环的等价性
 - 128 题的“只从起始点出发”剪枝, 将朴素 $O(n^2)$ 降到 $O(n)$
-

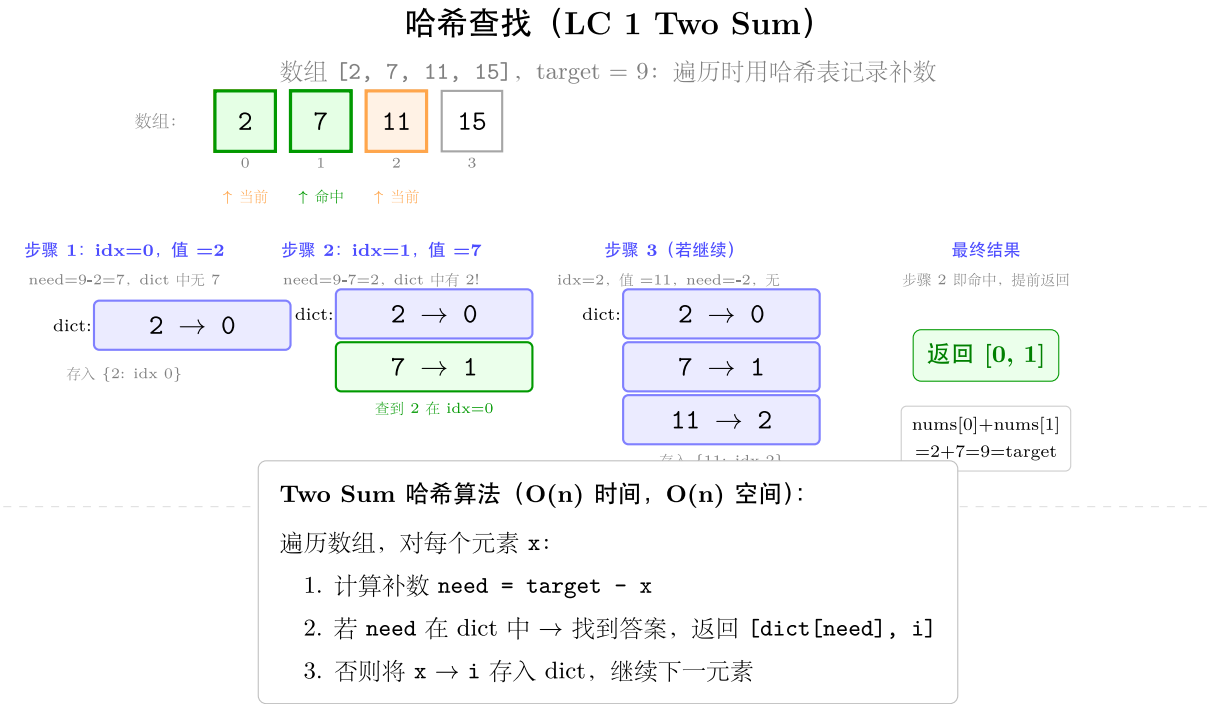


图 31: hash dict 4 步演化 + target 互补

几何示意

图哈希查找（LC 1 Two Sum）

图哈希分组（LC 49 Group Anagrams）

抽象成方法（标准模板代码）### 套路1：两数之和模式适用题：1

“python
from
typing
im-
port
List

```

def
twoSum(nums:
List[int],
target:
int) ->
List[int]:
""" “时
间
O(n),
空间
O(n)。
一次
扫描,
哈希
表记
录已
见值
→ 下
标。”
""" seen:
dict[int,
int] =
{} #
value
->
index
for i, x
in enu-
mer-
ate(nums):
com-
ple-
ment
=
target
- x if
com-
ple-
ment
in
seen:
return
[seen[complement],
i]

```

> 核
心思
路:
target
- x
是配
对数,
若它
已在
表中
则直
接返
回,
否则
先记
录当
前值
再继
续。>
无需
排序,
无需
双重
循环,
一次
线性
扫描
完成。

套路 2: 频次计数 (Counter / 26-bucket)

适用题: 242、383、49

```
from collections import Counter, defaultdict

# 242: 判断两个字符串是否互为 anagram
def isAnagram(s: str, t: str) -> bool:
    """ 时间  $O(n)$ , 空间  $O(1)$  (字符集固定为 26 个字母)。"""
    return Counter(s) == Counter(t)
```



图 32: 排序 key 分组哈希表

242 手写版 (面试偶尔要求不用 *Counter*)

```
def isAnagram_manual(s: str, t: str) -> bool:
    if len(s) != len(t):
        return False
    bucket = [0] * 26
    for c in s:
        bucket[ord(c) - ord('a')] += 1
    for c in t:
        bucket[ord(c) - ord('a')] -= 1
    return all(x == 0 for x in bucket)
```

383: 判断 *ransomNote* 能否由 *magazine* 中的字母构成

```
def canConstruct(ransomNote: str, magazine: str) -> bool:
    """ 时间  $O(n+m)$ , 空间  $O(1)$  (26 个字母)。"""
    mag_count = Counter(magazine)
    for c in ransomNote:
        mag_count[c] -= 1
        if mag_count[c] < 0:
            return False
    return True
```

49: 将字符串数组按 *anagram* 分组

```
def groupAnagrams(strs: List[str]) -> List[List[str]]:
    """ 时间  $O(n \cdot m \log m)$ , 空间  $O(n \cdot m)$ 。排序后的字符串作 key。"""
    groups: dict[str, List[str]] = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s)) # 排序后作为分组 key
        groups[key].append(s)
    return list(groups.values())
```

49 桶版 (不排序, $O(n \cdot m)$)

```
def groupAnagrams_bucket(strs: List[str]) -> List[List[str]]:
    """ 时间  $O(n \cdot m)$ , 空间  $O(n \cdot m)$ 。26-bucket tuple 作 key, 无需排序。"""
    groups: dict[tuple, List[str]] = defaultdict(list)
    for s in strs:
        bucket = [0] * 26
```

```

    for c in s:
        bucket[ord(c) - ord('a')] += 1
    groups[tuple(bucket)].append(s)
    return list(groups.values())

```

套路 3：双射映射（同构 / 单词模式）

适用题：205、290

205：判断两个字符串是否同构

```

def isIsomorphic(s: str, t: str) -> bool:
    """ 双向映射，时间  $O(n)$ ，空间  $O(n)$ 。 """
    s2t: dict[str, str] = {}
    t2s: dict[str, str] = {}
    for cs, ct in zip(s, t):
        if cs in s2t and s2t[cs] != ct:
            return False
        if ct in t2s and t2s[ct] != cs:
            return False
        s2t[cs] = ct
        t2s[ct] = cs
    return True

```

290：判断 *pattern* 和 *s* 是否符合相同的单词模式

```

def wordPattern(pattern: str, s: str) -> bool:
    """ 与 isIsomorphic 几乎完全相同，单位从字符变为单词。 """
    words = s.split()
    if len(pattern) != len(words):
        return False
    p2w: dict[str, str] = {}
    w2p: dict[str, str] = {}
    for p, w in zip(pattern, words):
        if p in p2w and p2w[p] != w:
            return False
        if w in w2p and w2p[w] != p:
            return False
        p2w[p] = w
        w2p[w] = p
    return True

```

关键：为什么要双向？若只维护 $s \rightarrow t$ ，'ab'/'aa' 中 $a \rightarrow a$ ， $b \rightarrow a$ 不会报错（两个字符映射到同一目标），但实际上 b 和 a 应该映射到不同字符。双向约束确保“一一对应”（双射）。

套路 4：滑动窗口哈希（固定大小窗口查重）

适用题：219

```
def containsNearbyDuplicate(nums: List[int], k: int) -> bool:
    """ 时间  $O(n)$ ，空间  $O(k)$ 。维护大小  $k+1$  的滑动窗口集合。 """
    window: set[int] = set()
    for i, x in enumerate(nums):
        if x in window:
            return True
        window.add(x)
        if len(window) > k:          # 窗口超过  $k$  时，删掉最旧的元素
            window.discard(nums[i - k])
    return False
```

窗口维护不变式：window 里恰好是 $\text{nums}[\max(0, i-k) \dots i-1]$ 的值集合，检查 $\text{nums}[i]$ 是否在其中即可判断距离 $\leq k$ 的重复。

套路 5：集合检环（Happy Number）

适用题：202

```
def isHappy(n: int) -> bool:
    """ 时间  $O(\log n)$  均摊，空间  $O(\log n)$ 。集合记录已出现的值检测循环。 """
    def next_val(x: int) -> int:
        total = 0
        while x:
            x, d = divmod(x, 10)
            total += d * d
        return total

    seen: set[int] = set()
    while n != 1:
        if n in seen:
            return False      # 出现循环，永远不会到 1
        seen.add(n)
```

```

    n = next_val(n)
return True

```

等价：快慢指针版 (Floyd 判环，空间 $O(1)$)

```

def isHappy_floyd(n: int) -> bool:
    def next_val(x: int) -> int:
        total = 0
        while x:
            x, d = divmod(x, 10)
            total += d * d
        return total

    slow, fast = n, next_val(n)
    while fast != 1 and slow != fast:
        slow = next_val(slow)
        fast = next_val(next_val(fast))
    return fast == 1

```

套路 6：集合去重 + 起始点剪枝（最长连续序列）

适用题：128

```

def longestConsecutive(nums: List[int]) -> int:
    """ 时间  $O(n)$ ，空间  $O(n)$ 。只从序列起点出发展开，避免重复。 """
    num_set = set(nums)
    best = 0
    for x in num_set:
        if x - 1 not in num_set:      # x 是某条连续序列的起始点
            length = 1
            while x + length in num_set:
                length += 1
            best = max(best, length)
    return best

```

剪枝关键：if `x - 1 not in num_set` 确保只从“起始点”出发，每个数至多被访问两次（一次判断是否起始点，一次在 while 里计数），总体 $O(n)$ 。

速查表

题型特征	套路	时间	空间
两数之和 / 配对	哈希表 seen{value→index}	$O(n)$	$O(n)$
字符频次比较	Counter / 26-bucket	$O(n)$	$O(1)$
字母构成判断 (单向)	Counter 相减检负	$O(n+m)$	$O(1)$
分组 (anagram 分组)	排序/桶 key + defaultdict	$O(nm \log m)$	$O(nm)$
同构 / 单词模式	双向字典双射	$O(n)$	$O(n)$
距离 k 内重复	滑动窗口 set	$O(n)$	$O(k)$
序列是否进入循环	集合记录已见 / 快慢指针	$O(\log n)$	$O(\log n)/O(1)$
最长连续序列	set + 起始点剪枝	$O(n)$	$O(n)$

方法变形 (4 类)

变形 1: 两数之和系列

- **1** (Two Sum) → 哈希表一次扫描, $O(n)$ 。
- **167** (Two Sum II, 有序数组) → 双指针对撞, $O(n)$, $O(1)$ 空间 (无需哈希表)。
- **15** (3Sum) → 排序 + 对每个元素用双指针, $O(n^2)$; 去重需跳过相同值。
- **18** (4Sum) → 3Sum 外再套一层循环, $O(n^3)$; 均可用哈希表减一层到 $O(n^2)$ 。
- 模式识别: 若数组已排序 → 优先双指针; 若未排序 → 哈希表。

变形 2: 频次计数扩展

- **242** (anagram 判断) → **49** (anagram 分组) → **438** (Find All Anagrams in a String, 滑窗 + 频次计数)。
- **383** (Ransom Note) → **76** (Minimum Window Substring, 滑窗 + 频次 + 双指针)。
- 26-bucket tuple 作 key 的 $O(nm)$ 方案在字符集固定时比排序 $O(nm \log m)$ 更优。

变形 3: 双射映射扩展

- **205** (Isomorphic Strings) ≡ **290** (Word Pattern): 两题解法框架完全相同, 仅”单位”不同 (字符 vs 单词)。
- 进阶: **726** (Number of Atoms) —嵌套字符串解析, 哈希表统计原子数量, 用栈处理括号。
- 双射约束失败场景速记: 'ab'/'aa' (a→a 且 b→a, 反向 a 对应了两个); 'aa'/'ab' (a 对应了两个不同字符)。

变形 4: 图 / 路径中的哈希去重

- **128** (最长连续序列) → `set` + 起始点剪枝, 等价于在图中找最长链而不重复访问节点。
- **202** (Happy Number) → 功能序列中的环检测, 等同于链表判环 (Floyd / 集合)。
- **684** (Redundant Connection) → Union-Find (并查集), 哈希表也可记录父节点, 但 UF 更优。
- AI 场景: **Bloom Filter** = 多个哈希函数 + bit array, 近似实现集合查询, 假阳性率可调; 用于 Redis 缓存穿透防护、推荐去重。

思考路标 (条件反射)

1. 看到 “**two sum** / 配对 / **target**” → 哈希表 `seen`, 一次扫描, $O(n)$
2. 看到 “**anagram** / 字母重排” → `Counter` 比较或 26-bucket
3. 看到 “**group** / 分组” → `defaultdict(list)` + 排序/桶 key
4. 看到 “同构 / 映射 / 模式匹配” → 双向字典, 两个方向同时检查
5. 看到 “距离 **k** 内重复” → 滑动窗口 `set`, 大小 $k+1$
6. 看到 “序列是否循环 / 无限循环” → `seen` 集合或 Floyd 快慢指针
7. 看到 “最长连续序列 **$O(n)$** ” → `set` + 只从起始点出发展开
8. 看到 “构成 / 组成 / 字符够不够” → `Counter` 单向, 相减看负
9. 看到 “字符集为小写字母” → 考虑 26-bucket 数组替代哈希表, 常数更小
10. 看到 “一一对应 / **bijection**” → 双向字典, 单向不够

易错点

1. 两数之和先查后记: 要先检查 `target - x` 是否在 `seen` 里, 再把 `x` 加入 `seen`; 顺序反了会让 `x` 匹配自身 (如 `target=6, x=3` 时 `3+3=6` 会误返回同一下标两次)。
2. **205 / 290** 仅单向映射: 只维护 `s→t` 会漏掉 `'ab'/'aa'` 这类场景 (`b` 和 `a` 同时映射到 `a`, 违反双射但单向不报错); 必须同时维护 `t→s`。
3. **219** 窗口删除时机: 先检查 `x in window`, 再加入, 最后判断窗口是否超过 `k` 时删 `nums[i-k]`; 顺序混乱会导致误判。
4. **128** 遍历 `num_set` 而非 `nums`: 若遍历 `nums` 且有重复元素, 仍然正确, 但遍历 `num_set` 避免对同一起点多次计数 (如 `[0,0,1,2]` 中 `0` 出现两次, 仅需展开一次)。
5. **49** 用可变对象作 `key`: `list` 不可哈希, 必须转成 `tuple(bucket)` 才能作 `dict key`; 排序版用 `''.join(sorted(s))` 得到字符串 `key`, 均可。
6. **202** 循环的数不一定立刻变大: 不要以为 “数变小就会到 1”, 实际上会在某个小的循环中来回 (如 `4→16→37→58→89→145→42→20→4`), 必须用 `set` 或快慢指针检环。
7. **383** 反向不成立: `canConstruct` 是 “ransomNote 能否由 magazine 构成”, 方向是 magazine 够不够, 不要写成比较两者计数相等。

8. 双射与等价类区分：同构字符串 / 单词模式要求双射（一一对应），而 anagram 要求等价类（相同字母集）；两者解法不同，不要混用。
-

典型应用例题

例 1: 1. Two Sum

题目：给定整数数组 `nums` 和目标值 `target`，返回两个下标使 `nums[i] + nums[j] == target (i != j)`。

思路：哈希表 `seen = {value: index}`，遍历时先查 `target - x` 是否已见过，是则返回；否则记录当前值。一次扫描， $O(n)$ 。

解：

```
# 参考: solutions/hash_table/p001_two_sum.py
def twoSum(nums: List[int], target: int) -> List[int]:
    seen: dict[int, int] = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

分析：每个元素最多访问一次，哈希表查找/插入 $O(1)$ ，总体 $O(n)$ 时间， $O(n)$ 空间。若改为暴力双重循环则 $O(n^2)$ ，哈希表以空间换时间。

例 2: 49. Group Anagrams

题目：给定字符串数组 `strs`，将互为 anagram 的字符串分到同一组，返回所有组。

思路：anagram 的充要条件是“字符频次完全相同”。对每个字符串，计算其频次特征（排序后的字符串或 26-bucket tuple）作为 key，`defaultdict(list)` 按 key 收集。

解：

```
# 参考: solutions/hash_table/p049_group_anagrams.py
def groupAnagrams(strs: List[str]) -> List[List[str]]:
    groups: dict[str, List[str]] = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s))
```

```

        groups[key].append(s)
    return list(groups.values())

```

分析：设 n 为字符串数量， m 为最长字符串长度。排序版 $O(nm \log m)$ ；桶版 $O(nm)$ (26 个桶，内层循环 m 次)，空间均 $O(nm)$ 。

例 3：128. Longest Consecutive Sequence

题目：给定未排序的整数数组，找到最长连续整数序列的长度，要求 $O(n)$ 时间。

思路：先建 `num_set = set(nums)`， $O(n)$ 。遍历时，只对“起始点”(x-1 不在 set 里) 展开计数，避免重复。每个数最多被访问两次，总体 $O(n)$ 。

解：

参考：solutions/hash_table/p128_longest_consecutive_sequence.py

```

def longestConsecutive(nums: List[int]) -> int:
    num_set = set(nums)
    best = 0
    for x in num_set:
        if x - 1 not in num_set:
            length = 1
            while x + length in num_set:
                length += 1
            best = max(best, length)
    return best

```

分析：外层循环遍历 set 中每个数，内层 while 只在起始点触发，所有 while 循环的总步数等于所有连续序列长度之和，即 $O(n)$ 。若用排序则 $O(n \log n)$ ，不满足题意。

自测题

自测 1 (1 Two Sum) ——`nums=[2,7,11,15]`，`target=9` 返回 `[0,1]`；`nums=[3,2,4]`，`target=6` 返回 `[1,2]`。提示：`seen = {}`，遍历时查 `target - x` 是否在 `seen` 里，在则返回，否则 `seen[x] = i`。参考 `solutions/hash_table/p001_two_sum.py`。

自测 2 (242 Valid Anagram) ——`s='anagram'`，`t='nagaram'` 返回 `True`；`s='rat'`，`t='car'` 返回 `False`。提示：`Counter(s) == Counter(t)` 或 26-bucket 两次遍历，`s` 时 +1，`t` 时 -1，全零则 `True`。参考 `solutions/hash_table/p242_valid_anagram.py`。

自测 3(49 Group Anagrams)——`strs=['eat','tea','tan','ate','nat','bat']` 返回 `[['eat','tea','ate'], ['tan',`
(顺序不限)。提示:`defaultdict(list)`,key 为 `''.join(sorted(s))`,遍历一次,返回 `list(groups.values())`。
参考 `solutions/hash_table/p049_group_anagrams.py`。

自测 4 (205 Isomorphic Strings) ——`s='egg', t='add'` 返回 `True`; `s='foo', t='bar'` 返回 `False`;
`s='paper', t='title'` 返回 `True`。提示: 双向字典 `s2t, t2s`, `zip` 遍历, 遇到冲突即 `False`。参考
`solutions/hash_table/p205_isomorphic_strings.py`。

自测 5 (219 Contains Duplicate II) ——`nums=[1,2,3,1]`, `k=3` 返回 `True`; `nums=[1,0,1,1]`, `k=1` 返回 `True`;
`nums=[1,2,3,1,2,3]`, `k=2` 返回 `False`。提示: 滑动窗口 `set`, 窗口大小 $\leq k$, 加入前先查, 超 `k` 后删 `nums[i-k]`。
参考 `solutions/hash_table/p219_contains_duplicate_ii.py`。

自测 6 (202 Happy Number) ——`n=19` 返回 `True` (`19→82→68→100→1`) ; `n=2` 返回 `False` (进入循
环) 。提示: `seen = set()`, `while n != 1` 时检查是否在 `seen` 里, 在则 `False`; 或快慢指针版。参考
`solutions/hash_table/p202_happy_number.py`。

自测 7(290 Word Pattern)——`pattern='abba', s='dog cat cat dog'` 返回 `True`; `pattern='abba', s='dog`
`cat cat fish'` 返回 `False`; `pattern='aaaa', s='dog cat cat dog'` 返回 `False`。提示: 与 205 同构, 双向
字典 `p2w, w2p`, `split` 后 `zip` 遍历。参考 `solutions/hash_table/p290_word_pattern.py`。

题目全览（9 题）

#	题目	套路分类	难度
1	Two Sum	哈希表 <code>seen{值→下标}</code>	Easy
49	Group Anagrams	频次 <code>key + defaultdict</code> 分组	Medium
128	Longest Consecutive Sequence	<code>set</code> + 起始点剪枝	Medium
202	Happy Number	集合检环 / 快慢指针	Easy
205	Isomorphic Strings	双向字典双射	Easy
219	Contains Duplicate II	滑动窗口 <code>set</code>	Easy
242	Valid Anagram	<code>Counter</code> / 26-bucket 频次比较	Easy
290	Word Pattern	双向字典双射（同 205）	Easy
383	Ransom Note	<code>Counter</code> 单向检负	Easy

融合版说明

段	来源	价值
一例速记	本文件	4 大套路一览 + AI 场景关联 (feature hashing / Bloom Filter)
思维路径还原	本文件	9 道题的解题内心独白，含关键判断点
抽象成方法	本文件	6 个标准模板（两数和/频次/双射/滑窗/检环/起始点剪枝）+ 速查表
方法变形	本文件	4 类变体（两数和系列/频次扩展/双射扩展/图路径去重）
思考路标	本文件	10 条题型识别条件反射
易错点	本文件	8 条高频踩坑（先查后记/单向映射/双射混淆等）
典型应用例题	solutions/	3 道精讲（1、49、128），代码 + 复杂度分析
自测题	leetcode	7 题带提示，链接 solutions 文件
题目全览	本文件	9 题完整列表，套路分类一览

跨 category 导航：- Two Sum 在数组已排序时优先用双指针（见 02-two-pointers.md）- 字符频次滑窗 → 见 03-sliding-window.md（76 Minimum Window Substring）- 集合检环 = 链表判环的函数式版本 → 见 12-linked-list.md 快慢指针套路 - Bloom Filter 的多哈希设计在 AI 推理引擎的 KV Cache 去重中广泛使用

15 —Intervals（融合版）

难度：□□□□□ 题数：4 核心套路：排序 + 合并、贪心插入、扫描线 / 差分数组、区间总结 本文件：覆盖 intervals 4 题的算法套路总结 + 典型题精讲 + 自测

一例速记

排序 + 合并：按区间左端点排序，遍历时若当前区间与结果集最后一个区间重叠（`cur[0] <= last[1]`），则合并（更新 `last[1] = max(last[1], cur[1])`）；否则直接追加（56 Merge Intervals）
贪心插入：插入新区间时，先将所有”在新区间左边不重叠”的区间直接加入结果，然后将所有”与新区间重叠”的区间逐个合并进 `newInterval`，最后追加新区间，再追加所有”在新区间右边”的区间（57 Insert Interval）
贪心射箭（按右端点排序）：按右端点升序排序，用贪心”一根箭尽量射穿更多气球”——箭放在当前气球右端点，若下一个气球左端点 > 当前箭位置，则需要新的箭（452）区

间总结：线性扫描连续段，遇到断点 (`nums[i] != nums[i-1] + 1`) 则输出一段 (228) AI 关联：时间窗口聚合 (Flink / Spark 滑动窗口) / GPU 显存碎片合并 / 任务调度区间冲突检测

思维路径还原

“看到‘合并区间’ (56) → 先按左端点排序 (`intervals.sort(key=lambda x: x[0])`)，然后用结果栈 `merged`，遍历每个区间：若 `merged` 为空或 `cur[0] > merged[-1][1]` (无重叠)，直接 `append`；否则更新 `merged[-1][1] = max(merged[-1][1], cur[1])` (合并，取右端点的较大值)。时间 $O(n \log n)$ (排序主导)，空间 $O(n)$ (输出)。

看到‘插入区间’ (57) → 不需要排序 (已排序)，三段处理：□ 左边不重叠 (`intervals[i][1] < newInterval[0]`) → 直接 `append`；□ 重叠 (`intervals[i][0] <= newInterval[1]`) → 合并到 `newInterval`: `newInterval[0] = min(newInterval[0], intervals[i][0])`, `newInterval[1] = max(newInterval[1], intervals[i][1])`；□ 把合并后的 `newInterval` `append`，再 `append` 剩余右边区间。时间 $O(n)$ ，空间 $O(n)$ 。

看到‘最少箭射气球’ (452) → 等价于找最多不重叠区间的个数 (贪心经典)：按右端点升序排序，维护当前箭的位置 `end` (初始化为第一个气球右端点)，遍历时若 `balloon[0] > end` (当前箭射不到)，则需要新箭，更新 `end = balloon[1]`；否则当前箭仍然有效 (不更新 `end`)。答案 = 箭的总数。时间 $O(n \log n)$ ，空间 $O(1)$ (不含排序空间)。

注意：452 的判断条件是 `balloon[0] > end` (严格大于)，因为题目说“触碰也算射穿”；若题目说“触碰不算”则改为 `>=`。

看到‘区间总结’ (228) → 一次线性扫描：若数组为空直接返回 `[]`；维护 `start = nums[0]`，遍历 `i=1..n-1`：若 `nums[i] != nums[i-1]+1` (断点)，输出 `[start, nums[i-1]]` 的字符串，更新 `start = nums[i]`；最后循环结束后别忘了输出最后一段。时间 $O(n)$ ，空间 $O(1)$ (不含输出)。”

学习目标

- 掌握“排序 + 双指针/栈合并”的区间合并模板 (56)
 - 理解三段式贪心插入 (57)：左边直接加 → 重叠合并 → 追加新区间 → 右边直接加
 - 熟练区分 56 (需先排序) 和 57 (已排序, $O(n)$ 即可) 两类题的时间复杂度差异
 - 掌握“按右端点贪心” (452)，能与“按左端点贪心” (56) 区分使用场景
 - 理解扫描线 / 差分数组思路 (会议室 II 类变形)，以及区间总结的线性扫描写法
-

几何示意

图区间合并（LC 56）

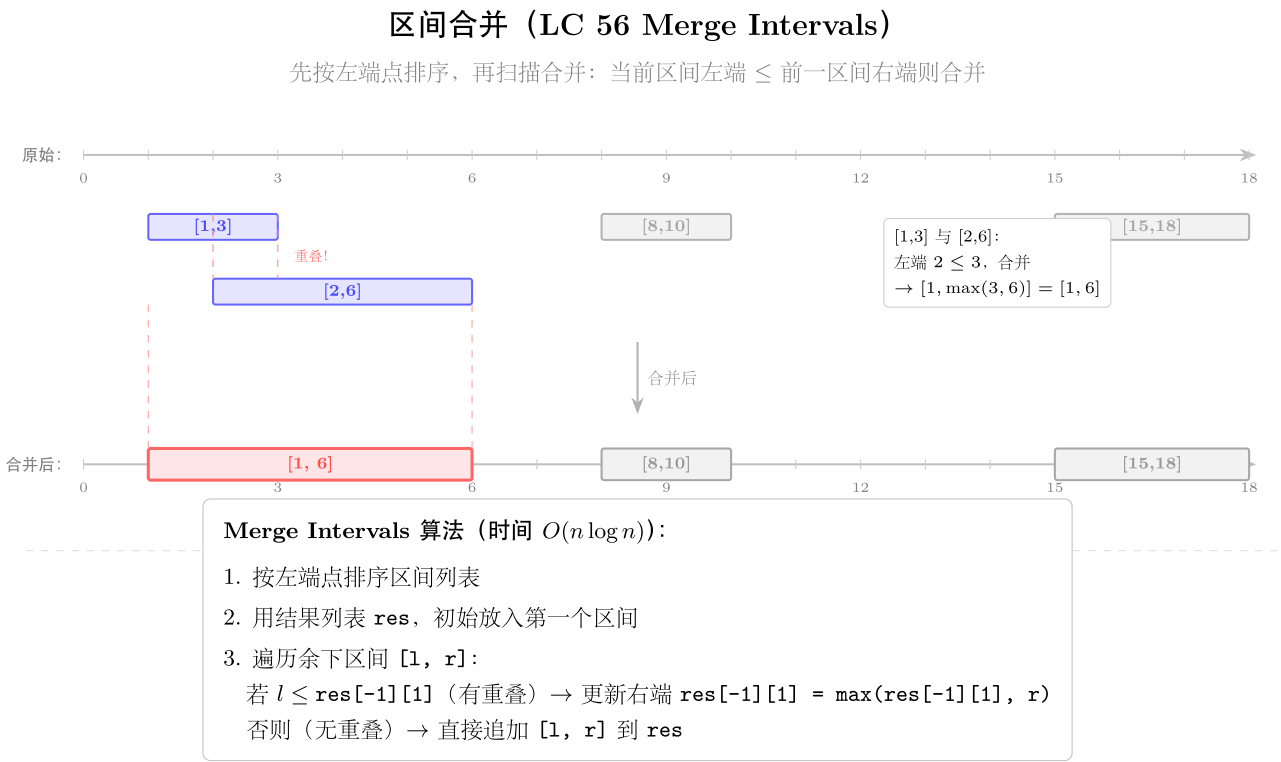


图 33: 时间轴 + 4 原始区间 + 合并

抽象成方法（标准模板代码）### 套路 1：排序 + 合并（Merge Intervals）

适用
题:
56
“python
from
typing
im-
port
List

```

def
merge(intervals:
List[List[int]])
->
List[List[int]]:
""" “时
间
O(n
log
n),
空间
O(n)。
排序
后逐
个合
并。”
"""

inter-
vals.sort(key=lambda
x:
x[0])
# 按
左端
点排
序
merged:
List[List[int]]
= []
for
cur in
inter-
vals:
if not
merged
or
cur[0]
>
merged[-
1][1]:
# 无
重叠:
当前
区间
在结
尾处

```

> 重
 叠判
 断:
`cur[0]`
`<=`
`merged[-1][1]`
 表示
 有重
 叠
 (注意:
`<=` 包
 含端
 点相
 接的
 情况,
 如
`[1,3]`
 和
`[3,5]`
 合并
 为
`[1,5]`)。
 > 合
 并只
 需更
 新右
 端点
 (左端
 点由
 排序
 保证
`cur[0]`
`>=`
`merged[-1][0]`)。

套路 2: 三段式贪心插入 (Insert Interval)

适用题: 57

```
def insert(intervals: List[List[int]],
          newInterval: List[int]) -> List[List[int]]:
    """ 时间  $O(n)$ , 空间  $O(n)$ 。三段: 左边不重叠 + 合并 + 右边不重叠。 """
    result: List[List[int]] = []
    i, n = 0, len(intervals)

    # 段 1: 在 newInterval 左边, 不重叠
    while i < n and intervals[i][1] < newInterval[0]:
        result.append(intervals[i])
        i += 1

    # 段 2: 与 newInterval 重叠, 合并
    while i < n and intervals[i][0] <= newInterval[1]:
        newInterval[0] = min(newInterval[0], intervals[i][0])
        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1
    result.append(newInterval)

    # 段 3: 在 newInterval 右边, 不重叠
    while i < n:
        result.append(intervals[i])
        i += 1

    return result
```

57 题的 intervals 已按左端点排好序, 所以 $O(n)$ 一次扫描即可, 无需再排序。三段的分界条件: intervals[i][1] < newInterval[0] (完全在左) 和 intervals[i][0] <= newInterval[1] (有重叠)。

套路 3: 按右端点贪心射击 (最少箭)

适用题: 452

```
def findMinArrowShots(points: List[List[int]]) -> int:
    """ 时间  $O(n \log n)$ , 空间  $O(1)$ 。按右端点排序, 贪心射穿最多气球。 """
    if not points:
        return 0
    points.sort(key=lambda x: x[1])  # 按右端点升序排序
    arrows = 1
    end = points[0][1]  # 第一根箭放在第一个气球右端点
    for start, right in points[1:]:
```

```

    if start > end:                # 当前气球射不到，需要新箭
        arrows += 1
        end = right               # 新箭放在当前气球右端点
    # start <= end: 当前箭能射到，不更新 end (贪心保留箭在尽量靠左的位置)
return arrows

```

为何按右端点排序？贪思路：“尽量让一根箭射穿更多气球”等价于“在当前气球右边缘射箭，能顺带覆盖所有左端点 $\leq$ 当前箭的后续气球”。

套路 4：线性扫描连续段 (Summary Ranges)

适用题：228

```

def summaryRanges(nums: List[int]) -> List[str]:
    """ 时间  $O(n)$ ，空间  $O(1)$  (不含输出)。扫描断点，输出区间字符串。 """
    if not nums:
        return []
    result: List[str] = []
    start = nums[0]
    for i in range(1, len(nums)):
        if nums[i] != nums[i - 1] + 1:    # 断点：连续性断裂
            # 输出 [start, nums[i-1]]
            if start == nums[i - 1]:
                result.append(str(start))
            else:
                result.append(f"{start}->{nums[i - 1]}")
            start = nums[i]    # 新段开始
    # 别忘了最后一段
    if start == nums[-1]:
        result.append(str(start))
    else:
        result.append(f"{start}->{nums[-1]}")
    return result

```

易错点：循环结束后，最后一段 $[start, nums[-1]]$ 需要单独输出（循环内只在遇到“断点”时输出前一段）。

套路 5：扫描线 / 差分数组（会议室类变形）

虽然 leetcode150 中 intervals category 没有直接包含 252/253（会议室 I/II），但扫描线思路是区间 category 的核心变形，常在变形题和面试中出现，值得掌握。

变形：会议室 II（253）—求同时进行的最多会议数，即需要多少个会议室

等价：最多有多少个区间在同一时刻重叠

```
def min_meeting_rooms(intervals: List[List[int]]) -> int:
    """ 扫描线：+1 表示会议开始，-1 表示会议结束，扫描事件找最大并发数。 """
    events: List[tuple] = []
    for start, end in intervals:
        events.append((start, 1))    # 开始：+1
        events.append((end, -1))     # 结束：-1（注意若 end==start 则 -1 先处理）
    events.sort(key=lambda x: (x[0], x[1])) # 同时刻：结束（-1）优先于开始（+1）
    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best
```

差分数组版（适合端点为整数且范围不大时）

```
def min_meeting_rooms_diff(intervals: List[List[int]]) -> int:
    """ 差分数组：在 start 位置 +1，在 end 位置 -1，前缀和求最大值。 """
    if not intervals:
        return 0
    max_end = max(e for _, e in intervals)
    diff = [0] * (max_end + 2)
    for start, end in intervals:
        diff[start] += 1
        diff[end] -= 1    # 结束时 -1（闭区间则 end+1 处 -1）
    current = best = 0
    for delta in diff:
        current += delta
        best = max(best, current)
    return best
```

速查表

题型特征	套路	时间	空间
合并重叠区间（无序）	排序 + 逐个合并	$O(n \log n)$	$O(n)$
插入新区间（已排序）	三段式贪心	$O(n)$	$O(n)$
最少覆盖数（最少箭）	按右端点排序 + 贪心	$O(n \log n)$	$O(1)$
连续段总结	线性扫描断点	$O(n)$	$O(1)$
最大并发数（会议室 II）	扫描线 / 差分数组	$O(n \log n) / O(n+R)$	$O(n)$

方法变形（4 类）

变形 1：合并区间系列

- **56**（合并区间，无序输入）→ 排序 $O(n \log n)$ ，再线性合并。
- **57**（插入区间，已排序输入）→ 三段式 $O(n)$ ，无需再排序。
- **252**（会议室 I，能否参加所有会议）→ 按左端点排序后检查相邻区间是否重叠（`intervals[i][0] < intervals[i-1][1]`），若有重叠则返回 `False`。
- 区别：56 合并输出结果；252 仅判断有无重叠（返回 `bool`）。

变形 2：贪心覆盖 / 区间调度

- **452**（最少箭）→ 按右端点贪心，等价于“最多不重叠区间”计数（Interval Scheduling Maximization）。
- **435**（Non-overlapping Intervals）→ 最少需要移除多少个区间使剩余不重叠 = n - 最多不重叠区间数。
- **253**（会议室 II）→ 最少需要多少间会议室 = 最大重叠数，用扫描线。
- 关键区分：
 - 按左端点贪心 → 合并、插入、判断重叠。
 - 按右端点贪心 → 最少箭数、最多不重叠、最少移除。

变形 3：扫描线 / 差分数组

- 扫描线：将每个区间拆成“开始事件 +1”和“结束事件 -1”，排序后计算前缀和，求最大值（会议室 II）。
- 差分数组：适合端点是整数且范围有界的情况（如时间戳在 $[0, T]$ 内）。
- 天际线问题（**218**）：扫描线 + 最大堆，维护当前最高建筑， $O(n \log n)$ （非本 category 但同框架）。
- AI 场景：Flink / Spark 的 sliding window 时间聚合本质是扫描线；GPU 任务调度按区间管理显存使用。

变形 4：区间总结 / 编码

- **228**（Summary Ranges）→ 线性扫描，维护段开始点，遇断点输出。
- **163**（Missing Ranges，非 LC150 但同类）→ 在 228 基础上改为输出“缺失”的区间段。

- 输出格式多变 (字符串 / 数组 / 计数), 但核心扫描逻辑不变。
-

思考路标 (条件反射)

1. 看到 “**merge intervals** / 合并区间” → 先按左端点排序, 再线性合并, `merged[-1][1] = max(...)`
 2. 看到 “**insert interval** / 插入区间 (已排序输入)” → 三段式 $O(n)$, 无需再排序
 3. 看到 “最少箭 / 最多不重叠” → 按右端点排序, 贪心射穿, `start > end` 时新箭
 4. 看到 “最多并发 / 最少会议室” → 扫描线 (开始 +1, 结束 -1, 前缀和最大值)
 5. 看到 “**summary ranges** / 连续段” → 线性扫描, 遇断点输出, 别忘最后一段
 6. 看到 “最少移除使不重叠” → 先求最多不重叠区间数 (按右端点贪心), 答案 = n - 最多不重叠数
 7. 看到 “端点相接算不算重叠” → 读题! 452 中 $[1, 3]$ 和 $[3, 5]$ 算重叠 (触碰即射穿); 56 中 $[1, 3]$ 和 $[3, 5]$ 也合并为 $[1, 5]$; 只有明确说 “开区间” 时才不包含端点
 8. 看到 “区间端点为整数且范围有界” → 考虑差分数组替代扫描线, 代码更简单
-

易错点

1. **56** 重叠判断用 `<=`: `cur[0] <= merged[-1][1]` 才算重叠 (包含端点相接), 不要用 `<`, 否则 $[1, 3], [3, 5]$ 无法合并。
 2. **57** 三段分界条件: 段 1 的分界是 `intervals[i][1] < newInterval[0]` (旧区间完全在左), 段 2 是 `intervals[i][0] <= newInterval[1]` (有重叠); 两个条件用的是不同端点, 易搞混。
 3. **452** 按右端点排序: 经典错误是按左端点排序后贪心, 会得到错误答案。记忆方法: 射箭贪的是 “右端点” (箭能覆盖到哪)。
 4. **452** 判断条件严格大于: `start > end` 时需要新箭; 若题目端点是整数, `start == end` 时当前箭仍能射到 (触碰即穿), 所以是严格 `>`。
 5. **228** 最后一段漏输出: `for` 循环内只在 “断点” 时输出 “上一段”, 循环结束后 `[start, nums[-1]]` 这最后一段必须单独 `append`, 初学者常漏。
 6. **56** 就地修改 `intervals` 的坑: `intervals.sort()` 会修改输入, 若题目要求不修改原数组则需先 `sorted(intervals, ...)`。
 7. 扫描线事件排序优先级: 同一时刻有 “开始” 和 “结束” 两个事件时, “结束 (-1)” 应排在 “开始 (+1)” 前面 (若开区间), 或后面 (若闭区间); 根据题目语义调整 `sort key` 中的第二维。
 8. **228** 单点区间: 若 `start == nums[i-1]` (单点), 输出 `str(start)` 而非 `"start->start"`, 不要忘记这个特判。
-

典型应用例题

例 1: 56. Merge Intervals

题目：给定区间列表，合并所有重叠区间，返回合并后的区间列表。

思路：按左端点排序，维护结果列表 merged。遍历时：若 merged 为空或当前区间左端点 > 结果最后区间右端点（无重叠），直接 append；否则更新结果最后区间右端点为两者右端点的最大值。

解：

```
# 参考: solutions/intervals/p056_merge_intervals.py
def merge(intervals: List[List[int]]) -> List[List[int]]:
    intervals.sort(key=lambda x: x[0])
    merged: List[List[int]] = []
    for cur in intervals:
        if not merged or cur[0] > merged[-1][1]:
            merged.append(cur)
        else:
            merged[-1][1] = max(merged[-1][1], cur[1])
    return merged
```

分析：排序 $O(n \log n)$ ，合并 $O(n)$ ，总体 $O(n \log n)$ ；空间 $O(n)$ （输出）。合并时只更新右端点，因为排序已保证 $\text{cur}[0] \geq \text{merged}[-1][0]$ ，左端点不可能缩小。

例 2: 57. Insert Interval

题目：给定已排序（按左端点升序，不重叠）的区间列表，插入一个新区间，返回合并后结果。

思路：三段式处理：□ 完全在新区间左边的旧区间直接追加；□ 与新区间重叠的旧区间逐个与 newInterval 合并（更新新区间的左右端点）；□ 追加合并后的新区间；□ 完全在右边的旧区间直接追加。

解：

```
# 参考: solutions/intervals/p057_insert_interval.py
def insert(intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
    result: List[List[int]] = []
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < newInterval[0]:
        result.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] <= newInterval[1]:
        newInterval[0] = min(newInterval[0], intervals[i][0])
        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1
    result.append(newInterval)
    return result
```

```

        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1
    result.append(newInterval)
    while i < n:
        result.append(intervals[i])
        i += 1
    return result

```

分析: $O(n)$ 时间 (每个区间访问一次), $O(n)$ 空间。不需要排序, 因为输入已排序, 三段式保证输出依然有序。

例 3: 452. Minimum Number of Arrows to Burst Balloons

题目: 气球在坐标轴上用区间 $[x_start, x_end]$ 表示, 垂直向上射箭可以射穿该坐标范围内所有气球, 求最少需要多少支箭。

思路: 按右端点升序排序, 贪心策略: 将箭放在当前气球的右端点, 能射穿所有“左端点 $\leq$ 当前箭位置”的气球。若下一个气球左端点 $>$ 当前箭位置, 则需要新箭。

解:

参考: [solutions/intervals/p452_minimum_number_of_arrows_to_burst_balloons.py](#)

```

def findMinArrowShots(points: List[List[int]]) -> int:
    if not points:
        return 0
    points.sort(key=lambda x: x[1])
    arrows = 1
    end = points[0][1]
    for start, right in points[1:]:
        if start > end:
            arrows += 1
            end = right
    return arrows

```

分析: 排序 $O(n \log n)$, 线性扫描 $O(n)$, 总体 $O(n \log n)$; 额外空间 $O(1)$ 。正确性: 每次新箭不可避免 (下一个气球的左端点已超出当前箭), 贪心将箭放在右端点使其覆盖面积最大。

自测题

自测 1 (56 Merge Intervals)——`intervals=[[1,3],[2,6],[8,10],[15,18]]` 返回 `[[1,6],[8,10],[15,18]]`;
`intervals=[[1,4],[4,5]]` 返回 `[[1,5]]`。提示: 先按 `x[0]` 排序, `merged = []`, 遍历时若无重叠则 `append`,
否则 `merged[-1][1] = max(...)`。参考 `solutions/intervals/p056_merge_intervals.py`。

自测 2 (57 Insert Interval) ——`intervals=[[1,3],[6,9]]`, `newInterval=[2,5]` 返回 `[[1,5],[6,9]]`;
`intervals=[[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval=[4,8]` 返回 `[[1,2],[3,10],[12,16]]`。
提示: 三段式, 注意段 1 条件 `intervals[i][1] < newInterval[0]`, 段 2 条件 `intervals[i][0] <= newInterval[1]`。参考 `solutions/intervals/p057_insert_interval.py`。

自测 3 (228 Summary Ranges)——`nums=[0,1,2,4,5,7]` 返回 `['0->2','4->5','7']`; `nums=[0,2,3,4,6,8,9]`
返回 `['0','2->4','6','8->9']`。提示: `start = nums[0]`, 遍历遇断点输出上一段, 循环后输出最后一段;
单点输出 `str(start)` 而非 `'start->start'`。参考 `solutions/intervals/p228_summary_ranges.py`。

自测 4 (452 Min Arrows)——`points=[[10,16],[2,8],[1,6],[7,12]]` 返回 2; `points=[[1,2],[3,4],[5,6],[7,8]]`
返回 4; `points=[[1,2],[2,3],[3,4],[4,5]]` 返回 2。提示: 按右端点排序, `arrows=1`, `end=points[0][1]`,
`start > end` 时 `arrows+=1`, `end=right`。参考 `solutions/intervals/p452_minimum_number_of_arrows_to_burst_bal`

题目全览（4 题）

#	题目	套路分类	难度
56	Merge Intervals	排序 + 逐个合并	Medium
57	Insert Interval	三段式贪心插入	Medium
228	Summary Ranges	线性扫描连续段	Easy
452	Minimum Number of Arrows to Burst Balloons	按右端点贪心	Medium

融合版说明

段	来源	价值
一例速记	本文件	4 大套路一览 + AI 场景关联（时间窗口/资源调度）
思维路径还原	本文件	4 道题的解题内心独白，含关键条件判断
抽象成方法	本文件	5 个标准模板（合并/插入/贪心射箭/区间总结/扫描线）+ 速查表

段	来源	价值
方法变形	本文件	4 类变体（合并系列/贪心覆盖/扫描线/区间编码）
思考路标	本文件	8 条题型识别条件反射
易错点	本文件	8 条高频踩坑（判断条件/排序方向/最后一段漏输出等）
典型应用例题	solutions/	3 道精讲（56、57、452），代码 + 正确性分析
自测题	leetcode	4 题带提示，链接 solutions 文件
题目全览	本文件	4 题完整列表，套路分类一览

跨 category 导航：- 排序是区问题的前置操作 → 见 04-binary-search.md（排序 + 二分）- 扫描线 + 最大堆（天际线问题）→ 见 16-heap.md - 区间 DP（如戳气球 312）→ 见 11-dp-multidim.md - Flink / Spark 的时间窗口聚合、GPU 显存碎片整理均以区间合并为核心操作

16—Heap（融合版）

难度：□□□□□ 题数：4 核心套路：Top-K（最小堆选 k 大）、双堆（中位数）、k 路合并、贪心 + 双堆 本文件：覆盖 heap 4 题的算法套路总结 + 典型题精讲 + 自测

一例速记

Python 堆全是最小堆：heapq 只提供最小堆；模拟最大堆需存负值（-x），取出时再取反 **Top-K 最大**：维护大小为 k 的最小堆，遍历时若新元素 > 堆顶则替换；堆顶即第 k 大元素（215）**Top-K 最小**：直接 heapq.nsmallest(k, nums) 或维护大小为 k 的最大堆（存负值）**双堆中位数**：lo（最大堆，存负值，保存较小一半）+ hi（最小堆，保存较大一半），平衡两堆大小（295）**k 路合并**：最小堆保存 k 个候选（值、来源列表索引、在该列表中的位置），每次弹出最小后推入其下一个候选（373）**贪心 + 双堆（IPO）**：先按 capital 排序，动态将“资本够用”的项目 profit 推入最大堆，每次选堆顶最大利润（502）**AI 关联**：beam search（维护大小为 k 的候选集）/ 优先级调度（任务队列）/ Top-K 采样（LLM 解码策略）

思维路径还原

“看到‘第 k 大元素’ (215) → 最小堆，大小维护为 k : `heap = nums[:k]; heapq.heapify(heap)`，遍历 `nums[k:]` 时，若 $x > \text{heap}[0]$ ，则 `heapq.heapreplace(heap, x)`（弹出堆顶并推入 x ）；遍历结束后 `heap[0]` 即第 k 大元素。时间 $O(n \log k)$ ，空间 $O(k)$ 。

为什么是最小堆？因为我们要“淘汰最小的”，保留最大的 k 个；最小堆的堆顶是当前 k 个里最小的，一旦新元素比堆顶大，就替换（淘汰当前最小，换入更大的），保证堆中始终是最大的 k 个。

替代方案：快速选择（Quickselect），平均 $O(n)$ ，最坏 $O(n^2)$ ；堆方法 $O(n \log k)$ 更稳定。

看到‘数据流中位数’ (295) → 双堆：`lo`（最大堆，用负值模拟，存较小一半）和 `hi`（最小堆，存较大一半）；维护不变式：`len(lo) == len(hi)` 或 `len(lo) == len(hi) + 1`；每次 `addNum`：先推入 `lo`（取反），将 `lo` 的最大值（即 `-heap[0]`）推入 `hi`，若 `len(hi) > len(lo)` 则将 `hi` 最小值推回 `lo`（取反）；`findMedian`：偶数时取两堆顶均值，奇数时取 `lo` 的堆顶（`lo` 多一个）。

看到‘ k 对最小和’ (373) → k 路合并：初始化最小堆：对每个 `nums1[i]`，将 `(nums1[i] + nums2[0], i, 0)` 推入堆（仅 `nums1` 的前 k 个或全部）；然后 `pop k` 次：弹出 `(sum, i, j)`，加入结果，若 `j+1 < len(nums2)` 则推入 `(nums1[i] + nums2[j+1], i, j+1)`。时间 $O(k \log k)$ ，空间 $O(k)$ 。

看到‘IPO 最大化资本’ (502) → 贪心 + 最大堆：按项目 `capital` 排序；维护最大堆（存负利润）；每次做项目前，把所有 `capital[i] <= w`（当前资本）的项目 `profit` 推入堆；然后弹出堆顶（最大利润）执行，更新 `w += profit`；重复 k 次。时间 $O(n \log n + k \log n)$ ，空间 $O(n)$ 。”

学习目标

- 熟练掌握 Python `heapq` 的最小堆 API: `heapify`, `heappush`, `heappop`, `heapreplace`, `nlargest`, `nsmallest`
- 理解最大堆的模拟方法（存负值）及取值时的取反
- 掌握 Top-K 最大的”最小堆维护 k 大”范式，以及快速选择的备选思路
- 掌握双堆中位数的不变式设计：两堆大小差 ≤ 1 ，`lo` 不少于 `hi`
- 能写 k 路合并的堆模板：（候选值，来源索引，位置）三元组推堆
- 理解 IPO 贪心的”先排序 + 动态解锁 + 最大堆贪心”组合

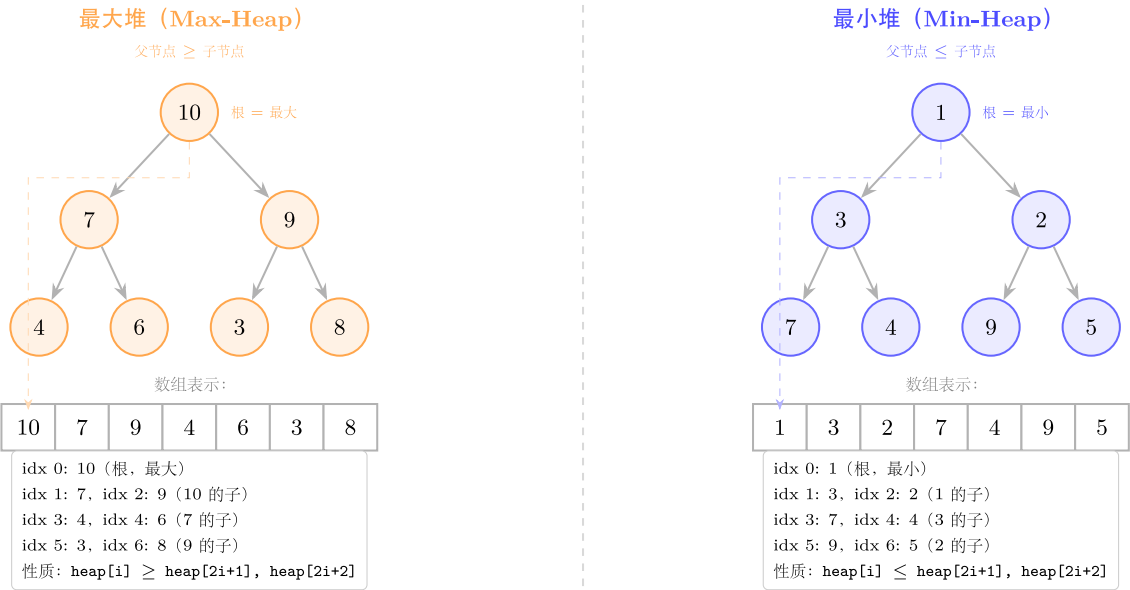
几何示意

图最大堆 vs 最小堆

图双堆中位数 (LC 295)

堆结构：最大堆 vs 最小堆

完全二叉树存储于数组；父节点下标 $(i-1)//2$ ，子节点 $2i+1, 2i+2$



堆操作复杂度:

插入 (push): $O(\log n)$, 加到末尾后上浮 (sift-up)

弹出 (pop): $O(\log n)$, 取出根后将末尾移到根再下沉 (sift-down)

建堆 (heapify): $O(n)$, 从最后一个非叶节点向上执行 sift-down

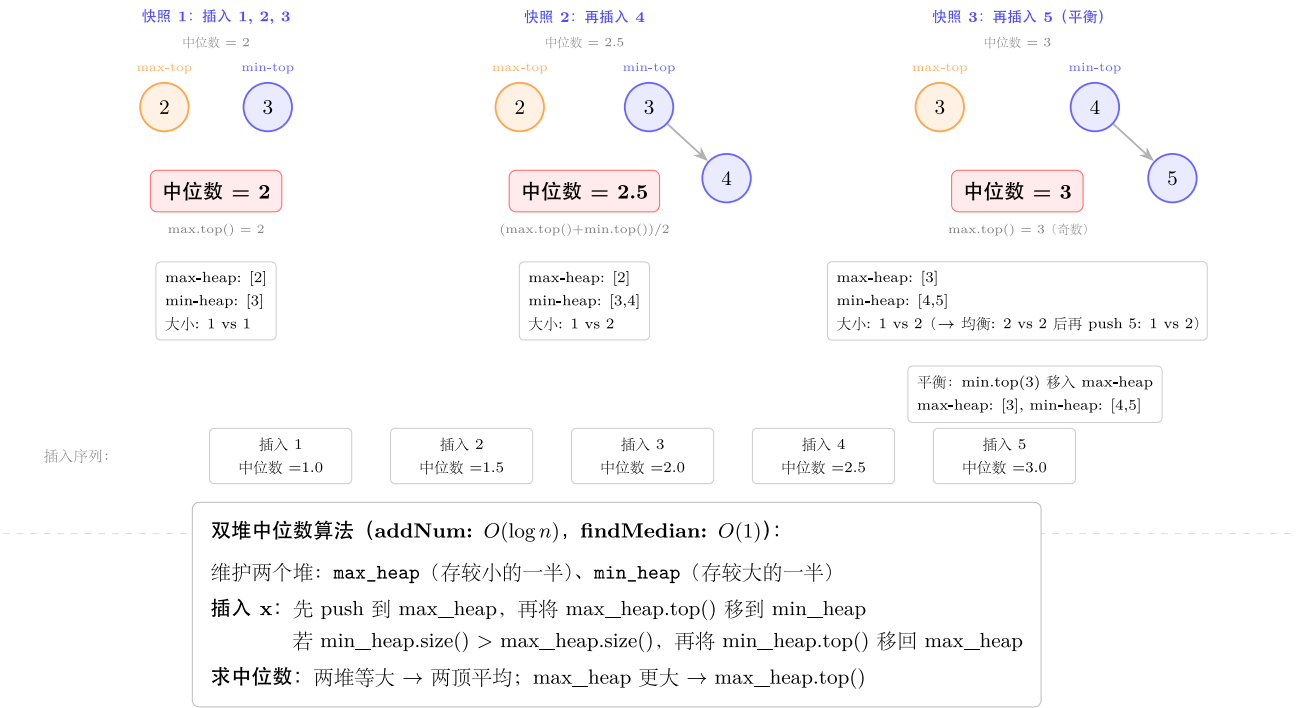
Python: heapq (最小堆); 最大堆取反: heappush(h, -x)

图 34: 二叉树 + 数组表示对比

数据流中位数（LC 295 Find Median）

max-heap（左半小数）+ min-heap（右半大数）；两堆大小差 ≤ 1

max-heap（左半） | min-heap（右半）



抽
象成
方法
(标准
模板
代码)

套路
1:
Top-K
最大
(最小
堆)
适用
题:
215
“python
im-
port
heapq
from
typing
im-
port
List

```

def
findK-
th-
Largest(nums:
List[int],
k: int)
-> int:
    """时
    间
    O(n
    log
    k),
    空间
    O(k)。
    最小
    堆维
    护 k
    大元
    素，
    堆顶
    即第
    k
    大。”
    """

    heap =
    nums[:k]
    heapq.heapify(heap)
    # O(k)
    建堆
    for x
    in
    nums[k:]:
    if x >
    heap[0]:
    # 新
    元素
    比当
    前第
    k 大
    还大
    heapq.heapreplace(heap,
    x) #
    弹出
    堆顶，
    让 x

```

```
# 库
函数
版
(更简洁,
但内部实
现类似)
def
findKth-
Largest_lib(nums:
List[int],
k: int)
-> int:
    return
    heapq.nlargest(k,
nums)[-1]
# 快速
选择版
(平均
O(n),
最坏
O(n2))
import
random
```

```

def
findK-
th-
Largest_quickselect(nums:
List[int],
k: int)
-> int:
    """ “随
    机化
    快速
    选择，
    平均
    O(n)。”
    """
    target
    =
    len(nums)
    - k #
    转换
    为”第
    target
    小”(0-
    indexed)

```

```

def
part-
tion(lo:
int, hi:
int) ->
int:
    pivot_idx
    = ran-
    dom.randint(lo,
    hi)
    nums[pivot_idx],
    nums[hi]
    =
    nums[hi],
    nums[pivot_idx]
    pivot
    =
    nums[hi]
    store
    = lo
    for i
    in
    range(lo,
    hi): if
    nums[i]
    <=
    pivot:
    nums[i],
    nums[store]
    =
    nums[store],
    nums[i]
    store
    += 1
    nums[store],
    nums[hi]
    =
    nums[hi],
    nums[store]
    return
    store

```

```
lo, hi
= 0,
len(nums)
- 1
while
lo <
hi: p
=
parti-
tion(lo,
hi) if
p <
target:
lo = p
+ 1
elif p
>
target:
hi = p
- 1
else:
break
return
nums[target]
““
```

> 选择策略： n 很大而 k 较小时，堆方法 $O(n \log k)$ 优于排序 $O(n \log n)$ ； $k \approx n/2$ 时，快速选择平均 $O(n)$ 更优，但最坏 $O(n^2)$ ；> 面试首选堆方法（稳定可控）。

套路 2：双堆中位数

适用题：295

```
class MedianFinder:
    """295: Find Median from Data Stream.
```

lo: 最大堆 (存负值), 维护较小一半。

hi: 最小堆, 维护较大一半。

不变式: $\text{len}(lo) == \text{len}(hi)$ 或 $\text{len}(lo) == \text{len}(hi) + 1$ (*lo* 可以多一个)。

"""

```
def __init__(self) -> None:
    self.lo: List[int] = []    # 最大堆 (负值)
    self.hi: List[int] = []    # 最小堆

def addNum(self, num: int) -> None:
    """ 均摊  $O(\log n)$ 。先推 lo, 平衡后再推 hi, 再平衡。 """
    heapq.heappush(self.lo, -num)    # 推入 lo (取负)
    # lo 的最大值必须  $\leq$  hi 的最小值
    if self.hi and (-self.lo[0]) > self.hi[0]:
        val = -heapq.heappop(self.lo)
        heapq.heappush(self.hi, val)
    # 平衡大小: lo 最多比 hi 多 1 个
    if len(self.lo) < len(self.hi):
        val = heapq.heappop(self.hi)
        heapq.heappush(self.lo, -val)
    elif len(self.lo) > len(self.hi) + 1:
        val = -heapq.heappop(self.lo)
        heapq.heappush(self.hi, val)

def findMedian(self) -> float:
    """  $O(1)$ 。 """
    if len(self.lo) == len(self.hi):
        return (-self.lo[0] + self.hi[0]) / 2.0
    return float(-self.lo[0])    # lo 多一个时, 中位数即 lo 堆顶
```

不变式关键: *lo* (最大堆, 较小一半) 的元素个数 $\geq$ *hi* (最小堆, 较大一半) 的元素个数, 且差值 ≤ 1 ; 同时 *lo* 的最大值 $\leq$ *hi* 的最小值 (维护有序性)。每次 addNum 先推 *lo*, 再通过至多两次平衡操作恢复不变式, 均摊 $O(\log n)$ 。

套路 3: k 路合并 (最小堆)

适用题: 373

```
def kSmallestPairs(nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
    """ 时间  $O(k \log k)$ , 空间  $O(k)$ 。
    思路:  $\text{nums1}[i] + \text{nums2}[j]$  的最小对 = 在 nums1 每一行中, 从 nums2[0] 开始的 "k 路合并"。
    初始堆:  $(\text{nums1}[i] + \text{nums2}[0], i, 0)$  对  $i=0..\min(k, \text{len}(\text{nums1}))-1$ 。
```

每次弹出后，将该行的下一个候选 ($nums1[i] + nums2[j+1]$, i , $j+1$) 推入堆。

```
"""
if not nums1 or not nums2:
    return []
heap: List[tuple] = []
for i in range(min(k, len(nums1))):
    heapq.heappush(heap, (nums1[i] + nums2[0], i, 0))

result: List[List[int]] = []
while heap and len(result) < k:
    total, i, j = heapq.heappop(heap)
    result.append([nums1[i], nums2[j]])
    if j + 1 < len(nums2):
        heapq.heappush(heap, (nums1[i] + nums2[j + 1], i, j + 1))
return result
```

理解：把问题看作 $\min(k, \text{len}(\text{nums1}))$ 路有序序列的合并（每路是 $\text{nums1}[i] + \text{nums2}[j]$ 按 j 递增的序列），最小堆从每路取一个候选，每次弹出最小值后补充该路下一个候选。

套路 4：贪心 + 最大堆（IPO）

适用题：502

```
def findMaximizedCapital(k: int, w: int,
                          profits: List[int], capital: List[int]) -> int:
    """ 时间  $O(n \log n + k \log n)$ ，空间  $O(n)$ 。
    思路：每次从“资本  $\leq w$  的项目”中选利润最大的（贪心），最大堆维护可选项目利润。
    """
    # 按 capital 升序排序，动态解锁可做项目
    projects = sorted(zip(capital, profits)) # [(cap, profit), ...]
    max_heap: List[int] = [] # 最大堆（存负利润）
    i = 0
    for _ in range(k):
        # 解锁所有 capital <= w 的项目，推入最大堆
        while i < len(projects) and projects[i][0] <= w:
            heapq.heappush(max_heap, -projects[i][1])
            i += 1
        if not max_heap:
            break # 没有可做的项目了
        w += -heapq.heappop(max_heap) # 取最大利润
    return w
```


贪心正确性：每次选能做的利润最大项目，长期资本增长最快。“按 capital 排序 + 指针 i 动态解锁”是避免每次都遍历所有项目的关键技巧。

速查表

题型特征	套路	时间	空间
第 k 大元素	最小堆（大小 k）	$O(n \log k)$	$O(k)$
第 k 大元素（平均最优）	快速选择	$O(n)$ 平均	$O(1)$
数据流中位数	双堆（lo 最大堆 + hi 最小堆）	$O(\log n)$ addNum	$O(n)$
k 对最小和	k 路合并最小堆	$O(k \log k)$	$O(k)$
最大化资本（带约束贪心）	排序 + 动态解锁 + 最大堆	$O(n \log n + k \log n)$	$O(n)$

方法变形（4 类）

变形 1：Top-K 系列

- 215（第 k 大，无序数组）→ 最小堆 $O(n \log k)$ ；快速选择 $O(n)$ 平均。
- 703（数据流第 k 大）→ 持久维护大小为 k 的最小堆，每次 add 后若超过 k 则 heappop。
- 347（前 k 个高频元素）→ 频次统计 + 最小堆（按频次维护 k 大）， $O(n \log k)$ ；或桶排序 $O(n)$ 。
- 973（最近 K 个点）→ 计算距离后 Top-K，同 215 堆模板，按距离排序。
- 关键区分：Top-K 最大用最小堆（淘汰最小）；Top-K 最小用最大堆（淘汰最大，存负值）。

变形 2：双堆扩展

- 295（中位数）→ 双堆，lo（最大堆）+ hi（最小堆），大小差 ≤ 1 。
- 480（滑动窗口中位数，非 LC150）→ 双堆 + 惰性删除：窗口滑动时，无效元素加入 delay_del 集合，弹堆顶时跳过无效元素。
- 4（Two Sorted Arrays 中位数）→ 二分法 $O(\log(m+n))$ ，不用堆；是“静态”中位数，而 295 是“动态流”。
- 双堆不变式强化：lo 的元素个数必须 $\geq$ hi 的元素个数，且 $-lo[0] \leq hi[0]$ （有序分割）。

变形 3：k 路合并扩展

- 373（k 对最小和）→ k 路合并，堆存 (sum, i, j)。
- 23（合并 k 个有序链表）→ 堆存 (node.val, i, node)，弹出后推入 node.next（见 12-linked-list.md）。

- **1439**（第 k 小的矩阵得分，非 LC150）→ 同 373，扩展到矩阵行列双索引。
- 通用模板：（候选值，来源 ID，位置）三元组 + 最小堆，每次弹出后推入该来源的下一个候选。

变形 4：贪心 + 堆（约束下的最优选择）

- **502**（IPO）→ 排序 + 动态解锁 + 最大堆贪心。
 - **1354**（多次操作后的数组，非 LC150）→ 最大堆 + 贪心。
 - **621**（Task Scheduler，非 LC150）→ 最大堆（按频次）+ 模拟冷却期。
 - AI 场景：
 - **Beam Search**：LLM 解码时维护 k 条候选序列（最大堆按 log-prob 排序），每步展开取 Top-K，本质是 k 路合并 + 优先队列。
 - **Top-K 采样**：LLM 的 top_k 参数即保留概率最高的 k 个 token（最小堆维护 k 大）。
 - 优先级队列调度：GPU 任务调度按优先级 + 资源约束，等同于 IPO 问题的多轮贪心。
-

思考路标（条件反射）

1. 看到“第 k 大 / 前 k 大”→ 最小堆（大小 k ），堆顶即第 k 大； $O(n \log k)$
 2. 看到“第 k 小 / 前 k 小”→ 最大堆（存负值，大小 k ），堆顶（取反）即第 k 小
 3. 看到“数据流 / 动态插入 + 查中位数”→ 双堆（lo 最大堆 + hi 最小堆）
 4. 看到“ k 对最小和 / k 路有序序列合并”→ 最小堆，三元组（value，来源，位置）
 5. 看到“带资本约束 / 条件解锁的最优选择”→ 排序解锁 + 最大堆贪心
 6. 看到“Python 最大堆”→ 存负值，`heappush(h, -x)`，取出时 `-heappop(h)`
 7. 看到“ k 个候选 / beam”→ 最小堆维护大小 k （若需最大则取负值）
 8. 看到“频率 Top-K”→ Counter 统计 + `heapq.nlargest(k, counter, key=counter.get)`
-

易错点

1. **Python** 只有最小堆：`heapq.heappush(h, x)` 是最小堆；模拟最大堆必须存 $-x$ ，取出后再取反；漏了取反会得到负值结果。
2. 双堆不变式破坏：addNum 时先推 lo 再平衡，顺序错误会导致不变式破坏；常见错误：推入 hi 后不检查是否 $hi > lo$ 导致大小失衡，findMedian 时堆顶不对。
3. 双堆有序性：lo（最大堆）的所有元素必须 $\leq$ hi（最小堆）的所有元素；若推入时没有先将 lo 的最大值与新元素比较，可能破坏有序性，中位数错误。
4. **373** 初始化范围：初始化堆时，对 nums1 的前 $\min(k, \text{len}(\text{nums1}))$ 个元素（不是全部）配对 `nums2[0]`，避免多余的堆操作。

5. **215 heapreplace vs heappushpop**: `heapreplace(heap, x)` 先弹出再推入（要求 $x \geq \text{heap}[0]$ 时才应替换，否则先检查条件）；`heappushpop(heap, x)` 先推入再弹出（结果始终是推入前后的最小值）。两者语义不同，Top-K 最大场景用 `heapreplace` + 前置条件检查。
6. **502 忘记 break**: 若最大堆为空（没有能做的项目）但 k 次还未完成，必须提前 `break`，否则会从空堆 `pop` 抛异常。
7. **373 推入下一个候选时检查边界**: `if j + 1 < len(nums2)` 才能推入 (`nums1[i] + nums2[j+1]`), $i, j+1$); 漏了边界检查会越界。
8. 双堆的 **len** 平衡方向: 设计为 `len(lo) >= len(hi)` 且 `len(lo) - len(hi) <= 1`, `findMedian` 时奇数个元素取 `lo` 堆顶; 若设计为 `len(hi) >= len(lo)` 方向相反, `findMedian` 逻辑也要对应调整。

典型应用例题

例 1: 215. Kth Largest Element in an Array

题目：给定整数数组，找到其中第 k 大的元素（不是第 k 个不同的元素）。

思路：维护大小为 k 的最小堆，堆顶是当前“前 k 大”中最小的（即第 k 大）。遍历时，若新元素 $>$ 堆顶，替换堆顶（`heapreplace`），否则跳过。遍历完毕后堆顶即答案。

解：

参考: [solutions/heap/p215_kth_largest_element_in_an_array.py](#)

```
def findKthLargest(nums: List[int], k: int) -> int:
    heap = nums[:k]
    heapq.heapify(heap)
    for x in nums[k:]:
        if x > heap[0]:
            heapq.heapreplace(heap, x)
    return heap[0]
```

分析：建堆 $O(k)$ ，遍历 $n - k$ 个元素每次 $O(\log k)$ ，总体 $O(n \log k)$ ；空间 $O(k)$ 。当 $k \ll n$ 时远优于排序 $O(n \log n)$ 。

例 2: 295. Find Median from Data Stream

题目：设计数据结构，支持动态添加数字并随时查询当前中位数。

思路：双堆维护数据流的两半：`lo`（最大堆，较小一半）和 `hi`（最小堆，较大一半）。不变式：两堆有序分割，且 `len(lo) - len(hi) ∈ {0, 1}`。`addNum` 时先推 `lo`，再平衡；`findMedian` 时根据两堆大小选择。

解：见模板代码“套路 2 双堆中位数”。

分析：每次 addNum 均摊 $O(\log n)$ （至多 2 次堆操作），findMedian $O(1)$ ；空间 $O(n)$ 。

例 3：502. IPO

题目：有 n 个项目，第 i 个项目需要 $\text{capital}[i]$ 资本才能启动，完成后获得 $\text{profit}[i]$ 。初始资本为 w ，最多做 k 个项目，求最终资本最大值。

思路：贪心：每次从“资本足够启动”的项目中选利润最大的。实现：先按 capital 排序；维护最大堆（存负利润）；每次做项目前，将所有 $\text{capital}[i] \leq w$ 的项目推入堆；弹出堆顶（最大利润）并加到 w ；重复 k 次。

解：见模板代码“套路 4 贪心 + 最大堆”。

分析：排序 $O(n \log n)$ ， k 轮循环中每个项目最多推入堆一次， $O(n \log n)$ ；总体 $O(n \log n + k \log n)$ ，空间 $O(n)$ 。

自测题

自测 1 (215 Kth Largest) —— $\text{nums}=[3,2,1,5,6,4]$ ， $k=2$ 返回 5； $\text{nums}=[3,2,3,1,2,4,5,5,6]$ ， $k=4$ 返回 4。提示： $\text{heap} = \text{nums}[:k]$ ； $\text{heapify}(\text{heap})$ ，遍历剩余元素， $x > \text{heap}[0]$ 时 $\text{heapreplace}(\text{heap}, x)$ ，返回 $\text{heap}[0]$ 。参考 `solutions/heap/p215_kth_largest_element_in_an_array.py`。

自测 2 (295 Find Median from Data Stream) ——依次 addNum(1)、addNum(2)，findMedian 返回 1.5；再 addNum(3)，findMedian 返回 2.0。提示：lo（最大堆，存负值）+ hi（最小堆），addNum 先推 lo，平衡有序性，再平衡大小；findMedian 根据两堆大小取堆顶。参考 `solutions/heap/p295_find_median_from_data_stream.py`。

自测 3 (373 K Pairs with Smallest Sums) —— $\text{nums1}=[1,7,11]$ ， $\text{nums2}=[2,4,6]$ ， $k=3$ 返回 $[[1,2],[1,4],[1,6]]$ ； $\text{nums1}=[1,1,2]$ ， $\text{nums2}=[1,2,3]$ ， $k=2$ 返回 $[[1,1],[1,1]]$ 。提示：初始化堆 $(\text{nums1}[i] + \text{nums2}[0], i, 0)$ 对 $i < \min(k, \text{len}(\text{nums1}))$ ，pop 后推入 $(\text{nums1}[i] + \text{nums2}[j+1], i, j+1)$ ，重复 k 次。参考 `solutions/heap/p373_find_k_pairs_with_smallest_sums.py`。

自测 4 (502 IPO) —— $k=2$ ， $w=0$ ， $\text{profits}=[1,2,3]$ ， $\text{capital}=[0,1,1]$ 返回 4（做项目 0 得 1，再做项目 2 得 3，共 4）； $k=3$ ， $w=0$ ， $\text{profits}=[1,2,3]$ ， $\text{capital}=[0,1,2]$ 返回 6。提示： $\text{projects} = \text{sorted}(\text{zip}(\text{capital}, \text{profits}))$ ， $i=0$ ，每轮将 $\text{capital}[i] \leq w$ 的项目推入最大堆（-profit），弹出堆顶更新 w ，重复 k 次。参考 `solutions/heap/p502_ipo.py`。

题目全览（4 题）

#	题目	套路分类	难度
215	Kth Largest Element in an Array	最小堆 Top-K	Medium
295	Find Median from Data Stream	双堆（lo 最大堆 + hi 最小堆）	Hard
373	Find K Pairs with Smallest Sums	k 路合并最小堆	Medium
502	IPO	排序 + 最大堆贪心	Hard

融合版说明

段	来源	价值
一例速记	本文件	4 大套路一览 + AI 场景关联 (beam search / 优先级调度)
思维路径还原	本文件	4 道题的解题内心独白，含 Python heap 细节
抽象成方法	本文件	4 个标准模板（Top-K/双堆/k 路合并/贪心堆）+ 快速选择备选 + 速查表
方法变形	本文件	4 类变体（Top-K 系列/双堆扩展/k 路合并扩展/贪心堆扩展）+ AI 关联
思考路标	本文件	8 条题型识别条件反射
易错点	本文件	8 条高频踩坑（最大堆取负/双堆有序性/边界检查等）
典型应用例题	solutions/	3 道精讲（215、295、502），代码 + 复杂度分析
自测题	leetcode	4 题带提示，链接 solutions 文件
题目全览	本文件	4 题完整列表，套路分类一览

跨 **category** 导航: - k 路合并链表(23 题) → 见 12-linked-list.md 套路 5(合并 k 个有序链表)- 快速选择与快速排序同根 → 见 09-divide-conquer.md - 堆排序 + 区间扫描线 → 见 15-intervals.md (会议室 II / 天际线) - beam search 在 LLM 解码中的应用: k 路候选序列按 log-prob 维护，每步展开取 Top-K，本质是 k 路合并 + 优先队列（373 的直接推广）

17—Kadane's Algorithm (融合版)

难度: □□□□ 题数: 2 核心套路: 维护 $cur_sum + global_max$, 线性扫描一次完成 本文件: 覆盖 kadane 2 题的算法套路总结 + 典型题精讲 + 自测

一例速记

标准 **Kadane**: $cur = \max(x, cur + x)$ (当前子数组和, 遇负则”重新开始”); $ans = \max(ans, cur)$ (更新全局最大) (53) 乘积变体: 乘积中负数会”变号”, 需同时维护 cur_max 和 cur_min (负 $\times$ 负 = 正), 每步更新时三者取 $\max/\min$ (152) 环形变体: 最大子数组和 = $\max(\text{线性最大}, \text{总和} - \text{线性最小})$; 特例: 全为负数时只取线性最大 (918) 核心直觉: 若已有子数组的和为负, 对后续子数组无正贡献, 丢弃从下一个元素重新开始 **AI** 关联: 在线极值流 (online $\max/\min$ tracking) / 时序信号峰值检测 / Streaming 数据的窗口聚合

思维路径还原

“看到‘最大子数组和’ (53) $\rightarrow$ 标准 Kadane: $cur = 0, ans = nums[0]$, 遍历每个元素 x : $cur = \max(x, cur + x)$ ($cur + x < x$ 时说明之前的和拖累了, 重置); $ans = \max(ans, cur)$ (更新全局最大)。时间 $O(n)$, 空间 $O(1)$ 。

注意: ans 初始化为 $nums[0]$ 而非 0, 因为数组可能全为负数, 答案应是最大的那个负数。等价写法: $cur = \max(x, cur + x)$ 等同于 $cur = cur + x$ if $cur > 0$ else x 。

看到‘最大乘积子数组’ (152) $\rightarrow$ 乘积特殊性: 负数 $\times$ 负数 = 正数, 一个当前最小值 (负的) 乘以下一个负数可能变成最大值。因此同时维护 cur_max (当前子数组最大乘积) 和 cur_min (当前子数组最小乘积):

```
new_max = max(x, x * cur_max, x * cur_min)
new_min = min(x, x * cur_max, x * cur_min)
cur_max, cur_min = new_max, new_min
ans = max(ans, cur_max)
```

注意: 计算 new_max 和 new_min 时必须用旧的 cur_max 和 cur_min , 先计算再更新 (否则 cur_max 被修改后影响 cur_min 的计算)。

看到‘环形数组最大子数组和’ (918) $\rightarrow$ 两种情况: 情况 1: 最大子数组不跨越边界 $\rightarrow$ 就是标准 Kadane 的答案 max_sum 。情况 2: 最大子数组跨越边界 (首尾相连) $\rightarrow$ 等价于”去掉中间一段连续子数组的最小和”, 即 $total - min_sum$ (min_sum 为最小子数组和, 用 Kadane 取 $\min$ 版本求得)。最终答案 = $\max(max_sum, total - min_sum)$; 特殊情况: 若所有元素均为负数 ($max_sum < 0$), $total - min_sum$ 等于 0 (去掉全部) 会给出错误答案, 此时应直接返回 max_sum 。”

学习目标

- 熟练写标准 Kadane 模板 (`cur = max(x, cur + x)`)，注意初始值为 `nums[0]` 而非 0
- 理解“cur 为负时重置”的直觉：负的前缀和对后续子数组无贡献
- 掌握乘积变体 (152)：同时维护 `cur_max / cur_min`，遇负数时两者可能翻转
- 掌握环形变体 (918)：两种情况取最大，全负时的特殊处理
- 理解 Kadane 与“在线极值”问题的关联

几何示意

图 Kadane 最大子数组 (LC 53)

```

## 抽象成方法 (标准模板代码)
###
套路
1: 标准 Kadane (最大子数组和) 适用题: 53
“python
from typing
import
List

```

```

def
max-
Sub-
Ar-
ray(nums:
List[int])
-> int:
""" “时
间
O(n),
空间
O(1)。
标准
Kadane
算
法。”
""" cur
= ans
=
nums[0]
for x
in
nums[1:]:
cur =
max(x,
cur +
x) #
cur +
x < x
时,
前缀
为负,
丢弃
重新
开始
ans =
max(ans,
cur)
return
ans

```

```

# 等
价写
法
(更显
式地
表达”
重置”
逻辑)
def
max-
Sub-
Ar-
ray_v2(nums:
List[int])
-> int:
    cur =
    0 ans
    =
    nums[0]
    for x
    in
    nums:
        cur =
        cur +
        x if
        cur >
        0 else
        x # 若
        之前
        的和
        为负
        则抛
        弃 ans
        =
        max(ans,
        cur)
    return
    ans

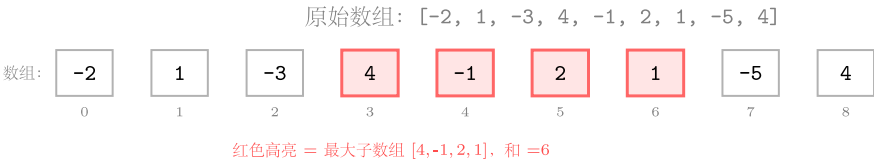
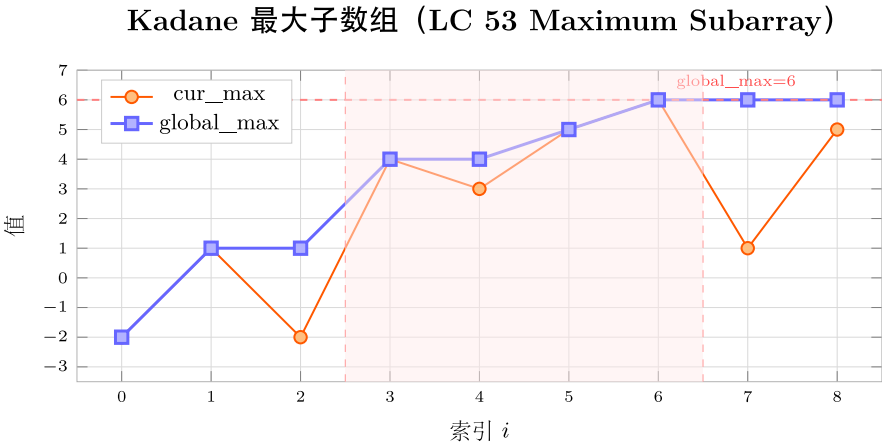
```

```

# DP
写法
(dp[i]
= 以
nums[i]
结尾
的最
大子
数组
和)
def
max-
Sub-
Ar-
ray_dp(nums:
List[int])
-> int:
"""
“dp[i]
=
max(nums[i],
dp[i-
1] +
nums[i]),
滚动
变量
优化
为
O(1)
空
间。”
"""dp =
nums[0]
ans =
dp for
x in
nums[1:]:
dp =
max(x,
dp +
x) ans
=
max(ans,
dp)

```

> 初
始化:
cur =
ans =
nums[0]
(不能
用 0
初始
化
ans,
否则
全负
数数
组时
ans 保
持 0
而非
正确
的负
数答
案)。
> 数
学保
证:
Kadane
正确
性来
自于”
子问
题最
优性”:
以
nums[i]
结尾
的最
大子
数组
和,
等于
nums[i]
本身
(从 i
重新
开始)



步骤明细:

idx	nums[i]	cur_max	global_max	操作
0	-2	-2	-2	初始化
1	1	1	1	重置 (1>-2+1)
2	-3	-2	1	延续 (-2>-3)
3	4	4	4	重置 (4>-2+4)
4	-1	3	4	延续 (3>-1)
5	2	5	5	延续 (5>3+2)
6	1	6	6	延续 (6>5+1) → 峰值
7	-5	1	6	延续 (1>6-5)
8	4	5	6	延续 (5>1+4)

Kadane 算法 ($O(n)$ 时间, $O(1)$ 空间):

```
cur_max = global_max = nums[0]
遍历 i = 1..n-1:
    cur_max = max(nums[i], cur_max + nums[i]) (当前子数组要或不要前缀)
    global_max = max(global_max, cur_max)
```

图 36: cur_max / global_max 折线 + 红色子数组

套路 2：乘积变体（同时维护 **max** 和 **min**）

适用题：152

```
def maxProduct(nums: List[int]) -> int:
    """ 时间  $O(n)$ ，空间  $O(1)$ 。同时维护 cur_max 和 cur_min（负数翻转）。 """
    cur_max = cur_min = ans = nums[0]
    for x in nums[1:]:
        # 先计算新值（必须用旧的 cur_max / cur_min!）
        new_max = max(x, x * cur_max, x * cur_min)
        new_min = min(x, x * cur_max, x * cur_min)
        cur_max, cur_min = new_max, new_min
        ans = max(ans, cur_max)
    return ans

# 另一种等价写法：遇到负数时主动交换 cur_max 和 cur_min
def maxProduct_v2(nums: List[int]) -> int:
    cur_max = cur_min = ans = nums[0]
    for x in nums[1:]:
        if x < 0:
            cur_max, cur_min = cur_min, cur_max  # 负数翻转后大小关系互换
        cur_max = max(x, x * cur_max)
        cur_min = min(x, x * cur_min)
        ans = max(ans, cur_max)
    return ans
```

为什么要同时维护 *cur_min*? 因为负数 $\times$ 负数 = 正数，一个很小的负值 (*cur_min*) 乘以下一个负数元素可能变成当前最大值。仅维护 *cur_max* 会漏掉这条“负 $\times$ 负”路径。注意先保存旧值再更新（或用临时变量），避免 *cur_max* 更新后影响 *cur_min* 的计算。

套路 3：环形变体（两路 Kadane）

适用题：918

```
def maxSubarraySumCircular(nums: List[int]) -> int:
    """ 时间  $O(n)$ ，空间  $O(1)$ 。
    两种情况：
    1. 最大子数组不跨越边界  $\rightarrow$  标准 Kadane 的 max_sum
    2. 最大子数组跨越边界  $\rightarrow total - min\_sum$ （去掉中间最小子数组）
    全负数特殊情况： $total - min\_sum == 0$  ( $min\_sum == total$ )，返回 max_sum。
    """
```

```
"""
total = 0
cur_max = cur_min = 0
max_sum = min_sum = nums[0]

for x in nums:
    cur_max = max(x, cur_max + x)
    max_sum = max(max_sum, cur_max)
    cur_min = min(x, cur_min + x)
    min_sum = min(min_sum, cur_min)
    total += x

# 全负数时 total - min_sum == 0, 应返回 max_sum（最大的那个负数）
if max_sum < 0:
    return max_sum
return max(max_sum, total - min_sum)
```

情况 2 的直觉：环形数组中”跨越首尾的子数组”= 整个数组去掉中间某段连续子数组；为使保留部分（首尾）最大，等价于让中间去掉的部分最小，即求最小子数组和 min_sum；跨越情况的最大值 = total - min_sum。

速查表

题型特征	套路	时间	空间
最大子数组和（线性）	标准 Kadane（cur + x / x 取大）	O(n)	O(1)
最大子数组乘积	双变量 Kadane（cur_max + cur_min）	O(n)	O(1)
环形数组最大子数组和	两路 Kadane（max + min），取 max	O(n)	O(1)
最大子数组（含下标）	Kadane + 记录起止下标	O(n)	O(1)

方法变形（3 类）

变形 1：Kadane 系列扩展

- 53（标准）→ 918（环形）→ 用”total - min_sum”转化环形为线性问题。

- **152** (最大乘积) → 双变量 Kadane, 同时维护 `cur_max` 和 `cur_min`。
- **1749** (绝对值最大子数组和, 非 LC150) → 最大子数组和与最小子数组和取绝对值的较大值。
- **363** (矩形区域不超过 K 的最大数值和, 非 LC150) → 将二维问题压缩为一维, 枚举上下行边界, 对每列前缀和用 Kadane + 有序集合 (`SortedList`)。

变形 2: 带约束的 Kadane

- 标准 **Kadane** 的限制: 子数组必须连续, 长度 ≥ 1 。
- 若允许空子数组 (长度 ≥ 0), `ans` 初始化为 0 即可 (最大和至少为 0)。
- **K 约束** (最大子数组和 $\leq K$) → 不能直接用 Kadane; 需前缀和 + 有序集合 (`SortedList.bisect_left`), $O(n \log n)$ 。
- **152 特例 (含 0)**: 遇到 0 时, `cur_max = max(0, ...)` = 0 (乘积清零), `cur_min = min(0, ...)` = 0; 0 将乘积链断开, 效果类似 Kadane 中负数前缀的重置。

变形 3: 在线数据流与滑动窗口

- **Kadane 本质**是在线算法: 每次 $O(1)$ 处理新元素, 适合 streaming 数据。
- 滑动窗口最大子数组 (固定窗口大小, 非 LC150) → 单调队列 (`deque`), $O(n)$ 。
- 在线极值流 (AI 特征监控): 实时监控时序数据中的最大连续增益段, Kadane 直接应用; 乘积变体用于波动率最大区间检测。
- AI 场景:
 - Transformer 注意力分数的时序片段选择 (最大相关性子序列) 类 Kadane。
 - 强化学习的奖励函数中, 累积最大奖励段的检测等同于 918 的环形变体。
 - 时序异常检测: 最大正偏差子数组 (Kadane) vs 最大负偏差子数组 (取反后 Kadane)。

思考路标 (条件反射)

1. 看到“最大子数组和 / **maximum subarray**” → 标准 Kadane, `cur = max(x, cur + x)`, `ans = max(ans, cur)`
2. 看到“最大子数组乘积 / **maximum product**” → 双变量 Kadane, 同时维护 `cur_max` 和 `cur_min`
3. 看到“环形数组 / **circular array**” → 两路 Kadane: `max_sum` 和 `min_sum`; 答案 = `max(max_sum, total - min_sum)`; 全负时特判
4. 看到“**ans 初始化为 0**” → 先检查题意: 若允许空子数组则初始化 0; 若要求非空子数组则初始化 `nums[0]`
5. 看到“包含负数的乘积” → 必须同时跟踪最大值和最小值, 负 $\times$ 负翻转
6. 看到“子数组 (连续)” → Kadane; “子序列 (可跳过)” → DP (不同类问题, 不要混用)

易错点

1. **ans** 初始化不能为 0：全负数数组时，Kadane 的 `cur` 永远不会达到 0，`ans = 0` 是错的；初始化 `ans = nums[0]` 才能正确处理全负数情况。
2. **152** 先算再赋值：`new_max = max(x, x * cur_max, x * cur_min)`；`new_min = ...` 必须用旧值算完 `new_max` 后才更新 `cur_max`；否则 `cur_max = max(x, x * cur_max, x * cur_min)`；`cur_min = min(x, x * cur_max, ...)` 中第二行用的 `cur_max` 已经是更新后的值，导致错误。
3. **918** 全负数特判：若 `max_sum < 0`（所有元素均负），则 `total - min_sum = 0`（`min_sum = total`），意味着去掉所有元素得 0，但子数组必须非空，因此应返回 `max_sum` 而非 `total - min_sum`。
4. **918 min_sum** 初始化：和 `max_sum` 一样，`min_sum = nums[0]`（不能为 `float('inf')`，因为 Kadane 变量 `cur_min` 从 0 开始会给出错误的 0 初始值）；但若 `cur_max / cur_min` 从 0 开始则需注意对应调整。统一在上面模板中 `cur_max = cur_min = 0`，`min_sum = max_sum = nums[0]` 是安全写法。
5. 子数组 vs 子序列：Kadane 解决的是“连续子数组”问题；若题目允许跳过元素（子序列），则应用 DP 而非 Kadane，混用会得到错误答案。
6. **152** 含 0 的情况：遇到 0 时，`x * cur_max = 0`，`x * cur_min = 0`，`max(0, 0, 0) = 0`；`cur_max` 和 `cur_min` 都变为 0，等价于从下一个元素重新开始，行为正确，不需要特判。

典型应用例题

例 1：53. Maximum Subarray

题目：给定整数数组，找到具有最大和的连续子数组，返回其和。

思路：标准 Kadane。维护 `cur`（以当前位置结尾的最大子数组和）和 `ans`（全局最大）。`cur = max(x, cur + x)`：若 `cur + x < x`，说明之前的前缀和为负，拖累了当前，应从当前元素重新开始。

解：

参考：[solutions/kadane/p053_maximum_subarray.py](#)

```
def maxSubArray(nums: List[int]) -> int:
    cur = ans = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        ans = max(ans, cur)
    return ans
```

分析： $O(n)$ 时间， $O(1)$ 空间。每个元素恰好访问一次，`cur` 和 `ans` 各 $O(1)$ 更新。对比暴力 $O(n^2)$ 和分治 $O(n \log n)$ ，Kadane 是最优解。

DP 视角：令 `dp[i]` = 以 `nums[i]` 结尾的最大子数组和，则 `dp[i] = max(nums[i], dp[i-1] + nums[i])`，答案 = `max(dp)`。Kadane 就是这个 DP 的滚动变量优化版本（`dp[i]` 只依赖 `dp[i-1]`）。

例 2: 918. Maximum Sum Circular Subarray

题目: 给定一个循环整数数组 (首尾相接), 找到具有最大和的非空连续子数组 (可以跨越首尾), 返回最大和。

思路: 分两种情况: - 情况 1: 最大子数组不跨越边界 → 标准 Kadane 求 `max_sum`。- 情况 2: 最大子数组跨越边界 → `total - min_sum` (去掉中间最小子数组, 用 Kadane 的“取 min”变体求 `min_sum`)。答案 = `max(max_sum, total - min_sum)`; 若 `max_sum < 0` (全负数), 返回 `max_sum`。

解:

参考: [solutions/kadane/p918_maximum_sum_circular_subarray.py](#)

```
def maxSubarraySumCircular(nums: List[int]) -> int:
    total = 0
    cur_max = cur_min = 0
    max_sum = min_sum = nums[0]
    for x in nums:
        cur_max = max(x, cur_max + x)
        max_sum = max(max_sum, cur_max)
        cur_min = min(x, cur_min + x)
        min_sum = min(min_sum, cur_min)
        total += x
    if max_sum < 0:
        return max_sum
    return max(max_sum, total - min_sum)
```

分析: $O(n)$ 时间, $O(1)$ 空间。两路 Kadane 在同一次遍历中完成, 无需两次 pass。全负数情况 `total - min_sum = total - total = 0`, 这是非法的 (空子数组), 因此 `max_sum < 0` 时直接返回 `max_sum`。

自测题

自测 1 (53 Maximum Subarray) ——`nums=[-2,1,-3,4,-1,2,1,-5,4]` 返回 6 (子数组 `[4,-1,2,1]`) ; `nums=[1]` 返回 1; `nums=[5,4,-1,7,8]` 返回 23。提示: `cur = ans = nums[0]`, 遍历从 `nums[1:]` 开始, `cur = max(x, cur + x)`, `ans = max(ans, cur)`, 注意全负数时 `ans` 应为最大负数。参考 [solutions/kadane/p053_maximum_subarray.py](#)。

自测 2 (152 Maximum Product Subarray, 同类练习) ——`nums=[2,3,-2,4]` 返回 6 (子数组 `[2,3]`) ; `nums=[-2,0,-1]` 返回 0; `nums=[-2,3,-4]` 返回 24 (子数组 `[-2,3,-4]`)。提示: `cur_max = cur_min = ans = nums[0]`, 遍历时先用临时变量保存 `new_max = max(x, x*cur_max, x*cur_min)`, `new_min = min(...)`, 同步更新, `ans = max(ans, cur_max)`。

自测 3 (918 Maximum Sum Circular Subarray) ——`nums=[1,-2,3,-2]` 返回 3 (子数组 `[3]`); `nums=[5,-3,5]` 返回 10 (子数组 `[5,5]`, 跨越首尾); `nums=[-3,-2,-3]` 返回 -2 (全负数, 返回最大负数)。提示: 同时跑 max Kadane

和 min Kadane，累加 total；max_sum < 0 时返回 max_sum，否则返回 max(max_sum, total - min_sum)。参考 solutions/kadane/p918_maximum_sum_circular_subarray.py。

题目全览（2 题）

#	题目	套路分类	难度
53	Maximum Subarray	标准 Kadane	Easy（但思想深刻）
918	Maximum Sum Circular Subarray	两路 Kadane（max + min）	Medium

融合版说明

段	来源	价值
一例速记	本文件	3 大变体一览（标准/乘积/环形）+ AI 场景
思维路径还原	本文件	3 道题（含 152 乘积变体）的解题内心独白
抽象成方法	本文件	3 个标准模板（标准 Kadane/乘积双变量/环形两路）+ 速查表
方法变形	本文件	3 类变体（Kadane 系列/带约束/在线数据流）+ AI 关联
思考路标	本文件	6 条题型识别条件反射
易错点	本文件	6 条高频踩坑（初始化/先算再赋值/全负特判等）
典型应用例题	solutions/	2 道精讲（53、918）+ 乘积变体说明，代码 + DP 视角
自测题	leetcode	3 题带提示（含 152 乘积变体练习），链接 solutions 文件
题目全览	本文件	2 题完整列表，套路分类一览

跨 category 导航：- Kadane 本质是 1D DP 的滚动变量优化 → 见 10-dp-1d.md（dp[i] 只依赖 dp[i-1]）
- 环形数组的另一类问题（如环形链表）→ 见 12-linked-list.md（快慢指针判环）- 滑动窗口最大值（固定窗口大小）→ 见 03-sliding-window.md（单调队列）- Kadane 在时序信号处理和在线特征监控中是”在线算法”的代表性例子，与 streaming 数据处理框架（Flink、Kafka Streams）的窗口聚合在概念上高度一致

18 —Graph BFS (融合版)

难度: □□□□ 题数: 3 核心套路: 最短路径 BFS (无权图)、字符串变换 BFS、蛇梯棋盘 BFS 本文件: 覆盖 graph_bfs 3 题的算法套路总结 + 典型题精讲 + 自测

一例速记

最短路径 **BFS** (无权图): 队列 + 已访问集合, 按”层”扩展节点, 每推进一层距离 +1; BFS 天然保证第一次到达终点时路径最短 (127 / 433) 字符串变换 **BFS**: 将每个单词/基因序列视为图中节点, 两节点有边当且仅当它们恰好相差一个字符; BFS 找最短变换链 (127 Word Ladder / 433 Minimum Genetic Mutation) 双向 **BFS** 优化: 从起点和终点同时扩展, 当两侧相遇时路径即为最短; 将搜索空间从 $O(b^d)$ 压缩到 $O(b^{d/2})$ (127 进阶优化) 棋盘 **BFS** / 状态压缩: 将二维棋盘格子编号, 记录已访问格子, 处理蛇和梯子的跳转 (909 Snakes and Ladders) **AI** 关联: 图神经网络 (GNN) 消息传递——每一轮聚合邻居信息, 与 BFS 按层传播结构完全对应; 知识图谱推理中的多跳问答 (multi-hop QA) 也依赖图的层次遍历

思维路径还原

“看到 ‘最短变换路径’ (127) → 立刻想 BFS: 把 wordList 转成 set ($O(1)$ 查找), 队列初始放入 (beginWord, 1) (单词, 当前步数)。对当前单词的每个字符位置, 逐一枚举 26 个字母, 若变换后在 word_set 中且未访问, 则入队, 并从 word_set 中删除 (标记已访问)。找到 endWord 时返回当前步数; 队列为空则返回 0。时间 $O(26 \times L \times N)$, 其中 L 为单词长度, N 为单词表大小。

看到 ‘433 最小基因变化’ → 与 127 完全相同框架: 8 个字符, 字符集为 {'A', 'C', 'G', 'T'} (只有 4 种), bank 作为合法节点集合。代码几乎可以直接复用, 只需把字符集从 26 个字母换成 4 个碱基。

看到 ‘909 蛇和梯子’ → 棋盘 BFS: 先将 $n \times n$ 棋盘”蛇形展开”为一维数组 (注意奇偶行方向交替), 然后从格子 1 开始 BFS, 每次从当前格子出发掷骰子 (1~6), 到达的格子若有蛇/梯子则跳转 (取 board 数组的值 -1), 若已访问则跳过, 否则入队。到达格子 $n^2 - 1$ 时返回步数。关键: 坐标变换——将格子编号 s 转为 (row, col), 注意行从底部向上、奇偶行列方向不同。”

学习目标

- 掌握无权图 BFS 最短路径模板: 队列 + 访问集合 + 按层扩展
- 熟练字符串变换 BFS: 逐字符枚举替换, 从 word_set 中删除标记已访问
- 理解双向 BFS 的剪枝原理及适用场景 (起终点已知、分支因子较大时)

- 掌握棋盘编号 ↔ 坐标变换（蛇形展开）的技巧
- 能识别”最短步数 / 最少操作”题型并直接套 BFS 模板

几何示意

图网格最短路径 BFS



图 37: 4x4 网格 + 起终点 + 蓝色层级渐变

抽象成方法（标准模板代码）

套路
1:
BFS
最短
路径
(无权
图通
用模
板)
适用
题:
127、
433、
909
“python
from
collec-
tions
im-
port
deque
from
typing
im-
port
List

```

def
bfs_shortest_path(start,
end,
neigh-
bors_fn)
-> int:
"""无
权图
BFS
最短
路径
通用
模板。
neigh-
bors_fn(node)
->
List[node]:
返回
邻居
节点
列表。
返回
从
start
到
end
的最
短距
离,
不可
达返
回
-1。"""
if start
==
end:
return
0 vis-
ited =
{start}
queue
=
deque([start])
dist =

```

> 关键不
变式:
每轮
循环
处理
同一”
层”的
所有
节点,
dist
代表
当前
层到
起点
的距
离。>
先检
查终
点再
入队
(Early
Exit)
可减
少一
层多
余扩
展。

套路 2: 字符串变换 BFS (Word Ladder / Genetic Mutation)

适用题: 127、433

```
def word_ladder_bfs(begin: str, end: str, word_set: set[str]) -> int:
    """
    127 / 433 通用框架: 每次变换一个字符, 求最短变换步数。
    时间  $O(26 * L * N)$ , 空间  $O(N)$ ,  $L$ = 单词长度,  $N$ = 词典大小。
    """
    if end not in word_set:
        return 0
```

```

queue = deque([(begin, 1)])
visited = {begin}
alphabet = 'abcdefghijklmnopqrstuvwxyz' # 127 用 26 字母

while queue:
    word, steps = queue.popleft()
    for i in range(len(word)):
        for c in alphabet:
            if c == word[i]:
                continue
            new_word = word[:i] + c + word[i+1:]
            if new_word == end:
                return steps + 1
            if new_word in word_set and new_word not in visited:
                visited.add(new_word)
                queue.append((new_word, steps + 1))
return 0

```

```
def min_mutation_bfs(start: str, end: str, bank: List[str]) -> int:
```

```
    """
```

433: 基因变化, 字符集为 {'A', 'C', 'G', 'T'}, bank 为合法节点集合。

时间 $O(4 * 8 * N)$, 空间 $O(N)$ 。

```
    """
```

```
    bank_set = set(bank)
```

```
    if end not in bank_set:
```

```
        return -1
```

```
    queue = deque([(start, 0)])
```

```
    visited = {start}
```

```
    gene_chars = 'ACGT'
```

```
    while queue:
```

```
        gene, steps = queue.popleft()
```

```
        for i in range(len(gene)):
```

```
            for c in gene_chars:
```

```
                if c == gene[i]:
```

```
                    continue
```

```
                new_gene = gene[:i] + c + gene[i+1:]
```

```
                if new_gene == end:
```

```
                    return steps + 1
```

```
                if new_gene in bank_set and new_gene not in visited:
```



```

        visited.add(new_gene)
        queue.append((new_gene, steps + 1))

    return -1

```

套路 3: 双向 BFS 优化 (Word Ladder 进阶)

适用题: 127 (大规模输入时)

```

def word_ladder_bidirectional(begin: str, end: str, word_list: List[str]) -> int:
    """
    双向 BFS: 从 begin 和 end 同时向中间扩展, 相遇即终止。
    将时间复杂度从  $O(b^d)$  降到  $O(b^{(d/2)})$ ,  $b$ = 分支因子,  $d$ = 最短路径长度。
    """
    word_set = set(word_list)
    if end not in word_set:
        return 0

    front, back = {begin}, {end}    # 两端的当前层节点集合
    visited = {begin, end}
    steps = 1

    while front and back:
        # 总是扩展较小的一侧 (BFS 平衡优化)
        if len(front) > len(back):
            front, back = back, front

        next_front = set()
        for word in front:
            for i in range(len(word)):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    new_word = word[:i] + c + word[i+1:]
                    if new_word in back:    # 两侧相遇
                        return steps + 1
                    if new_word in word_set and new_word not in visited:
                        visited.add(new_word)
                        next_front.add(new_word)

        front = next_front
        steps += 1
    return 0

```

套路 4: 棋盘 BFS + 坐标变换 (Snakes and Ladders)

适用题: 909

```
def snakes_and_ladders(board: List[List[int]]) -> int:
    """
    909:  $n \times n$  蛇形棋盘, 格子  $1..n^2$  编号, 蛇/梯子用 board 矩阵记录。
    BFS 求从格子 1 到格子  $n^2$  的最少步数。时间  $O(n^2)$ , 空间  $O(n^2)$ 。
    """
    n = len(board)
    total = n * n

    def label_to_coord(s: int):
        """ 将格子编号 s (1-indexed) 转为 board[row][col]。 """
        s -= 1  # 转为 0-indexed
        row_from_bottom = s // n  # 从底部算第几行
        col_in_row = s % n
        # 偶数行 (从底部) 从左到右, 奇数行从右到左
        if row_from_bottom % 2 == 0:
            col = col_in_row
        else:
            col = n - 1 - col_in_row
        row = n - 1 - row_from_bottom
        return row, col

    visited = {1}
    queue = deque([(1, 0)])  # (当前格子编号, 步数)

    while queue:
        pos, steps = queue.popleft()
        for dice in range(1, 7):  # 掷骰子 1~6
            nxt = pos + dice
            if nxt > total:
                break
            r, c = label_to_coord(nxt)
            dest = board[r][c]
            if dest != -1:  # 有蛇 / 梯子, 跳转
                nxt = dest
            if nxt == total:
                return steps + 1
            if nxt not in visited:
                visited.add(nxt)
```

```
        queue.append((nxt, steps + 1))
    return -1
```

速查表

题型特征	套路	时间	空间
无权图最短路径	BFS 按层扩展	$O(V + E)$	$O(V)$
字符串单字符变换最短链	BFS + 字符枚举	$O(26 \times L \times N)$	$O(N)$
字符集小（4 种）基因变化	BFS + 字符集枚举	$O(4 \times L \times N)$	$O(N)$
两端已知、分支因子大	双向 BFS	$O(b^{d/2})$	$O(b^{d/2})$
棋盘格子最短步数	BFS + 蛇形坐标变换	$O(n^2)$	$O(n^2)$

方法变形（3 类）

变形 1：Word Ladder 系列

- 127（Word Ladder）：字符集 26 字母，返回步数（含起始词），不可达返回 0。
- 433（Minimum Genetic Mutation）：字符集 4 碱基，返回步数（不含起始），bank 作为合法集合，不可达返回 -1。
- 框架完全相同，差异仅在字符集、合法节点集合、以及返回值定义（含不含起始节点）。

变形 2：BFS 与 Dijkstra 的选择

- 无权图（每条边权重相同）→ BFS 即可， $O(V + E)$ 。
- 有权图（边权不同）→ Dijkstra（最小堆）， $O((V + E) \log V)$ 。
- 本 category 3 题均为无权图（每次变换 = 步数 + 1），直接用 BFS。

变形 3：909 坐标变换的常见错误

- 棋盘蛇形展开：行号从底部开始（row_from_bottom = s // n），偶数行从左到右，奇数行从右到左。
- 访问判断要在跳转（蛇/梯子）之后记录，避免同一目标格子被重复入队。
- 掷骰子上限：nxt = pos + dice 可能超过 total，需要 if nxt > total: break（6 面骰，超过则后续也不用试）。

思考路标 (条件反射)

1. 看到“最短步数 / 最少操作次数 / 无权图” → BFS
 2. 看到“两个字符串相差一个字符” → 字符串变换 BFS, 枚举每个位置的替换
 3. 看到“word list / bank 作为合法节点” → 转成 set, BFS 时从 set 删除已访问 (等价于 visited 集合)
 4. 看到“双向 BFS” → 起终点都已知 + 分支因子较大时, 分别从两端扩展, 取较小的一侧推进
 5. 看到“棋盘 / 蛇和梯子” → BFS + 坐标变换, 注意蛇形展开奇偶行方向
 6. 看到“有权图最短路” → 跳到 Dijkstra (最小堆), 不要用 BFS
 7. 看到“GNN 消息传递 / 图卷积” → 类比 BFS 按层聚合邻居特征
-

易错点

1. 127 返回值含起始节点: 题目要求返回“变换序列的长度”(包含 beginWord), 所以初始 steps=1, 到达 endWord 时返回 steps+1; 不少人初始化为 0 导致差 1。
 2. 433 返回值不含起始节点: 与 127 相反, 返回的是变换次数 (不含 startGene), 注意两题定义差异。
 3. word_set 删除 vs visited 集合: 两者等价, 但直接从 word_set 中 discard 已访问节点可以省去 visited 集合; 若不删除则需单独维护 visited, 否则同一节点会被重复入队导致超时。
 4. 909 坐标变换: row_from_bottom 决定行的奇偶性 (从底部数), 易与“从顶部数”混淆; 建议单独写 label_to_coord 函数并用简单用例验证 (n=2 时格子 1 对应 board[1][0])。
 5. 909 掷骰子边界: pos + dice 可能超过 n^2 , 若超过则不合法, 需 break (因为 dice 从 1~6 递增, 一旦超过后续都超过)。
 6. 双向 BFS 步数计算: steps 变量代表“已扩展了多少层”, 相遇时答案是 steps + 1 (前向已走 steps 步, 后向再走 1 步相遇); 若两侧都已经扩展了 steps/2 层则相遇时答案为 steps, 根据实现方式仔细校验。
-

典型应用例题

例 1: 127. Word Ladder

题目: 给定 beginWord、endWord 和单词表 wordList, 每次只能改变一个字母, 且变换后的词必须在 wordList 中。求从 beginWord 变换到 endWord 的最短序列长度 (含首尾), 若不可达返回 0。

思路: 将每个单词视为图节点, 两词相差一个字母则有边 (无权)。BFS 从 beginWord 出发, 逐层扩展, 第一次到达 endWord 时的层数即为答案。枚举邻居时, 逐字符位置替换 26 个字母, 若在 word_set 中则为合法邻居。

解:

参考: [solutions/graph_bfs/p127_word_ladder.py](#)

```
def ladderLength(beginWord: str, endWord: str, wordList: List[str]) -> int:
```

```

word_set = set(wordList)
if endWord not in word_set:
    return 0
queue = deque([(beginWord, 1)])
visited = {beginWord}
while queue:
    word, steps = queue.popleft()
    for i in range(len(word)):
        for c in 'abcdefghijklmnopqrstuvwxyz':
            new_word = word[:i] + c + word[i+1:]
            if new_word == endWord:
                return steps + 1
            if new_word in word_set and new_word not in visited:
                visited.add(new_word)
                queue.append((new_word, steps + 1))
return 0

```

分析：单词长度 L ，词典大小 N 。每个单词生成 $26L$ 个候选邻居，每次哈希查找 $O(L)$ ，总体 $O(26L^2N)$ ；因 L 通常很小（5~8），实际运行快。BFS 保证最短路径。

例 2: 433. Minimum Genetic Mutation

题目：基因序列长度固定为 8，字符集为 {'A', 'C', 'G', 'T'}。给定 startGene、endGene 和合法基因库 bank，每次变换一个字符且结果必须在 bank 中。求最少变换次数，不可达返回 -1。

思路：与 127 框架相同，字符集缩小到 4 个字符，合法节点从 wordList 换成 bank。注意返回值定义与 127 不同（不含起始节点）。

解：

```

# 参考: solutions/graph_bfs/p433_minimum_genetic_mutation.py
def minMutation(startGene: str, endGene: str, bank: List[str]) -> int:
    bank_set = set(bank)
    if endGene not in bank_set:
        return -1
    queue = deque([(startGene, 0)])
    visited = {startGene}
    while queue:
        gene, steps = queue.popleft()
        for i in range(len(gene)):
            for c in 'ACGT':
                new_gene = gene[:i] + c + gene[i+1:]

```

```

    if new_gene == endGene:
        return steps + 1
    if new_gene in bank_set and new_gene not in visited:
        visited.add(new_gene)
        queue.append((new_gene, steps + 1))

return -1

```

分析：基因长度固定为 8，字符集 4 个，bank 大小 N 。时间 $O(4 \times 8 \times N) = O(32N)$ ，空间 $O(N)$ 。

例 3：909. Snakes and Ladders

题目： $n \times n$ 棋盘，格子从底部按蛇形编号 $1 \sim n^2$ 。board[r][c] 为 -1（正常格子）或目标格子编号（蛇/梯子）。从格子 1 出发，每次掷骰子（1~6），若落点有蛇/梯子则自动跳转。求到达格子 n^2 的最少步数，不可达返回 -1。

思路：BFS。将格子编号展开为状态，队列存储（当前格子，步数）。关键是将格子编号转为棋盘坐标（蛇形展开，偶数行从左到右，奇数行从右到左，行从底部向上）。

解：

参考：[solutions/graph_bfs/p909_snakes_and_ladders.py](#)

```
def snakesAndLadders(board: List[List[int]]) -> int:
```

```
    n = len(board)
```

```
    total = n * n
```

```
    def label_to_coord(s: int):
```

```
        s -= 1
```

```
        q, r = divmod(s, n)
```

```
        col = r if q % 2 == 0 else n - 1 - r
```

```
        row = n - 1 - q
```

```
        return row, col
```

```
visited = {1}
```

```
queue = deque([(1, 0)])
```

```
while queue:
```

```
    pos, steps = queue.popleft()
```

```
    for dice in range(1, 7):
```

```
        nxt = pos + dice
```

```
        if nxt > total:
```

```
            break
```

```
        r, c = label_to_coord(nxt)
```

```
        if board[r][c] != -1:
```

```
        nxt = board[r][c]
    if nxt == total:
        return steps + 1
    if nxt not in visited:
        visited.add(nxt)
        queue.append((nxt, steps + 1))

    return -1
```

分析：格子数 n^2 ，每个格子最多入队一次，每次最多扩展 6 个邻居。时间 $O(n^2)$ ，空间 $O(n^2)$ 。

自测题

自测 1(127 Word Ladder)——beginWord='hit', endWord='cog', wordList=['hot','dot','dog','lot','log','cog'] 应返回 5 (hit→hot→dot→dog→cog); wordList 不含 'cog' 时返回 0。提示：BFS, word_set 记录合法节点，逐字符枚举 26 字母替换，first reach endWord 时返回步数 +1。参考 solutions/graph_bfs/p127_word_ladder.py。

自测 2(433 Minimum Genetic Mutation)——startGene='AACCGGTT', endGene='AACCGGTA', bank=['AACCGGTA'] 返回 1; bank=[] 返回 -1。提示：框架同 127，字符集换成 'ACGT'，不可达返回 -1 而非 0。参考 solutions/graph_bfs/p433_minimum_genetic_mutation.py。

自测 3 (909 Snakes and Ladders) —— $n = 2$ 的棋盘格子编号为 [3,4],[1,2]（底行左起），board[1][0]=3 表示格子 1 有梯子跳到格子 3。手动验证坐标变换：格子 1 → (1,0)，格子 2 → (1,1)，格子 3 → (0,1)，格子 4 → (0,0)。提示：BFS, label_to_coord 函数须处理奇偶行方向，跳转后记录 nxt 再判断是否已访问。参考 solutions/graph_bfs/p909_snakes_and_ladders.py。

题目全览（3 题）

#	题目	套路分类	难度
127	Word Ladder	字符串变换 BFS + 26 字母枚举	Hard
433	Minimum Genetic Mutation	字符串变换 BFS + 4 碱基枚举	Medium
909	Snakes and Ladders	棋盘 BFS + 蛇形坐标变换	Medium

融合版说明

段	来源	价值
一例速记	本文件	3 题套路一览 + AI（GNN 消息传递）关联
思维路径还原	本文件	从题目条件到 BFS 实现的解题独白
抽象成方法	本文件	4 个标准模板（通用 BFS / 字符串变换 BFS / 双向 BFS / 棋盘 BFS）+ 速查表
方法变形	本文件	3 类变体（Word Ladder 系列 / BFS vs Dijkstra / 坐标变换易错）
思考路标	本文件	7 条题型识别条件反射
易错点	本文件	6 条高频踩坑（返回值定义 / 坐标变换 / 边界处理）
典型应用例题	solutions/	3 道精讲（127、433、909），代码 + 复杂度分析
自测题	leetcode	3 题带提示，链接 solutions 文件
题目全览	本文件	3 题完整列表，套路分类一览

跨 category 导航：- 有权图最短路 → Dijkstra（优先队列），见 heap category - BFS 按层思路同样适用于树的层序遍历 → 见 06-binary-tree-bfs.md - 图的 DFS 遍历 / 连通分量 → 见 19-graph-general.md - GNN 中每层聚合邻居信息 = BFS 每层扩展，是图深度学习的核心操作

19 —Graph General（融合版）

难度：□□□□□ 题数：6 核心套路：DFS 连通分量 / 岛屿计数、Union-Find、拓扑排序（Kahn / DFS）、带权图 BFS 求比值 本文件：覆盖 graph_general 6 题的算法套路总结 + 典型题精讲 + 自测

一例速记

DFS 连通分量 / 岛屿计数：遍历所有节点，对未访问的节点做 DFS/BFS，每次启动一次 DFS 即发现一个新连通分量（200 Number of Islands / 130 Surrounded Regions）**DFS 标记边界连通：**130 题先从边界的‘O’出发 DFS 标记”安全”区域，再把未标记的‘O’变成‘X’（染色法，避免判断每个‘O’是否连通到边界）**Union-Find（并查集）：**路径压缩 + 按秩合并，近乎 $O(1)$ 的 find/union；用于动态连通性（200 / 547 Friend Circles）克隆图 **DFS + 哈希表：**visited = {old_node: new_node}，DFS

遍历原图，克隆节点时先查哈希表防止重复克隆 (133) 拓扑排序 **Kahn**: 入度数组 + 队列，每次取入度为 0 的节点，处理后减少邻居入度，若最终处理节点数 = 总节点数则无环 (207 / 210 Course Schedule) 拓扑排序 **DFS** 三色: 白 (未访问) → 灰 (进栈中) → 黑 (已完成)，灰色节点再次被访问则有环 (207) 带权图 **BFS / DFS** 传递比值: 399 Evaluate Division, 将除法关系建成带权有向图 ($A/B = k$ 则 $A \rightarrow B$ 权 k , $B \rightarrow A$ 权 $1/k$), DFS/BFS 求路径权重之积 **AI** 关联: ML 流水线的 DAG 依赖管理 (TensorFlow 计算图 / PyTorch autograd)、编译器的指令调度 (Toposort)、微服务熔断检测 (连通性)

思维路径还原

“看到 ‘200 岛屿数量’ → DFS/BFS 遍历 + 计数: 双重循环扫描 grid, 遇到 ‘1’ 则岛屿数 +1, 然后 DFS 将整个连通的 ‘1’ 区域全部标记为已访问 (改为 ‘0’ 或 ‘#’)。时间 $O(m \cdot n)$, 空间 $O(m \cdot n)$ (递归栈最坏情况全是岛屿)。

看到 ‘130 被围绕的区域’ → 反向思维: 不是问哪些 ‘O’ 被围, 而是先找哪些 ‘O’ 安全 (连通到边界)。从四条边界上的所有 ‘O’ 出发 DFS, 标记为临时字符 ‘#’ (安全); 最后遍历全图: ‘#’ 恢复为 ‘O’ (安全的), ‘O’ 改为 ‘X’ (被围的), ‘X’ 保持不变。

看到 ‘133 克隆图’ → DFS + 哈希表: `visited = {original_node: cloned_node}` 记录已克隆的节点, 防止重复克隆导致死循环。DFS 时, 若邻居已在 `visited` 中则直接取克隆节点, 否则新建节点后递归克隆其邻居。

看到 ‘547 省份数量’ → 连通分量计数, 可用 DFS 或 Union-Find: DFS 版: 同 200 岛屿, 对邻接矩阵的每个未访问节点做 DFS; Union-Find 版: 遍历边, `union(i, j)`, 最后统计根节点数量。

看到 ‘207 / 210 课程表’ → 拓扑排序: 207 只问是否有环 (能否完成所有课程) → Kahn 算法, 若处理节点总数 < numCourses 则有环; 210 还要返回修课顺序 → Kahn 输出处理顺序, 若有环返回 []。

看到 ‘399 除法求值’ → 建带权图, DFS 求路径权重积: 每个变量是节点, $A/B=k$ 建边 $A \rightarrow B$ 权 k , $B \rightarrow A$ 权 $1/k$ 。对每个 query (C, D), DFS 从 C 出发找到 D 的路径并累乘权重, 找不到则返回 -1.0。”

学习目标

- 掌握 DFS 连通分量模板: 未访问节点启动 DFS, 递归标记所有连通点
 - 理解 “边界逆向标记” 技巧 (130), 避免逐个判断连通性
 - 实现路径压缩 + 按秩合并的 Union-Find, 熟悉其在连通性问题中的应用
 - 掌握 Kahn 拓扑排序 (入度队列法) 和 DFS 三色法 (环检测)
 - 理解带权图的构建方式 (399), DFS/BFS 在带权图上传递权重
-

几何示意

图 DFS 连通分量（LC 200 岛屿）

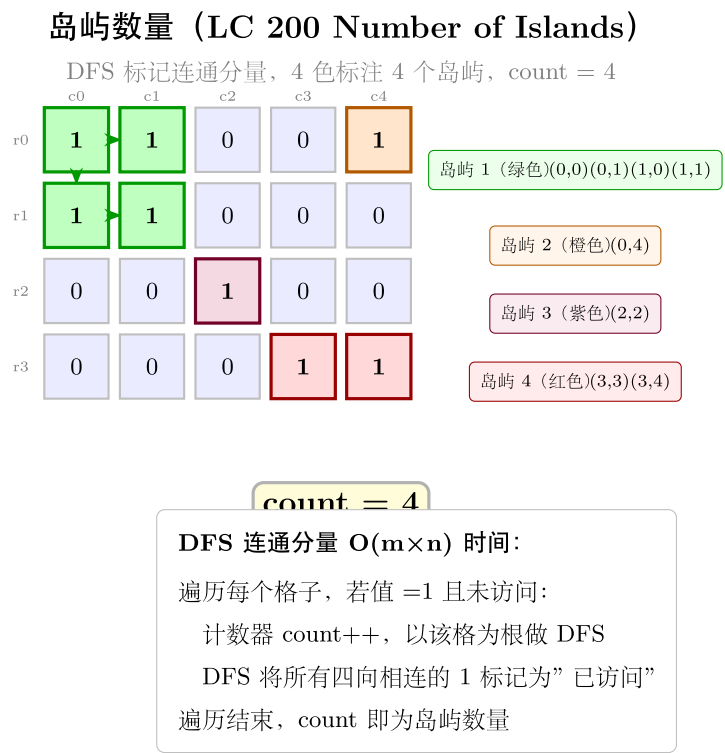


图 38: 4x5 网格 + 4 色岛屿

图 Union-Find 路径压缩

抽象方法（标准模板代码）

套路
1:
DFS
连通
分量
(岛屿
计数)
适用
题:
200、
547
“python
from
typing
im-
port
List

```

def
num_islands(grid:
List[List[str]])
-> int:
    """200:
DFS
连通
分量
计数,
时间
O(m ·
n),
空间
O(m ·
n)
(递归
栈)"""
    if not
grid:
return
0 m, n
=
len(grid),
len(grid[0])
count
= 0

```

```
def
dfs(r:
int, c:
int) ->
None:
if r <
0 or r
>= m
or c <
0 or c
>= n
or
grid[r][c]
!= '1':
return
grid[r][c]
= '#'#
标记
已访
问,
避免
回头
for dr,
dc in
[(-1,
0), (1,
0), (0,
-1), (0,
1)]:
dfs(r +
dr, c +
dc)
```

```

for r in
range(m):
for c
in
range(n):
if
grid[r][c]
== '1':
count
+= 1
dfs(r,
c)
return
count
"""

```

套路 2: 边界逆向 DFS (被围区域)

适用题: 130

```

def solve_surrounded(board: List[List[str]]) -> None:
    """
    130: 将非边界连通的 'O' 改为 'X', 原地修改。时间  $O(m \cdot n)$ , 空间  $O(m \cdot n)$ 。
    """
    if not board:
        return
    m, n = len(board), len(board[0])

    def dfs(r: int, c: int) -> None:
        if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != 'O':
            return
        board[r][c] = '#' # 临时标记: 安全的 'O'
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            dfs(r + dr, c + dc)

    # 第一步: 从四条边界的 'O' 出发, 标记所有连通到边界的 'O' 为 '#'
    for r in range(m):
        for c in [0, n - 1]:
            if board[r][c] == 'O':
                dfs(r, c)

```

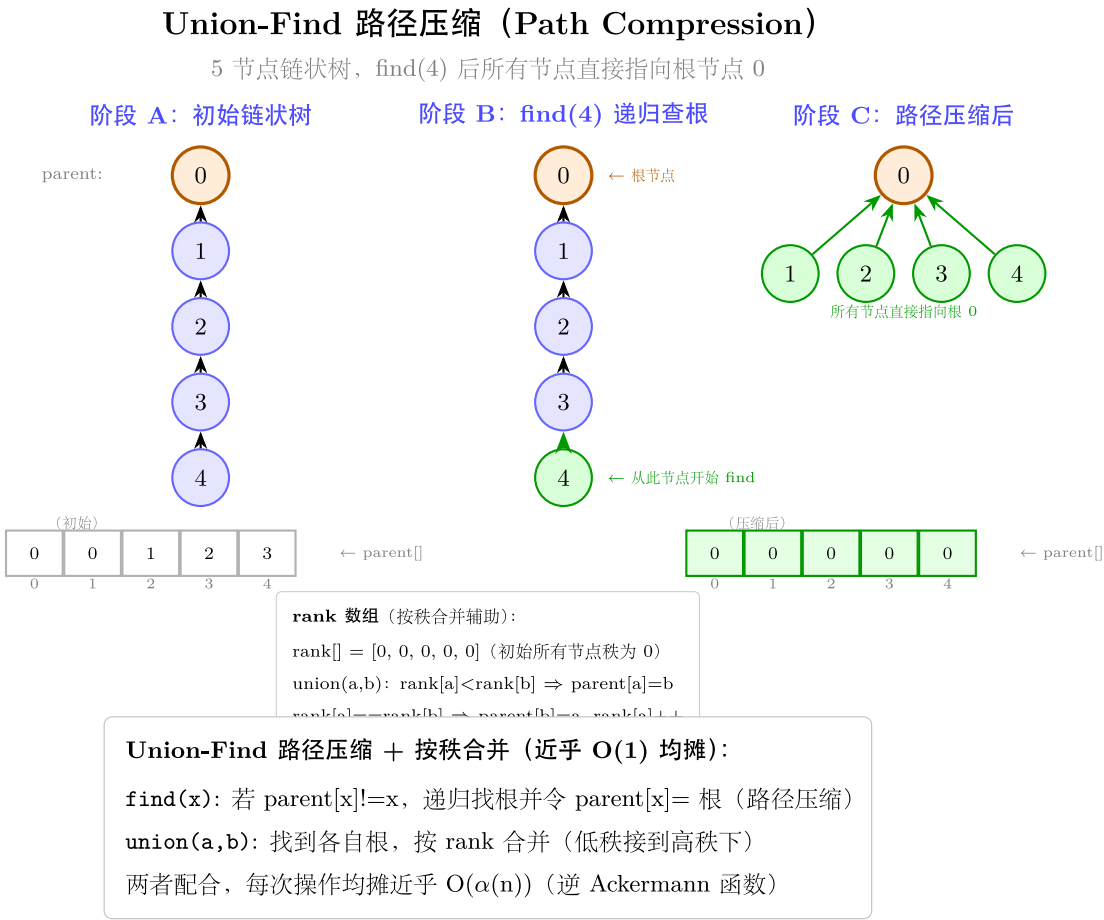


图 39: 5 节点链 → 扁平化 + parent/rank

```

for c in range(n):
    for r in [0, m - 1]:
        if board[r][c] == '0':
            dfs(r, c)

# 第二步：恢复与翻转
for r in range(m):
    for c in range(n):
        if board[r][c] == '0':
            board[r][c] = 'X'    # 被围，翻转
        elif board[r][c] == '#':
            board[r][c] = '0'    # 安全，恢复

```

套路 3: Union-Find (带路径压缩 + 按秩合并)

适用题: 200、547

```

class UnionFind:
    """ 路径压缩 + 按秩合并, find / union 近乎  $O(1)$  (反阿克曼函数)。 """

    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n                # 连通分量数量

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])    # 路径压缩
        return self.parent[x]

    def union(self, x: int, y: int) -> bool:
        """ 合并  $x$  和  $y$  所在集合, 若已在同一集合返回 False。 """
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return False
        if self.rank[rx] < self.rank[ry]:
            rx, ry = ry, rx        # 保证  $rx$  的秩  $\geq ry$ 
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]:
            self.rank[rx] += 1

```



```

        self.count -= 1
        return True

    def connected(self, x: int, y: int) -> bool:
        return self.find(x) == self.find(y)

# 547: 省份数量 (邻接矩阵, Union-Find 版)
def find_circle_num(isConnected: List[List[int]]) -> int:
    n = len(isConnected)
    uf = UnionFind(n)
    for i in range(n):
        for j in range(i + 1, n):
            if isConnected[i][j] == 1:
                uf.union(i, j)
    return uf.count

```

套路 4: 克隆图 (DFS + 哈希表)

适用题: 133

```

from typing import Optional

class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

def clone_graph(node: Optional[Node]) -> Optional[Node]:
    """
    133: DFS 克隆图, 时间  $O(V+E)$ , 空间  $O(V)$  (哈希表 + 递归栈)。
    """
    if not node:
        return None
    visited: dict[Node, Node] = {}

    def dfs(n: Node) -> Node:
        if n in visited:

```

```

        return visited[n]
    clone = Node(n.val)
    visited[n] = clone
    for neighbor in n.neighbors:
        clone.neighbors.append(dfs(neighbor))
    return clone

return dfs(node)

```

套路 5: 拓扑排序 Kahn (入度队列法)

适用题: 207、210

```
from collections import deque
```

```

def topo_sort_kahn(num_nodes: int, edges: List[List[int]]) -> List[int]:
    """
    Kahn 拓扑排序: 入度为 0 的节点入队, 处理后减少邻居入度。
    返回拓扑顺序, 若有环则返回空列表。时间  $O(V+E)$ , 空间  $O(V+E)$ 。
    """
    in_degree = [0] * num_nodes
    graph: List[List[int]] = [[] for _ in range(num_nodes)]
    for u, v in edges:
        graph[v].append(u)          # v 是 u 的先修课, v→u
        in_degree[u] += 1

    queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
    order: List[int] = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in graph[node]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0:
                queue.append(nxt)

    return order if len(order) == num_nodes else []

```

207: 判断是否可以完成所有课程 (是否有环)

```
def can_finish(numCourses: int, prerequisites: List[List[int]]) -> bool:
    order = topo_sort_kahn(numCourses, prerequisites)
    return len(order) == numCourses
```

210: 返回学习顺序

```
def find_order(numCourses: int, prerequisites: List[List[int]]) -> List[int]:
    return topo_sort_kahn(numCourses, prerequisites)
```

套路 6: 带权图 DFS (除法求值)

适用题: 399

```
from collections import defaultdict
```

```
def calc_equation(equations: List[List[str]], values: List[float],
                  queries: List[List[str]]) -> List[float]:
    """
    399: 建带权有向图, DFS 求路径权重之积。时间  $O((V+E) \cdot Q)$ , 空间  $O(V+E)$ 。
    V= 变量数, E= 方程数, Q= 查询数。
    """
    # 建图: graph[A][B] = A/B 的值
    graph: dict[str, dict[str, float]] = defaultdict(dict)
    for (A, B), val in zip(equations, values):
        graph[A][B] = val
        graph[B][A] = 1.0 / val

    def dfs(src: str, dst: str, visited: set) -> float:
        if src not in graph or dst not in graph:
            return -1.0
        if src == dst:
            return 1.0
        visited.add(src)
        for neighbor, weight in graph[src].items():
            if neighbor in visited:
                continue
            result = dfs(neighbor, dst, visited)
            if result != -1.0:
```

```
        return weight * result
    return -1.0

return [dfs(A, B, set()) for A, B in queries]
```

速查表

题型特征	套路	时间	空间
网格连通分量 / 岛屿计数	DFS 染色	$O(mn)$	$O(mn)$
边界连通的区域	边界逆向 DFS + 恢复	$O(mn)$	$O(mn)$
动态连通性 / 省份计数	Union-Find（路径压缩）	近乎 $O(n)$	$O(n)$
克隆图（含环）	DFS + 哈希表	$O(V + E)$	$O(V)$
有向图是否有环	Kahn 拓扑排序	$O(V + E)$	$O(V + E)$
拓扑顺序	Kahn 输出 order	$O(V + E)$	$O(V + E)$
除法关系传递求值	带权图 DFS 路径积	$O((V + E)Q)$	$O(V + E)$

方法变形（4 类）

变形 1：岛屿系列

- **200**（Number of Islands）：DFS 染色，统计启动次数。
- **547**（Number of Provinces）：邻接矩阵版连通分量，DFS 或 Union-Find。
- **130**（Surrounded Regions）：逆向标记边界连通，再批量翻转。
- 共同模式：visited 集合（或原地标记）+ 计数 / 染色。

变形 2：Union-Find 应用扩展

- **200 / 547**：连通分量计数。
- **684**（Redundant Connection，非本 category）：加边时若 find(u) == find(v) 则为冗余边。
- **399**：Union-Find 也可用于 399（带权并查集），但 DFS 更直观。
- 路径压缩确保每次 find 后树高度接近 1，单次近乎 $O(1)$ 。

变形 3：拓扑排序扩展

- **207**（能否完成）→ **210**（学习顺序）→ **269**（Alien Dictionary，非本 category）：逐步复杂化，核心都是 Kahn 或 DFS 三色。

- Kahn 更易于检测环 (`len(order) != V`)；DFS 三色 (WHITE/GRAY/BLACK) 更直观但实现略复杂。
- AI 场景：TensorFlow `tf.function` 编译计算图时会做拓扑排序，确保算子按依赖顺序执行。

变形 4：带权图扩展

- **399** (除法求值)：DFS 路径权重积。
- 若查询量大可用 Floyd-Warshall 预处理所有点对 ($O(V^3)$)，以 $O(1)$ 回答每次查询。
- Union-Find 带权版 (记录节点到根的权重) 也可解决 399，但实现复杂，不推荐面试首选。

思考路标 (条件反射)

1. 看到“连通分量 / 岛屿数量” → DFS 染色 + 计数
2. 看到“被围绕的区域 / 边界连通” → 逆向：先从边界出发标记安全，再处理其余
3. 看到“克隆图 / 深拷贝” → DFS + `visited = {old: new}` 哈希表防止重复克隆
4. 看到“动态合并集合 / 检查是否同一组” → Union-Find (路径压缩 + 按秩合并)
5. 看到“有向图 / 依赖关系 / 课程先修” → 拓扑排序 (Kahn 入度队列)
6. 看到“是否有环” → Kahn：处理节点数 $< V$ 则有环；DFS：灰色节点再次被访问则有环
7. 看到“等式 / 除法 / 比值传递” → 带权有向图，DFS 路径积
8. 看到“计算图 / 任务调度 / DAG” → 拓扑排序
9. 看到“**Union-Find vs DFS**” → 静态图且只需连通性 → Union-Find 更快；需要遍历路径 → DFS

易错点

1. **200 DFS 标记**：必须在进入 DFS 之前 (或进入时) 立刻标记已访问，否则 4 个方向的递归会重复访问同一格子导致无限循环或超时。
2. **130 逆向标记顺序**：必须先做边界 DFS 标记 (第一步)，再做翻转恢复 (第二步)；两步不能混在一次遍历中完成，否则翻转会破坏标记。
3. **133 克隆图含环**：图中可能有环，必须在 DFS 进入节点之前先创建克隆节点并放入 `visited`，然后再处理邻居；若先处理邻居再放入 `visited`，遇到环时会死循环。
4. **Kahn 拓扑排序边方向**：207 题 `[a, b]` 表示 `b` 是 `a` 的先修课 (先上 `b` 才能上 `a`)，建图时边方向为 `b → a` (`graph[b].append(a)`)，入度针对 `a`。方向搞反会导致结果错误。
5. **210 有环时返回 []**：若 `len(order) < numCourses` 说明有环，返回空列表；常见错误是忘记这个检查，直接返回不完整的 `order`。
6. **399 未知变量查询**：若 `query` 中的变量不在图中 (从未出现在 `equations` 里)，应返回 `-1.0`；在 `dfs` 函数起始处检查 `src not in graph or dst not in graph`。
7. **Union-Find 初始化计数**：`count = n` 初始化为所有节点各自独立；每次成功 `union` 时 `count -= 1`；若 `find(x) == find(y)` 则不减 (已经同一连通分量)。

8. **547 vs 200** 邻接结构: 200 是网格 (邻居是上下左右格子), 547 是邻接矩阵 (`isConnected[i][j] == 1`); 框架相同, 邻居获取方式不同。

典型应用例题

例 1: 200. Number of Islands

题目: 给定 $m \times n$ 的字符网格, '1' 为陆地, '0' 为水。计算岛屿数量 (由 4 连通的 '1' 构成的连通分量)。

思路: 双重循环扫描, 遇到 '1' 则岛屿数 +1, 同时 DFS 将该岛屿的所有格子标记为 '#' (已访问)。递归终止条件: 越界或当前格子不为 '1'。

解:

```
# 参考: solutions/graph_general/p200_number_of_islands.py
def numIslands(grid: List[List[str]]) -> int:
    m, n = len(grid), len(grid[0])
    count = 0

    def dfs(r: int, c: int) -> None:
        if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
            return
        grid[r][c] = '#'
        dfs(r - 1, c); dfs(r + 1, c); dfs(r, c - 1); dfs(r, c + 1)

    for r in range(m):
        for c in range(n):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count
```

分析: 每个格子最多被 DFS 访问一次 (标记后跳过), 时间 $O(mn)$, 空间 $O(mn)$ (递归栈, 最坏情况全为陆地)。

例 2: 207. Course Schedule

题目: numCourses 门课, prerequisites[i] = [a, b] 表示先修 b 才能修 a。判断能否完成所有课程 (即有向图是否无环)。

思路: Kahn 拓扑排序。建图: $b \rightarrow a$, 计算入度。队列初始放入所有入度为 0 的节点, 依次出队、减少邻居入度, 入度变 0 的邻居入队。最终处理节点数若等于 numCourses 则无环。

解:

参考: [solutions/graph_general/p207_course_schedule.py](#)

```
def canFinish(numCourses: int, prerequisites: List[List[int]]) -> bool:
    in_degree = [0] * numCourses
    graph = [[] for _ in range(numCourses)]
    for a, b in prerequisites:
        graph[b].append(a)
        in_degree[a] += 1
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    count = 0
    while queue:
        node = queue.popleft()
        count += 1
        for nxt in graph[node]:
            in_degree[nxt] -= 1
            if in_degree[nxt] == 0:
                queue.append(nxt)
    return count == numCourses
```

分析: 时间 $O(V + E)$, 空间 $O(V + E)$, $V = \text{numCourses}$, $E = \text{len}(\text{prerequisites})$ 。

例 3: 399. Evaluate Division

题目: 给定等式 $\text{equations}[i] = [A_i, B_i]$ 和对应的值 $\text{values}[i]$ (即 $A_i/B_i = \text{values}[i]$), 对每个 query $[C, D]$ 计算 C/D 的值, 无法计算则返回 -1.0。

思路: 将变量视为节点, 等式关系视为带权有向边 (双向)。对每个 query, DFS 从源节点出发找目标节点, 沿路径累乘权重。

解:

参考: [solutions/graph_general/p399_evaluate_division.py](#)

```
def calcEquation(equations: List[List[str]], values: List[float],
                 queries: List[List[str]]) -> List[float]:
    graph = defaultdict(dict)
    for (A, B), v in zip(equations, values):
        graph[A][B] = v
        graph[B][A] = 1.0 / v
```

```
def dfs(src: str, dst: str, visited: set) -> float:
    if src not in graph or dst not in graph:
        return -1.0
    if src == dst:
        return 1.0
    visited.add(src)
    for nb, w in graph[src].items():
        if nb not in visited:
            res = dfs(nb, dst, visited)
            if res != -1.0:
                return w * res
    return -1.0

return [dfs(A, B, set()) for A, B in queries]
```

分析: 变量数 V , 等式数 E , 查询数 Q 。每次 DFS $O(V+E)$, 总体 $O((V+E)Q)$ 。若查询量极大可用 Floyd-Warshall 预处理 $O(V^3)$ 后 $O(1)$ 查询。

自测题

自测 1 (200 Number of Islands) —— `grid = [['1','1','0'],['0','1','0'],['0','0','1']]` 应返回 2。提示: 双重循环, 遇到 '1' 则 `count+1` 并 DFS 染色为 '#', DFS 终止条件是越界或非 '1'。参考 `solutions/graph_general/p200_number_of_islands.py`。

自测 2 (130 Surrounded Regions) —— `board = [['X','X','X','X'],['X','0','0','X'],['X','X','0','X'],['X','0','X','X']]` 应将中间的 '0' 变为 'X', 但边界的 '0' 保留。提示: 先从边界 '0' 出发 DFS 标 '#', 再翻转 '0' / 恢复 '#'。参考 `solutions/graph_general/p130_surrounded_regions.py`。

自测 3 (133 Clone Graph) —— 给定一个 4 节点环形图, 克隆后验证克隆图与原图节点不共用 (`id(clone_node) != id(original_node)`) 但结构相同。提示: `visited = {}` 先建, DFS 进入时立刻创建克隆节点放入 `visited`, 再遍历邻居。参考 `solutions/graph_general/p133_clone_graph.py`。

自测 4 (207 Course Schedule) —— `numCourses=2, prerequisites=[[1,0]]` 返回 True; `prerequisites=[[1,0],[0,1]]` 返回 False (环)。提示: Kahn 拓扑排序, 处理节点总数是否等于 `numCourses`。参考 `solutions/graph_general/p207_course_schedule.py`。

自测 5 (210 Course Schedule II) —— `numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]`, 有效拓扑顺序为 `[0,1,2,3]` 或 `[0,2,1,3]`。提示: Kahn 算法直接输出 `order`, 若有环返回 `[]`。参考 `solutions/graph_general/p210_course_schedule_ii.py`。

自测 6 (399 Evaluate Division) —— `equations=[['a','b'],['b','c']], values=[2.0,3.0], queries=[['a','c'],['b','a']]` 期望 `[6.0, 0.5, -1.0]`。提示: 建双向带权图, DFS 传递权重积, 未知变量返回 -1.0。参考 `solutions/graph_general/p399_evaluate_division.py`。

题目全览（6 题）

#	题目	套路分类	难度
200	Number of Islands	DFS 连通分量计数	Medium
130	Surrounded Regions	边界逆向 DFS 标记	Medium
133	Clone Graph	DFS + 哈希表克隆	Medium
547	Number of Provinces	Union-Find / DFS 连通分量	Medium
207	Course Schedule	Kahn 拓扑排序（环检测）	Medium
210	Course Schedule II	Kahn 拓扑排序（输出顺序）	Medium
399	Evaluate Division	带权图 DFS 路径积	Medium

融合版说明

段	来源	价值
一例速记	本文件	6 题套路一览 + AI（DAG / 编译器） 关联
思维路径还原	本文件	6 道题的解题独白，含关键判断点
抽象成方法	本文件	6 个标准模板（DFS / 边界 DFS / Union-Find / 克隆 / Kahn / 带权 DFS）+ 速查表
方法变形	本文件	4 类变体（岛屿系列 / UF 扩展 / 拓 扑扩展 / 带权图扩展）
思考路标	本文件	9 条题型识别条件反射
易错点	本文件	8 条高频踩坑（环处理 / 边方向 / 标记时机）
典型应用例题	solutions/	3 道精讲（200、207、399），代码 + 分析
自测题	leetcode	6 题带提示，链接 solutions 文件
题目全览	本文件	6 题完整列表（含 399 备注）

跨 category 导航：- 无权图最短路径 → 18-graph-bfs.md（BFS 按层扩展）- 二叉树的 DFS → 07-binary-tree-dfs.md（preorder / inorder / postorder）- 回溯的 DFS → 08-backtracking.md（需要撤销选择）- TensorFlow / PyTorch 的计算图编译 = 对 DAG 做拓扑排序，算子按依赖顺序执行

20 — Trie (融合版)

难度: □□□□□ 题数: 3 核心套路: Trie 标准实现 (前缀树)、通配符 DFS 搜索、Trie + 回溯单词搜索 本文件: 覆盖 trie 3 题的算法套路总结 + 典型题精讲 + 自测

一例速记

Trie 标准实现: 每个节点包含 children[26] (或 dict) 和 is_end 标志; insert 逐字符插入, search 逐字符查找并检查末节点 is_end, startsWith 只查前缀是否存在 (208) **通配符 DFS 搜索:** 211 题中 '.' 匹配任意字符, 遇到 '.' 时对当前节点的所有非空子节点递归搜索, 普通字符则直接沿确定路径走 **Trie + 回溯 (Word Search II):** 先将所有词插入 Trie, 再对棋盘每个格子做 DFS + 回溯; 在 Trie 上同步走, 若当前 Trie 节点为 is_end 则收集该词, 回溯时撤销棋盘格子标记 (212) **剪枝优化:** 212 中找到一个单词后将对应 Trie 节点的 word 置为 None (防止重复收集); 若 Trie 节点已无子节点可剪去, 减少后续无效搜索 **AI 关联:** NLP 词典 / 输入法候选词 (前缀查询) / 搜索引擎自动补全 / 拼写检查; BERT tokenizer 的词表也基于前缀树管理子词 (subword) 词典

思维路径还原

“看到 ‘208 实现前缀树’ → 直接套 Trie 标准模板: 每个 TrieNode 有 children = {} 和 is_end = False。insert(word): 从根逐字符走, 不存在则新建节点, 最后标记 is_end = True。search(word): 逐字符走, 若中途找不到字符返回 False, 末尾返回 node.is_end。startsWith(prefix): 逐字符走, 成功走完 prefix 则返回 True, 中途断则返回 False。时间: 三个操作均 $O(L)$ (L 为字符串长度), 空间 $O(\sum L_i)$ (插入词的总长度)。”

看到 ‘211 通配符搜索’ → Trie + DFS: insert 正常插入; search 遇到普通字符走确定路径, 遇到 '.' 则对所有非空子节点递归搜索, 任一子路径搜索成功则返回 True。因为有通配符, 无法走确定路径, 必须用 DFS/回溯。

看到 ‘212 单词搜索 II’ → Trie + 棋盘回溯: 先把所有单词 insert 进 Trie, 再对棋盘每个格子启动 DFS (四方向), DFS 过程中同时在 Trie 上向下走: 若当前字符不在 Trie 子节点中则剪枝; 若 Trie 节点 is_end 则收集当前词, 将 is_end = False (防重复); 回溯时恢复棋盘格子 (board[r][c] = c_saved)。”

学习目标

- 掌握 Trie 节点的两实现 (dict vs 定长数组 children[26]) 及其权衡

- 熟练实现 insert / search / startsWith 三个核心操作
- 理解通配符 '.' 的 DFS 处理方式 (枚举所有子节点)
- 掌握 Trie + 棋盘回溯的组合技 (212)，包括剪枝策略
- 能识别“前缀匹配 / 自动补全 / 词典搜索”题型并直接套模板

几何示意

图 Trie 字典树结构

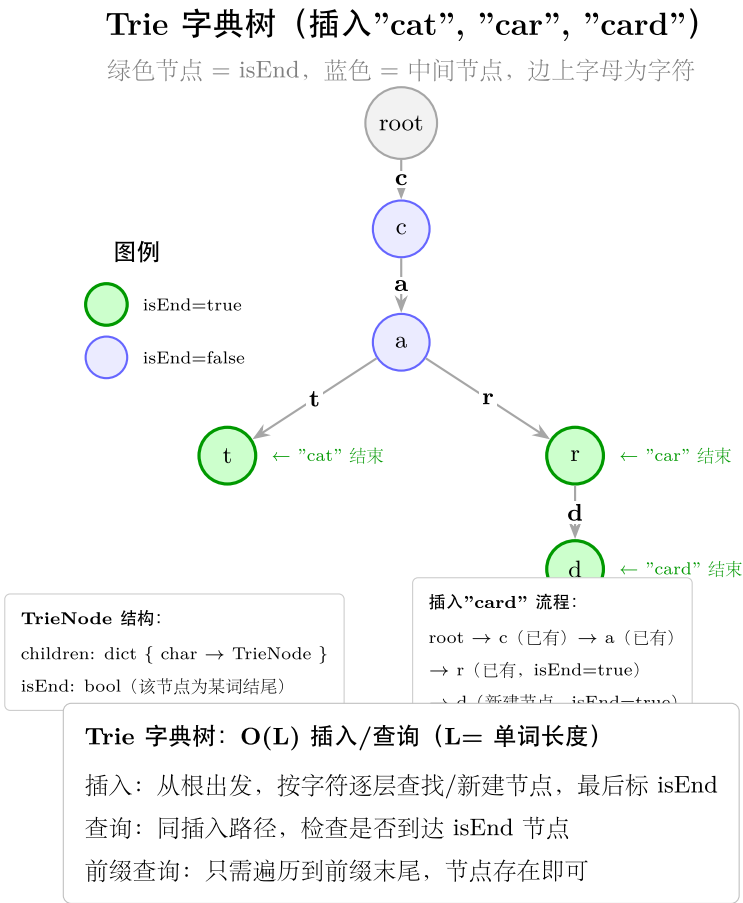


图 40: 插入 cat/car/card 后的 Trie

```
## 抽象方法  
(标准模板代码)  
####  
套路  
1:  
Trie  
标准实现  
(字典版)  
适用  
题:  
208  
“python  
class  
TrieNode:  
ode:  
def  
init(self):  
self.children:  
dict[str,  
‘TrieNode’]  
= {}  
self.is_end:  
bool =  
False
```

```
class
Trie:
    """208:
    前缀
    树标
    准实
    现。in-
    sert/search/startsWith
    均
    O(L),
    L 为
    字符
    串长
    度。"""
    def
    init(self):
    self.root
    =
    TrieN-
    ode()
```

```
def insert(self, word: str) -> None:
    node = self.root
    for c in word:
        if c not in node.children:
            node.children[c] = TrieNode()
        node = node.children[c]
    node.is_end = True

def search(self, word: str) -> bool:
    node = self._walk(word)
    return node is not None and node.is_end
```

```
def
startsWith(self,
prefix:
str) ->
bool:
return
self._walk(prefix)
is not
None
```

```

def
    _walk(self,
    s: str)
->
    "TrieN-
    ode |
    None":
    """沿
    s 逐字
    符走，
    返回
    末节
    点；
    中途
    断则
    返回
    None。”
    """
    node
    =
    self.root
    for c
    in s: if
    c not
    in
    node.children:
    return
    None
    node
    =
    node.children[c]
    return
    node
    """

```

套路 2：Trie 定长数组版（26 字符，性能更优）

适用题：208、212

```

class TrieNodeArray:
    """ 定长数组版，空间固定但访问更快（无哈希冲突）。 """

```



```

__slots__ = ['children', 'is_end']

def __init__(self):
    self.children: list['TrieNodeArray | None'] = [None] * 26
    self.is_end: bool = False

class TrieArray:
    def __init__(self):
        self.root = TrieNodeArray()

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = TrieNodeArray()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for c in prefix:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        return True

```

两种实现对比：字典版节省空间（字符集稀疏时），数组版访问速度快（下标直接寻址，无哈希）。字符集为全小写 26 字母时首选数组版。

套路 3: Trie + 通配符 DFS (Word Search)

适用题: 211

```
class WordDictionary:
    """
    211: 支持 '.' 通配符的单词搜索。insert  $O(L)$ , search  $O(26^L)$  最坏情况。
    """

    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end = True

    def search(self, word: str) -> bool:
        return self._dfs(self.root, word, 0)

    def _dfs(self, node: TrieNode, word: str, i: int) -> bool:
        """ 从 node 开始, 匹配 word[i:]; '.' 匹配任意字符。 """
        if i == len(word):
            return node.is_end
        c = word[i]
        if c == '.':
            # 枚举所有子节点
            for child in node.children.values():
                if self._dfs(child, word, i + 1):
                    return True
            return False
        else:
            if c not in node.children:
                return False
            return self._dfs(node.children[c], word, i + 1)
```

套路 4: Trie + 棋盘回溯 (Word Search II)

适用题: 212

```

from typing import List

class TrieNodeWord:
    """ 扩展 TrieNode, 存储到达该节点所拼成的单词 (仅在 is_end 节点)。 """
    def __init__(self):
        self.children: dict[str, 'TrieNodeWord'] = {}
        self.word: str | None = None # 非 None 时表示此处是一个完整单词

def find_words(board: List[List[str]], words: List[str]) -> List[str]:
    """
    212: Trie + 棋盘 DFS 回溯。时间  $O(M \cdot N \cdot 4^L)$ ,  $L$ = 最长单词长度。
    """
    # 构建 Trie
    root = TrieNodeWord()
    for w in words:
        node = root
        for c in w:
            if c not in node.children:
                node.children[c] = TrieNodeWord()
            node = node.children[c]
        node.word = w

    m, n = len(board), len(board[0])
    result: List[str] = []

    def dfs(r: int, c: int, trie_node: TrieNodeWord) -> None:
        ch = board[r][c]
        if ch not in trie_node.children:
            return
        nxt = trie_node.children[ch]
        if nxt.word is not None:
            result.append(nxt.word)
            nxt.word = None # 防止同一单词被重复收集

        board[r][c] = '#' # 标记已访问
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:

```

```
        nr, nc = r + dr, c + dc
        if 0 <= nr < m and 0 <= nc < n and board[nr][nc] != '#':
            dfs(nr, nc, nxt)
    board[r][c] = ch          # 回溯：恢复格子

    for r in range(m):
        for c in range(n):
            dfs(r, c, root)
    return result
```

速查表

题型特征	套路	时间（每次操作）	空间
插入 / 精确搜索 / 前缀搜索	Trie 标准实现	$O(L)$	$O(\sum L_i)$
支持 '.' 通配符搜索	Trie + DFS 枚举子节点	$O(26^L)$ 最坏	$O(\sum L_i)$
棋盘中搜索多个单词	Trie + 棋盘回溯 + 剪枝	$O(MN \cdot 4^L)$	$O(\sum L_i + MN)$

方法变形（3 类）

变形 1: Trie 节点设计扩展

- 208: 最基础，仅需 children + is_end。
- 211: 同 208，但 search 改为 DFS 以支持 '.'。
- 212: TrieNode 增加 word 字段（或 is_end + 另存单词到列表），收集整词更方便。
- 进阶：自动补全场景中，每个节点可额外存 top-k 热词列表（LRU 缓存思路）。

变形 2: 棋盘 DFS 剪枝策略

- 212 基础剪枝：发现单词后置 `nxt.word = None` 防重复；Trie 节点若 children 为空且 word 为 None 则可删除，减少后续无效搜索（但实现稍复杂，面试中通常只做第一层剪枝即可）。
- 79 (Word Search I, 非本 category)：仅需在棋盘找一个单词，不用 Trie，直接 DFS + 回溯。
- 212 vs 79：单词 → DFS 回溯；多单词 → Trie + DFS 回溯（避免对每个词各做一次完整 DFS）。

变形 3: 前缀树应用场景

- 输入法候选词: `startsWith(prefix)` 找所有匹配前缀的单词 (BFS / DFS Trie 子树)。
- 最长公共前缀 (14, 非本 category): 也可用 Trie 解决 (走到分叉点为止), 但排序 + 比首尾更简洁。
- XOR 最大化 (421 非本 category): 二进制 Trie, 每位贪心选择与当前位相反的路径, 最大化 XOR。

思考路标 (条件反射)

1. 看到“前缀树 / 插入 / 精确搜索 / **startsWith**” → Trie 标准实现, 三个操作各 $O(L)$
2. 看到“通配符 '.' / 模式匹配” → Trie + DFS, '.' 时枚举所有子节点
3. 看到“棋盘找多个单词” → Trie + 棋盘 DFS 回溯 (而非对每个词单独 DFS)
4. 看到“棋盘找单个单词” → 直接 DFS + 回溯 (79 Word Search), 不需要 Trie
5. 看到“字符集为小写字母” → 考虑定长数组版 `children[26]`, 性能优于 `dict`
6. 看到“防止重复输出” → 找到单词后将 Trie 节点的 `word` 置为 `None` (而非用结果集 `set`)
7. 看到“自动补全 / 输入法 / 词典前缀” → Trie, AI 场景中 `subword tokenizer` 也用前缀树管理词表

易错点

1. **208 search vs startsWith** 的区别: `search` 要求完整匹配 (必须检查末节点 `is_end`), `startsWith` 只要前缀存在即可 (不检查 `is_end`); 两者代码相差仅最后一行, 容易混淆。
2. **211** 递归终止条件: `i == len(word)` 时返回 `node.is_end`, 而不是 `True`; 若此时节点不是词尾则返回 `False`。
3. **212** 回溯标记: 进入 DFS 前必须将 `board[r][c]` 标记为 '#', DFS 结束后必须恢复为原字符 `ch`; 常见错误是忘记恢复导致后续路径无法使用该格子。
4. **212** 防重复收集: 同一单词可能在棋盘多处找到, 用 `nxt.word = None` (而非 `result` 用 `set`) 来防重复, 因为 `set` 需要额外的包含检查。
5. **Trie** 根节点: 根节点本身不代表任何字符, 是所有字符的入口; `insert` 和 `search` 从 `root.children` 开始, 不要把根节点本身当作字符节点使用。
6. 定长数组越界: 若题目包含非小写字母字符 (如大写字母、数字), 定长数组 `[None]*26` 会越界 (`ord(c) - ord('a')` 可能为负或超过 25), 此时改用 `dict` 版。
7. **211** 最坏时间复杂度: 全为 '.' 时 DFS 遍历整个 Trie, 时间 $O(26^L)$; 实际上 Trie 中插入的词数量有限, 不会退化到理论最坏。

典型应用例题

例 1：208. Implement Trie (Prefix Tree)

题目：实现 Trie 数据结构，支持 insert(word)、search(word)（精确匹配）、startsWith(prefix)（前缀匹配）三个操作。

思路：每个 TrieNode 维护 children（字典或定长数组）和 is_end 标志。insert 逐字符创建路径，search 逐字符走并检查末尾 is_end，startsWith 只验证路径存在性。

解：

参考：[solutions/trie/p208_implement_trie_prefix_tree.py](#)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end = True

    def search(self, word: str) -> bool:
        node = self.root
        for c in word:
            if c not in node.children:
                return False
            node = node.children[c]
        return node.is_end

    def startsWith(self, prefix: str) -> bool:
        node = self.root
        for c in prefix:
            if c not in node.children:
                return False
```

```

        node = node.children[c]
    return True

```

分析：三个操作时间均 $O(L)$ ， L 为字符串长度。空间 $O(\sum L_i)$ ，即所有插入单词长度之和（最坏情况下无公共前缀）。

例 2：211. Design Add and Search Words Data Structure

题目：实现数据结构支持 `addWord(word)` 和 `search(word)`，`search` 中 '.' 可匹配任意单个字母。

思路：`addWord` 同标准 Trie 的 `insert`；`search` 在 Trie 上 DFS，遇到普通字符走确定路径，遇到 '.' 则枚举所有非空子节点递归搜索。

解：

参考：[solutions/trie/p211_design_add_and_search_words_data_structure.py](#)

```

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        node = self.root
        for c in word:
            if c not in node.children:
                node.children[c] = TrieNode()
            node = node.children[c]
        node.is_end = True

    def search(self, word: str) -> bool:
        def dfs(node: TrieNode, i: int) -> bool:
            if i == len(word):
                return node.is_end
            c = word[i]
            if c == '.':
                return any(dfs(child, i + 1) for child in node.children.values())
            if c not in node.children:
                return False
            return dfs(node.children[c], i + 1)
        return dfs(self.root, 0)

```

分析：`addWord` $O(L)$ ；`search` 最坏 $O(26^L)$ （全 '.'），实际取决于 Trie 中实际存储的词，远小于理论上界。

例 3：212. Word Search II

题目：给定 $m \times n$ 棋盘和单词列表 `words`，找出所有出现在棋盘中的单词（字符需 4 连通且不重复使用）。

思路：将所有单词插入 Trie。对棋盘每个格子做 DFS，同时在 Trie 上行走；若 Trie 节点有 word（词尾）则收集该词并清除（防重复）；DFS 时回溯恢复格子。

解：

参考：[solutions/trie/p212_word_search_ii.py](#)

```
def findWords(board: List[List[str]], words: List[str]) -> List[str]:
```

```
    root = TrieNodeWord()
```

```
    for w in words:
```

```
        node = root
```

```
        for c in w:
```

```
            if c not in node.children:
```

```
                node.children[c] = TrieNodeWord()
```

```
            node = node.children[c]
```

```
        node.word = w
```

```
m, n = len(board), len(board[0])
```

```
result = []
```

```
def dfs(r, c, trie):
```

```
    ch = board[r][c]
```

```
    if ch not in trie.children:
```

```
        return
```

```
    nxt = trie.children[ch]
```

```
    if nxt.word:
```

```
        result.append(nxt.word)
```

```
        nxt.word = None
```

```
    board[r][c] = '#'
```

```
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
```

```
        nr, nc = r+dr, c+dc
```

```
        if 0<=nr<m and 0<=nc<n and board[nr][nc] != '#':
```

```
            dfs(nr, nc, nxt)
```

```
    board[r][c] = ch
```

```
for r in range(m):
```

```
    for c in range(n):
```

```
        dfs(r, c, root)
```

```
return result
```


分析:棋盘格子 MN 个,每个格子启动 DFS,最坏深度为最长单词长度 L ,每层最多 4 个方向。时间 $O(MN \cdot 4^L)$, Trie 剪枝大幅减少实际搜索量。

自测题

自测 1 (208 Implement Trie) ——依次调用 `insert('apple')`、`search('apple')` (True)、`search('app')` (False)、`startsWith('app')` (True)、`insert('app')`、`search('app')` (True)。提示: `search` 需检查末节点 `is_end`, `startsWith` 不检查。参考 `solutions/trie/p208_implement_trie_prefix_tree.py`。

自测 2 (211 Add and Search) ——`addWord('bad')`、`addWord('dad')`、`addWord('mad')`、`search('pad')` → False, `search('bad')` → True, `search('.ad')` → True, `search('b..')` → True。提示: DFS, 遇 '.' 枚举所有子节点, 普通字符走确定路径。参考 `solutions/trie/p211_design_add_and_search_words_data_structure.py`。

自测 3 (212 Word Search II) ——`board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']]`, `words=['oath','pea','eat','rain']`, 应返回 `['oath','eat']` (顺序不限)。提示: 先建 Trie, 棋盘每格启动 DFS + 回溯, 找到词后清除 Trie 节点 word 防重复。参考 `solutions/trie/p212_word_search_ii.py`。

题目全览（3 题）

#	题目	套路分类	难度
208	Implement Trie (Prefix Tree)	Trie 标准实现 (insert / search / startsWith)	Medium
211	Design Add and Search Words	Trie + 通配符 DFS	Medium
212	Word Search II	Trie + 棋盘回溯 + 剪枝	Hard

融合版说明

段	来源	价值
一例速记	本文件	3 题套路一览 + AI (NLP 词典 / 输入法) 关联
思维路径还原	本文件	3 道题的解题独白, 含关键判断点

段	来源	价值
抽象成方法	本文件	4 个标准模板 (字典版 Trie / 数组版 Trie / 通配符 DFS / 棋盘回溯) + 速查表
方法变形	本文件	3 类变体 (节点设计扩展 / 棋盘剪枝 / 前缀树应用场景)
思考路标	本文件	7 条题型识别条件反射
易错点	本文件	7 条高频踩坑 (is_end / 回溯恢复 / 防重复)
典型应用例题	solutions/	3 道精讲 (208、211、212), 代码 + 分析
自测题	leetcode	3 题带提示, 链接 solutions 文件
题目全览	本文件	3 题完整列表

跨 category 导航: - 棋盘 DFS 回溯 (79 Word Search 单词) → 08-backtracking.md - 字符串前缀最长公共前缀 (14) → 01-array-string.md - 图的 DFS (连通分量) → 19-graph-general.md - BERT / GPT tokenizer 的 BPE (Byte Pair Encoding) 也需要维护前缀词表, 与 Trie 高度相关

21 — Bit Manipulation (融合版)

难度: □□□□□ 题数: 6 核心套路: XOR 消消乐 (单数)、位计数 (hamming weight / popcount)、位运算技巧 (最低位 / 反转 / 区间公共前缀) 本文件: 覆盖 bit_manipulation 6 题的算法套路总结 + 典型题精讲 + 自测

一例速记

XOR 消消乐 (Single Number I): 全部异或, 相同数对消为 0, 结果即为出现奇数次的数 (136) **Single Number II** (每数出现 3 次): 对每一位统计 1 的个数, 模 3 取余, 余数即为目标数对应位 (137); 或用两个变量 ones/twos 模拟三进制计数器 **Hamming Weight** (位计数): $n \& (n-1)$ 消去最低有效位, 循环次数 = 1 的个数 (191); 或 `bin(n).count('1')`; 或 Brian Kernighan 算法 **Number of 1 Bits** 进阶: 338 Counting Bits, $O(n)$ 用 DP: $dp[i] = dp[i \gg 1] + (i \& 1)$ (右移一位的结果已知, 加上当前最低位) **Reverse Bits**: 逐位取最低位, 左移到目标位置, 累计拼接 (190); 或分治翻转 (16→8→4→2→1 位块交换) **Bitwise AND of Range**: 两数一直右移到相等 (即找公共前缀), 左移回去即为答案 (201); 等价于 Brian Kernighan: 不断用 `right & (right-1)` 消去 right 最低位直到 `right <= left` **Add Binary**: 从低位逐位模拟加法进位 (67); 或 Python 直接 `bin(int(a,2) +`

`int(b,2))[2:]` AI 关联: 哈希压缩 (MinHash / SimHash 用位运算加速) / 集合编码 (bitmap / bitset) / CUDA 原子操作 / 量化模型中的整数位运算 (INT8 / INT4 推理)

思维路径还原

“看到‘136 只有一个数出现一次，其余出现两次’ → XOR 消消乐: `result = 0; for x in nums: result ^= x`, $O(n)$ 时间 $O(1)$ 空间。性质: $x \oplus x = 0$, $x \oplus 0 = x$, XOR 满足交换律和结合律。

看到‘137 只有一个数出现一次，其余出现三次’ → 不能直接用 XOR (3 次不对消)。方法 1: 对 32 位每一位统计 1 的个数, `count[bit] % 3` 就是答案对应位 ($O(32n)$)。方法 2: ones/twos 两个变量, 模拟每一位的三进制计数器: `ones = (ones ^ x) & ~twos`, `twos = (twos ^ x) & ~ones` 最终 ones 即为仅出现一次的数。

看到‘191 位 1 的个数’ → Brian Kernighan: `while n: n &= n-1; count += 1`, 每次操作消去 n 最低有效位 (`n & (n-1)` 将最低 1 变为 0)。循环次数 = 1 的个数, 比逐位移位快 (跳过所有 0 位)。

看到‘338 位计数 (0~n 每个数的 1 个数)’ → DP: `bits[i] = bits[i >> 1] + (i & 1)`: i 右移一位得 $i/2$ (已知), 当前最低位贡献 $i \& 1$ 。 $O(n)$ 时间, $O(n)$ 空间 (输出数组, 无额外空间)。

看到‘190 反转 32 位整数的位’ → 逐位操作: 右移 n 取最低位, 左移到目标位置, 32 轮循环。注意 Python 整数无溢出, 最后用 `& 0xFFFFFFFF` 截取 32 位。

看到‘201 区间位与 (left AND left+1 AND ... AND right)’ → 公共前缀法: left 和 right 同时右移, 直到 `left == right`, 记录右移次数 `shift`, 结果为 `right << shift`。直觉: 若 `left != right`, 则区间内必然包含相差 1 的相邻数, 它们在某一位上一个为 0 一个为 1, AND 后该位为 0; 一直右移到 `left == right`, 即找到公共前缀。”

学习目标

- 掌握 XOR 消消乐原理及其在 136 / 137 中的应用与局限
 - 熟练使用 `n & (n-1)` 消去最低有效位, 实现 $O(k)$ 的位计数 ($k = 1$ 的个数)
 - 理解 338 的 DP 位计数: 右移 + 最低位, $O(n)$ 预处理所有数
 - 掌握 190 的位反转: 32 轮逐位提取 + 拼接, 注意 Python 整数截断
 - 理解 201 的公共前缀思路 (同步右移直到相等)
 - 能识别“位运算 / 奇偶性 / 区间 AND”题型并直接套对应模板
-

几何示意

图 XOR 单数 (LC 136)



图 41: 5 步累计 XOR + 二进制 + 真值

图 Brian Kernighan 位计数 (LC 191)

抽象成方法（标准模板代码）

套路
1:
XOR
消消
乐
(Sin-
gle
Num-
ber I)
适用
题:
136
“python
from
typing
im-
port
List

```

def
single_number(nums:
List[int])
-> int:
    """
    找出
    只出
    现一
    次的
    数,
    其余
    数出
    现恰
    好两
    次。
    XOR
    性质:
     $x \wedge x = 0$ ,
     $x \wedge 0 = x$ 。
    时间
     $O(n)$ ,
    空间
     $O(1)$ 。"""
    result
    = 0
    for x
    in
    nums:
        result
        ^= x
    return
    result

```

```

# 等
价的
函数
式写
法
(Python)
im-
port
func-
tools,
opera-
tor def
sin-
gle_number_functional(nums:
List[int])
-> int:
return
func-
tools.reduce(operator.xor,
nums)
"""

```

套路 2: 三进制位计数 (Single Number II)

适用题: 137

```

def single_number_ii(nums: List[int]) -> int:
    """
    137: 找出只出现一次的数, 其余数出现恰好三次。
    方法 1 (直观): 对每一位统计 1 的个数, mod 3 取余。时间  $O(32n)$ , 空间  $O(1)$ 。
    """
    result = 0
    for bit in range(32):
        total = sum((x >> bit) & 1 for x in nums)
        if total % 3:
            result |= 1 << bit
    # Python 需处理负数 (32 位有符号)
    if result >= (1 << 31):
        result -= (1 << 32)
    return result

```



图 42: n & (n-1) 每步消最低位 1


```
def single_number_ii_fast(nums: List[int]) -> int:
    """
    137: ones/twos 模拟三进制计数器, 时间  $O(n)$ , 空间  $O(1)$ 。
    ones[bit] = 1 表示该位累计 1 次, twos[bit] = 1 表示累计 2 次, 3 次时两者归零。
    """
    ones, twos = 0, 0
    for x in nums:
        ones = (ones ^ x) & ~twos
        twos = (twos ^ x) & ~ones
    return ones
```

套路 3: Brian Kernighan 位计数

适用题: 191

```
def hamming_weight(n: int) -> int:
    """
    191: 统计  $n$  的二进制表示中 1 的个数 (popcount)。
     $n \& (n-1)$  消去最低有效位, 时间  $O(k)$ ,  $k = 1$  的个数 32。
    """
    count = 0
    while n:
        n &= n - 1    # 消去最低有效位
        count += 1
    return count
```

等价写法 1: 逐位移位 ($O(32)$, 固定 32 轮)

```
def hamming_weight_shift(n: int) -> int:
    count = 0
    for _ in range(32):
        count += n & 1
        n >>= 1
    return count
```

等价写法 2: Python 内置 (推荐实际使用)

```
def hamming_weight_builtin(n: int) -> int:
```

```
return bin(n).count('1')
```

套路 4: DP 位计数 (0~n 所有数)

适用题: 338

```
def count_bits(n: int) -> List[int]:
    """
    338: 返回 0..n 每个数的 1 的个数。时间  $O(n)$ , 空间  $O(n)$  (即输出数组)。
    状态转移:  $bits[i] = bits[i \gg 1] + (i \& 1)$ 
    直觉:  $i$  右移一位得  $i/2$  (1 的个数已知), 再加上当前最低位是否为 1。
    """
    bits = [0] * (n + 1)
    for i in range(1, n + 1):
        bits[i] = bits[i >> 1] + (i & 1)
    return bits
```

关键: $i \gg 1 = i // 2$, 对应 i 去掉最低位后的数, 其 popcount 已被计算。这是最优解: $O(n)$ 时间, $O(1)$ 额外空间 (不含输出)。

套路 5: 位反转 (Reverse Bits)

适用题: 190

```
def reverse_bits(n: int) -> int:
    """
    190: 反转 32 位无符号整数的二进制位。时间  $O(32)$ , 空间  $O(1)$ 。
    """
    result = 0
    for _ in range(32):
        result = (result << 1) | (n & 1)  # 取 n 最低位, 追加到 result 末尾
        n >>= 1
    return result & 0xFFFFFFFF  # Python 截取 32 位
```

分治翻转 (高级, 可缓存 16 位半字)

```
def reverse_bits_cache():
    """ 若需多次反转同一 32 位整数, 可缓存 16 位半字的反转结果,  $O(1)$  完成。 """
    cache: dict[int, int] = {}
```

```
def _reverse_16(x: int) -> int:
    if x not in cache:
        res = 0
        for _ in range(16):
            res = (res << 1) | (x & 1)
            x >>= 1
        cache[x] = res
    return cache[x]

def reverse(n: int) -> int:
    lo = n & 0xFFFF
    hi = (n >> 16) & 0xFFFF
    return (_reverse_16(lo) << 16) | _reverse_16(hi)

return reverse
```

套路 6: 区间位与 (公共前缀)

适用题: 201

```
def range_bitwise_and(left: int, right: int) -> int:
    """
    201: 计算 [left, right] 内所有整数的位与。
    同步右移直到 left == right, 即找公共二进制前缀。时间  $O(\log n)$ , 空间  $O(1)$ 。
    """
    shift = 0
    while left != right:
        left >>= 1
        right >>= 1
        shift += 1
    return right << shift
```

Brian Kernighan 变体: 不断消去 right 最低位直到 right <= left

```
def range_bitwise_and_bk(left: int, right: int) -> int:
    while right > left:
        right &= right - 1    # 消去 right 最低有效位
    return right
```

速查表

题型特征	套路	时间	空间
找出现奇数次的数（其余偶数次）	XOR 消消乐	$O(n)$	$O(1)$
找出现 1 次的数（其余 3 次）	按位 mod 3 或 ones/twos	$O(n)$	$O(1)$
单个数的 popcount	Brian Kernighan $n \& (n-1)$	$O(k)$	$O(1)$
0~n 所有数的 popcount	DP $bits[i] = bits[i >> 1] + (i \& 1)$	$O(n)$	$O(n)$
反转 32 位整数的位	逐位取低位拼接	$O(32)$	$O(1)$
区间 AND	同步右移找公共前缀	$O(\log n)$	$O(1)$
二进制字符串相加	逐位模拟进位	$O(\max(L_a, L_b))$	$O(\max(L_a, L_b))$

方法变形（3 类）

变形 1: XOR 扩展

- **136**（单个奇数次）→ 基础 XOR。
- **137**（单个，其余 3 次）→ 按位 mod 3（可泛化为任意 k 次）。
- **260**（两个奇数次，非本 category）：XOR 得到两数之 XOR，找任一非零位将数组分组，分组后分别对每组 XOR 即得两个数。
- 泛化规律：若其余数出现 k 次，对每一位统计 1 的个数 mod k 即为目标数对应位。

变形 2: 位运算技巧速查

- $x \& (x-1)$ ：消去最低有效位（1）
- $x \& (-x)$ 或 $x \& \sim(x-1)$ ：提取最低有效位（Lowest Set Bit）
- $x \mid (x-1)$ ：将最低有效位及以下全置 1
- $x \wedge (x-1)$ ：将最低有效位及以下全置 0（另一形式）
- $\sim x + 1$ 或 $-x$ （Python 直接用负号）：二进制补码取反
- $(x \gg k) \& 1$ ：取第 k 位
- $x \mid (1 \ll k)$ ：将第 k 位置 1
- $x \& \sim(1 \ll k)$ ：将第 k 位置 0

变形 3: 338 DP 变体

- $bits[i] = bits[i \& (i-1)] + 1$ ：另一种等价 DP（消去最低位后 +1），与 Brian Kernighan 对应。

- `bits[i] = bits[i // 2] + (i % 2)`: 同 `bits[i>>1] + (i&1)`, 仅写法不同。
- 集合枚举: 对 0 到 $2^n - 1$ 的所有子集, 338 的预处理可为子集枚举时的位计数提供 $O(1)$ 查表。

思考路标 (条件反射)

1. 看到“出现偶数次 / 其余出现 2 次” → XOR 消消乐, $O(n) O(1)$
2. 看到“其余出现 3 次 / k 次” → 按位 mod k, 或 ones/twos 三进制计数器
3. 看到“统计单个数的 1 位个数” → `n & (n-1)` 循环, 或 `bin(n).count('1')`
4. 看到“统计 0~n 所有数的 1 位个数” → DP, `bits[i] = bits[i>>1] + (i&1)`
5. 看到“反转 32 位整数” → 32 轮循环, 每轮取最低位追加到结果, 末尾 & `0xFFFFFFFF`
6. 看到“区间 AND / 区间公共前缀” → 同步右移直到相等, 或 Brian Kernighan 消低位
7. 看到“二进制字符串相加” → 逐位模拟进位 (类比十进制加法), 或 Python int 转换
8. 看到“集合编码 / 子集枚举” → 位运算: `i` 的所有非空子集 `s = i; while s: ... s = (s-1)&i`
9. 看到“INT8 / INT4 推理加速” → 量化模型中, 矩阵乘法用整数位运算代替浮点 (AI 关联)

易错点

1. **136 XOR** 顺序无关: XOR 满足交换律和结合律, 可以任意顺序异或, 结果不变; 不需要先排序。
2. **137 Python** 负数处理: Python 整数不溢出, 但题目语境是 32 位有符号整数; 按位统计后若结果 $\geq 2^{31}$ 需减去 2^{32} 转为负数表示。ones/twos 方法在 Python 中天然正确 (无溢出问题)。
3. **190 Python** 截断: Python 无符号整数概念, `n >>= 1` 对负数 (Python 表示为任意精度负整数) 会有符号扩展; 题目给的是 32 位无符号整数, 逐位操作后最后 & `0xFFFFFFFF` 截取 32 位。
4. **191 vs 338** 的区别: 191 对单个数求 popcount; 338 对 0~n 每个数求 popcount, 用 DP 而非循环重复计算。
5. **201** 右移到相等: `while left != right` 而非 `left < right`; 若 `left == right` 则区间内只有一个数, AND 就是它本身, 不需要右移。
6. **67 Add Binary** 前导零: 用 int 转换时 `bin(...)` 结果前缀为 '0b', 用 `[2:]` 截取; 逐位模拟时从末尾向前加, 最后 carry 若为 1 需补到最前。
7. **ones/twos** 顺序: 137 中必须先更新 ones 再更新 twos, 顺序反了会导致逻辑错误 (两者互依赖)。

典型应用例题

例 1: 136. Single Number

题目: 数组中除某个数外, 其余每个数都恰好出现两次, 找出那个出现一次的数。要求 $O(n)$ 时间, $O(1)$ 空间。

思路：XOR 消消乐。 $x \oplus x = 0$ (相同数对消)， $x \oplus 0 = x$ (零不改变)。全部异或后，出现两次的数对消为 0，只剩出现一次的数。

解：

```
# 参考: solutions/bit_manipulation/p136_single_number.py
def singleNumber(nums: List[int]) -> int:
    result = 0
    for x in nums:
        result ^= x
    return result
```

分析：一次遍历，时间 $O(n)$ ，空间 $O(1)$ 。若用哈希表则 $O(n)$ 空间；用排序则 $O(n \log n)$ 时间。XOR 是最优解。

例 2: 191. Number of 1 Bits

题目：返回正整数 n 的 32 位二进制表示中 1 的个数（又称 Hamming Weight）。

思路：Brian Kernighan 算法： $n \& (n-1)$ 每次消去最低有效位（最右边的 1），循环次数等于 1 的个数，比逐位移位（固定 32 轮）更快。

解：

```
# 参考: solutions/bit_manipulation/p191_number_of_1_bits.py
def hammingWeight(n: int) -> int:
    count = 0
    while n:
        n &= n - 1    # 消去最低有效位
        count += 1
    return count
```

分析：时间 $O(k)$ ， k 为 1 的个数（最多 32），在稀疏位（1 较少）时比逐位移位快。 $n \& (n-1)$ 的原理： $n-1$ 将 n 的最低有效位（比如第 k 位）变为 0，其下方所有位变为 1；AND 后第 k 位以下全为 0，第 k 位以上不变。

例 3: 201. Bitwise AND of Numbers Range

题目：给定区间 $[left, right]$ ，计算该区间内所有整数的位与结果。

思路：若 $left \neq right$ ，区间内相邻整数必然在某位上一个为 0 一个为 1，该位 AND 后为 0。同步右移直到 $left == right$ ，即找到两端共同的二进制前缀，再左移回去即为答案。

解:

```
# 参考: solutions/bit_manipulation/p201_bitwise_and_of_numbers_range.py
def rangeBitwiseAnd(left: int, right: int) -> int:
    shift = 0
    while left != right:
        left >>= 1
        right >>= 1
        shift += 1
    return right << shift
```

分析: 时间 $O(\log n)$ (最多右移 32 次), 空间 $O(1)$ 。当 `left = 0` 时区间包含 0, AND 结果必为 0; 当 `left == right` 时结果即为 left 本身。

自测题

自测 1 (136 Single Number) ——`nums=[2,2,1]` 返回 1; `nums=[4,1,2,1,2]` 返回 4。提示: `result = 0; for x in nums: result ^= x`, 一行搞定。参考 `solutions/bit_manipulation/p136_single_number.py`。

自测 2 (137 Single Number II) ——`nums=[2,2,3,2]` 返回 3; `nums=[0,1,0,1,0,1,99]` 返回 99。提示: 按位统计, `count[bit] % 3` 即目标数该位; 或 ones/twos 三进制计数器 (先更新 ones 再更新 twos)。参考 `solutions/bit_manipulation/p137_single_number_ii.py`。

自测 3 (190 Reverse Bits) ——`n=43261596` (二进制 00000010100101000001111010011100), 反转后应为 964176192 (00111001011110000010100101000000)。提示: 32 轮循环, `result = (result << 1) | (n & 1)`; `n >>= 1`, 最后 `& 0xFFFFFFFF`。参考 `solutions/bit_manipulation/p190_reverse_bits.py`。

自测 4 (191 Number of 1 Bits) ——`n=11` (二进制 1011) 返回 3; `n=128` (二进制 10000000) 返回 1。提示: `while n: n &= n-1; count += 1` (Brian Kernighan) 或 `bin(n).count('1')`。参考 `solutions/bit_manipulation/p191_number_of_1.py`。

自测 5 (201 Bitwise AND Range) ——`left=5, right=7` (二进制 101, 110, 111), AND 结果为 100 = 4。`left=0, right=0` 返回 0。提示: 同步右移直到 `left==right`, shift 次数记录, 结果 `right << shift`。参考 `solutions/bit_manipulation/p201_bitwise_and_of_numbers_range.py`。

自测 6 (67 Add Binary) ——`a='11', b='1'` 返回 '100'; `a='1010', b='1011'` 返回 '10101'。提示: 从末尾逐位加, 维护 carry, 最后若 `carry=1` 则在最前加 '1'; 或 `bin(int(a,2)+int(b,2))[2:]`。参考 `solutions/bit_manipulation/p067_add_binary.py`。

题目全览 (6 题)

#	题目	套路分类	难度
136	Single Number	XOR 消消乐	Easy
137	Single Number II	按位 mod 3 / ones-twos 计数	Medium
190	Reverse Bits	32 轮逐位反转	Easy
191	Number of 1 Bits	Brian Kernighan n & (n-1)	Easy
201	Bitwise AND of Numbers Range	同步右移公共前缀	Medium
67	Add Binary	逐位模拟进位	Easy

融合版说明

段	来源	价值
一例速记	本文件	6 题套路一览 + AI（量化推理 / 哈希压缩）关联
思维路径还原	本文件	6 道题的解题独白，含关键公式
抽象成方法	本文件	6 个标准模板（XOR / 三进制 / Brian Kernighan / DP 位计数 / 位反转 / 公共前缀）+ 速查表
方法变形	本文件	3 类变体（XOR 扩展 / 位运算技巧速查 / DP 变体）
思考路标	本文件	9 条题型识别条件反射
易错点	本文件	7 条高频踩坑（Python 截断 / 负数 / 顺序依赖）
典型应用例题	solutions/	3 道精讲（136、191、201），代码 + 分析
自测题	leetcode	6 题带提示，链接 solutions 文件
题目全览	本文件	6 题完整列表

跨 category 导航：- 哈希表 → 14-hash-table.md（位运算常作为快速哈希的构件）- 数学 → 22-math.md（进制转换、快速幂与位运算紧密相关）- 子集枚举 → 08-backtracking.md（位掩码枚举所有子集）- CUDA Warp 级原子操作（atomicOr, atomicAnd）和 INT8 推理中的矩阵量化均大量使用本 category 的位运算技巧

22—Math (融合版)

难度: □□□□□ 题数: 6 核心套路: 快速幂 (分治)、整数平方根 (二分)、数字特性分析 (回文 / 尾零 / 共线)、进制模拟 (Plus One) 本文件: 覆盖 math 6 题的算法套路总结 + 典型题精讲 + 自测

一例速记

快速幂 (50 Pow(x,n)): $x^n = x^{n/2} \times x^{n/2}$, 指数折半, $O(\log n)$ 次乘法; 注意 $n < 0$ 时取倒数, n 为奇数时额外乘一次 x **整数平方根 (69 Sqrt(x))**: 二分查找 $[0, x]$ 内最大满足 $m^2 \leq x$ 的 m ; 注意用 $m \leq x/m$ 而非 $m^2 \leq x$ 防止整数溢出 (Python 不溢出可直接比较) **回文数 (9 Palindrome Number)**: 负数和末尾为 0 的数 (0 除外) 直接返回 False; 反转数字的后半段, 与前半段比较; 避免转字符串 **Plus One (66)**: 从末位向前逐位进位, 全 9 的情况退出循环后在首部追加 1 阶乘尾零 **(172 Trailing Zeroes)**: 尾零 = 10 的因子数 = $\min(2 \text{ 的因子数}, 5 \text{ 的因子数})$; 因 2 的因子远多于 5, 只需统计 5 的因子: $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \dots$ **直线上最多点数 (149 Max Points on a Line)**: 枚举每对点确定斜率, 斜率相同的点共线; 用 $(dy/gcd, dx/gcd)$ 的分数形式作 key 精确表示斜率, 避免浮点误差 **AI 关联**: 快速幂用于模幂运算 (密码学 / 同余运算); 数值稳定性 (浮点精度) 是深度学习训练中损失函数计算的核心问题; Softmax 减去 max 即为数值稳定性优化

思维路径还原

“看到 ‘50 Pow(x,n)’ → 快速幂 (分治): 若 $n == 0$ 返回 1.0; 若 $n < 0$ 则 $x = 1/x$, $n = -n$; 若 $n \% 2 == 0$: $half = myPow(x, n//2)$; $return half * half$; 若 $n \% 2 == 1$: $return x * myPow(x, n-1)$ 。递归深度 $O(\log n)$, 总乘法次数 $O(\log n)$ 。迭代版: 从低位到高位检查 n 的每一位, 若当前位为 1 则累乘当前 x , x 每轮平方。

看到 ‘69 Sqrt(x)’ → 二分: $lo, hi = 0, x$, $while\ lo \leq hi$: $mid = (lo+hi)//2$; $if\ mid*mid \leq x$: $ans=mid$; $lo=mid+1$; $else$: $hi=mid-1$ 。最终 ans 即为答案。Python 中 int 不溢出, 可以直接 $mid*mid$; 其他语言用 $mid \leq x // mid$ 防溢出。

看到 ‘9 Palindrome Number’ → 反转后半数字: 负数或 $(x \% 10 == 0 \text{ and } x != 0)$ → False; 将 x 后半数字逐位放入 rev ($rev = rev*10 + x\%10$; $x //= 10$), 直到 $x \leq rev$ (反转了一半); 比较: 偶数位 $x == rev$, 奇数位 $x == rev // 10$ (去掉中间数字)。

看到 ‘172 Factorial Trailing Zeroes’ → 统计 5 的因子: $n!$ 中 5 的因子个数 = $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \dots$; 循环: $result = 0$; $while\ n \geq 5$: $n //= 5$; $result += n$ 。

看到 ‘149 Max Points on a Line’ → 枚举 + 哈希斜率: 对每个点 i , 用哈希表统计其他点与 i 的斜率频次, 斜率用约分后的 $(dy/gcd, dx/gcd)$ 表示 (处理垂直线 $dx=0$ 时特殊处理), 每轮结果 = 哈希表中最大频次 + 1 (+1 是点 i 本身); 取所有轮次中的最大值, 时间 $O(n^2)$ 。”

学习目标

- 掌握快速幂 (50): 分治递归版和迭代位运算版, $O(\log n)$ 乘法
- 熟练二分搜索整数平方根 (69), 注意溢出保护
- 理解 172 尾零计数的数学原理 (只需统计 5 的因子)
- 掌握 9 的回文数判断: 反转后半段, 避免转字符串
- 熟练 149 的斜率哈希技巧: 用最简分数避免浮点误差
- 能对简单进位模拟 (66) 快速实现并处理全 9 边界

几何示意

图快速幂 (LC 50)

抽象成方法 (标准模板代码) ### 套路 1: 快速幂 (分治递归 + 迭代) 适用题: 50

```

“python
def
my_pow_recursive(x:
float,
n: int)
->
float:
"""50:
快速
幂,
时间
O(log
n),
空间
O(log
n)
(递归
栈)"""
if n
== 0:
return
1.0 if
n < 0:
return
my_pow_recursive(1
/ x, -n)
half =
my_pow_recursive(x,
n // 2)
if n %
2 ==
0:
return
half *
half
else:
return
half *
half *
x

```

```
def
```

```
my_pow_iterative(x:
```

```
float,
```

```
n: int)
```

```
->
```

```
float:
```

```
"""50:
```

```
快速
```

```
幂迭
```

```
代版,
```

```
时间
```

```
 $O(\log$ 
```

```
 $n)$ ,
```

```
空间
```

```
 $O(1)$ 。
```

```
从  $n$ 
```

```
的二
```

```
进制
```

```
低位
```

```
向高
```

```
位逐
```

```
位处
```

```
理。"""
```

```
if  $n <$ 
```

```
0:  $x, n$ 
```

```
 $= 1 /$ 
```

```
 $x, -n$ 
```

```
result
```

```
 $= 1.0$ 
```

```
while
```

```
 $n$ : if  $n$ 
```

```
 $\& 1$ : #
```

```
当前
```

```
位为
```

```
1, 将
```

```
当前
```

```
 $x$  累
```

```
乘到
```

```
结果
```

```
result
```

```
 $= x \times$ 
```

```
 $= x \times$ 
```

```
 $x$  平
```

```
方,
```

> 迭
代版：
 n 的
二进
制从
低位
到高
位，
每位
对应
一个
 x^{2^k} ，
若该
位为
1 则
累乘。
 $O(\log n)$
次乘
法，
 $O(1)$
空间。

套路 2：二分搜索整数平方根

适用题：69

```
def my_sqrt(x: int) -> int:
    """
    69: 计算  $x$  的整数平方根（向下取整）。时间  $O(\log x)$ ，空间  $O(1)$ 。
    """
    if x < 2:
        return x
    lo, hi = 1, x // 2    #  $\text{sqrt}(x) \leq x/2$  ( $x \geq 2$  时)
    ans = 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if mid <= x // mid:    #  $\text{mid} * \text{mid} \leq x$  (防溢出写法)
            ans = mid
            lo = mid + 1
        else:
```

快速幂（LC 50 Pow(x,n)）

计算 x^{13} ，二进制 $13 = 1101$ ，分治跳过 bit=0 的步骤

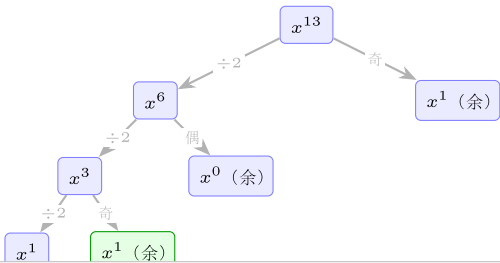
13 二进制: $\begin{matrix} 1 & 1 & 0 & 1 \\ 2^3 & 2^2 & 2^1 & 2^0 \end{matrix} = 8+4+1 = 13$

快速幂迭代步骤（从低位到高位）

步骤	当前位	base	操作	result
初始	—	base = x	n = 13 = 1101	result = 1
i=0	bit=1	x^1	result ×= base (x^1)	result = x^1
i=1	bit=0	x^2	跳过 (bit=0)	result = x^1
i=2	bit=1	x^4	result ×= base (x^4)	result = x^5
i=3	bit=1	x^8	result ×= base (x^8)	result = x^{13}

$x^{13} = x^8 \cdot x^4 \cdot x^1$ （对应二进制 **1 1 0 1** 中值为 1 的位）
base 每步平方: $x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8 \rightarrow \dots$

分治递归树（等价视角）



快速幂 $O(\log n)$ 时间：
迭代法: base=x, 每轮 base=base*base; n 为奇时 result*=base; n»=1
递归法: pow(x,n)=pow(x*x,n/2) (n 偶), 或 x*pow(x,n-1) (n 奇)
核心: 利用二进制分解, $x^n = \prod_{bit_i=1} x^{2^i}$

图 43: $x^{13} = x^8 \cdot x^4 \cdot x^1$ 二进制分解

```

        hi = mid - 1
    return ans

```

牛顿迭代法 (更快收敛, 面试进阶)

```

def my_sqrt_newton(x: int) -> int:
    """ 牛顿迭代:  $r = (r + x/r) / 2$ , 收敛到  $\sqrt{x}$ 。 """
    if x < 2:
        return x
    r = x
    while r * r > x:
        r = (r + x // r) // 2
    return r

```

套路 3: 回文数 (反转后半段)

适用题: 9

```

def is_palindrome(x: int) -> bool:
    """
    9: 判断整数是否为回文数, 不转字符串。时间  $O(\log x)$ , 空间  $O(1)$ 。
    """
    # 特判: 负数不是回文; 非零且末位为 0 不是回文
    if x < 0 or (x % 10 == 0 and x != 0):
        return False
    rev = 0
    while x > rev:          # 只反转后半段, 直到  $x \leq rev$ 
        rev = rev * 10 + x % 10
        x //= 10
    # 偶数位:  $x == rev$ ; 奇数位:  $x == rev // 10$  (跳过中间数字)
    return x == rev or x == rev // 10

```

套路 4: 加一 (进位模拟)

适用题: 66

```

from typing import List

```

```
def plus_one(digits: List[int]) -> List[int]:
    """
    66: 数组表示的十进制整数加 1。时间  $O(n)$ , 空间  $O(1)$  (除全 9 情况)。
    """
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0
    # 走到这里说明全是 9 (如 [9,9,9]), 在最前补 1
    return [1] + digits
```

套路 5: 阶乘尾零

适用题: 172

```
def trailing_zeroes(n: int) -> int:
    """
    172:  $n!$  的尾零个数 = 5 的因子个数。时间  $O(\log n)$ , 空间  $O(1)$ 。
    公式:  $\text{floor}(n/5) + \text{floor}(n/25) + \text{floor}(n/125) + \dots$ 
    """
    result = 0
    while n >= 5:
        n //= 5
        result += n
    return result
```

原理: 尾零 = 因子 10 = $\min(\text{因子 } 2 \text{ 数}, \text{因子 } 5 \text{ 数})$ 。因子 2 的数量远多于因子 5, 所以只统计因子 5。
 $n!$ 中 5 的因子: 每隔 5 个数贡献 1 个 5, 每隔 25 个数再贡献 1 个 (因为 $25 = 5^2$), 依此类推。

套路 6: 直线上最多点数 (斜率哈希)

适用题: 149

```
from math import gcd
from collections import defaultdict

def max_points(points: List[List[int]]) -> int:
```



```
"""
149: 枚举每对点，用约分斜率作哈希 key。时间  $O(n^2)$ ，空间  $O(n)$ 。
"""
n = len(points)
if n <= 2:
    return n
ans = 2
for i in range(n):
    slope_count: dict[tuple, int] = defaultdict(int)
    for j in range(i + 1, n):
        dy = points[j][1] - points[i][1]
        dx = points[j][0] - points[i][0]
        if dx == 0:
            key = (1, 0)          # 垂直线
        else:
            g = gcd(abs(dy), abs(dx))
            # 规范化：确保分母 dx 为正（统一斜率表示方向）
            if dx < 0:
                dy, dx = -dy, -dx
            key = (dy // g, dx // g)
        slope_count[key] += 1
    ans = max(ans, slope_count[key] + 1)  # +1 是点 i 本身
return ans
```

速查表

题型特征	套路	时间	空间
x^n (n 可负)	快速幂 (分治/迭代)	$O(\log n)$	$O(1)$ (迭代)
整数平方根 (向下取整)	二分搜索 / 牛顿迭代	$O(\log x)$	$O(1)$
判断回文数 (不转字符串)	反转后半段比较	$O(\log x)$	$O(1)$
数组表示的整数 +1	从末位进位模拟	$O(n)$	$O(1)$
$n!$ 的尾零个数	统计 5 的因子	$O(\log n)$	$O(1)$
直线上最多点数	枚举 + 约分斜率哈希	$O(n^2)$	$O(n)$

方法变形 (3 类)

变形 1: 快速幂扩展

- 50 (实数幂) → 基础模板。
- 模幂 (`pow(x, n, mod)`): 在每次乘法后取模, 用于密码学 (RSA 解密)。
- 矩阵快速幂: 将标量乘法换成矩阵乘法, 用于加速线性递推 (如斐波那契 $O(\log n)$)。
- Python 内置 `pow(x, n, mod)` 即为快速幂 + 模运算, 面试可直接用。

变形 2: 整数平方根 vs 二分搜索

- 69 (`Sqrt(x)`): 经典二分, 右边界可初始化为 $x//2$ (因为 $\sqrt{x} \leq x/2$ 对 $x \geq 4$ 成立)。
- 374 (Guess Number Higher or Lower, 非本 category): 同样是二分框架, 只是比较函数换为 `guess(mid)` API。
- 溢出保护: C++/Java 中 `mid * mid` 可能溢出 `int`, 改为 `mid <= x / mid`; Python 无溢出, 直接写 `mid * mid <= x`。

变形 3: 数论 / 精度

- 149 斜率精确表示: 浮点除法会有精度损失 (如 $1/3$ vs $2/6$ 不等), 用 `(dy//gcd, dx//gcd)` 的整数元组作 key 避免浮点误差。
- 172 质因数法推广: $n!$ 中质数 p 的因子个数 $= \lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \dots$ (勒让德公式)。
- 数值稳定性 (AI 关联): Softmax 计算中先减去最大值 ($x - \max(x)$), 避免 e^x 溢出, 是数值稳定性的典型应用; 与本 category 的整数精度控制思路一脉相承。

思考路标 (条件反射)

1. 看到 “计算 x^n / 快速幂” → 分治 $O(\log n)$; 迭代版更省空间
2. 看到 “整数平方根 / `floor(sqrt)`” → 二分, 右边界 $x//2$, 用 `mid <= x//mid` 防溢出
3. 看到 “回文数 / 不能转字符串” → 反转后半段, 与前半段比较
4. 看到 “数组表示整数 +1” → 从末位向前进位, 全 9 情况在最前加 1
5. 看到 “阶乘尾零 / 因子 5” → 勒让德公式, 循环除 5 累加
6. 看到 “斜率 / 共线” → 枚举 + 约分斜率哈希 (gcd 化简), 避免浮点
7. 看到 “负指数” → 快速幂: $n < 0$ 时 $x = 1/x$, $n = -n$
8. 看到 “溢出风险 / C++/Java” → 改为除法比较 `mid <= x // mid` (Python 无此问题)
9. 看到 “模幂 / 密码学” → `pow(x, n, mod)` (Python 内置快速幂)

易错点

1. **50 负指数**: $n = -2147483648$ (Python 最小整数, 但无溢出); $-n$ 直接可得, 无需担心; 但 Java/C++ 中 `INT_MIN` 取反会溢出, 需先转 `long`。
2. **50 n 为奇数的处理**: `half * half * x` 而非 `half * half + x`, 乘法不是加法。
3. **69 搜索区间**: `hi = x // 2` (而非 `x`) 可减少一半搜索范围; $x=1$ 时 $x//2 = 0$, 需特判 $x < 2$ 直接返回 x 。
4. **9 末尾为 0 的判断**: $x == 0$ 是特例 (回文), $x \% 10 == 0$ and $x != 0$ 才返回 `False`; 否则 100、10 等非回文数无法被过滤。
5. **9 奇数位 vs 偶数位**: 循环结束条件是 $x \leq \text{rev}$; 偶数位时 $x == \text{rev}$; 奇数位时多转移了中间数字, 应比较 $x == \text{rev} // 10$ 。
6. **66 原地修改**: Python 的 `list` 原地修改返回原数组 (最常见情况); 全 9 时创建新数组 `[1] + digits`, 注意不是 `digits.insert(0, 1)` (也可以, 但 `insert` 是 $O(n)$)。
7. **149 斜率规范化方向**: `gcd` 只保证约分, 但 $(-1, 2)$ 和 $(1, -2)$ 表示同一斜率; 规范化方案: 统一让分母 dx 为正 (若 $dx < 0$ 则 dy 和 dx 同时取反)。
8. **149 重复点**: 若两点坐标完全相同 ($dy=dx=0$), 需单独计数 (每对重复点都在任意直线上, 应加到所有斜率的计数中); 题目保证点各异时此情况不存在。

典型应用例题

例 1: 50. Pow(x, n)

题目: 实现 `pow(x, n)` 即 x^n , x 为浮点数, n 为整数 (可为负)。

思路: 快速幂。 $x^n = (x^{n/2})^2$ (n 为偶数) 或 $x \cdot (x^{(n-1)/2})^2$ (n 为奇数)。 $n < 0$ 时转化为 $(1/x)^{-n}$ 。迭代版枚举 n 的二进制位, 从低位到高位, 每次平方 x , 若当前位为 1 则累乘。

解:

```
# 参考: solutions/math/p050_powx_n.py
def myPow(x: float, n: int) -> float:
    if n < 0:
        x, n = 1 / x, -n
    result = 1.0
    while n:
        if n & 1:
            result *= x
        x *= x
        n >>= 1
    return result
```

分析: $O(\log n)$ 次乘法, $O(1)$ 空间。暴力循环 $O(n)$ 次乘法在 $n = 2^{31}$ 时超时。

例 2: 172. Factorial Trailing Zeroes

题目：给定整数 n ，返回 $n!$ 末尾 0 的个数。要求 $O(\log n)$ 时间。

思路：尾零 = 因子 10 的个数 = $\min(\text{因子 } 2 \text{ 的个数}, \text{因子 } 5 \text{ 的个数})$ 。因子 2 远多于因子 5，故只统计 5 的因子： $\lfloor n/5 \rfloor$ 个数倍数含 1 个 5， $\lfloor n/25 \rfloor$ 个数倍数额外含 1 个 5（共 2 个），依此类推。

解：

参考: `solutions/math/p172_factorial_trailing_zeroes.py`

```
def trailingZeroes(n: int) -> int:
    result = 0
    while n >= 5:
        n //= 5
        result += n
    return result
```

分析：循环执行 $O(\log_5 n)$ 次，时间 $O(\log n)$ ，空间 $O(1)$ 。例：100! 的尾零 = $\lfloor 100/5 \rfloor + \lfloor 100/25 \rfloor = 20 + 4 = 24$ 。

例 3: 149. Max Points on a Line

题目：给定平面上 n 个点，求最多有多少个点在同一条直线上。

思路：枚举基准点 i ，对其余每个点 j 计算与 i 的斜率（用约分分数表示），用哈希表统计各斜率的点数。取最大值即为过 i 的直线上的最多点数。对所有 i 取最大值。

解：

参考: `solutions/math/p149_max_points_on_a_line.py`

```
def maxPoints(points: List[List[int]]) -> int:
    n = len(points)
    if n <= 2:
        return n
    ans = 2
    for i in range(n):
        count: dict[tuple, int] = defaultdict(int)
        for j in range(i + 1, n):
            dy = points[j][1] - points[i][1]
            dx = points[j][0] - points[i][0]
            if dx == 0:
                key = (1, 0)
```

```

else:
    g = gcd(abs(dy), abs(dx))
    if dx < 0:
        dy, dx = -dy, -dx
    key = (dy // g, dx // g)
    count[key] += 1
    ans = max(ans, count[key] + 1)
return ans

```

分析：双重循环 $O(n^2)$ ，每次计算 gcd $O(\log(\max |dy|, |dx|))$ ，整体 $O(n^2 \log C)$ 。n <= 300 的约束下可通过。

自测题

自测 1 (9 Palindrome Number) ——x=121 返回 True, x=-121 返回 False, x=10 返回 False, x=0 返回 True。提示：负数直接 False；末尾为 0 且非 0 直接 False；反转后半段，循环终止条件 x <= rev；偶数位 x==rev，奇数位 x==rev//10。参考 solutions/math/p009_palindrome_number.py。

自测 2 (50 Pow(x,n)) ——x=2.0, n=10 返回 1024.0; x=2.1, n=3 返回约 9.261; x=2.0, n=-2 返回 0.25。提示：迭代版，n<0 时 x=1/x, n=-n; while n 循环，奇数位乘 x，每轮 x 平方，n 右移。参考 solutions/math/p050_powx_n.py。

自测 3 (66 Plus One) ——digits=[1,2,3] 返回 [1,2,4]; digits=[4,3,2,1] 返回 [4,3,2,2]; digits=[9,9,9] 返回 [1,0,0,0]。提示：从末位逐位检查，< 9 则 +1 直接返回，== 9 则置 0 继续，循环结束后前置 1。参考 solutions/math/p066_plus_one.py。

自测 4 (69 Sqrt(x)) ——x=4 返回 2, x=8 返回 2, x=0 返回 0, x=1 返回 1。提示：二分 [1, x//2], mid <= x//mid 时更新 ans 并收缩左边界，否则收缩右边界；x<2 时直接返回 x。参考 solutions/math/p069_sqrtx.py。

自测 5 (172 Trailing Zeroes) ——n=3 返回 0, n=5 返回 1, n=25 返回 6。提示：while n >= 5: n //= 5; result += n; 25 对应 5+1=6 (25//5=5, 5//5=1)。参考 solutions/math/p172_factorial_trailing_zeroes.py。

自测 6 (149 Max Points on a Line) ——points=[[1,1],[2,2],[3,3]] 返回 3; points=[[1,1],[3,2],[5,3],[4,1],[2,3]] 返回 4。提示：枚举基准点，约分斜率作 key，哈希表统计，结果加 1(基准点自身)。参考 solutions/math/p149_max_points_o

题目全览 (6 题)

#	题目	套路分类	难度
9	Palindrome Number	反转后半段比较	Easy
50	Pow(x, n)	快速幂 (分治 / 迭代)	Medium
66	Plus One	从末位进位模拟	Easy

#	题目	套路分类	难度
69	Sqrt(x)	二分搜索 / 牛顿迭代	Easy
149	Max Points on a Line	枚举 + 约分斜率哈希	Hard
172	Factorial Trailing Zeroes	勒让德公式（统计 5 因子）	Medium

融合版说明

段	来源	价值
一例速记	本文件	6 题套路一览 + AI（数值稳定性）关联
思维路径还原	本文件	6 道题的解题独白，含关键公式
抽象成方法	本文件	6 个标准模板（快速幂 / 二分平方根 / 回文数 / Plus One / 尾零 / 斜率哈希）+ 速查表
方法变形	本文件	3 类变体（快速幂扩展 / 二分变体 / 精度控制）
思考路标	本文件	9 条题型识别条件反射
易错点	本文件	8 条高频踩坑（负指数 / 奇偶位 / 斜率方向 / 溢出）
典型应用例题	solutions/	3 道精讲（50、172、149），代码 + 分析
自测题	leetcode	6 题带提示，链接 solutions 文件
题目全览	本文件	6 题完整列表

跨 category 导航：- 整数平方根 = 二分搜索变体 → 04-binary-search.md - 位运算版快速幂 → 21-bit-manipulation.md（迭代版快速幂逐位检测指数）- 回文字符串 → 01-array-string.md（151 / 5 等）- Softmax 数值稳定性（减去 max）是深度学习框架（PyTorch F.softmax）的标准实现，与本 category 的数值精度控制一脉相承

23 —Matrix（融合版）

难度：□□□□□ 题数：5 核心套路：原地修改（标记法 / 状态压缩）、螺旋遍历（边界缩进）、矩阵旋转（转置 + 翻转）、数独验证（哈希 / 位集）、生命游戏（编码历史态）本文件：覆盖 matrix 5 题的算法套路总结 + 典型题精讲 + 自测

一例速记

Set Matrix Zeroes (73): 用矩阵第一行 / 第一列作为标记空间, 记录哪行哪列需要置零; 需单独变量记录第一行 / 第一列本身是否含零, 最后统一处理, 避免标记与实际数据互相干扰, $O(1)$ 额外空间 **Rotate Image (48)**: 先沿主对角线转置 ($\text{matrix}[i][j], \text{matrix}[j][i] = \text{matrix}[j][i], \text{matrix}[i][j]$), 再水平翻转每行 ($\text{row.reverse}()$) $\rightarrow$ 顺时针旋转 90° ; 逆时针: 先水平翻转再转置 **Spiral Matrix (54)**: 四个边界指针 $\text{top} / \text{bottom} / \text{left} / \text{right}$, 按“右 $\rightarrow$ 下 $\rightarrow$ 左 $\rightarrow$ 上”顺序遍历, 每遍历一条边后收缩对应边界 **Valid Sudoku (36)**: 同时维护行、列、 3×3 宫格的已见数字集合, 遍历一次完成验证; 宫格编号 ($r//3, c//3$) **Game of Life (289)**: 规则同时应用到所有格, 用额外状态位编码“历史态”: 当前为 1 将变 0 $\rightarrow$ 标记为 2; 当前为 0 将变 1 $\rightarrow$ 标记为 3; 第二次遍历用 $\% 2$ 还原最终态 **AI 关联**: CNN 卷积核在 feature map 上的滑动窗口 = 矩阵分块遍历; 转置 / 旋转对应数据增强 (flip / rotate augmentation); 生命游戏是元胞自动机, 与 GPU 上的并行 SIMD 更新模型高度相关

思维路径还原

“看到 ‘73 将含零元素的行列全置零’ $\rightarrow O(1)$ 额外空间标记法: 第一步: 扫描第一行是否有 0 (标记 row0_has_zero), 扫描第一列是否有 0 (标记 col0_has_zero)。第二步: 对非第一行 / 列的元素, 若 $\text{matrix}[i][j] == 0$, 则 $\text{matrix}[i][0] = 0$ 且 $\text{matrix}[0][j] = 0$ 。第三步: 根据第一行 / 列标记, 将对应行 / 列置零 (从内向外, 避免覆盖标记)。第四步: 若 row0_has_zero 则第一行全置零, 若 col0_has_zero 则第一列全置零。

看到 ‘48 原地顺时针旋转 90° ’ $\rightarrow$ 转置 + 翻转: $\square$ 对每对 (i, j) ($i < j$) 交换 $\text{matrix}[i][j]$ 和 $\text{matrix}[j][i]$ (转置); $\square$ 对每行调用 $\text{row.reverse}()$ (水平翻转)。不要想着四角环形替换 (虽然也对), 转置 + 翻转更容易记忆且不易出错。

看到 ‘54 螺旋遍历’ $\rightarrow$ 四边界循环: 初始化 $\text{top}=0, \text{bottom}=m-1, \text{left}=0, \text{right}=n-1$ 。循环: 遍历顶行 ($\text{left} \rightarrow \text{right}$), $\text{top}++$; 遍历右列 ($\text{top} \rightarrow \text{bottom}$), $\text{right}--$; 若 $\text{top} \leq \text{bottom}$: 遍历底行 ($\text{right} \rightarrow \text{left}$), $\text{bottom}--$; 若 $\text{left} \leq \text{right}$: 遍历左列 ($\text{bottom} \rightarrow \text{top}$), $\text{left}++$; 直到结果数组填满。

看到 ‘36 数独验证’ $\rightarrow$ 三组集合同时维护: 遍历每个格子 (r, c) , 维护 $\text{rows}[r]$ 、 $\text{cols}[c]$ 、 $\text{boxes}[r//3][c//3]$ 三个集合, 若数字已在对应集合中则无效; 否则加入。

看到 ‘289 生命游戏’ $\rightarrow$ 编码多状态避免额外空间: $1 \rightarrow 0$ (活 $\rightarrow$ 死) 用 2 标记, $0 \rightarrow 1$ (死 $\rightarrow$ 活) 用 3 标记; 计算邻居时: $\text{cell} \& 1$ 取原始值 ($2 \& 1 = 0, 3 \& 1 = 1$); 第二轮: $2 \rightarrow 0, 3 \rightarrow 1$ (或统一用 $\text{cell} \% 2$)。”

学习目标

- 掌握矩阵原地标记法（73）：用第一行 / 列作标记空间，避免额外 $O(mn)$ 数组
- 熟练矩阵旋转的”转置 + 翻转”方法（48），以及四角环形替换的另一写法
- 掌握螺旋遍历的四边界指针模板（54），避免 off-by-one 错误
- 理解数独验证（36）的三维集合检查（行 / 列 / 宫格）
- 理解生命游戏（289）的多状态编码技巧，原地 $O(1)$ 额外空间实现
- 能识别 CNN feature map 操作与矩阵遍历的类比关系

几何示意

图矩阵旋转 90°（LC 48）

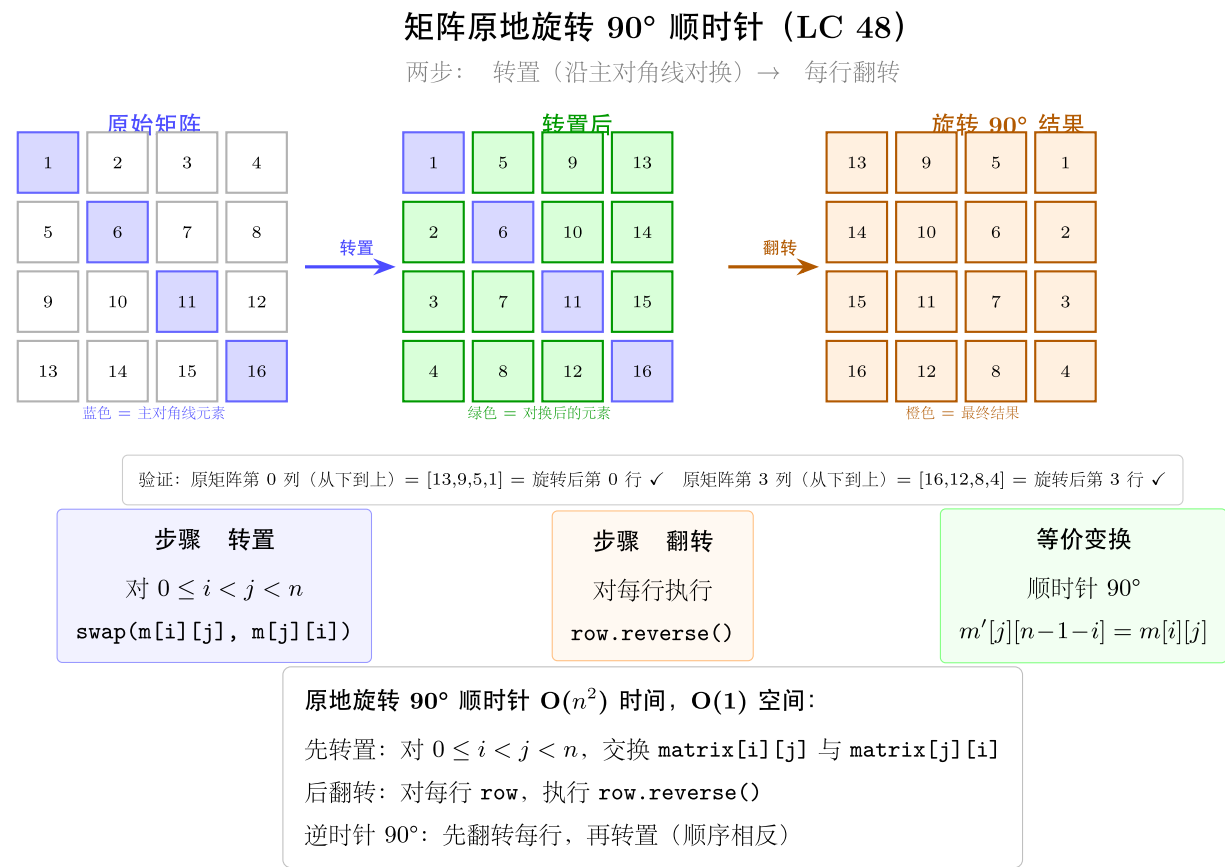


图 44: 转置 + 行翻转两步

图螺旋遍历（LC 54）

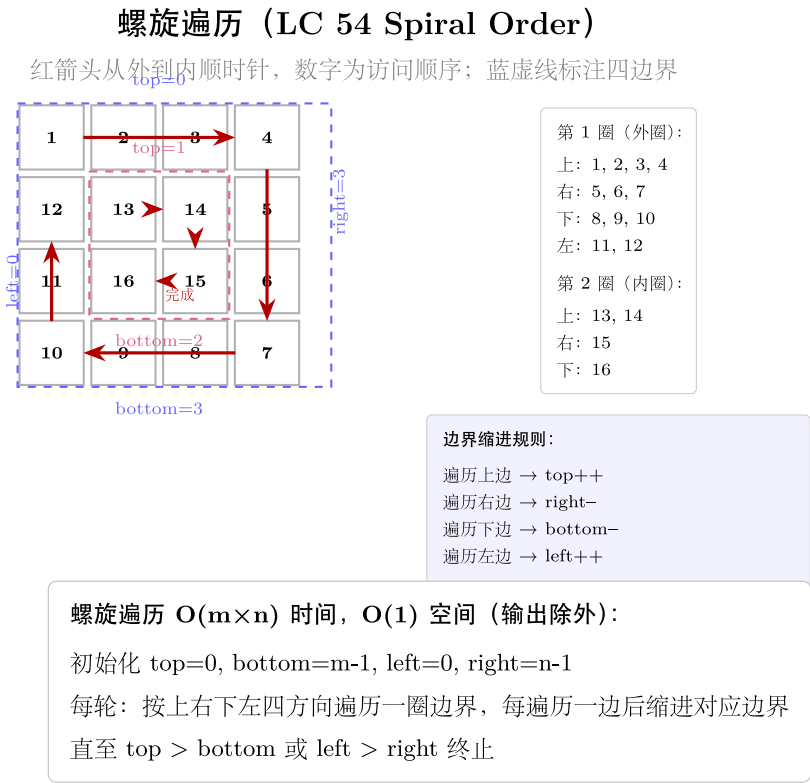


图 45: 4x4 + 红箭头螺旋 + 4 边界

抽象成方法 (标准模板代码) ### 套路 1: 原地标记法 (Set Matrix Zeros) 适用题: 73

```
“python
from
typing
im-
port
List
```

```

def
set_zeroes(matrix:
List[List[int]])
->
None:
"""73:
将含
零元
素所
在行
列全
部置
零，
原地
修改，
O(1)
额外
空间。
用第
一行
和第
一列
作为
标记
数
组。"""
m, n =
len(matrix),
len(matrix[0])
# 记
录第
一行 /
第一
列本
身是
否含
0
row0_zero
=
any(matrix[0][j]
== 0
for j
in
range(n))

```

```
# 用
第一
行 /
列标
记内
部零
元素
for i
in
range(1,
m):
for j
in
range(1,
n): if
ma-
trix[i][j]
== 0:
ma-
trix[i][0]
= 0
ma-
trix[0][j]
= 0
```

```
# 根
据标
记,
将内
部行
列置
零
(先处
理内
部,
再处
理第
一
行/列)
for i
in
range(1,
m): if
ma-
trix[i][0]
== 0:
for j
in
range(1,
n):
ma-
trix[i][j]
= 0
for j
in
range(1,
n): if
ma-
trix[0][j]
== 0:
for i
in
range(1,
m):
ma-
trix[i][j]
= 0
```

```

# 最
后处
理第
一行 /
第一
列 if
row0_zero:
for j
in
range(n):
ma-
trix[0][j]
= 0 if
col0_zero:
for i
in
range(m):
ma-
trix[i][0]
= 0 ““

```

套路 2: 矩阵旋转 (转置 + 翻转)

适用题: 48

```

def rotate_image(matrix: List[List[int]]) -> None:
    """
    48: 原地顺时针旋转 90°, 时间  $O(n^2)$ , 空间  $O(1)$ 。
    步骤: 沿主对角线转置; 水平翻转每行。
    """
    n = len(matrix)
    # 步骤 1: 转置 ( $matrix[i][j] \leftrightarrow matrix[j][i]$ ,  $i < j$  的部分)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # 步骤 2: 水平翻转每行
    for row in matrix:
        row.reverse()

```

逆时针旋转 90° (先水平翻转, 再转置)

```
def rotate_ccw(matrix: List[List[int]]) -> None:
    n = len(matrix)
    for row in matrix:
        row.reverse()
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

四角环形替换 (另一写法, 直接模拟环形旋转)

```
def rotate_image_ring(matrix: List[List[int]]) -> None:
    """
    逐层从外到内, 四角依次旋转。同样  $O(n^2)$  时间,  $O(1)$  空间。
    """
    n = len(matrix)
    for layer in range(n // 2):
        first, last = layer, n - 1 - layer
        for i in range(first, last):
            offset = i - first
            top = matrix[first][i]
            # 左 → 顶
            matrix[first][i] = matrix[last - offset][first]
            # 底 → 左
            matrix[last - offset][first] = matrix[last][last - offset]
            # 右 → 底
            matrix[last][last - offset] = matrix[i][last]
            # 顶 → 右
            matrix[i][last] = top
```

套路 3: 螺旋遍历 (四边界缩进)

适用题: 54

```
def spiral_order(matrix: List[List[int]]) -> List[int]:
    """
    54: 螺旋顺序遍历矩阵, 时间  $O(m \cdot n)$ , 空间  $O(1)$  (不含输出)。
    四边界: top, bottom, left, right, 按层收缩。
    """
    result: List[int] = []
```

```

top, bottom = 0, len(matrix) - 1
left, right = 0, len(matrix[0]) - 1

while top <= bottom and left <= right:
    # 顶行：从左到右
    for c in range(left, right + 1):
        result.append(matrix[top][c])
    top += 1
    # 右列：从上到下
    for r in range(top, bottom + 1):
        result.append(matrix[r][right])
    right -= 1
    # 底行：从右到左（需判断仍有行）
    if top <= bottom:
        for c in range(right, left - 1, -1):
            result.append(matrix[bottom][c])
        bottom -= 1
    # 左列：从下到上（需判断仍有列）
    if left <= right:
        for r in range(bottom, top - 1, -1):
            result.append(matrix[r][left])
        left += 1
return result

```

套路 4：数独验证（三组集合）

适用题：36

```

def is_valid_sudoku(board: List[List[str]]) -> bool:
    """
    36：验证数独是否有效（不需要能解出），时间  $O(81)=O(1)$ ，空间  $O(81)=O(1)$ 。
    三组集合：rows[9]、cols[9]、boxes[3][3]，各存储已出现的数字。
    """
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [[set() for _ in range(3)] for _ in range(3)]

    for r in range(9):
        for c in range(9):
            val = board[r][c]

```



```

    if val == '.':
        continue
    # 宫格编号
    br, bc = r // 3, c // 3
    if val in rows[r] or val in cols[c] or val in boxes[br][bc]:
        return False
    rows[r].add(val)
    cols[c].add(val)
    boxes[br][bc].add(val)
return True

```

套路 5: 生命游戏 (多状态编码)

适用题: 289

```

def game_of_life(board: List[List[int]]) -> None:
    """
    289: 按规则同时更新所有细胞, 原地修改,  $O(1)$  额外空间。
    状态编码:  $1 \rightarrow 0$  标记为 2 (活死),  $0 \rightarrow 1$  标记为 3 (死活)。
    读取原态:  $cell \& 1$  ( $2 \& 1 = 0$ ,  $3 \& 1 = 1$ , 保持原义)。
    """
    m, n = len(board), len(board[0])
    dirs = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]

    def count_live(r: int, c: int) -> int:
        return sum(
            1 for dr, dc in dirs
            if 0 <= r+dr < m and 0 <= c+dc < n and board[r+dr][c+dc] & 1
        )

    # 第一轮: 标记变化
    for r in range(m):
        for c in range(n):
            live = count_live(r, c)
            if board[r][c] == 1 and live not in (2, 3):
                board[r][c] = 2    # 活 → 死
            elif board[r][c] == 0 and live == 3:
                board[r][c] = 3    # 死 → 活

    # 第二轮: 还原最终态

```

```
for r in range(m):
    for c in range(n):
        board[r][c] %= 2      # 2→0（死），3→1（活），0/1 不变
```

速查表

题型特征	套路	时间	空间
含零行列全置零	第一行/列作标记 + 两轮遍历	$O(mn)$	$O(1)$
顺时针旋转 90°	转置 + 水平翻转	$O(n^2)$	$O(1)$
逆时针旋转 90°	水平翻转 + 转置	$O(n^2)$	$O(1)$
螺旋遍历	四边界缩进循环	$O(mn)$	$O(1)$
数独合法性验证	三组集合（行 / 列 / 宫格）	$O(1)$ （固定 81 格）	$O(1)$
生命游戏原地更新	多状态编码（2/3）+ 两轮遍历	$O(mn)$	$O(1)$

方法变形（3 类）

变形 1：原地标记扩展

- 73（Set Matrix Zeroes）→ 用第一行 / 列作标记， $O(1)$ 空间。
- 289（Game of Life）→ 用额外状态位（ $l=2$ ）标记旧态， $O(1)$ 空间。
- 两者共同思路：不能同时读旧值写新值——必须先“标记”再“还原”，或保证读写顺序不互相影响。
- 若允许 $O(m+n)$ 额外空间：73 可用两个布尔数组 `zero_rows[m]` 和 `zero_cols[n]`，代码更简单。

变形 2：旋转 / 翻转系列

- 顺时针 90°：转置 + 水平翻转（`row.reverse()`）。
- 逆时针 90°：水平翻转 + 转置（两步顺序互换）。
- 旋转 180°：垂直翻转（每行 `reverse`）+ 水平翻转（反转行顺序），或连续两次 90° 旋转。
- 数据增强（AI）：PyTorch `torchvision.transforms.RandomHorizontalFlip / RandomRotation` 本质上就是矩阵翻转 / 旋转，与 48 题完全相同的操作。

变形 3: 边界处理技巧

- **54 螺旋遍历**: 底行遍历前需判断 $top \leq bottom$ (防止单行矩阵重复遍历), 左列遍历前需判断 $left \leq right$ (防止单列矩阵重复遍历)。
- **59 (Spiral Matrix II, 填充螺旋, 非本 category)**: 框架完全相同, 改为写入而非读取。
- **36 宫格编号**: $(r//3, c//3)$ 将 9×9 网格划分为 9 个 3×3 宫格, 编号范围 $(0,0) \sim (2,2)$, 是固定技巧。

思考路标 (条件反射)

1. 看到“含零行列全置零 / $O(1)$ 空间” → 第一行 / 第一列作标记, 两阶段处理 (先内部后边界)
2. 看到“顺时针旋转 90° ” → 转置 + 水平翻转; 逆时针 → 水平翻转 + 转置
3. 看到“螺旋顺序 / 按层遍历” → 四边界指针 ($top / bottom / left / right$) 收缩
4. 看到“数独 / 九宫格验证” → 三组集合: 行 set + 列 set + $[r//3][c//3]$ 宫格 set
5. 看到“生命游戏 / 同时更新” → 多状态编码 (历史值嵌入当前格), $\& 1$ 读旧值, $\% 2$ 还原新值
6. 看到“矩阵搜索 (240 Search 2D Matrix II)” → 从右上角出发, 比目标大则左移, 比目标小则下移, $O(m+n)$ (此题虽不在本 category 但常与矩阵题一起出现)
7. 看到“CNN feature map” → 卷积 = 滑动矩阵窗口, 与螺旋 / 边界遍历思路类比
8. 看到“数据增强 flip / rotate” → 矩阵翻转 / 旋转, 与 48 完全相同

易错点

1. **73 标记顺序**: 必须先读内部区域再处理第一行 / 列; 若先处理第一行 / 列, 内部零元素写入的标记可能被错误置零, 导致标记信息丢失。
2. **73 第一行第一列需单独标记**: `row0_has_zero` 和 `col0_has_zero` 需在第二步 (用第一行 / 列作标记) 之前记录, 否则内部元素的标记会污染第一行 / 列的原始状态。
3. **48 转置范围**: 转置只交换上三角部分 (`j in range(i+1, n)`), 否则 (i, j) 和 (j, i) 会被交换两次回到原始状态。
4. **54 底行和左列的条件判断**: 处理底行前需判断 $top \leq bottom$ (否则单行矩阵会重复添加), 处理左列前需判断 $left \leq right$ (否则单列矩阵会重复添加); 两个条件是必需的, 不能省略。
5. **289 读旧值方式**: 用 `board[r][c] & 1` 而非 `board[r][c]` (因为可能已被标记为 2 或 3); 若直接读 `board[r][c]` 会把 2 误判为活细胞 (2 非 0)。
6. **289 还原新值**: 第二轮 `board[r][c] %= 2`: $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 0, 3 \rightarrow 1$, 正好对应最终状态。常见错误是用 `board[r][c] //= 2` 或忘记还原。
7. **36 宫格索引**: `boxes[r//3][c//3]` 而非 `boxes[r//3 * 3 + c//3]` (扁平化也可, 但矩阵形式更直观); 注意 `board` 中数字是字符串 `'1'~'9'`, 不是整数, `set` 中存字符串 `key`。

典型应用例题

例 1: 73. Set Matrix Zeroes

题目：给定 $m \times n$ 整数矩阵，若某元素为 0，则将其所在行和列全部置零。原地修改， $O(1)$ 额外空间。

思路：用矩阵第一行标记哪列有零，第一列标记哪行有零；单独记录第一行和第一列自身是否含零。分四步：

□ 记录第一行/列是否有零；□ 扫描内部标记；□ 根据标记置零内部；□ 最后处理第一行/列。

解：

```
# 参考: solutions/matrix/p073_set_matrix_zeroes.py
def setZeroes(matrix: List[List[int]]) -> None:
    m, n = len(matrix), len(matrix[0])
    row0 = any(matrix[0][j] == 0 for j in range(n))
    col0 = any(matrix[i][0] == 0 for i in range(m))
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0
                matrix[0][j] = 0
    for i in range(1, m):
        if matrix[i][0] == 0:
            for j in range(1, n):
                matrix[i][j] = 0
    for j in range(1, n):
        if matrix[0][j] == 0:
            for i in range(1, m):
                matrix[i][j] = 0
    if row0:
        for j in range(n): matrix[0][j] = 0
    if col0:
        for i in range(m): matrix[i][0] = 0
```

分析：时间 $O(mn)$ ，空间 $O(1)$ （仅使用两个布尔变量和矩阵自身的第一行 / 列）。

例 2: 48. Rotate Image

题目：给定 $n \times n$ 整数矩阵，将其顺时针旋转 90° ，原地修改。

思路：顺时针旋转 90° = 先沿主对角线转置，再水平翻转每行。数学推导：旋转后 $(i, j) \rightarrow (j, n - 1 - i)$ ；转置后 $(i, j) \rightarrow (j, i)$ ，再水平翻转后 $(j, i) \rightarrow (j, n - 1 - i)$ ，恰好吻合。

解:

```
# 参考: solutions/matrix/p048_rotate_image.py
def rotate(matrix: List[List[int]]) -> None:
    n = len(matrix)
    # 转置 (上三角交换)
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # 水平翻转每行
    for row in matrix:
        row.reverse()
```

分析: 转置 $O(n^2/2)$ 次交换, 翻转 $O(n^2/2)$ 次交换, 总体 $O(n^2)$ 时间, $O(1)$ 空间。

例 3: 54. Spiral Matrix

题目: 给定 $m \times n$ 矩阵, 按螺旋顺序 (顺时针) 返回所有元素。

思路: 四边界指针 (top / bottom / left / right), 按“右 → 下 → 左 → 上”顺序遍历每条边, 每遍历一条边后收缩对应边界。注意底行和左列在遍历前需检查边界有效性 (防止单行 / 单列重复)。

解:

```
# 参考: solutions/matrix/p054_spiral_matrix.py
def spiralOrder(matrix: List[List[int]]) -> List[int]:
    result = []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        for c in range(left, right + 1): result.append(matrix[top][c])
        top += 1
        for r in range(top, bottom + 1): result.append(matrix[r][right])
        right -= 1
        if top <= bottom:
            for c in range(right, left - 1, -1): result.append(matrix[bottom][c])
            bottom -= 1
        if left <= right:
            for r in range(bottom, top - 1, -1): result.append(matrix[r][left])
            left += 1
    return result
```

分析: 时间 $O(mn)$ (每个元素恰好访问一次), 空间 $O(1)$ (不含输出数组)。

自测题

自测 1（73 Set Matrix Zeroes）——`matrix=[[1,1,1],[1,0,1],[1,1,1]]` 应得 `[[1,0,1],[0,0,0],[1,0,1]]`；
`matrix=[[0,1,2,0],[3,4,5,2],[1,3,1,5]]` 应得 `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`。提示：先记录第一
行/列有无零，再用第一行/列标记内部，最后处理内部行列，最后处理第一行/列。参考 `solutions/matrix/p073_set_matrix`

自测 2(48 Rotate Image)——`matrix=[[1,2,3],[4,5,6],[7,8,9]]` 旋转后应为 `[[7,4,1],[8,5,2],[9,6,3]]`。
提示：先转置(只遍历 $j > i$ 的半三角)，再 `row.reverse()`。参考 `solutions/matrix/p048_rotate_image.py`。

自测 3（54 Spiral Matrix）——`matrix=[[1,2,3],[4,5,6],[7,8,9]]` 应返回 `[1,2,3,6,9,8,7,4,5]`；单行
`[[1,2,3,4]]` 应返回 `[1,2,3,4]`；单列 `[[1],[2],[3]]` 应返回 `[1,2,3]`。提示：四边界指针，底行左列前
需判断 `top<=bottom / left<=right`。参考 `solutions/matrix/p054_spiral_matrix.py`。

自测 4（36 Valid Sudoku）——标准数独盘面返回 True；若某行有两个 '5' 返回 False；某 3×3 宫格有重复返回
False。提示：三组集合，宫格编号 `(r//3, c//3)`，跳过 '.'。参考 `solutions/matrix/p036_valid_sudoku.py`。

自测 5(289 Game of Life)——`board=[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]`，一步后应为 `[[0,0,0],[1,0,1],[0,1,1],[0`
提示：第一轮标记 1→0 为 2, 0→1 为 3; `cell & 1` 读原值；第二轮 `% 2` 还原。参考 `solutions/matrix/p289_game_of_life.py`。

题目全览（5 题）

#	题目	套路分类	难度
36	Valid Sudoku	三组集合（行 / 列 / 宫格）验证	Medium
48	Rotate Image	转置 + 水平翻转	Medium
54	Spiral Matrix	四边界缩进循环	Medium
73	Set Matrix Zeroes	第一行 / 列作标记 + O(1) 空间	Medium
289	Game of Life	多状态编码原地更新	Medium

融合版说明

段	来源	价值
一例速记	本文件	5 题套路一览 + AI（CNN feature map / 数据增强）关联
思维路径还原	本文件	5 道题的解题独白，含关键技巧

段	来源	价值
抽象成方法	本文件	5 个标准模板（原地标记 / 旋转 / 螺旋 / 数独 / 生命游戏）+ 速查表
方法变形	本文件	3 类变体（原地标记扩展 / 旋转系列 / 边界处理）
思考路标	本文件	8 条题型识别条件反射
易错点	本文件	7 条高频踩坑（标记顺序 / 转置范围 / 条件判断 / 读旧值）
典型应用例题	solutions/	3 道精讲（73、48、54），代码 + 分析
自测题	leetcode	5 题带提示，链接 solutions 文件
题目全览	本文件	5 题完整列表

跨 category 导航：- 矩阵 BFS（岛屿 / 连通区域）→ 19-graph-general.md（200 Number of Islands）
- 矩阵二分搜索（74 / 240）→ 04-binary-search.md（矩阵有序性利用）- CNN 卷积核在 feature map 上的滑动 = 矩阵分块遍历，padding 和 stride 决定边界处理方式 - 生命游戏是冯·诺依曼元胞自动机的代表，GPU 上的并行细胞更新 = SIMD 向量化的矩阵原地操作